

SEM 广告投放策略

摘要

本文以某互联网公司 2025 年的 SEM 投放数据（12 个推广单元的 2627 条单元-日记录、365 天新注册数、2227 条关键词年度统计）为对象，建立“描述-分类-优化-预测”四层模型，全部计算在固定镜像的云端流水线上完成，论文中每个数字均可追溯到同一次运行。

问题一以漏斗指标、Lorenz 曲线和带周几、月份、法定节假日与调休哑变量的对数线性回归（HC3 稳健标准误）刻画投放策略。全年消费 142.59 万元、点击 834,815 次，CTR 3.70%，CPC 1.7081 元；消费高度集中（前 10 个词占 58.41%，Gini 系数 0.9676），40.0% 的关键词全年零消费，290 个关键词 ID 在多个单元重复投放。法定节假日日注册数变化 -73.72% （95% CI $[-79.99, -65.47]$ ， $p = 8.0 \times 10^{-22}$ ；Mann-Whitney 检验 $p = 4.56 \times 10^{-14}$ ），其中控制当日消费后的需求侧效应为 -24.64% ，其余来自公司在假日主动减投（消费 -76.68% ）；周一与周日的注册下降几乎完全由减投解释。非负最小二乘归因模型（留出期 $R^2 = 0.7375$ ）给出各单元注册率，单元级点击-消费弹性 $\gamma_u \in [0.6502, 0.9301]$ 证实边际收益递减。

问题二以对数消费额为成本、熵权复合指数（点击、浏览、注意力时长、归因注册）为效益，用两类 Fisher-Jenks 自然断点划分（成本阈值 41.59 元、效益阈值 0.2758）：黄金词 40、重点词 271、潜力词 977、问题词 49、无效词 890；与中位数法、高斯混合法的标签一致率为 78.36% 与 93.53%，独立校验器逐词复核通过。

问题三以幂函数响应 $q_i(x) = c_i(x/s_i)^{\gamma_u}$ 与每次点击的期望注册 ρ_i 建立每个单元-日的可分离凹规划（含最小投放额的关键词选择机制），两阶段拉格朗日-注水法精确求解；独立校验器复核可行性与 KKT 条件，凸松弛给出总体最优性界 0.080%。同预算下预期注册量较 2025 年“按历史比例”的做法提升 12.35%（2 月 1-8 日）与 28.00%（8 月 1-8 日），弹性自助 95% 区间为 $[11.67, 13.39]\%$ 与 $[27.53, 28.57]\%$ ，各灵敏度情形下提升均为正。

问题四以单元级日历回归预测效率乘子、CPC、CTR 与展位（滚动回测 80% 区间覆盖率 78.34%-87.26%），预算取 2025 年同期投入 23,488.02 元，两层分配后用 500 个蒙特卡罗情景给出 2026 年 9 月 11-17 日的期望范围：点击 17,487.9（10%-90% 分位 13,547.4-21,970.2）、注册 1,194.4（922.9-1,495.3）人、CPC 1.3431（1.0691-1.7337）元；若按全年日均口径安排同额预算，预期注册可再增加 96.0%。

关键词：搜索引擎营销；假日效应；熵权法与自然断点；可分离凹规划；滚动回测；蒙特卡罗模拟

1 问题重述

1.1 问题背景

搜索引擎营销 (SEM) 按关键词竞价购买搜索结果页的广告位：广告主为每个关键词出价并设定预算，搜索引擎按出价与质量决定展现位置，广告主按点击付费。竞价即时变化、展现与点击具有随机性，而关键词竞价成本持续攀升，单纯增加预算抢占流量的做法已不可持续。某互联网公司针对某产品投放 SEM 广告，设计了 5 个方案、12 个推广单元（每个单元是一组相关关键词的集合）以及数千个关键词，2025 年投入约 142 万元。公司希望通过优化关键词及其投放策略提高投放效益。

1.2 给定数据

附件 1 提供三张表：Sheet1 为 2025 年每个（日期、方案 ID、推广单元 ID）的日粒度记录，含展现量、点击量、消费额、上方位（前三名）展现量、上方首位展现量、上方位点击量与上方位消费额，共 2627 行；Sheet2 为每日新注册用户数（含 SEM 投放等渠道），共 365 行；Sheet3 为每个关键词的年度统计（序号、匿名关键词 ID、方案 ID、推广单元 ID、消费额、点击量、浏览量、跳出率、平均访问时长），共 2227 行。附件 2 为三个结果模板文件。

1.3 待解决的问题

1. **问题一**：从广告设计质量与创意、关键词管理与运用、出价策略与预算、投放策略与时间等方面分析该公司投放策略的合理性，并分析投放效益随时间的变化规律与假日效应。
2. **问题二**：按投入成本与效益把关键词分为黄金词（低成本、高效益）、重点词（高成本、高效益）、潜力词（低成本、低效益）、问题词（高成本、低效益）、无效词（无成本、无效益）五类，主要结果写入论文，详细结果按模板写入 `result2.xlsx`（方案 ID、推广单元、序号与五个类别标记列）。
3. **问题三**：在问题一、二的基础上，根据关键词的类型与关联关系，在各推广单元低成本、高效益、用完但不超过预算的条件下选择关键词，给出每个推广单元及其关键词的投放模型；给出 2025 年 2 月 1–8 日与 8 月 1–8 日每天的最优投放策略并说明其优越性，详细结果写入 `result3.xlsx`（日期、方案 ID、推广单元、关键词、投入金额、预期展位、预期点击量、预期浏览量、预期注册量）。
4. **问题四**：2026 年沿用 2025 年的方案、单元与关键词，预算不超过 2025 年的投入；考虑竞价逐日变化以及展现量、展现位、点击量、浏览量、注册量的不确定性，给出 2026 年 9 月 11–17 日每天的最优投放策略，并估计上述各量的期望范围，详细结果写入 `result4.xlsx`（结构同 `result3.xlsx`）。

2 问题分析

四个问题构成一条递进的链：问题一从数据中提取描述性事实与两组结构参数（各单元每次点击的注册率 r_u 与点击对消费的弹性 γ_u ）；问题二用这些参数定义关键词的效益并分类；问题三把响应函数、点击价值与类别约束组合为每个单元-日的预算分配模型；问题四在问题三的模型上叠加对未来日效率、竞价与展位的预测及其不确定性。图 1 给出技术路线，本节说明各问题的核心矛盾与处理思路。

2.1 问题一的分析

“合理性”没有唯一的量化定义，本文把它落实为四组可检验的事实：(i) 设计质量与创意由漏斗转化指标反映——点击率 CTR、平均点击成本 CPC、上方位与首位展现占比，以及上方位与非上方位的 CTR 差异；(ii) 关键词管理由长尾结构反映——零消费词占比、消费的集中度（Lorenz 曲线与 Gini 系数）、同一关键词在多个单元重复投放的数量；(iii) 出价与预算由消费在单元间的分布和边际收益反映——单元级弹性 $\gamma_u < 1$ 意味着边际收益递减，预算越集中于单一单元，边际注册成本越高；(iv) 投放时间由日历规律反映。假日效应的难点在于它混合了两种机制：需求侧（用户在假日搜索与注册减少）与供给侧（公司在假日主动减投）。本文用带日历哑变量的对数线性回归估计总效应，再在回归中控制当日消费额分离需求侧效应，并用 Mann-Whitney 非参数检验作稳健性对照。注册数只有日总量，本文用非负最小二乘把它归因到各单元的点击，得到后续问题所需的注册率 r_u 。

2.2 问题二的分析

题目给出的五类是“成本 $\times$ 效益”二维象限加一个退化类（无成本无效益）。需要决定三件事：效益如何度量、“高/低”的界限如何定、界限是否稳健。数据中关键词消费跨越六个数量级，因此成本取对数尺度；效益应综合点击、浏览深度、停留时长与注册贡献，本文用极差标准化后的熵权加权和（权重由数据的信息量决定，并与等权对照）；阈值采用两类 Fisher-Jenks 自然断点（一维精确解），并以中位数与高斯混合后验交点作对照，报告标签一致率。分类结果由与分类程序分离的校验器按原始列逐词重推。

2.3 问题三的分析

附件只有关键词的年度汇总而无逐日曲线，“投入-产出”关系必须借助单元级日序列估计：单元的日点击对日消费的弹性 γ_u 由 2025 年数据回归得到，关键词的响应曲线以其年度日均工作点 (s_i, c_i) 为锚、以 γ_u 为形状参数外推。目标函数取最终业务指标——预期注册量，等于点击量乘以每次点击的期望注册 ρ_i （单元注册率 $\times$ 跳出率质量因子）。预算在题目中未给出，本文取该单元当日 2025 年实际消费（使“最优策略”与“实际做法”在同样的钱上可比），并以窗口预算跨日调配作备选。每个单元-日的问题是带上界与最小投放额的可分离凹规划：不含最小投放额时 KKT 条件充要，解为注水

形式；含最小投放额时是非凸的选择问题，本文用拉格朗日价格确定入选集合再精确求解，并用凸松弛给出最优性界。优越性通过与“按历史比例分配”和“均匀分配”两条基线在同一响应模型、同一预算下的反事实比较来说明，弹性的抽样不确定性用参数自助法传播到提升幅度。

2.4 问题四的分析

预测目标点在样本末 8.5 个月之后，近期水平的持续性不可用，日历结构（周几、月份、节假日）是数据中唯一可外推的信息；因此本文对每个单元的效率乘子、CPC、CTR、上方位与首位占比分别建立日历回归模型，并用 2025 年第四季度的滚动起点回测（与季节朴素、28 日均值两条基线比较 MAE、pinball 损失与 80% 区间覆盖率）检验点预测与区间的校准。预算取 2025 年同期（9 月 11–17 日）各单元的实际消费，同时给出按全年日均折算的备选口径。不确定性以单元–日共同冲击、关键词特异冲击、CTR 与展位冲击和注册的泊松噪声进入响应函数；由于这些冲击以与投放额无关的乘性方式进入，期望注册最大化问题与用期望乘子的确定性问题同解，最优策略可先按日、再按关键词两层求解，期望范围由蒙特卡罗情景给出。

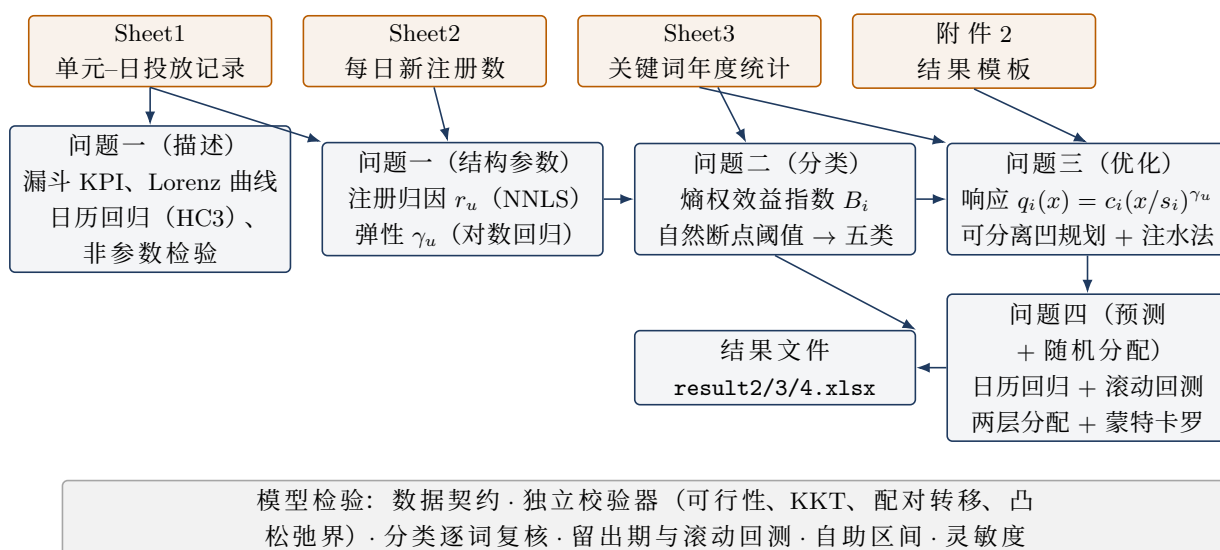


图 1 技术路线：问题一产出的注册率 r_u 与弹性 γ_u 贯穿后续三个问题；问题三的分配模型被问题四复用；全部结果经独立检验后写入结果文件。

3 模型假设与符号说明

3.1 模型假设

下列假设中，A1–A2 来自题目与附件的给定信息，A3–A8 为本文附加；每条给出依据与被违反时的后果。

假设 1 (数据口径). 附件 1 的记录完整地代表了 2025 年的投放：Sheet3 各单元的消费与点击合计与 Sheet1 完全对账（见第 4.1 节）；题述“6000 多个关键词”指账户内的全

部关键词，附件仅提供其中有统计记录的 2227 条，分析以附件为准。

假设 2 (节假日日历). 法定节假日与调休上班日按国务院办公厅 2025 年安排 [1] (元旦 1/1; 春节 1/28–2/4; 清明 4/4–4/6; 劳动节 5/1–5/5; 端午 5/31–6/2; 国庆中秋 10/1–10/8; 调休 1/26、2/8、4/27、9/28、10/11)。2026 年 9 月 11–17 日为周五至周四，不含法定节假日。

假设 3 (注册的线性可加归因). 日注册数等于基线、各单元点击的线性贡献与日历效应之和，单元 u 每次点击带来的期望注册 $r_u \geq 0$ 在年内不变。依据：注册只有日总量，线性可加是能 365 天数据识别 12 个单元贡献的最简结构；若违反（如渠道间存在协同）， r_u 只是平均意义上的边际率，分类与分配的相对结论仍成立，绝对注册量有偏。

假设 4 (幂函数响应与单元共享弹性). 关键词 i 在单日投放 x 下的期望点击为 $q_i(x) = c_i(x/s_i)^{\gamma_u}$ ，其中 (s_i, c_i) 为其 2025 年活跃日的日均消费与日均点击，形状参数 $\gamma_u \in (0, 1)$ 由所属单元的日序列估计并在单元内共享。依据：单元级回归的 $\hat{\gamma}_u \in [0.6502, 0.9301]$ 显著小于 1，幂函数在 11/12 个单元上优于指数饱和函数 (MDR-0005)；曲线过历史工作点保证“按历史比例分配”恰好复现历史点击。若违反（关键词弹性异质），最优解的具体投放额有偏，但凹性保证的分散化结论不变。

假设 5 (点击价值与浏览量). 关键词每次点击的期望注册 $\rho_i = r_u \kappa_i$ ， $\kappa_i = (1 - b_i)/(1 - \bar{b}_u)$ 为以单元点击加权平均跳出率 $\bar{b}_u$ 归一的跳出率质量因子（截断到 $[0.25, 2.5]$ ）；浏览量等于点击量乘以该词历史每次点击的浏览页数 d_i （无访问记录时用单元均值）。依据：跳出即离开，未跳出的访问才可能注册； κ_i 的截断避免极端小样本词主导分配。

假设 6 (日效应可分离). 单元 u 在日期 t 的实际点击等于响应函数值乘以日效率乘子 $\varepsilon_{u,t}$ ， $\varepsilon_{u,t}$ 只取决于日历（周几、月份、节假日）与随机冲击，与当日的关键词分配无关。依据： $\log \varepsilon_{u,t}$ 是弹性回归的残差结构；可分离性使跨日与日内分配可以分层求解。

假设 7 (预算与投放上限). 问题三中单元 u 在日期 t 的预算 $B_{u,t}$ 等于其 2025 年当日实际消费；问题四中单元 u 的 7 日预算等于其 2025 年 9 月 11–17 日的实际消费之和。单个关键词单日投放不超过其历史日均消费的 $\mu = 4.0$ 倍（问题词减半），且不低于其历史比例份额；低于 $m = 0.50$ 元的投放视为不投放。依据：题目未给预算，与实际消费相同的预算使“最优 vs 实际”可比；上限反映单个关键词的流量天花板（年度日消费 95% 分位约为均值的 2–4 倍），下限保证入选词有实际意义。上限倍数、预算与最小投放额的影响在灵敏度分析中给出。

假设 8 (2026 年的结构稳定性). 2026 年沿用 2025 年的方案、单元与关键词，各单元的弹性、注册率、CTR 与展位模型不变；未来的日效率、CPC、CTR 与展位仅由日历结构与 2025 年残差、月份间波动刻画。依据：题目明确沿用同样的方案与关键词；2026 年的市场趋势无法从一年的数据识别，本文以区间而非点值报告并在评价中说明。

3.2 符号说明

符号	含义	单位
$u \in \mathcal{U}, t, i \in \mathcal{K}_u$	推广单元、日期、单元 u 内的关键词	—
$I_{u,t}, C_{u,t}, S_{u,t}$	单元-日展现量、点击量、消费额	次、次、元
$I_{u,t}^{\text{top}}, I_{u,t}^1, C_{u,t}^{\text{top}}, S_{u,t}^{\text{top}}$	上方位展现量、首位展现量、上方位点击量与消费额	次、次、次、元
R_t	日新注册数	人
$S_i, C_i, V_i, b_i, \tau_i$	关键词年度消费额、点击量、浏览量、跳出率、平均访问时长	元、次、次、—、s
H_t, A_t, P_t, Q_t	法定节假日、调休上班日、假前一日、假后一日哑变量	—
δ_H, θ	假日效应（对数尺度）、注册对消费的弹性	—
$r_u, \bar{r}$	单元 u 每次点击的期望注册、合并注册率	人/次
η	点击的 adstock 衰减系数	—
γ_u, n_u	单元 u 的点击-消费弹性、2025 年活跃天数	—、d
s_i, c_i, d_i	关键词日均消费、日均点击、每次点击浏览页数	元、次、页/次
κ_i, ρ_i	跳出率质量因子、每次点击的期望注册	—、人/次
$q_i(x), \varepsilon_{u,t}$	关键词响应函数、单元-日效率乘子	次、—
z_{ij}, w_j, B_i	效益分量、熵权、综合效益指数	—
$x_i^c = \log_{10} S_i$	成本尺度	—
$B_{u,t}, x_i, \bar{x}_i$	单元-日预算、关键词投放额、投放上限	元
m, μ, ϕ_i	最小投放额、上限倍数、类型因子（问题词 0.5）	元、—、—
$a_i = \rho_i c_i s_i^{-\gamma_u}$	分配模型的目标系数	人/元 γ_u
λ	预算约束的影子价格	人/元
p^{top}, p^1, π	上方位占比、首位占比、预期展位指数	—、—、—
$X_{u,t}$	问题四中单元 u 分到日期 t 的预算	元
σ_u, σ_u^m	日历回归的残差标准差、月份效应标准差	—

4 问题一：投放策略的合理性与时间规律

本节先说明数据口径与质量，再从设计质量与创意、关键词管理、出价与预算、投放时间四个方面给出可检验的事实，最后估计注册归因模型与单元级弹性，为后续问题提供结构参数。

4.1 数据说明与质量

所有数据经契约校验（列类型、取值范围、漏斗一致性：点击 $\leq$ 展现、上方位展现 $\leq$ 展现、首位展现 $\leq$ 上方位展现、上方位点击 $\leq$ 点击、上方位消费 $\leq$ 消费）全部通过。Sheet1 有 2627 条单元-日记录，覆盖 5 个方案、12 个推广单元和 2025 年全部 365 天，（日期、方案、单元）无重复；各单元的投放起止不同（四个单元自 5 月 21 日起投，一个单元 11 月 28 日停投），春节等假期多数单元停投，293 条记录有展现而消费为零。全年合计消费 1,425,949.79 元（题述“142 余万元”）、点击 834,815 次、展现 22,580,381 次。Sheet2 的新注册数合计 85,313 人。Sheet3 有 2227 条关键词记录（题述“6000 多个关键词”多于附件行数，本文以附件为准，假设 1），关键词 ID 为匿名整数，共

1884 个不同 ID，其中 290 个 ID 被 2-4 个单元共用（涉及 633 条记录）；890 条（40.0%）全年零消费零点击，其跳出率与时长以 “/” 标记；跳出率为 0-1 的小数，平均访问时长为 “HH:MM:SS” 或数值 0（84 行）。Sheet3 分单元的消费与点击合计与 Sheet1 完全对账（总消费差异 0.02 元，为四舍五入）。

4.2 指标体系与四个方面的事实

以单元 u 的年度合计定义漏斗指标

$$CTR_u = \frac{\sum_t C_{u,t}}{\sum_t I_{u,t}}, \quad CPC_u = \frac{\sum_t S_{u,t}}{\sum_t C_{u,t}}, \quad p_u^{top} = \frac{\sum_t I_{u,t}^{top}}{\sum_t I_{u,t}}, \quad p_u^1 = \frac{\sum_t I_{u,t}^1}{\sum_t I_{u,t}}, \quad (1)$$

关键词消费的集中度用按消费降序的 Lorenz 曲线与 Gini 系数度量：对有消费的 n 个词按 $S_{(1)} \leq \dots \leq S_{(n)}$ 排序，

$$G = \frac{n + 1 - 2 \sum_{k=1}^n (\sum_{j \leq k} S_{(j)}) / \sum_j S_{(j)}}{n}. \quad (2)$$

表 2 给出各单元的年度 KPI，图 2 给出单元消费占比、CPC-CTR 散点与关键词 Lorenz 曲线。

表 2 各推广单元 2025 年投放 KPI（按消费额降序；CTR、CPC、上方位与首位占比按式 (1)）

方案 ID	推广单元 ID	投放天数	消费额/元	占比/%	展现量	点击量	CTR/%	CPC/元	上方位/%	首位/%
500635396	9657930100	321	808,153	56.7	8,769,221	358,634	4.09	2.25	44.0	11.7
525368335	9811363528	259	181,497	12.7	4,780,540	100,380	2.10	1.81	8.9	7.2
495817671	9630806627	309	171,870	12.1	4,417,936	107,520	2.43	1.60	17.8	8.4
495403620	8878077181	225	124,226	8.7	255,414	168,752	66.07	0.74	88.6	81.9
495403620	9628223555	321	91,605	6.4	3,454,523	72,103	2.09	1.27	18.5	8.7
500635396	9700005700	148	26,940	1.9	163,203	12,173	7.46	2.21	33.1	30.9
500635396	9773660250	161	6,434	0.5	180,446	4,206	2.33	1.53	28.6	8.6
63563817	4190988310	172	4,982	0.3	125,307	3,283	2.62	1.52	14.4	3.3
63563817	1841014603	178	4,768	0.3	113,857	4,023	3.53	1.19	19.1	14.4
63563817	2874903000	178	2,848	0.2	87,675	2,019	2.30	1.41	21.0	7.1
63563817	4190988309	178	2,478	0.2	222,330	1,635	0.74	1.52	9.7	1.8
63563817	2526158329	177	149	0.0	9,929	87	0.88	1.71	3.9	1.1

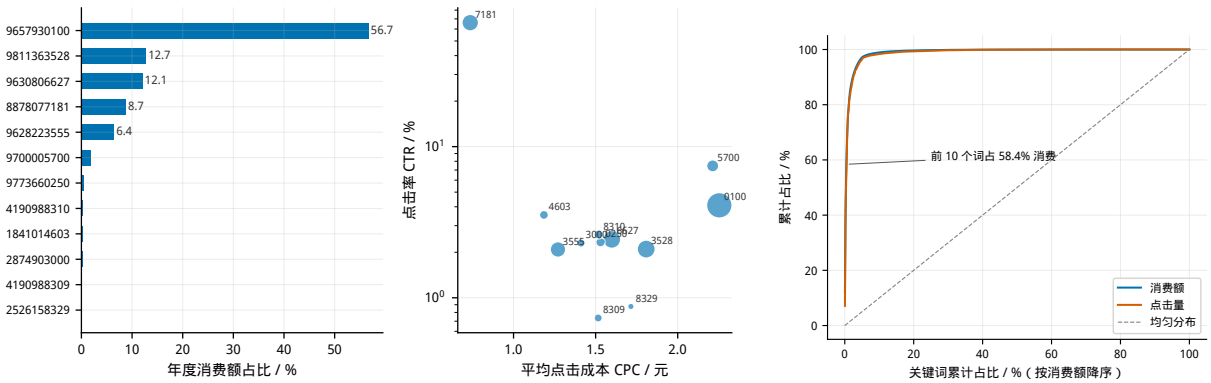


图 2 左：各推广单元年度消费额占比；中：单元 CPC-CTR 散点（对数纵轴，气泡面积 $\propto$ 消费占比，标签为单元 ID 后四位）；右：关键词消费额与点击量的 Lorenz 曲线（按消费额降序累计），虚线为均匀分布。

设计质量与创意 账户整体 CTR 为 3.70%，CPC 1.7081 元。上方位（前三名）展现只占 27.11%（首位 10.39%），却贡献了 67.06% 的点击与 70.35% 的消费：上方位 CTR 为 9.15%，非上方位仅 1.67%，相差五倍以上，而上方位 CPC（1.7918 元）只比非上方位（1.5377 元）高约 17%。因此创意本身在能进入首页上方区域时是有效的，问题在于近四分之三的展现落在上方区域之外且几乎不产生点击——这些展现对应的关键词匹配或质量度不足。单元 8878077181 只有 4 个关键词，CTR 高达 66.1%、上方位占比 88.6%、CPC 仅 0.74 元，属品牌/导航型流量，说明品牌词的创意与匹配是充分的；相反，单元 4190988309 的 CTR 仅 0.74%（表 2），其创意或关键词匹配需要重新审视。

关键词管理与运用 关键词呈极端长尾：40.0% 的词全年零消费零点击；有消费的 1337 个词中，前 10 个词占消费 58.41%，前 100 个占 96.40%，Gini 系数 0.9676（图 2 右）。290 个关键词 ID 同时出现在多个单元中，同一账户的不同单元会在同一查询上相互竞价，抬高自身 CPC；关键词库中约四成的词从未获得流量，说明词库缺乏定期清理与匹配方式调整。

出价策略与预算 消费在单元间同样集中：单元 9657930100 占全年消费 56.67%，各单元 CPC 从 0.74 元到 2.25 元不等（表 2）。第 4.4 节估计的点击-消费弹性 γ_u 全部小于 1（合并估计 0.8355，标准误 0.0113），即边际收益递减；而消费最大的单元 9657930100 的弹性最低（表 4），其 CPC 也最高，说明预算在该单元上已进入边际成本快速上升的区域，把部分预算转移到弹性更高、CPC 更低的单元能以同样的钱换取更多点击与注册——这正是问题三、四的优化空间。

投放策略与时间 图 3 显示日消费、点击与注册的全年走势：三条序列高度同步（注册与消费的相关系数约 0.8），春节后 2-3 月出现全年高峰，6-7 月为低谷；每周内呈明显的“周一、周日低”模式，法定节假日（阴影）期间三者同时深跌。这些规律与假日效应在下一小节用回归量化。

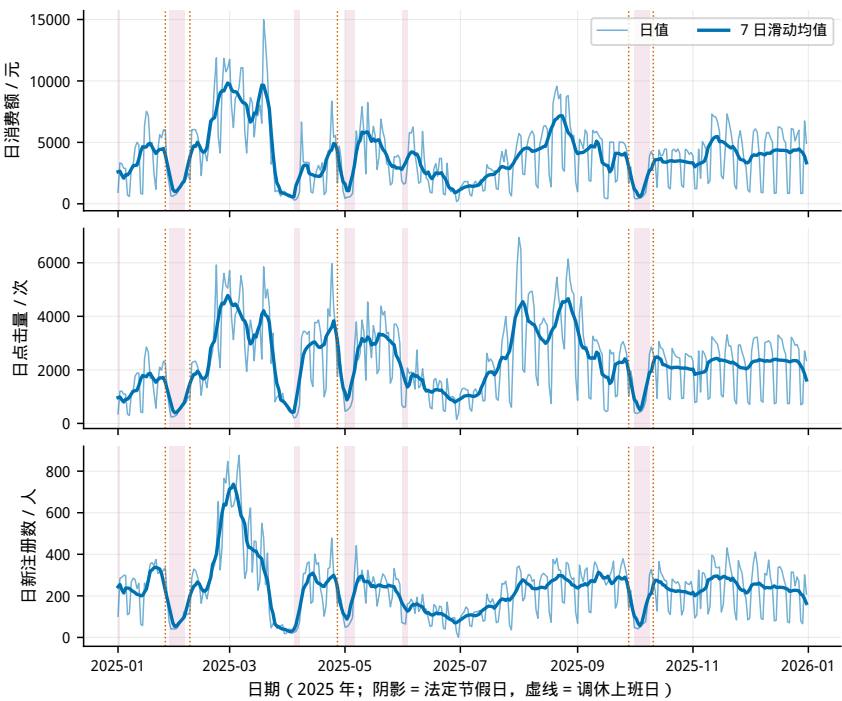


图 3 2025 年日消费额、日点击量与日新注册数（细线为日值，粗线为 7 日滑动均值；阴影为法定节假日，竖虚线为调休上班日）。

4.3 日历回归与假日效应

对账户日总量 $y_t \in \{R_t, S_t, C_t, I_t\}$ 建立对数线性回归

$$\log y_t = \alpha + \sum_{w \neq \text{周三}} \beta_w D_t^w + \sum_{m \neq 1} \mu_m M_t^m + \delta_H H_t + \delta_A A_t + \delta_P P_t + \delta_Q Q_t + e_t, \quad (3)$$

其中 D_t^w 、 M_t^m 为周几与月份哑变量（基准为周三与 1 月）， H_t, A_t, P_t, Q_t 分别为法定节假日、调休上班日、假前一日、假后一日的指示变量（假设 2）。系数 δ 的效应以 $100(e^\delta - 1)\%$ 报告，标准误采用 MacKinnon–White 的 HC3 异方差稳健形式 [2]。对 CPC、CTR 与上方位占比以水平值代替对数。为把假日的需求侧效应与公司主动减投分离，再估计控制当日消费额的模型

$$\log R_t = \alpha' + \theta \log S_t + \sum_w \beta'_w D_t^w + \sum_m \mu'_m M_t^m + \delta'_H H_t + \delta'_A A_t + \delta'_P P_t + \delta'_Q Q_t + e'_t, \quad (4)$$

其中 θ 为注册对消费的弹性， δ'_H 是同样投入下假日仍然存在的注册变化。作为非参数对照, 用 Mann–Whitney U 检验 [3] 比较法定节假日（28 天）与普通工作日（243 天）的日注册数分布。

表 3 日历回归 (3) 的效应估计（相对周三 / 普通日，%；HC3 稳健 95% 置信区间与 p 值）

因素	注册效应/%	95% CI	p	消费效应/%	95% CI	p	点击效应/%	p
法定节假日	-73.7	[-80.0, -65.5]	0.000	-76.7	[-83.2, -67.6]	0.000	-75.8	0.000
调休上班日	-2.4	[-49.5, 88.6]	0.942	-1.0	[-58.6, 136.4]	0.982	0.6	0.989
假前一日	-54.3	[-84.2, 32.1]	0.148	-49.3	[-77.5, 14.2]	0.101	-41.4	0.253
假后一日	-17.3	[-47.3, 29.9]	0.410	-25.9	[-53.3, 17.7]	0.204	-26.3	0.234
周一	-53.8	[-65.2, -38.8]	0.000	-59.7	[-68.9, -47.9]	0.000	-53.9	0.000
周二	-3.4	[-21.5, 18.9]	0.743	-9.7	[-30.5, 17.4]	0.448	-7.1	0.513
周四	-5.0	[-23.5, 18.0]	0.644	1.6	[-19.6, 28.4]	0.896	1.3	0.902
周五	-4.7	[-21.5, 15.7]	0.627	-1.1	[-21.4, 24.5]	0.925	-1.8	0.848
周六	-12.6	[-27.5, 5.3]	0.156	-3.9	[-22.9, 19.8]	0.725	-6.1	0.489
周日	-47.4	[-58.5, -33.2]	0.000	-52.5	[-63.4, -38.2]	0.000	-46.9	0.000

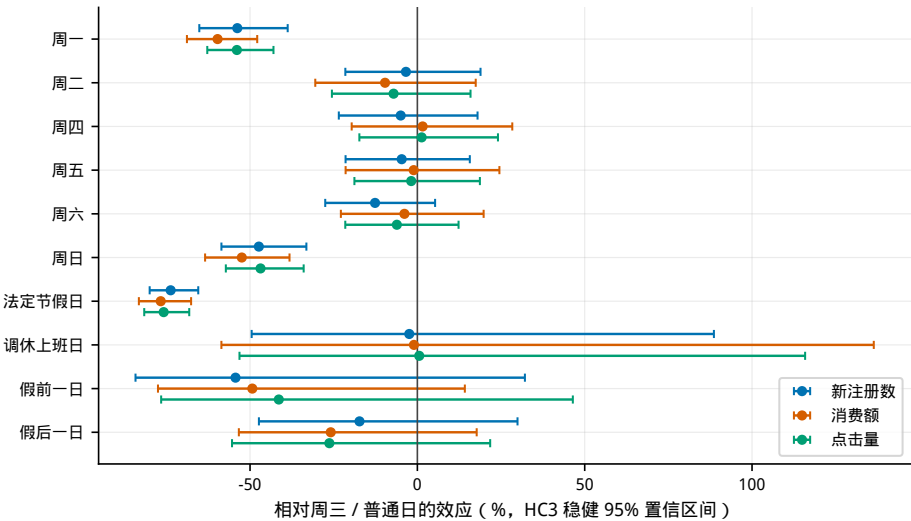


图 4 周几、法定节假日、调休、假前与假后对日新注册数、消费额与点击量的效应（相对周三 / 普通日，%），横线为 HC3 稳健 95% 置信区间。

表 3 与图 4 给出估计结果。法定节假日的注册数效应为 -73.72% (95% CI $[-79.99, -65.47]$, $p = 8.0 \times 10^{-22}$), 消费额 -76.68% 、点击量 -75.81% 、展现量 -70.78% 同样显著; Mann-Whitney 检验给出假日与工作日注册中位数 58 与 271 人 (比值 0.2140), $p = 4.56 \times 10^{-14}$, 两种方法结论一致。价格与质量指标没有假日效应: CPC 的假日系数为 -0.0240 元 ($p = 0.8043$), CTR 为 -0.242 个百分点 ($p = 0.5740$); 上方位占比在假日下降 -4.64 个百分点 ($p = 0.0432$), 与减投后排位后移一致。周几效应显著 (Wald 检验 $p = 3.53 \times 10^{-12}$): 周一注册 -53.81% 、周日 -47.36% , 周六 -12.64% ($p = 0.1561$), 周二至周五与周三无显著差异; 调休上班日与普通日无差异 (-2.42% , $p = 0.9419$); 假前一日、假后一日的点估计为负但因样本少不显著。

控制消费额的模型 (4) 把上述总效应分解: 注册对消费的弹性 $\theta = 0.7235$ (SE 0.0631), R^2 由 0.4802 升至 0.7829; 同样投入下假日注册仍低 -24.64% (95% CI $[-40.01, -5.32]$, $p = 0.0151$), 即约三分之一的假日跌幅来自需求侧, 其余来自公司假日减投 -76.68% ; 而周一 (-10.85% , $p = 0.2665$) 与周日 (-9.86% , $p = 0.2684$) 在控制消费后不再显著, 说明这两天的注册下降几乎完全是公司减投的结果。

假日减投是否合理, 可用边际收益均衡原则判断。设注册对消费的响应为 $R = KS^\theta$, 假日需求侧乘子为 $e^{\delta'_H}$, 则假日与普通日边际注册相等要求 $e^{\delta'_H} \theta K S_H^{\theta-1} = \theta K S_N^{\theta-1}$, 即最优的假日/普通日消费比

$$\frac{S_H^*}{S_N^*} = \exp\left(\frac{\delta'_H}{1-\theta}\right) = 36.0\%, \quad (5)$$

即最优减投幅度约 64.0%, 与实际减投 -76.68% 相比略偏保守但基本合理; 相反, 周一与周日的请求侧效应不显著, 实际减投幅度明显超出请求侧效应所能解释的范围, 这两天以同样的边际成本放弃了注册量, 是投放时间安排中最值得改进之处。

4.4 注册归因与单元级结构参数

附件只有日注册总量, 为得到各单元每次点击的价值, 建立带日历项的归因回归 (假设 3)

$$R_t = c_0 + \sum_{u \in \mathcal{U}} r_u \tilde{C}_{u,t} + \beta^\top D_t + e_t, \quad \tilde{C}_{u,t} = C_{u,t} + \eta \tilde{C}_{u,t-1}, \quad r_u \geq 0, \quad (6)$$

其中 D_t 为式 (3) 的日历哑变量, $\tilde{C}$ 为 adstock 点击 (衰减 η 在 $\{0, 0.2, 0.4, 0.6\}$ 中由滚动起点交叉验证的 MAE 选取), 系数 r_u 用非负最小二乘 [4] 估计。点击数不足 10 000 或估计值落在合并估计 $\bar{r}$ 的 $[0.25, 4]$ 倍之外的单元使用合并估计 (把所有单元点击加总后同样回归得到)。式 (6) 选出 $\eta = 0.0$ (无显著滞后效应), 全样本 $R^2 = 0.7951$, 以前 70% 天数训练、后 30% 留出的 $R^2 = 0.7375$ 、MAPE 30.19% (图 5)。基线注册 151.39 人/天, 合并注册率 7.9475 人/百次点击; 5 个单元保留自身估计, 归因到 SEM 点击的注册合计 60,269 人 (占全年注册 70.65%), 平均归因注册成本 23.66 元 (表 4)。

单元级点击-消费弹性由活跃日的对数回归估计 (假设 4):

$$\log C_{u,t} = a_u + \gamma_u \log S_{u,t} + \sum_w \beta_{u,w} D_t^w + e_{u,t}, \quad (7)$$

合并模型 (单元固定效应) 给出 $\gamma = 0.8355$ (SE 0.0113); 活跃天数不少于 60 且估计落在 $[0.3, 0.95]$ 内的 10 个单元采用自身估计 (0.6502–0.9301), 其余 (品牌型单元 8878077181 的 $\hat{\gamma} > 1$ 源于品牌流量的爆发式波动, 另一单元活跃天数不足) 采用合并估计。图 6 表明双对数尺度上的线性关系在各单元都成立, R^2 在 0.65–0.97 之间 (表 4)。

表 4 各单元的注册率（式 (6)）、归因注册与注册成本，以及弹性估计（式 (7)）与采用值

推广单元 ID	注册率/(人/百次点击)	来源	归因注册/人	注册成本/元	$\hat{\gamma}_u$	SE	R^2	采用 γ_u	γ 来源
1841014603	7.947	合并估计	320	14.9	0.930	0.048	0.89	0.930	单元
2526158329	7.947	合并估计	7	21.6	0.494	0.096	0.65	0.836	合并
2874903000	7.947	合并估计	160	17.7	0.811	0.036	0.92	0.811	单元
4190988309	7.947	合并估计	130	19.1	0.650	0.033	0.87	0.650	单元
4190988310	7.947	合并估计	261	19.1	0.882	0.026	0.94	0.882	单元
8878077181	4.346	单元估计	7,334	16.9	1.325	0.066	0.88	0.836	合并
9628223555	2.785	单元估计	2,008	45.6	0.836	0.018	0.87	0.836	单元
9630806627	7.369	单元估计	7,923	21.7	0.853	0.013	0.96	0.853	单元
9657930100	7.759	单元估计	27,828	29.0	0.699	0.025	0.82	0.699	单元
9700005700	7.947	合并估计	967	27.8	0.896	0.033	0.93	0.896	单元
9773660250	7.947	合并估计	334	19.2	0.750	0.085	0.86	0.750	单元
9811363528	12.948	单元估计	12,997	14.0	0.825	0.010	0.97	0.825	单元

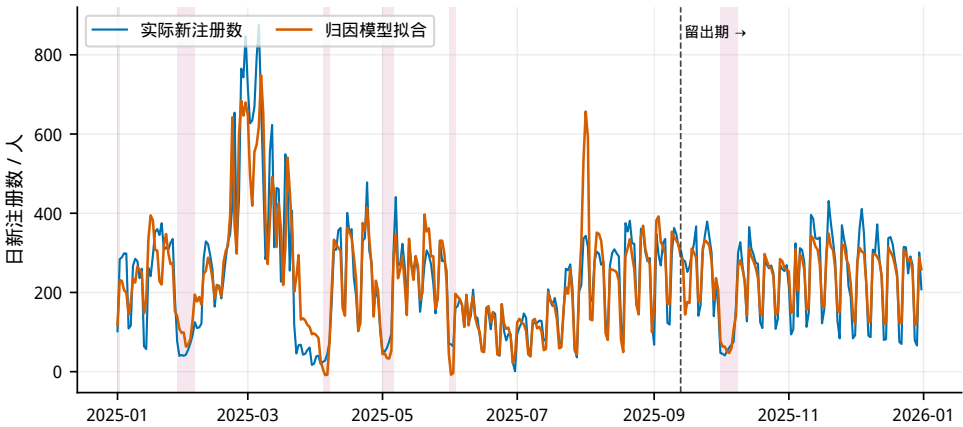


图 5 日新注册数与归因回归 (6) 的拟合值（竖虚线右侧为留过期；阴影为法定节假日）。

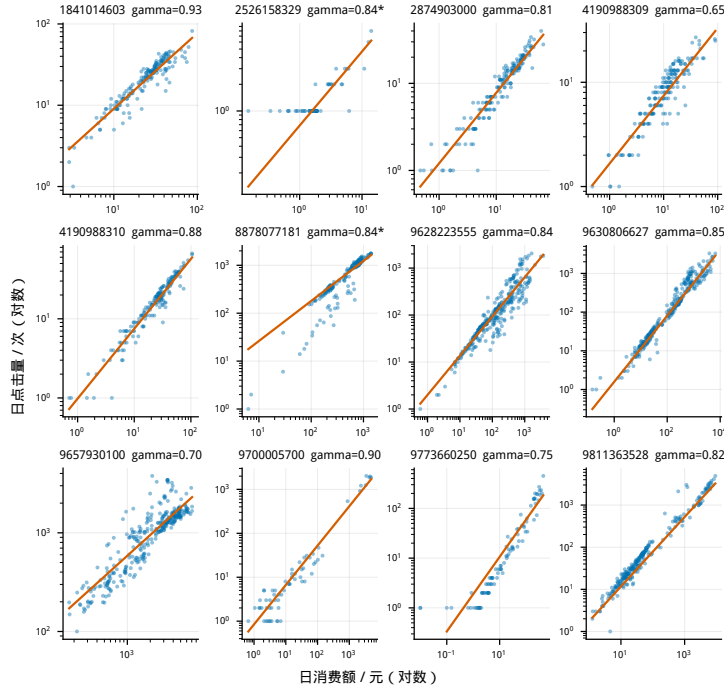


图 6 各推广单元日点击量-日消费额散点（双对数）与幂函数响应曲线 $c_u(S/s_u)^{\gamma_u}$ ；标题中带 * 的单元使用合并弹性。

4.5 合理性小结

综合以上事实：(i) 创意与匹配在上方位有效，但约四分之三的展现落在上方位之外且 CTR 极低，质量度与匹配方式有改进空间；(ii) 关键词库长尾严重、四成的词零消费、290 个词跨单元重复投放，缺乏清理与去重；(iii) 预算过度集中于弹性最低、CPC 最高的单元，违背边际收益均衡；(iv) 假日减投幅度与需求侧效应大体匹配，但周一、周日的大幅减投没有需求侧依据。后三点分别由问题二（分类清理）、问题三与四（预算再分配与时间安排）给出改进方案。

5 问题二：关键词的成本-效益分类

本节把题目的五类落实为“成本尺度 $\times$ 效益指数”平面上的象限划分：先定义成本与复合效益指数，再用数据驱动的阈值划分并检验阈值的稳健性，最后由独立校验器复核每个标签。

5.1 成本与效益的度量

关键词的年度消费 S_i 跨越六个数量级，成本取对数尺度 $x_i^c = \log_{10} S_i$ （仅对 $S_i > 0$ 的词定义）。效益综合四个维度（表 5）：点击量、浏览量、注意力时长与归因注册，

$$z_{i1} = \log(1 + C_i), \quad z_{i2} = \log(1 + V_i), \quad z_{i3} = \log(1 + C_i(1 - b_i)\tau_i), \quad z_{i4} = \log(1 + r_{u(i)}C_i), \quad (8)$$

其中 $C_i(1 - b_i)\tau_i$ 是未跳出访问产生的总停留时间（人·秒）， $r_{u(i)}$ 为所属单元的注册率（第 4.4 节）；无访问记录（“/”）的词取 $b_i = 1, \tau_i = 0$ 。对每个分量做极差标准化 $\tilde{z}_{ij} = (z_{ij} - \min_k z_{kj}) / (\max_k z_{kj} - \min_k z_{kj}) \in [0, 1]$ ，权重由熵权法 [5] 确定：

$$p_{ij} = \frac{\tilde{z}_{ij}}{\sum_k \tilde{z}_{kj}}, \quad e_j = -\frac{1}{\ln n} \sum_i p_{ij} \ln p_{ij}, \quad w_j = \frac{1 - e_j}{\sum_l (1 - e_l)}, \quad (9)$$

信息量越大（分布越分散）的分量权重越高。综合效益指数

$$B_i = \sum_{j=1}^4 w_j \tilde{z}_{ij} \in [0, 1]. \quad (10)$$

熵权为：点击 0.1966、浏览 0.2093、注意力时长 0.2849、归因注册 0.3092；等权指数与熵权指数的 Spearman 秩相关为 0.9995，说明权重选择对排序几乎没有影响。

表 5 效益指数的四个分量及其业务含义

分量	定义	含义	熵权
点击量	$\log(1 + C_i)$	流量规模	0.1966
浏览量	$\log(1 + V_i)$	浏览深度	0.2093
注意力时长	$\log(1 + C_i(1 - b_i)\tau_i)$	未跳出访问的总停留时间	0.2849
归因注册	$\log(1 + r_{u(i)}C_i)$	最终转化	0.3092

5.2 阈值与分类规则

“高/低”的界限在有消费的词上由数据决定。主方法为两类 Fisher-Jenks 自然断点 [6-7]：对排序后的样本 $v_{(1)} \leq \dots \leq v_{(n)}$ 求

$$k^* = \arg \min_{1 \leq k \leq n} \left[\sum_{i \leq k} (v_{(i)} - \bar{v}_{\leq k})^2 + \sum_{i > k} (v_{(i)} - \bar{v}_{> k})^2 \right], \quad \text{阈值} = \frac{1}{2} (v_{(k^*)} + v_{(k^*+1)}), \quad (11)$$

利用前缀和可在排序后 $O(n)$ 精确求解，它等价于一维两类 k -means 的全局最优解。作为对照，另取中位数与二分量高斯混合模型后验概率 0.5 的交点作阈值。分类规则为

$$\text{标签}_i = \begin{cases} \text{无效词,} & S_i = 0 \text{ 且 } C_i = 0, \\ \text{黄金词,} & x_i^c \leq \tau_c, B_i \geq \tau_b, \\ \text{重点词,} & x_i^c > \tau_c, B_i \geq \tau_b, \\ \text{潜力词,} & x_i^c \leq \tau_c, B_i < \tau_b, \\ \text{问题词,} & x_i^c > \tau_c, B_i < \tau_b, \end{cases} \quad (12)$$

无效词按“零消费且零点击”判定 (MDR-0001)：未购买任何点击的词不可能通过广告产生效益，两条零消费、零点击但浏览量为 1 的记录视为统计噪声。主方法阈值为成本 41.59 元/年 ($\log_{10}$ 尺度 1.6190)、效益指数 0.2758；中位数法与高斯混合法的的成本阈值分别为 8.06 元与 107.51 元 (图 8)。

5.3 分类结果与稳健性

分类结果为：黄金词 40、重点词 271、潜力词 977、问题词 49、无效词 890 (表 6, 图 7；各单元的分类分布见附录表 15)。271 个重点词承担了 99.11% 的消费，CPC 1.7143 元，是账户的流量与注册主体；40 个黄金词的 CPC 仅 0.7192 元，以极低成本取得高于阈值的效益，是加大出价与预算的首选；49 个问题词 CPC 高达 2.2313 元而效益低于阈值，应降低出价、收紧匹配或暂停；977 个潜力词消费与效益都低 (CPC 1.0599 元)，适合小预算测试以发现新的黄金词；890 个无效词占词库四成，应清理或改用更宽的匹配方式。

表 6 五类关键词的数量、消费、点击、CPC、平均效益指数与归因注册

类别	数量	占比/%	消费额/元	消费占比/%	消费中位数/元	点击量	CPC/元	平均效益指数	归因注册/人
黄金词	40	1.8	1,072	0.1	26.8	1,491	0.72	0.307	169
重点词	271	12.2	1,413,276	99.1	300.0	824,389	1.71	0.452	59,372
潜力词	977	43.9	7,542	0.5	4.3	7,116	1.06	0.113	604
问题词	49	2.2	4,059	0.3	60.0	1,819	2.23	0.232	124
无效词	890	40.0	0	0.0	0.0	0	—	0.000	0

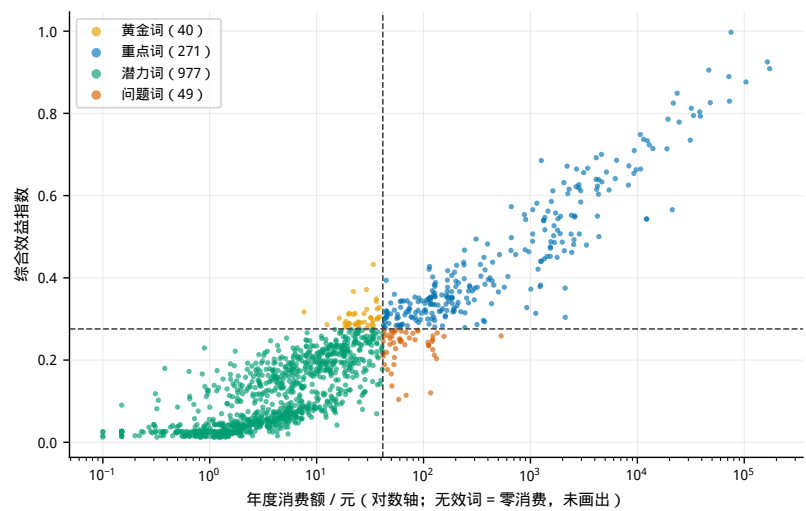


图 7 关键词的成本-效益散点：横轴为年度消费额（对数轴），纵轴为综合效益指数 (10)，虚线为自然断点阈值，颜色为类别（无效词零消费未画出）。

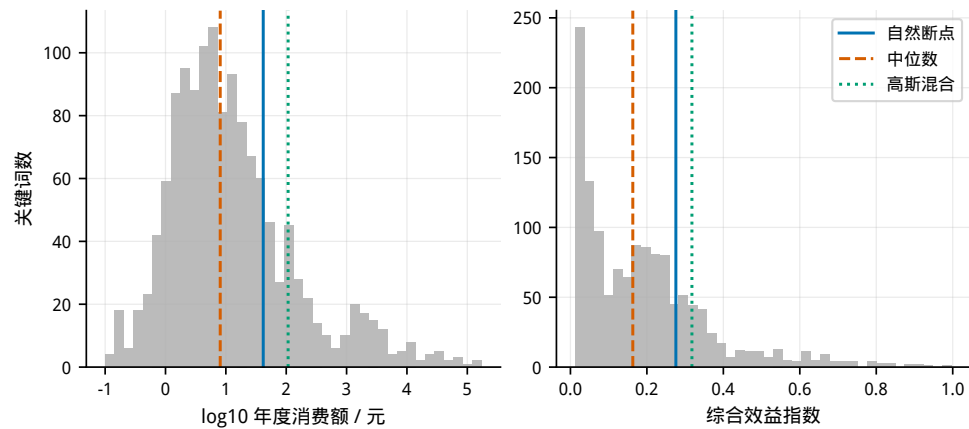


图 8 $\log_{10}$ 年度消费额（左）与综合效益指数（右）的直方图及三种阈值方法的位置。

阈值方法的稳健性见表 7：主方法与中位数法的标签一致率为 78.36%（中位数把阈值压到 8.06 元，使大量长尾词进入“高成本”），与高斯混合法为 93.53%，与等权效益指数为 99.19%。无效词在所有方法下都是 890 个，重点词、问题词的成员在三种方法下高度重合，差异集中在阈值附近的潜力词与黄金词之间。

表 7 不同阈值方法与权重下的类别数量（成本阈值为原尺度）

阈值方法	成本阈值/元	效益阈值	黄金词	重点词	潜力词	问题词	无效词
自然断点（主）	41.59	0.276	40	271	977	49	890
中位数	8.06	0.163	92	577	579	89	890
高斯混合	107.51	0.318	32	195	1,078	32	890
等权效益 + 自然断点	41.59	0.275	47	282	970	38	890

5.4 独立复核与结果文件

与分类程序分离的校验器从原始列重新推导每个标签：标签在词表内、无效词当且仅当零消费且零点击、象限规则 (12) 逐词复现（0 个不一致）、五类计数之和等于 2227、黄金词的最大消费不超过问题词的最小消费且最小效益不低于问题词的最大效益、效益指数落在 $[0, 1]$ ；全部通过（第 8 节）。详细结果按模板写入 result2.xlsx（2227 行，五个类别列以 1/0 标记）。

6 问题三：关键词选择与投放模型

本节在问题一的结构参数 (r_u, γ_u) 与问题二的类别之上，为每个推广单元建立“投入 $\rightarrow$ 点击 $\rightarrow$ 浏览、注册”的响应模型，把每个单元-日的关键词选择与投放额写成带最小投放额的可分离凹规划，给出精确算法与最优性证书，并用同预算的反事实对照说明策略的优越性。

6.1 关键词响应函数、点击价值与展位模型

附件只有关键词的年度汇总，本文以其活跃日的日均工作点 $(s_i, c_i) = (S_i/n_u, C_i/n_u)$ 为锚、以单元弹性为形状参数建立幂函数响应（假设 4）：

$$q_i(x) = c_i \left(\frac{x}{s_i} \right)^{\gamma_u}, \quad \text{CPC}_i(x) = \frac{x}{q_i(x)} = \frac{s_i}{c_i} \left(\frac{x}{s_i} \right)^{1-\gamma_u}, \quad x \geq 0, \quad (13)$$

它过历史工作点 $q_i(s_i) = c_i$ ，且 $\gamma_u < 1$ 使平均与边际点击成本随投入上升——这是 SEM 竞价市场的基本事实 [8-9]：更多点击只能通过更高排位、更高出价获得。文献中的广告响应模型（如 ADBUDG 与幂函数）同样以凹响应刻画边际收益递减 [10-11]；指数饱和函数在 11/12 个单元上的拟合残差平方和大于幂函数，故不采用（MDR-0005）。每次点击的期望注册与浏览量为（假设 5）

$$\rho_i = r_u \kappa_i, \quad \kappa_i = \text{clip} \left(\frac{1 - b_i}{1 - \bar{b}_u}, 0.25, 2.5 \right), \quad v_i(x) = d_i q_i(x), \quad (14)$$

72 个有点击但浏览量为 0 且跳出率为 0 的词属访问追踪缺失，其 b_i, d_i 回退到单元均值。日期 t 的期望点击为 $\hat{\varepsilon}_{u,t} q_i(x)$ （假设 6）， $\hat{\varepsilon}_{u,t} = E[\varepsilon_{u,t}]$ 由第 7.1 节的日历模型给出（问题三中为样本内拟合值）。

展位由单元日序列上的加权 logit 回归刻画（按展现量加权，斜率限制为非负，不显著或为负时退化为常数）：

$$\begin{aligned} \text{logit } p_u^{\text{top}} &= a_u + b_u \log \text{CPC}, & \text{logit } p_u^1 &= a'_u + b'_u \log \text{CPC}, \\ \pi &= 1 \cdot p^1 + 2.5 (p^{\text{top}} - p^1) + 4(1 - p^{\text{top}}), \end{aligned} \quad (15)$$

关键词在投放 x 下的 CPC 由式 (13) 代入得到其上方位、首位占比与预期展位指数 π (首位记 1, 上方位其余记 2.5, 上方位之外记 4; 数值越小越靠前, MDR-0007)。

6.2 单元-日分配模型

记 $\mathcal{K}_u^e$ 为单元 u 的候选词集合: 黄金词、重点词、潜力词与问题词参与, 无效词不参与; 问题词的投放上限减半 ($\phi_i = 0.5$, 其余 $\phi_i = 1$)。关键词的“关联关系”体现为: 同一单元的词共享弹性 γ_u 与注册率 r_u 、共用该单元的预算, 类型决定上限; 同一 ID 在多个单元重复投放时各份独立参与 (数据无法识别跨单元的相互蚕食, 重复投放作为问题一的发现提示合并)。单元 u 在日期 t 的模型为

$$(P_{u,t}) : \max_x \sum_{i \in \mathcal{K}_u^e} \hat{\varepsilon}_{u,t} \rho_i c_i \left(\frac{x_i}{s_i} \right)^{\gamma_u} \quad \text{s.t.} \quad \sum_i x_i \leq B_{u,t}, \quad x_i \in \{0\} \cup [m, \bar{x}_i], \quad (16)$$

$$\bar{x}_i = \max \left\{ \mu \phi_i s_i, \frac{B_{u,t} s_i}{\sum_{j \in \mathcal{K}_u^e} s_j}, m \right\}, \quad \mu = 4.0, \quad m = 0.50 \text{ 元}, \quad (17)$$

其中 $B_{u,t}$ 为该单元当日的 2025 年实际消费 (假设 7), 目标是当日预期注册量 (点击、浏览随之由式 (13)、(14) 给出)。关键词预算分配在文献中通常写成以凹响应为目标的背包型或凸规划问题 [12-13], 式 (16) 属于这一类, 其特点是用最小投放额显式地把“选词”与“定额”合并在一个问题中。上限不低于历史比例份额保证“按历史比例分配”总是可行, 从而最优值不低于历史做法; 若 $\sum_i \bar{x}_i < 1.2B_{u,t}$ 则等比例放大上限以保证预算可用完。最小投放额 m 是关键词选择机制: 投放低于 m 的词视为不投放, 避免“每个词分几分钱”的无意义解。写 $a_i = \hat{\varepsilon}_{u,t} \rho_i c_i s_i^{-\gamma_u}$, 目标即 $\sum_i a_i x_i^{\gamma_u}$ 。

6.3 求解算法与最优性

定理 6.1 (松弛问题的注水解). 设 $a_i > 0$, $\gamma \in (0, 1)$, $\bar{x}_i > 0$ 。问题 $\max\{\sum_i a_i x_i^\gamma : \sum_i x_i \leq B, 0 \leq x_i \leq \bar{x}_i\}$ 有唯一最优解。若 $\sum_i \bar{x}_i \leq B$, 则 $x^* = \bar{x}$; 否则

$$x_i^*(\lambda^*) = \min \left\{ \bar{x}_i, (\gamma a_i / \lambda^*)^{1/(1-\gamma)} \right\}, \quad (18)$$

其中 $\lambda^* > 0$ 是方程 $\sum_i x_i(\lambda) = B$ 的唯一根。

证明. 目标是严格凹函数之和, 可行域是紧凸集, 故最优解存在且唯一; Slater 条件成立, KKT 条件充要 [14-15]。因 $d(a_i x^\gamma)/dx \rightarrow +\infty$ ($x \rightarrow 0^+$), 下界 $x_i \geq 0$ 的乘子恒为零; 对预算乘子 $\lambda \geq 0$ 与上界乘子 $\nu_i \geq 0$ 有 $\gamma a_i x_i^{\gamma-1} = \lambda + \nu_i$, $\nu_i(x_i - \bar{x}_i) = 0$, 解得式 (18)。当 $\sum_i \bar{x}_i > B$ 时预算必然取等 (否则可增加某个未达上限的 x_i 而改进), 而 $\lambda \mapsto \sum_i x_i(\lambda)$ 连续、在未全部达上限处严格递减、 $\lambda \rightarrow 0^+$ 时趋于 $\sum_i \bar{x}_i > B$ 、 $\lambda \rightarrow \infty$ 时趋于 0, 故根唯一。□

定理 6.1 是连续非线性资源分配问题的经典结论 [16], λ^* 用对数尺度二分法求到相对精度 10^{-12} 。含最小投放额时 $\{0\} \cup [m, \bar{x}_i]$ 非凸, 本文用拉格朗日松弛确定入选集合、再在该集合上精确求解 (算法 1): 对价格 λ , 每个词的拉格朗日子问题 $\max\{a_i x^\gamma - \lambda x : x \in \{0\} \cup [m, \bar{x}_i]\}$ 的解为 $x_i(\lambda) = \text{clip}((\gamma a_i / \lambda)^{1/(1-\gamma)}, m, \bar{x}_i)$ (当其值 $a_i x_i^\gamma \geq \lambda x_i$) 或 0, 总投放随 λ 单调不减, 二分得到预算恰好可用的价格及其入选集合; 第二阶段在入选集合上以下界 m 、上界 $\bar{x}_i$ 做有界注水 (定理 6.1 的推广, KKT 条件在该集合上精确成立)。

每次二分需 $O(n)$ 运算, 二分次数 $O(\log(1/\epsilon))$, 故复杂度 $O(n \log(1/\epsilon))$; 107 个单元-日问题 (最多 577 个候选词) 合计求解不到一分钟。最优性由与求解器分离的校验器证明 (第 8.4 节): 可行性、KKT 驻点条件 (内点坐标边际价值相等、达上限坐标的边际不小于价格、达下限坐标的平均价值覆

Algorithm 1: 两阶段拉格朗日-注水法求解 ($P_{u,t}$)

输入: 系数 $a_i > 0$, 弹性 γ , 上限 $\bar{x}_i$, 预算 B , 最小投放额 m , 精度 ϵ

输出: 投放向量 x^* 、入选集合 $\mathcal{S}$

- 1 **if** $\sum_i \bar{x}_i \leq B$ **then**
 - 2 **return** $x^* = \bar{x}$;
 - 3 **阶段一 (选择):** $x_i(\lambda) \leftarrow \text{clip}((\gamma a_i / \lambda)^{1/(1-\gamma)}, m, \bar{x}_i)$, 若 $a_i x_i(\lambda)^\gamma < \lambda x_i(\lambda)$ 则
 $x_i(\lambda) \leftarrow 0$;
 - 4 在 $[\lambda_{\text{lo}}, \lambda_{\text{hi}}]$ 上对 λ 做对数尺度二分, 直到 $\lambda_{\text{hi}} / \lambda_{\text{lo}} - 1 < \epsilon$ 且
 $\sum_i x_i(\lambda_{\text{hi}}) \leq B < \sum_i x_i(\lambda_{\text{lo}})$;
 - 5 $\mathcal{S} \leftarrow \{i : x_i(\lambda_{\text{hi}}) > 0\}$;
 - 6 **阶段二 (精确求解):** 在 $\mathcal{S}$ 上求解 $\max\{\sum_{i \in \mathcal{S}} a_i x_i^\gamma : \sum_{i \in \mathcal{S}} x_i = B, m \leq x_i \leq \bar{x}_i\}$
 ——以 $x_i(\lambda) = \text{clip}((\gamma a_i / \lambda)^{1/(1-\gamma)}, m, \bar{x}_i)$ 二分求 λ , 再在自由坐标上按比例修正预算余量;
 - 7 **return** x^* ($\mathcal{S}$ 外为 0)
-

盖价格)、随机配对转移 / 剔除 / 插入均不能改进, 以及用通用凸求解器 (CVXPY/Clarabel[17]) 求得的去掉最小投放额约束的松弛问题最优值作为上界。松弛问题是 ($P_{u,t}$) 的放松, 其最优值 $\geq$ 任意可行解的值, 因此

$$\text{总体最优性界} = \frac{\sum_{(u,t)} [\text{松弛上界}_{u,t} - \text{目标值}_{u,t}]}{\sum_{(u,t)} \text{目标值}_{u,t}} = 0.080\%, \quad (19)$$

即两阶段解与全局最优的差距不超过总注册量的千分之一; 去掉 13 个预算不足 20 倍最小投放额的微小问题后, 单个问题的最大相对差为 3.404% (微小问题上松弛界不具信息量, 最大相对差 20.1%, 但它们对总量的贡献可忽略)。

6.4 反事实基线、不确定性与备选预算口径

优越性通过同一响应模型、同一日乘子、同一预算下的反事实比较说明。基线一“按历史比例”复现 2025 年的做法: $x_i^{\text{prop}} = B_{u,t} s_i / \sum_{j \in \mathcal{K}_u^e} s_j$ (对该单元全部候选词, 因此比优化器多用了被剔除的词); 基线二“均匀分配”: $x_i^{\text{eq}} = B_{u,t} / |\mathcal{K}_u^e|$ 。弹性的抽样不确定性用参数自助法 [18] 传播: 每次抽 $\gamma_u^{(b)} \sim N(\hat{\gamma}_u, \text{se}_u)$ (截断到 $[0.3, 0.95]$), 重解最优与基线, 取提升比的 2.5% 与 97.5% 分位 (100 次)。作为预算口径的备选, “窗口预算”允许单元在 8 天内跨日调配: 先按各日期望效率乘子求解 $\max\{\sum_t \hat{\epsilon}_{u,t} X_t^{\gamma_u} : \sum_t X_t = \sum_t B_{u,t}, X_t \leq 1.5 \max_t B_{u,t}\}$ (当日内上限不起作用时内层最优值恰为 $K_u X_t^{\gamma_u}$, 两层分解精确), 再按 ($P_{u,t}$) 分配到关键词。

6.5 结果与优越性

表 8 汇总两个窗口各单元的结果, 逐日结果见附录表 16, 图 9 给出逐日对照与自助分布。2 月 1-8 日处于春节假期, 仅 2 个单元有实际投放 (16 个单元-日, 预算 13,681.15 元, 候选词 286 个, 入选 73 个); 8 月 1-8 日 12 个单元全部投放 (91 个单元-日, 预算 37,483.78 元, 候选词 1337 个, 入选 535 个)。同预算下最优策略的预期注册量为 516.42 与 2237.43 人, 较按历史比例的 459.65 与

1748.04 人提升 12.35% 与 28.00%（自助 95% 区间 [11.67, 13.39]% 与 [27.53, 28.57]%），较均匀分配提升 88.54% 与 77.01%；预期点击提升 12.05% 与 15.87%，平均 CPC 由 2.1595 与 1.5124 元降至 1.9273 与 1.3052 元。

表 8 两个窗口各推广单元的分配结果：预算、候选与平均入选词数、按历史比例与最优的预期注册、提升及弹性自助 95% 区间、CPC

窗口	推广单元	天数	预算/元	候选词	平均入选词	历史比例注册/人	最优注册/人	提升/%	95% 自助区间	历史 CPC/元	最优 CPC/元
2025-02-01 至 02-08	9628223555	8	1,853	247	15.8	18.0	31.8	76.7		2.88	2.22
2025-02-01 至 02-08	9657930100	8	11,829	39	25.1	441.7	484.7	9.7		2.08	1.89
2025-02-01 至 02-08	合计	16	13,681	286	20.4	459.6	516.4	12.4	[11.7, 13.4]	2.16	1.93
2025-08-01 至 08-08	1841014603	8	305	35	4.2	18.9	22.8	20.7		1.29	1.17
2025-08-01 至 08-08	2526158329	4	13	14	5.2	0.6	1.0	59.6		1.53	1.71
2025-08-01 至 08-08	2874903000	8	229	60	22.8	11.1	15.9	43.7		1.44	1.46
2025-08-01 至 08-08	4190988309	8	244	19	4.4	10.1	10.6	5.3		1.93	1.94
2025-08-01 至 08-08	4190988310	8	235	45	18.4	10.5	16.2	54.4		1.71	1.89
2025-08-01 至 08-08	8878077181	8	4,319	4	3.0	275.1	314.2	14.2		0.68	0.60
2025-08-01 至 08-08	9628223555	8	548	247	7.4	27.4	49.6	81.2		0.56	0.41
2025-08-01 至 08-08	9630806627	8	2,870	281	53.8	242.3	331.6	36.8		0.87	0.79
2025-08-01 至 08-08	9657930100	8	23,969	39	27.9	822.4	894.5	8.8		2.26	2.08
2025-08-01 至 08-08	9700005700	7	141	11	1.0	5.7	17.6	210.5		1.98	0.70
2025-08-01 至 08-08	9773660250	8	2,116	5	5.0	76.5	77.0	0.7		2.20	2.17
2025-08-01 至 08-08	9811363528	8	2,494	577	96.8	247.5	486.4	96.5		1.32	0.80
2025-08-01 至 08-08	合计	91	37,484	1,337	21.7	1748.0	2237.4	28.0	[27.5, 28.6]	1.51	1.31

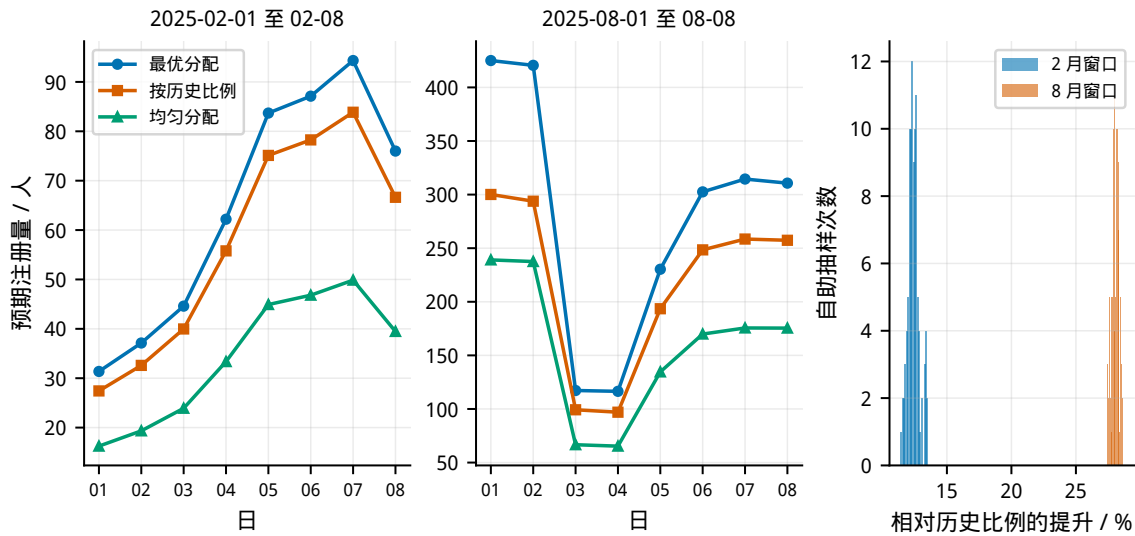


图 9 左、中：两个窗口逐日预期注册量——最优分配、按历史比例、均匀分配；右：弹性参数自助抽样下最优分配相对历史比例的提升分布。

提升的来源见图 10。最优解把预算从 CPC 高、注册率低的词移向 ρ_i/CPC_i 高的词：一部分重点词被推到上限（沿上限线分布，339 个入选词-日达到上限），另一部分被压缩或剔除（271 个重点词中约半数至少入选一天），8 月窗口每个单元-日平均入选 21.7 个词。图中位于上限线之上的一簇点是历史日均消费低于 m/μ 的长尾词（主要是潜力词与黄金词）：按式 (17) 其上限就是最小投放额 m ，它们以每天 m 元的试探性投入入选——8 月窗口这类词-日共 1259 个，只占预算的 1.68%，贡献预期注册的 5.49%；幂函数在远离历史工作点处的外推可靠性有限，对这些词的预期只应理解为“值得小额测试”，主要提升仍来自重点词之间的再分配。提升幅度随单元的候选词数与弹性差异而不同：8 月窗口单元 9773660250 的提升最小（0.7%），单元 9700005700 最大（210.5%）；消费最大的单元 9657930100 候选词最少且弹性最低，提升幅度也接近最低（表 8）。因此“低成本、高效益”

的实现路径主要是长尾单元内部的词级再分配，而不是改变类别间的消费结构（重点词在两种方案下都占消费的绝大部分）。

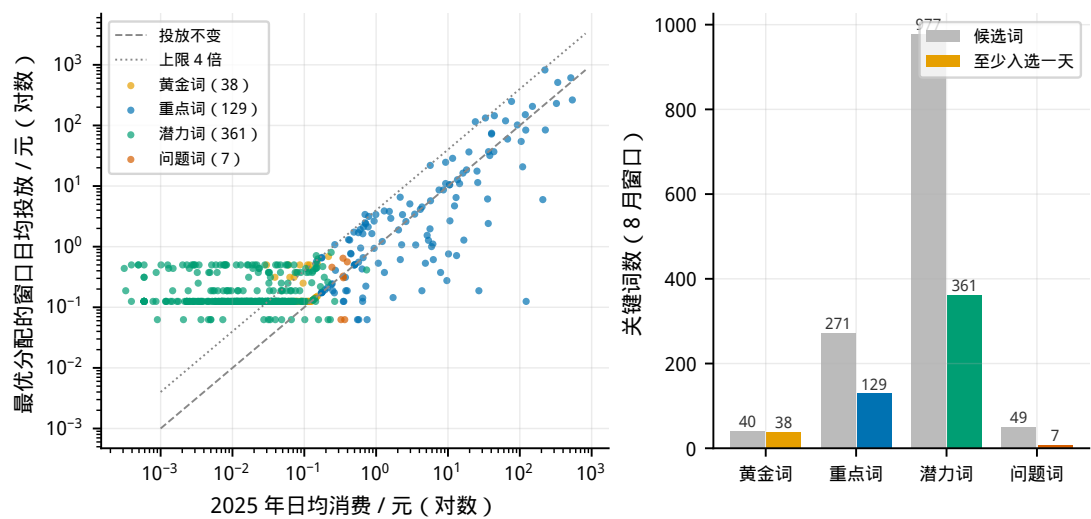


图 10 8 月窗口最优分配的机制。左：每个入选关键词的窗口日均最优投放与其 2025 年日均消费（双对数，颜色为类别；虚线为投放不变，点线为上限倍数）；右：各类别的候选词数与至少入选一天的词数。

窗口预算的备选口径（表 9）显示：允许跨日调配时，2 月窗口再提升 0.82%（春节期间各日效率相近），8 月窗口再提升 11.25%——把 8 月 3、4 日（周日、周一，效率乘子最低）的预算移到工作日；这与问题一“周一、周日减投过度”的结论一致，但方向相反：模型建议在效率低的日子少投、效率高的日子多投，而 2025 年的做法是在效率高的工作日与效率低的周日都按惯性投放。

表 9 窗口预算跨日调配（备选口径）相对逐日预算的额外提升

窗口	逐日预算最优注册/人	窗口预算最优注册/人	额外提升/%	跨日调动预算比例/%
2 月窗口	516.4	520.6	0.8	16.0
8 月窗口	2237.4	2489.1	11.2	12.5

详细结果按模板写入 result3.xlsx（2303 行：日期、方案 ID、推广单元、关键词、投入金额、预期展位、预期点击量、预期浏览量、预期注册量；投入金额为式 (16) 的解，预期展位为式 (15) 的 π ）。策略的优越性归结为四点：(i) 每个单元-日问题的解是带全局最优性证书的最优解（式 (19)）；(ii) 同预算下相对 2025 年做法的注册提升为正且区间不含零；(iii) 灵敏度分析（第 8.5 节）中上限倍数、弹性、预算、问题词处理与最小投放额的各种情形下提升均为正；(iv) 响应模型在历史工作点附近经过校准（第 8.4 节）。

7 问题四：2026 年 9 月 11–17 日的投放策略与期望范围

本节先为每个单元建立日效率、CPC、CTR 与展位的日历预测模型并用滚动回测检验其点预测与区间，再在问题三的分配模型上叠加不确定性：证明期望注册最大化问题与用期望乘子的确定性问题的同解，从而以两层分配给出逐日策略，并用蒙特卡罗情景给出竞价（CPC）、展现量、展现位、点击量、浏览量与注册量的期望范围。

7.1 预测对象、日历模型与滚动回测

对单元 u 有投放的每一天定义六个目标：效率乘子 $\log \varepsilon_{u,t} = \log C_{u,t} - \gamma_u \log(S_{u,t}/\bar{s}_u) - \log \bar{c}_u$ ($\bar{s}_u, \bar{c}_u$ 为活跃日均值, 即式 (7) 的残差结构)、 $\log \text{CPC}_{u,t}$ 、 $\text{logit } \text{CTR}_{u,t}$ 、 $\text{logit } p_{u,t}^{\text{top}}$ 、 $\text{logit } p_{u,t}^1$ 与 $\log C_{u,t}$ 。每个目标 $y_{u,t}$ 用日历回归

$$y_{u,t} = \beta_{u,0} + \sum_{w \neq \text{周三}} \beta_{u,w} D_t^w + \sum_m \mu_{u,m} M_t^m + \delta_{u,H} H_t + \delta_{u,A} A_t + e_{u,t}, \quad e_{u,t} \sim (0, \sigma_u^2), \quad (20)$$

以岭正则 (10^{-3} , 不含截距) 稳定估计。预测点在样本末 8.5 个月之后, 近期水平不可用, 因此 2026 年的预测标准差取 $\sqrt{\sigma_u^2 + (\sigma_u^m)^2}$, 其中 σ_u^m 为月份效应的标准差, 用以刻画年际间月份水平的漂移 (假设 8); 9 月效应取 2025 年 9 月的估计 (该月无投放的单元取基准月)。点预测经 exp 或 expit 反变换, 效率乘子的期望取对数正态均值 $\hat{\varepsilon}_{u,t} = \exp(\mu_{u,t} + \frac{1}{2}\sigma^2)$ 。

模型用 2025 年第四季度的滚动起点回测检验 [19]: 起点为 10 月 9 日起每周一个 (共 12 个), 预测步长 7 天, 训练集至少 40 天; 对照两条基线——季节朴素 (上周同一周几的值, 区间取 7 阶差分的 10%/90% 分位) 与 28 日均值 (区间取近 28 日偏差的分位); 指标为变换尺度上的 MAE、RMSE、分位数 0.1 与 0.9 的 pinball 损失 [20-21] 以及 80% 区间覆盖率。附录表 18 与图 11 给出结果: 日历回归的 80% 区间覆盖率为 78.34%–87.26%, 接近名义水平; 28 日均值的区间过窄 (63.63%–80.31%), 季节朴素的区间过宽 (约 90%)。点预测方面, 对效率乘子、CPC、CTR 与上方位占比, 28 日均值在 7 天步长内的 MAE 更小 (近期水平的持续性强于全年日历结构), 而对点击量日历回归最优。由于 2026 年 9 月的目标点没有可用的“近期水平”, 本文采用日历回归给出点预测, 其区间宽度已由回测校准; 这一取舍与代价在第 9 节中说明。

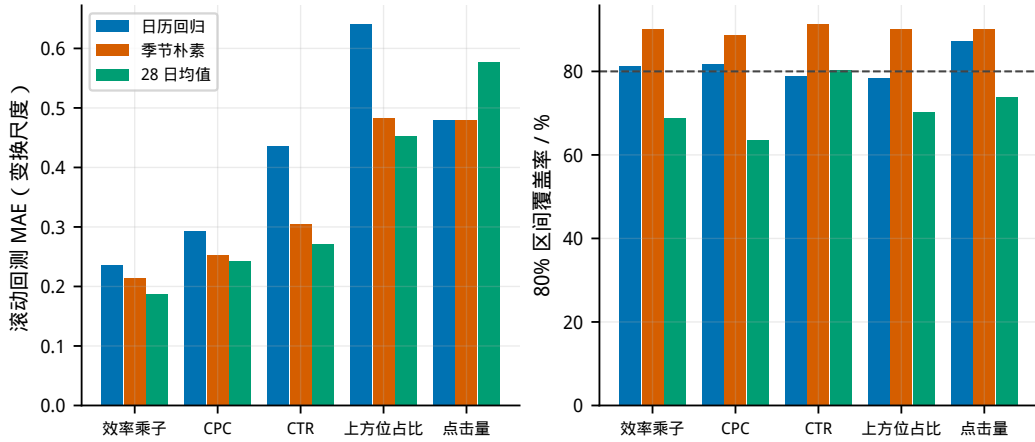


图 11 滚动回测：日历回归、季节朴素与 28 日均值对各目标的 MAE (左) 与 80% 区间覆盖率 (右, 虚线为名义水平 80%)。

7.2 预算、不确定性模型与随机分配

预算按“不超过 2025 年的投入”取 2025 年同期 (9 月 11–17 日) 各单元的实际消费之和 B_u (合计 23,488.02 元), 同期无消费的 2 个单元不安排投放, 其余 10 个单元参与; 按全年日均折算 ($\sum_t S_{u,t}/365 \times 7$, 合计 27,346.98 元) 作为备选口径 (MDR-0008)。不确定性以四类冲击进入响应 (假设 6): 单元-日共同冲击 $\varepsilon_{u,t} = \exp(\mu_{u,t} + \sigma z_t)$, $z_t \sim N(0, 1)$; 关键词特异冲击 $\eta_i = \exp(\sigma_u^{\text{kw}} \zeta_i - \frac{1}{2}(\sigma_u^{\text{kw}})^2)$ (均值 1, σ_u^{kw} 取该单元 $\log \text{CPC}$ 的残差标准差); CTR 与上方位占比的 logit

日冲击；注册的泊松噪声。于是

$$\begin{aligned} \text{点击}_i &= q_i(x_i) \varepsilon_{u,t} \eta_i, & \text{CPC}_i &= \frac{x_i}{\text{点击}_i}, & \text{展现}_i &= \frac{\text{点击}_i}{\text{CTR}_{u,t}}, \\ \text{浏览}_i &= d_i \text{点击}_i, & \text{注册}_i &\sim \text{Poisson}(\rho_i \text{点击}_i), \end{aligned} \quad (21)$$

展位由式 (15) 在情景 CPC 下计算。关键词的“竞价”在附件中不可观测，本文以其平均点击成本 CPC_i 作为竞价水平的度量。

命题 7.1 (确定性等价). 若冲击 $\varepsilon_{u,t}$ 、 η_i 与投放额无关且 $E[\eta_i] = 1$ ，则期望注册最大化问题 $\max_x E[\sum_i \rho_i q_i(x_i) \varepsilon_{u,t} \eta_i]$ 与用期望乘子 $\hat{\varepsilon}_{u,t} = E[\varepsilon_{u,t}]$ 的确定性问题 $(P_{u,t})$ 同解。

证明. 期望与求和交换， $E[\varepsilon_{u,t} \eta_i] = E[\varepsilon_{u,t}] E[\eta_i] = \hat{\varepsilon}_{u,t}$ ，目标函数逐点等于确定性目标。□

因此最优策略无需情景优化，蒙特卡罗只用于给出各量的期望范围；样本平均近似的期望值与确定性目标一致，是模拟正确性的内部检验。策略分两层求解：(i) 跨日分配 $\max\{\sum_t \hat{\varepsilon}_{u,t} X_t^{\gamma_u} : \sum_t X_t \leq B_u, X_t \in \{0\} \cup [m, \bar{X}_u]\}$ ， $\bar{X}_u = \max\{1.5 \max_{t \in \text{同期}} S_{u,t}, B_u/7\}$ ，用算法 1（把“日”视为关键词， $s = c = 1$ 、系数 $\hat{\varepsilon}_{u,t}$ ）求解；(ii) 日内分配即 $(P_{u,t})$ ，预算取 X_t 。两层共 75 个问题全部通过独立校验器。随后对每个单元-日按式 (21) 生成 500 个情景，报告每个关键词、每个单元-日与合计的期望值与 10%–90% 分位。

7.3 结果

表 10 给出逐日策略的汇总，表 11 给出各单元的 7 日结果，图 12 与图 13 给出期望与范围。7 天合计投入 23,488.02 元，入选关键词 146 个（845 个关键词-日），预期点击 17,487.9 次（10%–90% 分位 13,547.4–21,970.2）、注册 1,194.4 人（922.9–1,495.3）、展现 721,553.1 次（362,488.7–1,168,717.5）、浏览 93,916.4 次（69,297.8–121,848.9）、平均 CPC 1.3431 元（1.0691–1.7337）、点击加权的预期展位指数 2.6126（介于第 2 与第 3 位之间）。蒙特卡罗期望注册 1,194.4 人与确定性目标 1187.4 人一致（命题 7.1）。跨日分配给 9 月 13、14 日（周六、周日）的预算明显低于工作日（表 10），与问题一的周几效应一致。展现量的范围远宽于点击量，因为它同时承受 CTR 与效率乘子的冲击；CPC 的范围（约 $\pm 30\%$ ）反映了竞价的逐日波动。

表 10 2026 年 9 月 11–17 日逐日策略汇总：投入、CPC、展现量、展位、点击量、浏览量、注册量的期望与 10%–90% 范围

日期	单元数	投入/元	CPC/元	CPC 范围	展现量	展现范围	展位	点击量	点击范围	浏览量	注册量	注册范围
2026-09-11	9	3,836.7	1.41	[0.88, 2.71]	109,243	[21,364, 225,027]	2.62	2,730	[1,418, 4,342]	14,506	185	[93, 292]
2026-09-12	9	3,333.4	1.27	[0.79, 2.47]	110,063	[20,431, 228,681]	2.55	2,628	[1,348, 4,213]	14,981	175	[87, 282]
2026-09-13	10	2,206.8	1.36	[0.85, 2.59]	78,011	[13,886, 153,622]	2.65	1,622	[853, 2,609]	8,417	114	[56, 189]
2026-09-14	9	2,142.4	1.38	[0.87, 2.64]	74,265	[13,888, 164,617]	2.72	1,548	[812, 2,468]	7,606	113	[55, 182]
2026-09-15	10	3,813.7	1.31	[0.81, 2.56]	109,971	[21,051, 231,030]	2.58	2,914	[1,489, 4,684]	16,073	197	[96, 318]
2026-09-16	9	3,982.7	1.29	[0.81, 2.54]	124,030	[23,272, 256,321]	2.59	3,095	[1,567, 4,936]	17,044	209	[101, 335]
2026-09-17	9	4,171.9	1.41	[0.88, 2.76]	115,971	[22,739, 251,327]	2.65	2,951	[1,513, 4,723]	15,289	201	[100, 323]

同期预算口径把 81.1% 的预算留给单元 9657930100（2025 年 9 月中旬其他单元几乎停投），而注册率最高、CPC 最低的长尾单元 9811363528 只分到二百余元却贡献了一百多人的预期注册（表 11）。若改按全年日均口径安排同量级的预算（27,346.98 元），确定性预期注册由 1187.4 人升至 2327.0 人（+96.0%）：这不是模型的技巧，而是 2025 年 9 月投放节奏本身的问题——效率最高的单元在该时段被停投。本文建议公司在 2026 年 9 月不要沿用同期的单元停投安排。与 2025 年同期实际点击

表 11 各推广单元的 7 日预算、弹性、候选词数、CPC 与点击量、注册量的期望和 10%-90% 范围

推广单元 ID	7 日预算/元	γ	候选词	CPC/元	点击量	点击范围	注册量	注册范围
9657930100	19,048.3	0.699	39	2.22	8,568	[7,038, 10,326]	665.4	[541, 802]
8878077181	3,394.7	0.836	4	0.47	7,164	[5,115, 9,507]	310.1	[223, 411]
9628223555	258.5	0.836	247	0.50	518	[385, 658]	19.1	[12, 27]
9811363528	233.8	0.825	577	0.39	598	[487, 716]	127.3	[101, 154]
9630806627	186.8	0.853	281	0.63	298	[240, 359]	33.0	[23, 42]
1841014603	146.0	0.930	35	0.89	163	[133, 195]	14.8	[9, 21]
4190988310	100.6	0.882	45	1.47	69	[61, 77]	11.3	[7, 16]
2874903000	68.8	0.811	60	1.22	56	[46, 68]	8.5	[5, 13]
4190988309	48.6	0.650	19	0.97	50	[41, 61]	4.3	[2, 7]
2526158329	1.8	0.836	14	0.50	3	[1, 5]	0.5	[0, 2]

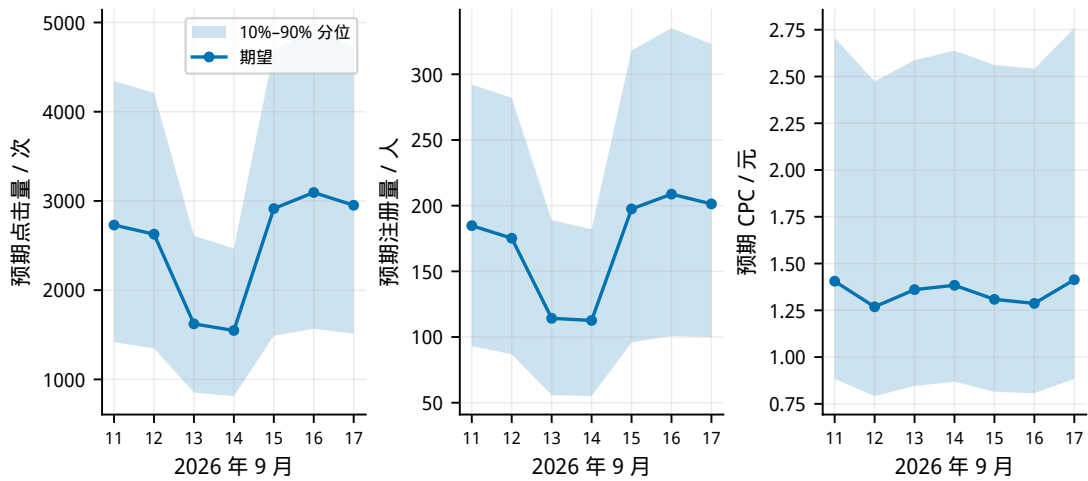


图 12 2026 年 9 月 11-17 日逐日预期点击量、注册量与 CPC（点为期望，带为 10%-90% 分位）。

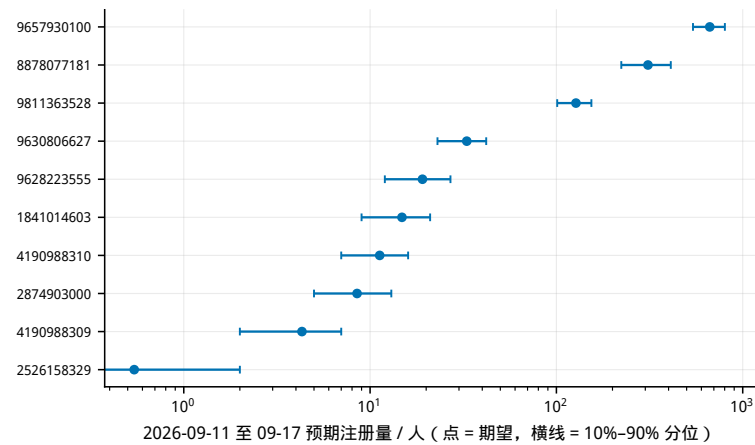


图 13 各推广单元 7 天预期注册量及 10%-90% 范围（对数横轴）。

12,843 次相比，同样的钱在最优策略下预期点击 17,487.9 次 (+36.17%)，其中包含了日历模型对 9 月效率的估计，应结合区间理解。详细结果（期望值）按模板写入 `result4.xlsx` (845 行)；每个关键词-日的 10%、50%、90% 分位与上方位、首位占比在支撑材料 `forecast/plan.csv` 中给出。

8 模型检验

本文对每一个求解结果都用与求解程序分离的校验器复核，并对统计与预测模型做样本外检验 (验证协议见支撑材料 `docs/mdr/0009-verification-protocol.md`)；表 12 汇总检验结论，下面分项说明。

表 12 独立检验汇总（全部检验在同一次云端运行中自动执行，任一失败即阶段失败）

检验项	结果
Q3 单元-日分配问题数	107
Q3 独立校验（可行性 / KKT / 配对转移与插入 / 通用凸求解器上界）	全部通过
Q3 凸松弛总体最优性界 ($\sum$ 上界 - $\sum$ 目标值) / $\sum$ 目标值	0.080%
Q3 单个问题的最大相对差（不含 13 个预算低于 20 倍最小投放额的微小问题）	3.404%
Q3 微小问题的最大相对差（松弛界在此处不具信息量，见正文）	20.1%
响应模型校准：历史比例分配预测点击 vs 实际 (MAPE / 中位 APE)	32.9% / 22.0%
Q4 分配问题数（跨日 + 单元-日）	75
Q4 独立校验	全部通过
Q2 分类独立复核（规则重推 / 无效词定义 / 计数）	通过

8.1 数据与假日效应

三张表通过列类型、取值范围与漏斗一致性契约（第 4.1 节），Sheet3 与 Sheet1 的分单元合计对账一致。假日效应的两种检验相互印证：HC3 稳健回归给出注册效应 -73.72% ($p = 8.0 \times 10^{-22}$)，Mann-Whitney 检验给出中位数比 0.2140 ($p = 4.56 \times 10^{-14}$)；控制消费额后需求侧效应 -24.64% 仍显著 ($p = 0.0151$)，而周一、周日效应消失，分解结论不依赖单一模型设定。日历设定的正确性由单元测试对照国务院办公厅通知逐日核对。

8.2 注册归因与弹性的样本外检验

归因模型的 `adstock` 衰减由滚动起点（50%、60%、70%、80% 处）交叉验证选取，留出期（后 30% 天数） $R^2 = 0.7375$ 、MAPE 30.19%（图 5）；留出期覆盖国庆假期，模型仍复现了假日跌幅与周内节律。弹性回归的单元 R^2 见表 4，合并估计的标准误仅 0.0113；弹性的抽样不确定性通过参数自助法进入问题三的提升区间（表 8）。

8.3 分类的独立复核与稳健性

校验器从原始列重新推导每个标签，六项检查（标签在词表内、无效词当且仅当零消费且零点击、象限规则逐词复现、计数守恒、黄金词严格优于问题词、效益指数在 $[0, 1]$ ）全部通过。阈值方法的一致率为 78.36%（中位数）、93.53%（高斯混合）、99.19%（等权效益），熵权与等权指数的秩相关 0.9995（表 7）；自然断点算法与暴力枚举在单元测试中逐例一致，性质测试（随机生成的 20-200 个词）验证每个词恰得一个标签。

8.4 问题三分配的最优性证书与响应模型校准

校验器对全部 107 个单元-日问题检查：(i) 非负、不超上限、入选词不低于最小投放额、总投放不超预算且预算用尽或全部达上限；(ii) KKT 条件——内点坐标的边际价值 $\gamma a_i x_i^{\gamma-1}$ 相对离差不超过 10^{-6} ，达上限坐标的边际不小于影子价格，达下限坐标的边际不大于价格且其平均价值 $a_i x_i^{\gamma-1}$ 覆盖价格；(iii) 200 次随机配对转移或整词剔除、50 次未选词按最小投放额插入均不能改进目标；(iv) 用 CVXPY/Clarabel 求解去掉最小投放额的凸松弛作为上界。全部通过；总体最优性界为 0.080% (式 (19))，去掉 13 个预算不足 20 倍最小投放额的微小问题后单个问题最大相对差 3.404%。单元测试另从三个方向核对求解器：无上限时与闭式解一致；小规模实例与网格穷举一致；含最小投放额时与枚举全部入选子集并逐个用 SLSQP 求解的结果一致；基于随机实例的性质测试验证“校验器接受求解器的输出、拒绝人为劣化的解”。

响应模型的校准用“历史比例分配 + 实际预算”应复现实际点击这一事实检验：107 个单元-日的 MAPE 为 32.92%，中位绝对百分误差 22.00%。误差主要来自关键词只有年度汇总——响应曲线由年均工作点与单元弹性外推，无法刻画单个词在特定日期的竞争环境；由于最优策略与基线在同一模型、同一日乘子下比较，模型的系统性水平误差在提升幅度中一阶抵消，这也是本文以相对提升而非绝对注册量作为优越性证据的原因。

8.5 灵敏度分析

附录表 17 给出十种情形下最优分配相对历史比例的提升。上限倍数是最敏感的参数：它决定预算能向高效率词集中的程度，8 月窗口的提升随 μ 从 2 升到 8 而增大，但即使在最保守的 $\mu = 2$ 下提升仍接近基准的三分之二；弹性上调（凹性减弱）使再分配的收益更大，下调则相反，两者与自助区间的方向一致；预算 $\pm 20\%$ 几乎不改变相对提升，说明结论对预算口径不敏感；剔除问题词或取消其上限折减对结果几乎没有影响（问题词仅占消费 0.28%）；取消最小投放额（ $m = 0$ ）使平均入选词数增至一百多个而提升略有下降，说明选择机制以极小的目标代价换来了可执行的稀疏方案。所有情形下提升均为正。

8.6 问题四的回测、审计与不确定性

预测模型的滚动回测（附录表 18，图 11）表明日历回归的 80% 区间覆盖率为 78.34%–87.26%，接近名义水平，而两条基线分别过窄与过宽；点预测在 7 天步长内不如 28 日均值，但目标点在样本末 8.5 个月之后，日历结构是唯一可外推的信息，本文如实报告这一取舍。跨日与日内共 75 个分配问题全部通过与问题三相同的校验器。蒙特卡罗期望注册 1,194.4 人与确定性目标 1187.4 人一致，验证了命题 7.1 与模拟的实现；预算口径的备选（全年日均）作为情景分析给出（第 7.3 节）。结果文件 result2/3/4.xlsx 均通过模板契约检查（表名与表头与模板一致、行数、数值区、小数位不超过 4 位）。

9 模型评价与推广

9.1 模型的优点

(1) 四个问题共用一套结构参数（注册率 r_u 、弹性 γ_u 、日效率乘子），分类、分配与预测在同一响应模型下相互一致，问题三的分配模型被问题四原样复用。(2) 分配问题有精确算法与最优性证书：松弛问题的注水解有唯一性证明，含选择机制的问题用拉格朗日价格确定入选集合后精确求解，凸松弛给出的总体界仅 0.080%。(3) 每个结论都有独立证据：假日效应有参数与非参数两种检验并

分解出需求侧成分；分类有逐词复核与三种阈值的一致率；分配有反事实对照、自助区间与十种灵敏度情形；预测有滚动回测的覆盖率。(4) 全部计算在固定镜像的云端流水线上执行，论文中的数字、表格与图形由同一次运行生成并记录散列，可完整复现（附录 B）。

9.2 局限与可能的偏差方向

(1) 关键词只有年度汇总，响应曲线由年均工作点与单元弹性外推，校准 MAPE 约 32.92%；若某些词的真实弹性高于单元弹性，最优解对这些词的投放偏低，反之偏高。(2) 注册只有日总量， r_u 是回归推断而非真实追踪，5 个单元使用自身估计，其余使用合并估计；若各单元的真实注册率差异更大，长尾单元的提升可能被低估或高估。(3) 假日效应中的需求侧成分依赖“控制当日消费”的识别假设，若公司减投本身响应了需求的实时信号，需求侧效应会被高估。(4) 同一关键词跨单元重复投放的相互蚕食无法从数据识别，最优解可能对重复词略为乐观。(5) 2026 年的市场趋势、竞争对手行为与平台规则变化无法从一年的数据识别，问题四的区间只反映 2025 年的残差与月份间波动；2 月窗口仅两个单元有实际投放，其结论的外推性有限。(6) “预期展位”是按首位 / 上方位其余 / 上方位之外三档定义的排位指数，绝对数值依赖这一约定，但排序不受影响。

9.3 改进方向与推广

若能获得关键词的逐日消费与点击、真实的转化追踪与出价记录，响应函数可按词估计并直接以出价为决策变量，本文的分配框架（可分离凹规划 + 拉格朗日选择 + 独立证书）无需改动；跨单元的重复词可加入共享流量约束，跨日分配可加入 CVaR 形式的超支约束以控制预算风险。方法本身可推广到任何“预算 → 凹响应 → 转化”的资源分配场景，如信息流广告、渠道预算分配与库存促销。

AI 工具使用声明

本研究在问题分析、方案比较、程序编写与调试、云端实验组织、数值核验、图表绘制以及论文文字组织等环节使用了生成式 AI 工具（Anthropic Claude Code，模型 Claude Fable 5.1）；所有数据处理、计算与排版在 Modal 云平台执行。使用过程与人工核验方式详见支撑材料中的《AI 工具使用详情.pdf》。

参考文献

- [1] 国务院办公厅. 国务院办公厅关于 2025 年部分节假日安排的通知（国办发明电〔2024〕12 号）[Z]. 2024.
- [2] MACKINNON J G, WHITE H. Some heteroskedasticity-consistent covariance matrix estimators with improved finite sample properties[J]. Journal of Econometrics, 1985, 29(3): 305-325.
- [3] MANN H B, WHITNEY D R. On a test of whether one of two random variables is stochastically larger than the other[J]. The Annals of Mathematical Statistics, 1947, 18(1): 50-60.
- [4] LAWSON C L, HANSON R J. Solving least squares problems[M]. Philadelphia: SIAM, 1995.
- [5] SHANNON C E. A mathematical theory of communication[J]. The Bell System Technical Journal, 1948, 27(3): 379-423.
- [6] FISHER W D. On grouping for maximum homogeneity[J]. Journal of the American Statistical Association, 1958, 53(284): 789-798.

- [7] JENKS G F. The data model concept in statistical mapping[J]. International Yearbook of Cartography, 1967, 7: 186-190.
- [8] EDELMAN B, OSTROVSKY M, SCHWARZ M. Internet advertising and the generalized second-price auction: Selling billions of dollars worth of keywords[J]. American Economic Review, 2007, 97(1): 242-259.
- [9] VARIAN H R. Position auctions[J]. International Journal of Industrial Organization, 2007, 25(6): 1163-1178.
- [10] LITTLE J D C. Aggregate advertising models: The state of the art[J]. Operations Research, 1979, 27(4): 629-667.
- [11] HANSSENS D M, PARSONS L J, SCHULTZ R L. Market response models: Econometric and time series analysis[M]. 2nd ed. Boston: Kluwer Academic Publishers, 2001.
- [12] FELDMAN J, MUTHUKRISHNAN S, PÁL M, et al. Budget optimization in search-based advertising auctions[C]//Proceedings of the 8th ACM Conference on Electronic Commerce. New York: ACM, 2007: 40-49.
- [13] ZHANG W, ZHANG Y, GAO B, et al. Joint optimization of bid and budget allocation in sponsored search[C]//Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. New York: ACM, 2012: 1177-1185.
- [14] BOYD S, VANDENBERGHE L. Convex optimization[M]. Cambridge: Cambridge University Press, 2004.
- [15] BERTSEKAS D P. Nonlinear programming[M]. 2nd ed. Belmont: Athena Scientific, 1999.
- [16] PATRIKSSON M. A survey on the continuous nonlinear resource allocation problem[J]. European Journal of Operational Research, 2008, 185(1): 1-46.
- [17] DIAMOND S, BOYD S. CVXPY: A Python-embedded modeling language for convex optimization[J]. Journal of Machine Learning Research, 2016, 17(83): 1-5.
- [18] EFRON B, TIBSHIRANI R J. An introduction to the bootstrap[M]. New York: Chapman & Hall, 1993.
- [19] HYNDMAN R J, ATHANASOPOULOS G. Forecasting: Principles and practice[M]. 3rd ed. Melbourne: OTexts, 2021.
- [20] KOENKER R, BASSETT G. Regression quantiles[J]. Econometrica, 1978, 46(1): 33-50.
- [21] GNEITING T, RAFTERY A E. Strictly proper scoring rules, prediction, and estimation[J]. Journal of the American Statistical Association, 2007, 102(477): 359-378.

A 支撑材料文件清单

下表列出支撑材料压缩包内的全部文件（相对路径）及其用途。压缩包内另含 `MANIFEST.json`，记录每个文件的 SHA-256。

文件	说明
<code>AI_USAGE.md</code>	AI 工具使用记录（源文件）
<code>AI 工具使用详情.pdf</code>	AI 工具使用详情
<code>MANIFEST.json</code>	支撑材料成员清单与 SHA-256
<code>Makefile</code>	本地快捷命令(只提交云端任务)
<code>README.md</code>	仓库说明与复现入口
<code>REPRODUCE.md</code>	复现说明
<code>UNLICENSE</code>	公有领域声明（Unlicense）
<code>docs/ARCHITECTURE.md</code>	工程规范、建模标准、决策记录
<code>docs/CONTRIBUTING.md</code>	工程规范、建模标准、决策记录
<code>docs/DATA_NOTES.md</code>	工程规范、建模标准、决策记录
<code>docs/ENGINEERING_STANDARD.md</code>	工程规范、建模标准、决策记录
<code>docs/MODELING_STANDARD.md</code>	工程规范、建模标准、决策记录
<code>docs/RESULTS.md</code>	工程规范、建模标准、决策记录
<code>docs/RUNBOOK.md</code>	工程规范、建模标准、决策记录
<code>docs/adr/0001-cloud-only-execution.md</code>	工程规范、建模标准、决策记录
<code>docs/adr/0002-stage-run-manifest-model.md</code>	工程规范、建模标准、决策记录
<code>docs/adr/0003-independent-repos-vendored-runtime.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0000-template.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0001-data-scope-and-keys.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0002-holiday-calendar-and-regression.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0003-registration-attribution.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0004-classification-benefit-and-thresholds.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0005-response-function.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0006-allocation-model.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0007-expected-position.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0008-forecast-and-uncertainty.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0009-verification-protocol.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/0010-paper-reporting-conventions.md</code>	工程规范、建模标准、决策记录
<code>docs/mdr/README.md</code>	工程规范、建模标准、决策记录
<code>docs/problem_brief.md</code>	工程规范、建模标准、决策记录
<code>forge/__init__.py</code>	云端运行时(阶段执行、清单、契约、排版、打包)

文件	说明
forge/config.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/context.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/contracts.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/events.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/hashing.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/manifest.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/modal_app.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/packaging.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/pdfqa.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/plotting.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/runner.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/tex.py	云端运行时 (阶段执行、清单、契约、排版、打包)
forge/xlsx.py	云端运行时 (阶段执行、清单、契约、排版、打包)
manuscript/ai_usage.tex	论文 LaTeX 源码
manuscript/main.tex	论文 LaTeX 源码
manuscript/references.bib	论文 LaTeX 源码
manuscript/sections/00_abstract.tex	论文 LaTeX 源码
manuscript/sections/01_restatement.tex	论文 LaTeX 源码
manuscript/sections/02_analysis.tex	论文 LaTeX 源码
manuscript/sections/03_assumptions.tex	论文 LaTeX 源码
manuscript/sections/04_models.tex	论文 LaTeX 源码
manuscript/sections/04_q1.tex	论文 LaTeX 源码
manuscript/sections/05_q2.tex	论文 LaTeX 源码
manuscript/sections/06_q3.tex	论文 LaTeX 源码
manuscript/sections/07_q4.tex	论文 LaTeX 源码
manuscript/sections/08_validation.tex	论文 LaTeX 源码

文件	说明
manuscript/sections/09_evaluation.tex	论文 LaTeX 源码
manuscript/sections/10_ai_statement.tex	论文 LaTeX 源码
manuscript/sections/ai_usage_body.tex	论文 LaTeX 源码
manuscript/sections/appendix_repro.tex	论文 LaTeX 源码
manuscript/sections/appendix_tables.tex	论文 LaTeX 源码
pipelines/__init__.py	本题建模与求解程序
pipelines/common/__init__.py	通用阶段：数据接入、门禁、论文、质量检查、打包发布
pipelines/common/stages.py	通用阶段：数据接入、门禁、论文、质量检查、打包发布
pipelines/common/validation.py	通用阶段：数据接入、门禁、论文、质量检查、打包发布
pipelines/e/__init__.py	本题建模与求解程序
pipelines/e/allocate.py	本题建模与求解程序
pipelines/e/calendar.py	本题建模与求解程序
pipelines/e/classify.py	本题建模与求解程序
pipelines/e/contracts.py	本题建模与求解程序
pipelines/e/data.py	本题建模与求解程序
pipelines/e/eda.py	本题建模与求解程序
pipelines/e/figures.py	本题建模与求解程序
pipelines/e/forecast.py	本题建模与求解程序
pipelines/e/response.py	本题建模与求解程序
pipelines/e/stages.py	本题建模与求解程序
pipelines/e/tables.py	本题建模与求解程序
pyproject.toml	项目元数据与静态检查配置
runs/20260912-041501-ae773aa/allocate/allocation.csv	
runs/20260912-041501-ae773aa/allocate/allocation.parquet	
runs/20260912-041501-ae773aa/allocate/allocation_audit.json	
runs/20260912-041501-ae773aa/allocate/allocation_summary.csv	
runs/20260912-041501-ae773aa/allocate/allocation_summary.parquet	
runs/20260912-041501-ae773aa/allocate/bootstrap.csv	
runs/20260912-041501-ae773aa/allocate/bootstrap.parquet	
runs/20260912-041501-ae773aa/allocate/calibration_check.json	

文件	说明
runs/20260912-041501-ae773aa/allocate/events.jsonl	结构化运行日志
runs/20260912-041501-ae773aa/allocate/keyword_parameters.csv	
runs/20260912-041501-ae773aa/allocate/keyword_parameters.parquet	阶段清单（输入输出散列、环境、参数）
runs/20260912-041501-ae773aa/allocate/manifest.json	
runs/20260912-041501-ae773aa/allocate/numbers.json	
runs/20260912-041501-ae773aa/allocate/position_models.json	
runs/20260912-041501-ae773aa/allocate/sensitivity.csv	
runs/20260912-041501-ae773aa/allocate/sensitivity.parquet	
runs/20260912-041501-ae773aa/allocate/window_budget_alternative.csv	
runs/20260912-041501-ae773aa/allocate/window_budget_alternative.parquet	
runs/20260912-041501-ae773aa/classify/classification.csv	
runs/20260912-041501-ae773aa/classify/classification.parquet	
runs/20260912-041501-ae773aa/classify/classification_by_label.csv	
runs/20260912-041501-ae773aa/classify/classification_by_label.parquet	
runs/20260912-041501-ae773aa/classify/classification_check.json	
runs/20260912-041501-ae773aa/classify/classification_per_unit.csv	
runs/20260912-041501-ae773aa/classify/classification_per_unit.parquet	
runs/20260912-041501-ae773aa/classify/classification_summary.json	
runs/20260912-041501-ae773aa/classify/events.jsonl	结构化运行日志
runs/20260912-041501-ae773aa/classify/manifest.json	阶段清单（输入输出散列、环境、参数）
runs/20260912-041501-ae773aa/classify/numbers.json	
runs/20260912-041501-ae773aa/eda/attribution.json	
runs/20260912-041501-ae773aa/eda/attribution_fit.csv	

文件	说明
runs/20260912-041501-ae773aa/eda/attribution_fit.parquet	
runs/20260912-041501-ae773aa/eda/calendar_regression.csv	
runs/20260912-041501-ae773aa/eda/calendar_regression.parquet	
runs/20260912-041501-ae773aa/eda/calendar_regression_summary.json	
runs/20260912-041501-ae773aa/eda/concentration.json	
runs/20260912-041501-ae773aa/eda/daily_totals.csv	
runs/20260912-041501-ae773aa/eda/daily_totals.parquet	
runs/20260912-041501-ae773aa/eda/day_type_profile.csv	
runs/20260912-041501-ae773aa/eda/day_type_profile.parquet	
runs/20260912-041501-ae773aa/eda/events.jsonl	结构化运行日志
runs/20260912-041501-ae773aa/eda/keyword_per_unit.csv	
runs/20260912-041501-ae773aa/eda/keyword_per_unit.parquet	
runs/20260912-041501-ae773aa/eda/lorenz.csv	
runs/20260912-041501-ae773aa/eda/lorenz.parquet	
runs/20260912-041501-ae773aa/eda/manifest.json	阶段清单（输入输出散列、环境、参数）
runs/20260912-041501-ae773aa/eda/numbers.json	
runs/20260912-041501-ae773aa/eda/unit_daily.csv	
runs/20260912-041501-ae773aa/eda/unit_daily.parquet	
runs/20260912-041501-ae773aa/eda/unit_kpis.csv	
runs/20260912-041501-ae773aa/eda/unit_kpis.parquet	
runs/20260912-041501-ae773aa/eda/unit_response.csv	
runs/20260912-041501-ae773aa/eda/unit_response.parquet	
runs/20260912-041501-ae773aa/figures/events.jsonl	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_all_location_shift.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_all_location_shift.png	论文图(矢量 PDF 与预览 PNG)

文件	说明
runs/20260912-041501-ae773aa/figures/figures/fig_att tribution.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_att tribution.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_bac ktest.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_bac ktest.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_cal endar_effects.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_cal endar_effects.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_cla ss_thresholds.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_cla ss_thresholds.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_cos t_benefit.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_cos t_benefit.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_dai ly_series.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_dai ly_series.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_lor enz.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_lor enz.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_q3_ gain.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_q3_ gain.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_q4_ plan.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_q4_ plan.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_q4_ units.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_q4_ units.png	论文图(矢量 PDF 与预览 PNG)

文件	说明
runs/20260912-041501-ae773aa/figures/figures/fig_res ponse_curves.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_res ponse_curves.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_uni t_kpis.pdf	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures/fig_uni t_kpis.png	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/figures_index.j son	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/figures/manifest.json	论文图(矢量 PDF 与预览 PNG)
runs/20260912-041501-ae773aa/forecast/allocation_aud it.json	
runs/20260912-041501-ae773aa/forecast/backtest.csv	
runs/20260912-041501-ae773aa/forecast/backtest.parqu et	
runs/20260912-041501-ae773aa/forecast/backtest_summa ry.csv	
runs/20260912-041501-ae773aa/forecast/backtest_summa ry.parquet	
runs/20260912-041501-ae773aa/forecast/budget_alterna tives.json	
runs/20260912-041501-ae773aa/forecast/budgets.csv	
runs/20260912-041501-ae773aa/forecast/budgets.parque t	
runs/20260912-041501-ae773aa/forecast/events.jsonl	结构化运行日志
runs/20260912-041501-ae773aa/forecast/keyword_parame ters.csv	
runs/20260912-041501-ae773aa/forecast/keyword_parame ters.parquet	
runs/20260912-041501-ae773aa/forecast/manifest.json	阶段清单 (输入输出散列、环境、 参数)
runs/20260912-041501-ae773aa/forecast/numbers.json	
runs/20260912-041501-ae773aa/forecast/plan.csv	
runs/20260912-041501-ae773aa/forecast/plan.parquet	
runs/20260912-041501-ae773aa/forecast/plan_by_unit.j son	
runs/20260912-041501-ae773aa/forecast/plan_by_unit_d ay.csv	

文件	说明
runs/20260912-041501-ae773aa/forecast/plan_by_unit_day.parquet	
runs/20260912-041501-ae773aa/forecast/unit_forecasts.csv	
runs/20260912-041501-ae773aa/forecast/unit_forecasts.parquet	
runs/20260912-041501-ae773aa/ingest/data/附件1_Sheet1.parquet	件 原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-041501-ae773aa/ingest/data/附件1_Sheet2.parquet	件 原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-041501-ae773aa/ingest/data/附件1_Sheet3.parquet	件 原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-041501-ae773aa/ingest/events.jsonl	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-041501-ae773aa/ingest/inventory.json	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-041501-ae773aa/ingest/manifest.json	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-041501-ae773aa/ingest/problem_text.txt	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-041501-ae773aa/lint/events.jsonl	静态检查报告
runs/20260912-041501-ae773aa/lint/lint_report.json	静态检查报告
runs/20260912-041501-ae773aa/lint/manifest.json	静态检查报告
runs/20260912-041501-ae773aa/lint/mypy.txt	静态检查报告
runs/20260912-041501-ae773aa/lint/ruff_check.txt	静态检查报告
runs/20260912-041501-ae773aa/lint/ruff_format.txt	静态检查报告
runs/20260912-041501-ae773aa/qa/events.jsonl	论文与结果文件质量门禁报告
runs/20260912-041501-ae773aa/qa/manifest.json	论文与结果文件质量门禁报告
runs/20260912-041501-ae773aa/qa/qa_report.json	论文与结果文件质量门禁报告
runs/20260912-041501-ae773aa/results/events.jsonl	结果工作簿与求解输出
runs/20260912-041501-ae773aa/results/manifest.json	结果工作簿与求解输出
runs/20260912-041501-ae773aa/results/result2.xlsx	结果工作簿与求解输出
runs/20260912-041501-ae773aa/results/result3.xlsx	结果工作簿与求解输出
runs/20260912-041501-ae773aa/results/result4.xlsx	结果工作簿与求解输出
runs/20260912-041501-ae773aa/results/results_summary.json	结果工作簿与求解输出
runs/20260912-041501-ae773aa/tables/events.jsonl	论文表（LaTeX 片段与 CSV）
runs/20260912-041501-ae773aa/tables/manifest.json	论文表（LaTeX 片段与 CSV）

文件	说明
runs/20260912-041501-ae773aa/tables/tables/tab_attri bution_response.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_attri bution_response.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_backt est.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_backt est.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_calen dar_effects.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_calen dar_effects.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_class _per_unit.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_class _per_unit.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_class _robustness.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_class _robustness.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_class _summary.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_class _summary.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q3_da ily.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q3_da ily.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q3_se nsitivity.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q3_se nsitivity.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q3_su mmary.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q3_su mmary.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q3_wi ndow_budget.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q3_wi ndow_budget.tex	论文表 (LaTeX 片段与 CSV)

文件	说明
runs/20260912-041501-ae773aa/tables/tables/tab_q4_da_ily.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q4_da_ily.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q4_units.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_q4_units.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_unit_kpis.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_unit_kpis.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_verification.csv	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables/tab_verification.tex	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/tables/tables_index.json	论文表 (LaTeX 片段与 CSV)
runs/20260912-041501-ae773aa/test/coverage.json	测试报告与覆盖率
runs/20260912-041501-ae773aa/test/events.jsonl	测试报告与覆盖率
runs/20260912-041501-ae773aa/test/junit.xml	测试报告与覆盖率
runs/20260912-041501-ae773aa/test/manifest.json	测试报告与覆盖率
runs/20260912-041501-ae773aa/test/pytest.txt	测试报告与覆盖率
runs/20260912-041501-ae773aa/test/test_report.json	测试报告与覆盖率
runs/20260912-041501-ae773aa/validate/data_profile.json	数据契约校验报告与数据画像
runs/20260912-041501-ae773aa/validate/events.jsonl	数据契约校验报告与数据画像
runs/20260912-041501-ae773aa/validate/manifest.json	数据契约校验报告与数据画像
runs/20260912-041501-ae773aa/validate/validation_report.json	数据契约校验报告与数据画像
tests/__init__.py	单元 / 性质 / 集成测试
tests/conftest.py	单元 / 性质 / 集成测试
tests/integration/__init__.py	单元 / 性质 / 集成测试
tests/integration/test_ingest_validate.py	单元 / 性质 / 集成测试
tests/unit/__init__.py	单元 / 性质 / 集成测试
tests/unit/test_config.py	单元 / 性质 / 集成测试
tests/unit/test_contracts.py	单元 / 性质 / 集成测试
tests/unit/test_e_allocate.py	单元 / 性质 / 集成测试
tests/unit/test_e_calendar_data.py	单元 / 性质 / 集成测试

文件	说明
tests/unit/test_e_classify.py	单元 / 性质 / 集成测试
tests/unit/test_e_forecast.py	单元 / 性质 / 集成测试
tests/unit/test_hashing.py	单元 / 性质 / 集成测试
tests/unit/test_manifest.py	单元 / 性质 / 集成测试
tests/unit/test_packaging_tex.py	单元 / 性质 / 集成测试
tests/unit/test_runner.py	单元 / 性质 / 集成测试
tests/unit/test_xlsx.py	单元 / 性质 / 集成测试
tools/__init__.py	本地控制面：提交云端任务、下载、校验散列
tools/cli.py	本地控制面：提交云端任务、下载、校验散列

B 复现记录

论文中的全部数值、表格与图形均由同一次云端运行（run 20260912-041501-ae773aa，代码版本 9affd17f540e7631cb8c1804c8911d4000d89b0b，构建日期 2026-09-12）生成。各阶段状态与输出摘要如下。

阶段	状态	用时/s	输出摘要 (SHA-256 前 12 位)
allocate	completed	51.0	0ea04bf6f53a
classify	completed	1.0	51c608417cda
eda	completed	1.0	edfe1358e716
figures	completed	23.0	77c7e5184b34
forecast	completed	11.0	1d5db747a8e0
ingest	completed	1.0	02e38a853d4c
lint	completed	20.0	cd5e1ca31d0d
paper	running	0.0	
qa	failed	2.0	
results	completed	1.0	bbad2b8a99ec
tables	completed	1.0	3aa4744a0e04
test	completed	19.0	6d3e14a65892
validate	completed	0.0	8ad89df6a5ec

复现步骤：安装 Modal 客户端并登录后，在仓库根目录依次执行 `python3tools/cli.pyprovision`（验证云端工具链）、`python3tools/cli.pyruningest,validate--new-run`（接入与契约校验）、`python3tools/cli.pyruneda,classify,allocate,forecast,results,figures,tables--sizemedium`（四个问题的科学阶段与结果、图表）、`python3tools/cli.pyrunlint,test,paper,qa,package,release`（门禁、论文、质检、打包与发布）。每个阶段目录下的 `manifest.json` 记录输入输出散列、参数、依赖阶段的输出摘要与环境；`events.jsonl` 为结构化日志；`numbers.json` 为该

阶段登记的论文数字。随机性只经 `ctx.seed_everything` 派生（种子见 `configs/default.toml`），同一代码与配置下的重跑得到相同的输出摘要。问题三与问题四的独立校验报告分别为 `allocate/allocation_audit.json` 与 `forecast/allocation_audit.json`，分类复核报告为 `classify/classification_check.json`，回测明细为 `forecast/backtest.csv`。

C 补充表格

本附录收录正文引用的大表：各推广单元的关键词类别分布（问题二）、两个窗口的逐日分配结果与灵敏度分析（问题三）以及滚动回测的完整指标（问题四）。

表 15 各推广单元的关键词类别分布（问题二）

推广单元 ID	黄金词	重点词	潜力词	问题词	无效词	合计
1841014603	0	8	24	3	41	76
2526158329	0	0	14	0	12	26
2874903000	0	9	46	5	12	72
4190988309	2	2	14	1	28	47
4190988310	1	15	27	2	28	73
8878077181	0	4	0	0	0	4
9628223555	2	44	187	14	117	364
9630806627	3	85	187	6	161	442
9657930100	0	28	10	1	6	45
9700005700	0	5	6	0	6	17
9773660250	0	3	2	0	1	6
9811363528	32	68	460	17	478	1,055

表 16 问题三两个窗口的逐日汇总：预算、实际点击、按历史比例与最优的预期点击和注册、均匀分配的预期注册及提升

日期	单元数	预算/元	实际点击	历史比例点击	最优点击	历史比例注册	均匀注册	最优注册	提升/%
2025-08-01	11	5,722.9	6,942	3,828.4	4,710.6	300.1	239.1	425.1	41.6
2025-08-02	12	5,515.8	6,480	3,650.0	4,537.0	293.8	237.5	420.6	43.1
2025-08-03	11	2,422.2	2,003	1,425.6	1,597.5	99.2	66.6	117.2	18.1
2025-08-04	11	2,364.3	1,912	1,380.7	1,550.6	97.0	65.3	116.5	20.1
2025-08-05	11	4,530.6	3,797	2,956.9	3,312.5	193.5	134.6	230.4	19.0
2025-08-06	12	5,300.0	4,827	3,785.1	4,273.8	248.4	169.9	302.5	21.8
2025-08-07	11	5,703.8	4,919	3,913.9	4,422.0	258.5	175.6	314.5	21.6
2025-08-08	12	5,924.1	4,932	3,844.1	4,314.8	257.4	175.4	310.7	20.7
2025-02-01	2	803.0	313	393.6	450.6	27.4	16.2	31.4	14.4
2025-02-02	2	1,155.0	479	462.8	527.9	32.6	19.3	37.1	13.9
2025-02-03	2	1,403.3	586	536.4	600.3	40.0	23.9	44.6	11.6
2025-02-04	2	1,806.4	739	750.2	838.5	55.8	33.4	62.2	11.5
2025-02-05	2	2,007.8	834	1,010.7	1,128.7	75.1	44.9	83.7	11.4
2025-02-06	2	2,044.0	785	1,050.8	1,172.3	78.3	46.8	87.1	11.3
2025-02-07	2	2,523.3	998	1,174.0	1,303.3	83.9	49.9	94.3	12.5
2025-02-08	2	1,938.3	921	956.8	1,076.7	66.6	39.5	76.0	14.0

表 17 问题三的灵敏度分析：不同建模选择下最优分配相对历史比例的预期注册提升

情形	窗口	最优注册/人	历史比例注册/人	提升/%	平均入选词
基准	2 月	516.4	459.6	12.4	20.4
基准	8 月	2237.4	1748.0	28.0	21.7
上限倍数 2	2 月	507.2	459.6	10.3	34.8
上限倍数 2	8 月	2064.5	1748.0	18.1	27.2
上限倍数 8	2 月	517.9	459.6	12.7	18.1
上限倍数 8	8 月	2342.7	1748.0	34.0	17.3
弹性 $\gamma \times 0.9$	2 月	517.3	474.6	9.0	21.2
弹性 $\gamma \times 0.9$	8 月	2111.6	1733.3	21.8	20.9
弹性 $\gamma \times 1.1$	2 月	523.0	445.4	17.4	20.9
弹性 $\gamma \times 1.1$	8 月	2411.9	1768.6	36.4	23.4
预算 -20%	2 月	441.9	392.8	12.5	17.5
预算 -20%	8 月	1925.7	1472.7	30.8	19.1
预算 +20%	2 月	585.7	522.6	12.1	25.3
预算 +20%	8 月	2519.6	2011.2	25.3	23.6
剔除问题词	2 月	516.4	459.6	12.4	20.4
剔除问题词	8 月	2237.2	1748.0	28.0	21.5
问题词不降上限	2 月	516.4	459.6	12.4	20.4
问题词不降上限	8 月	2237.5	1748.0	28.0	21.7
最小投放 0	2 月	516.1	459.6	12.3	143.0
最小投放 0	8 月	2197.4	1748.0	25.7	116.8

表 18 滚动回测（12 个起点 $\times$ 7 天，12 个单元）：各目标在变换尺度上的 MAE、RMSE、pinball 损失与 80% 区间覆盖率

目标	模型	MAE	RMSE	Pinball(0.1+0.9)	80% 覆盖率/%
log 点击量	日历回归	0.480	0.568	0.253	87.3
log 点击量	28 日均值	0.577	0.661	0.237	74.0
log 点击量	季节朴素	0.480	0.581	0.304	90.2
log CPC	日历回归	0.293	0.328	0.199	81.8
log CPC	28 日均值	0.243	0.263	0.166	63.6
log CPC	季节朴素	0.253	0.289	0.191	88.7
效率乘子 $\log \varepsilon$	日历回归	0.236	0.268	0.157	81.3
效率乘子 $\log \varepsilon$	28 日均值	0.187	0.211	0.113	68.8
效率乘子 $\log \varepsilon$	季节朴素	0.214	0.244	0.140	90.1
logit CTR	日历回归	0.437	0.516	0.272	78.9
logit CTR	28 日均值	0.271	0.319	0.129	80.3
logit CTR	季节朴素	0.305	0.370	0.179	91.2
logit 首位占比	日历回归	0.584	0.678	0.347	84.5
logit 首位占比	28 日均值	0.406	0.464	0.167	72.3
logit 首位占比	季节朴素	0.444	0.529	0.240	91.0
logit 上方位占比	日历回归	0.640	0.727	0.394	78.3
logit 上方位占比	28 日均值	0.452	0.514	0.240	70.2
logit 上方位占比	季节朴素	0.483	0.563	0.285	90.2

D 完整程序

以下按文件收录建模、求解、检验、绘图、排版与打包的全部源程序，文件名与附录 A 一致。运行方式见附录 B。

Makefile

```
1 # Thin wrappers over tools/cli.py. Nothing here computes locally.
2 PY ?= python3
3 CLI := $(PY) tools/cli.py
4 STAGE ?= ingest
5 SIZE ?= small
6 RUN ?=
7 RUNFLAG := $(if $(RUN),--run-id $(RUN),)
8 FORCEFLAG := $(if $(FORCE),--force,)
9
10 .PHONY: fmt help provision run pipeline exec status logs runs download release lint test paper qa package smoke
11
12 help:
13     @echo "make provision          # verify cloud toolchain"
14     @echo "make run STAGE=q1 SIZE=medium # run one stage (RUN=<id> to continue a run, FORCE=1 to rerun)"
15     @echo "make pipeline          # ingest,validate,lint,test in one container"
16     @echo "make status | logs STAGE=q1 | runs | download STAGE=results"
17     @echo "make paper | qa | package | release VERSION=v1.0.0"
18
19 provision:
20     $(CLI) provision
21
22 run:
23     $(CLI) run $(STAGE) $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
24
25 pipeline:
26     $(CLI) run ingest,validate,lint,test $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
27
28 exec:
29     $(CLI) exec --code '$(CODE)' $(RUNFLAG)
30
31 status:
32     $(CLI) status $(RUNFLAG)
33
34 logs:
35     $(CLI) logs $(STAGE) $(RUNFLAG)
36
37 runs:
38     $(CLI) runs
39
40 download:
41     $(CLI) download $(RUNFLAG) $(if $(STAGE),--stage $(STAGE),)
42
43 lint:
44     $(CLI) run lint $(RUNFLAG) $(FORCEFLAG)
45
46 test:
47     $(CLI) run test $(RUNFLAG) $(FORCEFLAG)
48
49 paper:
50     $(CLI) run paper $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
51
52 qa:
53     $(CLI) run qa $(RUNFLAG) $(FORCEFLAG)
54
```



```

55 package:
56     $(CLI) run package $(RUNFLAG) $(FORCEFLAG)
57
58 release:
59     $(CLI) run release $(RUNFLAG) $(FORCEFLAG) && $(CLI) release --version $(VERSION) $(RUNFLAG) --force
60
61 smoke:
62     $(CLI) run ingest,validate,lint,test,paper,qa,package,release --new-run --param require_results=false --param
        ↪ allow_missing_results=true
63
64 fmt:
65     $(CLI) fmt $(RUNFLAG)

```

configs/default.toml

```

1  # Forge 运行时配置（默认档）。Stage 代码通过 ctx.cfg("a.b.c") 读取；
2  # `--profile <name>` 会叠加 configs/<name>.toml；`--param config_overrides='{...}'` 可临时覆盖。
3
4  [run]
5  seed = 20260912
6
7  [gates]
8  lint_required = ["ruff_check", "ruff_format", "mypy"]
9  mypy_paths = ["forge"]
10
11 [paper]
12 main = "main.tex"
13 # 需在同一 run 中已完成的 stage；其 tables/*.tex 与 figures/*.pdf/png 会被并入排版
14 sources = ["figures", "tables"]
15 max_body_pages = 30
16 max_overfull_pt = 20.0
17 code_globs = [
18     "forge/**/*.py", "pipelines/**/*.py", "tools/**/*.py", "tests/**/*.py",
19     "configs/*.toml", "configs/*.json", "Makefile", "pyproject.toml",
20 ]
21
22 [qa]
23 required_stages = ["lint", "test"]
24 results_stage = "results"
25 require_results = true
26 identity_denylist = [] # 硬门禁（正则）：出现即失败
27 identity_watchlist = ["大学", "学院", "赛区", "队号", "指导教师", "[\\w.-]+@[\\w.-]+\\.\\w+"] # 软提示：写入报告供复核
28
29 [package]
30 stages = ["ingest", "validate", "eda", "classify", "allocate", "forecast", "results", "figures", "tables", "lint", "
        ↪ test", "qa"]
31 code_globs = [
32     "forge/**/*.py", "pipelines/**/*.py", "tools/**/*.py", "tests/**/*.py", "configs/**",
33     "manuscript/**/*.tex", "manuscript/**/*.bib", "docs/**/*.md", "Makefile", "pyproject.toml",
34     "README.md", "UNLICENSE", "AI_USAGE.md",
35 ]
36 max_bytes = 20971520
37
38 [release]
39 paper_name = "论文.pdf"
40
41 [classify]
42 methods = ["jenks", "median", "gmm"] # threshold methods on log10(cost) and on the benefit index
43 main_method = "jenks" # reported labels; the others are robustness checks
44
45 [allocate]

```

```

46 cap_multiplier = 4.0          # daily spend cap per keyword = multiplier x mean daily spend
47 min_spend = 0.5              # yuan; smaller allocations mean the keyword is not selected
48 problem_cap_factor = 0.5     # 问题词 keep only half of the normal cap
49 bootstrap = 100              # elasticity bootstrap draws for the improvement interval
50
51 [forecast]
52 scenarios = 500               # Monte Carlo scenarios per unit-day for expected ranges

```

configs/project.json

```

1 {
2   "letter": "E",
3   "package": "e",
4   "slug": "sem-advertising",
5   "title": "SEM 广告投放策略",
6   "app": "cumcm2026-e-sem-advertising",
7   "volume": "cumcm2026-e-artifacts",
8   "problem_pdf": "E题.pdf",
9   "attachments_dir": "附件",
10  "templates_dir": "附件/附件2",
11  "result_files": [
12    "result2.xlsx",
13    "result3.xlsx",
14    "result4.xlsx"
15  ],
16  "github_repo": "cumcm2026-e-sem-advertising"
17 }

```

forge/__init__.py

```

1 """Forge — cloud-only research engineering runtime (CUMCM 2026).
2
3 Every computation (data parsing, modelling, solving, tests, figures, LaTeX, PDF QA,
4 packaging) runs inside Modal. The local machine only edits text, drives git,
5 submits jobs, downloads artifacts and verifies transport hashes.
6 """
7
8 __version__ = "1.0.0"

```

forge/config.py

```

1 """TOML configuration with profile overlays and dotted access."""
2
3 from __future__ import annotations
4
5 import json
6 import tomllib
7 from pathlib import Path
8 from typing import Any
9
10 from .hashing import sha256_json
11
12 _MISSING = object()
13
14
15 def deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:

```

```

16     out = dict(base)
17     for key, value in override.items():
18         if isinstance(value, dict) and isinstance(out.get(key), dict):
19             out[key] = deep_merge(out[key], value)
20         else:
21             out[key] = value
22     return out
23
24
25 class Config:
26     """Read-only dotted-access view over a merged configuration mapping."""
27
28     def __init__(self, data: dict[str, Any]) -> None:
29         self._data = json.loads(json.dumps(data))
30
31     def get(self, key: str, default: Any = None) -> Any:
32         cursor: Any = self._data
33         for part in key.split("."):
34             if not isinstance(cursor, dict) or part not in cursor:
35                 return default
36             cursor = cursor[part]
37         return cursor
38
39     def __getitem__(self, key: str) -> Any:
40         value = self.get(key, _MISSING)
41         if value is _MISSING:
42             raise KeyError(key)
43         return value
44
45     def section(self, key: str) -> dict[str, Any]:
46         value = self.get(key, {})
47         return dict(value) if isinstance(value, dict) else {}
48
49     def as_dict(self) -> dict[str, Any]:
50         return json.loads(json.dumps(self._data))
51
52     @property
53     def digest(self) -> str:
54         return sha256_json(self._data)
55
56
57 def load_project(repo: Path) -> dict[str, Any]:
58     return json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
59
60
61 def load_config(repo: Path, profile: str | None = None, overrides: dict[str, Any] | None = None) -> Config:
62     data = tomlib.loads((repo / "configs" / "default.toml").read_text(encoding="utf-8"))
63     if profile:
64         data = deep_merge(data, tomlib.loads((repo / "configs" / f"{profile}.toml").read_text(encoding="utf-8")))
65     if overrides:
66         data = deep_merge(data, overrides)
67     return Config(data)

```

forge/context.py

```

1 """Stage execution context: paths, config, params, logging, seeds and paper numbers."""
2
3 from __future__ import annotations
4
5 import json
6 import random

```

```

7 import re
8 import zlib
9 from pathlib import Path
10 from typing import Any
11
12 from .config import Config, load_config, load_project
13 from .events import EventLogger
14
15 _NUMBER_KEY = re.compile(r"[A-Za-z]+$")
16
17
18 class StageContext:
19     """Everything a stage needs. Stages must write only under ``stage_dir``."""
20
21     def __init__(self, run_id: str, stage: str, params: dict[str, Any], repo: Path, vol: Path) -> None:
22         self.run_id = run_id
23         self.stage = stage
24         self.params = dict(params)
25         self.repo = repo
26         self.vol = vol
27         self.project = load_project(repo)
28         overrides = self.params.get("config_overrides") or None
29         self.config: Config = load_config(repo, self.params.get("profile"), overrides)
30         self.run_dir = vol / "runs" / run_id
31         self.stage_dir = self.run_dir / stage
32         self.stage_dir.mkdir(parents=True, exist_ok=True)
33         self.log = EventLogger(self.stage_dir / "events.jsonl", run_id, stage)
34         self._numbers: dict[str, str] = {}
35
36     # ---- raw inputs (immutable, at repo root, exactly as delivered by the organisers) ----
37     @property
38     def problem_pdf(self) -> Path:
39         return self.repo / self.project["problem_pdf"]
40
41     @property
42     def attachments(self) -> Path:
43         return self.repo / self.project["attachments_dir"]
44
45     @property
46     def templates(self) -> Path:
47         return self.repo / self.project["templates_dir"]
48
49     # ---- outputs ----
50     def out(self, *parts: str) -> Path:
51         path = self.stage_dir.joinpath(*parts)
52         path.parent.mkdir(parents=True, exist_ok=True)
53         return path
54
55     def dep(self, stage: str) -> Path:
56         """Directory of a completed dependency stage in the same run (raises otherwise)."""
57         directory = self.run_dir / stage
58         manifest = directory / "manifest.json"
59         if not manifest.exists():
60             raise RuntimeError(f"dependency stage '{stage}' has not run in run {self.run_id}")
61         status = json.loads(manifest.read_text(encoding="utf-8")).get("status")
62         if status != "completed":
63             raise RuntimeError(f"dependency stage '{stage}' has status {status!r} in run {self.run_id}")
64         return directory
65
66     def has_stage(self, stage: str) -> bool:
67         manifest = self.run_dir / stage / "manifest.json"
68         return manifest.exists() and json.loads(manifest.read_text(encoding="utf-8")).get("status") == "completed"
69

```

```

70 def dep_data(self, stage: str, name: str) -> Any:
71     return json.loads((self.dep(stage) / name).read_text(encoding="utf-8"))
72
73 # ---- parameters / configuration ----
74 def param(self, key: str, default: Any = None) -> Any:
75     return self.params.get(key, default)
76
77 def cfg(self, key: str, default: Any = None) -> Any:
78     return self.config.get(key, default)
79
80 # ---- determinism ----
81 def seed(self, salt: str = "") -> int:
82     base = int(self.cfg("run.seed", 0))
83     return (base + zlib.crc32(salt.encode("utf-8"))) % (2**31 - 1)
84
85 def seed_everything(self, salt: str = "") -> int:
86     value = self.seed(salt)
87     random.seed(value)
88     try:
89         import numpy as np
90
91         np.random.seed(value) # noqa: NPY002 - legacy global RNG used by third-party libraries
92     except ImportError: # pragma: no cover
93         pass
94     self.log.info("seed", salt=salt, value=value)
95     return value
96
97 # ---- numbers that flow into the paper as \val<Key> macros ----
98 def number(self, key: str, value: Any, fmt: str | None = None) -> str:
99     if not _NUMBER_KEY.match(key):
100         raise ValueError(f"paper number keys must be letters only (TeX macro names); got {key!r}")
101     if fmt is not None:
102         text = format(value, fmt)
103     elif isinstance(value, bool):
104         text = str(value)
105     elif isinstance(value, int):
106         text = str(value)
107     elif isinstance(value, float):
108         text = f"{value:.4f}"
109     else:
110         text = str(value)
111     self._numbers[key] = text
112     return text
113
114 def flush_numbers(self) -> None:
115     if not self._numbers:
116         return
117     path = self.stage_dir / "numbers.json"
118     merged: dict[str, str] = {}
119     if path.exists():
120         merged.update(json.loads(path.read_text(encoding="utf-8")))
121     merged.update(self._numbers)
122     path.write_text(json.dumps(merged, ensure_ascii=False, indent=2, sort_keys=True), encoding="utf-8")
123
124 # ---- json helpers ----
125 def write_json(self, name: str, obj: Any) -> Path:
126     path = self.out(name)
127     path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str), encoding="utf-8")
128     return path
129
130 def read_json(self, path: Path) -> Any:
131     return json.loads(path.read_text(encoding="utf-8"))

```

forge/contracts.py

```
1 """Lightweight DataFrame contracts (schema + range + monotonicity + custom checks)."""
2
3 from __future__ import annotations
4
5 import re
6 from collections.abc import Callable
7 from dataclasses import dataclass
8 from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13
14 @dataclass(frozen=True)
15 class Column:
16     name: str
17     kind: str = "any" # int | float | number | str | datetime | any
18     nullable: bool = False
19     min: float | None = None
20     max: float | None = None
21     regex: str | None = None
22     unique: bool = False
23     monotonic: str | None = None # increasing | non_decreasing | decreasing | non_increasing
24     allowed: tuple[Any, ...] | None = None
25     step: float | None = None # constant difference between consecutive values
26
27
28 @dataclass(frozen=True)
29 class FrameContract:
30     name: str
31     columns: tuple[Column, ...]
32     min_rows: int = 1
33     max_rows: int | None = None
34     extra_columns: str = "forbid" # forbid | allow
35     checks: tuple[Callable[[pd.DataFrame], list[str]], ...] = ()
36
37
38 def normalise_columns(df: pd.DataFrame) -> pd.DataFrame:
39     out = df.copy()
40     out.columns = pd.Index([str(c).strip() for c in out.columns])
41     return out
42
43
44 def validate_frame(df: pd.DataFrame, contract: FrameContract) -> dict[str, Any]:
45     df = normalise_columns(df)
46     errors: list[str] = []
47     warnings: list[str] = []
48     expected = [c.name for c in contract.columns]
49     missing = [c for c in expected if c not in df.columns]
50     extra = [c for c in df.columns if c not in expected]
51     if missing:
52         errors.append(f"missing columns: {missing}")
53     if extra and contract.extra_columns == "forbid":
54         errors.append(f"unexpected columns: {extra}")
55     n = len(df)
56     if n < contract.min_rows:
57         errors.append(f"rows {n} < min_rows {contract.min_rows}")
58     if contract.max_rows is not None and n > contract.max_rows:
59         errors.append(f"rows {n} > max_rows {contract.max_rows}")
60
61     for col in contract.columns:
```

```

62     if col.name not in df.columns:
63         continue
64     series = df[col.name]
65     nulls = int(series.isna().sum())
66     if nulls and not col.nullable:
67         errors.append(f"{col.name}: {nulls} null values")
68     values = series.dropna()
69     if col.kind in {"int", "float", "number"}:
70         numeric = pd.to_numeric(values, errors="coerce")
71         bad = int(numeric.isna().sum())
72         if bad:
73             errors.append(f"{col.name}: {bad} non-numeric values")
74         numeric = numeric.dropna().astype(float)
75         if col.kind == "int" and len(numeric) and not np.all(np.mod(numeric.to_numpy(), 1) == 0):
76             errors.append(f"{col.name}: non-integer values present")
77         if col.min is not None and len(numeric) and float(numeric.min()) < col.min:
78             errors.append(f"{col.name}: min {numeric.min()} < {col.min}")
79         if col.max is not None and len(numeric) and float(numeric.max()) > col.max:
80             errors.append(f"{col.name}: max {numeric.max()} > {col.max}")
81         if col.monotonic and len(numeric) > 1:
82             diff = np.diff(numeric.to_numpy())
83             ok = {
84                 "increasing": bool((diff > 0).all()),
85                 "non_decreasing": bool((diff >= 0).all()),
86                 "decreasing": bool((diff < 0).all()),
87                 "non_increasing": bool((diff <= 0).all()),
88             }[col.monotonic]
89             if not ok:
90                 errors.append(f"{col.name}: not {col.monotonic}")
91         if col.step is not None and len(numeric) > 1:
92             diff = np.diff(numeric.to_numpy())
93             if not np.allclose(diff, col.step):
94                 errors.append(f"{col.name}: step is not constant {col.step} (min {diff.min()}, max {diff.max()})")
95     elif col.kind == "str":
96         if col.regex:
97             pattern = re.compile(col.regex)
98             bad_values = [str(v) for v in values.astype(str) if not pattern.fullmatch(str(v))]
99             if bad_values:
100                 errors.append(f"{col.name}: {len(bad_values)} values violate {col.regex}/, e.g. {bad_values[:3]}")
101     elif col.kind == "datetime":
102         parsed = pd.to_datetime(values, errors="coerce")
103         bad = int(parsed.isna().sum())
104         if bad:
105             errors.append(f"{col.name}: {bad} unparsable datetimes")
106     if col.unique and values.duplicated().any():
107         errors.append(f"{col.name}: {int(values.duplicated().sum())} duplicated values")
108     if col.allowed is not None:
109         unexpected = sorted(set(values.unique()) - set(col.allowed), key=str)
110         if unexpected:
111             errors.append(f"{col.name}: values outside allowed set: {unexpected[:5]}")
112
113 for check in contract.checks:
114     try:
115         errors.extend(check(df))
116     except Exception as exc:
117         errors.append(f"check {getattr(check, '__name__', check)!r} raised {exc!r}")
118
119 return {
120     "contract": contract.name,
121     "ok": not errors,
122     "rows": n,
123     "columns": list(df.columns),
124     "errors": errors,

```



```

125     "warnings": warnings,
126 }

```

forge/events.py

```

1  """Structured JSONL event logging with a human-readable mirror on stdout."""
2
3  from __future__ import annotations
4
5  import json
6  import time
7  from datetime import UTC, datetime
8  from pathlib import Path
9  from typing import Any
10
11
12  def _short(value: Any, limit: int = 200) -> str:
13      text = value if isinstance(value, str) else json.dumps(value, ensure_ascii=False, default=str)
14      return text if len(text) <= limit else text[: limit - 1] + "..."
15
16
17  class EventLogger:
18      """Append-only event stream: one JSON object per line, mirrored to stdout."""
19
20      def __init__(self, path: Path, run_id: str, stage: str) -> None:
21          path.parent.mkdir(parents=True, exist_ok=True)
22          self._fh = path.open("a", encoding="utf-8")
23          self.path = path
24          self.run_id = run_id
25          self.stage = stage
26          self._t0 = time.monotonic()
27
28      def emit(self, event: str, level: str = "info", **fields: Any) -> None:
29          record = {
30              "ts": datetime.now(UTC).isoformat(timespec="milliseconds"),
31              "elapsed_s": round(time.monotonic() - self._t0, 3),
32              "run_id": self.run_id,
33              "stage": self.stage,
34              "level": level,
35              "event": event,
36              **fields,
37          }
38          self._fh.write(json.dumps(record, ensure_ascii=False, default=str) + "\n")
39          self._fh.flush()
40          extra = " ".join(f"{k}={_short(v)}" for k, v in fields.items())
41          print(f"[{record['elapsed_s']:9.3f}s] [{level.upper():5s}] [{self.stage}:{event}] {extra}", flush=True)
42
43      def info(self, event: str, **fields: Any) -> None:
44          self.emit(event, "info", **fields)
45
46      def warn(self, event: str, **fields: Any) -> None:
47          self.emit(event, "warn", **fields)
48
49      def error(self, event: str, **fields: Any) -> None:
50          self.emit(event, "error", **fields)
51
52      def metric(self, name: str, value: Any, **fields: Any) -> None:
53          self.emit("metric", "info", name=name, value=value, **fields)
54
55      def close(self) -> None:
56          if not self._fh.closed:

```

```
57         self._fh.close()
```

forge/hashing.py

```
1  """Content hashing helpers (SHA-256 everywhere; JSON hashed in canonical form)."""
2
3  from __future__ import annotations
4
5  import hashlib
6  import json
7  from collections.abc import Iterable
8  from pathlib import Path
9  from typing import Any
10
11  CHUNK = 1 << 20
12
13
14  def sha256_file(path: Path) -> str:
15      h = hashlib.sha256()
16      with path.open("rb") as fh:
17          for chunk in iter(lambda: fh.read(CHUNK), b""):
18              h.update(chunk)
19      return h.hexdigest()
20
21
22  def sha256_bytes(data: bytes) -> str:
23      return hashlib.sha256(data).hexdigest()
24
25
26  def sha256_json(obj: Any) -> str:
27      payload = json.dumps(obj, sort_keys=True, ensure_ascii=False, separators=(",", ":"), default=str)
28      return sha256_bytes(payload.encode("utf-8"))
29
30
31  def hash_tree(root: Path, ignore: Iterable[str] = ()) -> dict[str, dict[str, Any]]:
32      """Return ``{relative_posix_path: {"sha256": ..., "bytes": ...}}`` for every file under ``root``.
33
34      Any path component contained in ``ignore`` excludes the file (directory names or file names).
35      The result is sorted by path so that its canonical JSON digest is stable.
36
37      """
38      ignored = set(ignore)
39      out: dict[str, dict[str, Any]] = {}
40      if not root.exists():
41          return out
42      for path in sorted(root.rglob("*")):
43          if not path.is_file():
44              continue
45          rel = path.relative_to(root)
46          if ignored & set(rel.parts):
47              continue
48          out[rel.as_posix()] = {"sha256": sha256_file(path), "bytes": path.stat().st_size}
49      return out
50
51
52  def tree_digest(tree: dict[str, dict[str, Any]]) -> str:
53      return sha256_json({k: v["sha256"] for k, v in sorted(tree.items())})
```

forge/manifest.py

```
1 """Run/stage manifests: the unit of lineage, reproducibility and audit."""
2
3 from __future__ import annotations
4
5 import importlib.metadata
6 import json
7 import os
8 import platform
9 import sys
10 import traceback
11 from dataclasses import asdict, dataclass, field, fields
12 from datetime import UTC, datetime
13 from pathlib import Path
14 from typing import Any
15
16 from .hashing import hash_tree, tree_digest
17
18 TRACKED_PACKAGES = (
19     "numpy",
20     "scipy",
21     "pandas",
22     "pyarrow",
23     "openpyxl",
24     "matplotlib",
25     "scikit-learn",
26     "statsmodels",
27     "ortools",
28     "highspy",
29     "numba",
30     "cvxpy",
31     "networkx",
32     "pymupdf",
33     "pytest",
34     "hypothesis",
35     "ruff",
36     "mypy",
37 )
38 RECORD_FILES = ("manifest.json", "events.jsonl") # run records, excluded from the outputs digest
39
40
41 def utc_now() -> str:
42     return datetime.now(UTC).isoformat(timespec="seconds")
43
44
45 def runtime_info() -> dict[str, Any]:
46     packages: dict[str, str] = {}
47     for name in TRACKED_PACKAGES:
48         try:
49             packages[name] = importlib.metadata.version(name)
50         except importlib.metadata.PackageNotFoundError:
51             continue
52     return {
53         "python": sys.version.split()[0],
54         "platform": platform.platform(),
55         "cpu_count": os.cpu_count(),
56         "modal_task_id": os.getenv("MODAL_TASK_ID"),
57         "modal_is_remote": os.getenv("MODAL_IS_REMOTE") == "1" or os.getenv("MODAL_TASK_ID") is not None,
58         "packages": packages,
59         "env": {k: os.environ.get(k) for k in ("OMP_NUM_THREADS", "OPENBLAS_NUM_THREADS", "PYTHONHASHSEED")},
60     }
61
```

```

62
63 @dataclass
64 class Manifest:
65     schema: str = "forge.manifest/1"
66     run_id: str = ""
67     stage: str = ""
68     status: str = "running" # running | completed | failed
69     started_at: str = field(default_factory=utc_now)
70     finished_at: str | None = None
71     duration_s: float | None = None
72     code_ref: dict[str, Any] = field(default_factory=dict)
73     params: dict[str, Any] = field(default_factory=dict)
74     config_digest: str = ""
75     deps: dict[str, str] = field(default_factory=dict)
76     inputs: dict[str, dict[str, Any]] = field(default_factory=dict)
77     outputs: dict[str, dict[str, Any]] = field(default_factory=dict)
78     outputs_digest: str = ""
79     metrics: dict[str, Any] = field(default_factory=dict)
80     runtime: dict[str, Any] = field(default_factory=dict)
81     error: dict[str, Any] | None = None
82
83 @classmethod
84 def start(
85     cls,
86     run_id: str,
87     stage: str,
88     params: dict[str, Any],
89     code_ref: dict[str, Any],
90     config_digest: str,
91     deps: dict[str, str],
92     inputs: dict[str, dict[str, Any]],
93 ) -> Manifest:
94     manifest = cls(
95         run_id=run_id,
96         stage=stage,
97         params=params,
98         code_ref=code_ref,
99         config_digest=config_digest,
100         deps=deps,
101         inputs=inputs,
102     )
103     manifest.runtime = runtime_info()
104     return manifest
105
106 def complete(self, stage_dir: Path, metrics: dict[str, Any]) -> None:
107     self.outputs = hash_tree(stage_dir, ignore=RECORD_FILES)
108     self.outputs_digest = tree_digest(self.outputs)
109     self.metrics = json.loads(json.dumps(metrics, default=str))
110     self.status = "completed"
111     self._finish()
112
113 def fail(self, exc: BaseException) -> None:
114     self.error = {"type": type(exc).__name__, "message": str(exc), "traceback": traceback.format_exc()}
115     self.status = "failed"
116     self._finish()
117
118 def _finish(self) -> None:
119     self.finished_at = utc_now()
120     started = datetime.fromisoformat(self.started_at)
121     self.duration_s = round((datetime.fromisoformat(self.finished_at) - started).total_seconds(), 3)
122
123 def write(self, path: Path) -> None:
124     path.parent.mkdir(parents=True, exist_ok=True)

```

```

125     path.write_text(json.dumps(asdict(self), ensure_ascii=False, indent=2, default=str), encoding="utf-8")
126
127     @classmethod
128     def read(cls, path: Path) -> Manifest:
129         raw = json.loads(path.read_text(encoding="utf-8"))
130         known = {f.name for f in fields(cls)}
131         return cls(**{k: v for k, v in raw.items() if k in known})
132
133     def verify_outputs(self, stage_dir: Path) -> list[str]:
134         """Return a list of mismatches between the manifest and the files on disk (empty == verified)."""
135         problems: list[str] = []
136         current = hash_tree(stage_dir, ignore=RECORD_FILES)
137         for rel, meta in self.outputs.items():
138             if rel not in current:
139                 problems.append(f"missing: {rel}")
140             elif current[rel]["sha256"] != meta["sha256"]:
141                 problems.append(f"sha256 mismatch: {rel}")
142         for rel in current:
143             if rel not in self.outputs:
144                 problems.append(f"unexpected file: {rel}")
145         return problems

```

forge/modal_app.py

```

1  """Modal application — the only compute surface of this repository.
2
3  Local side (tools/cli.py) only submits:
4      modal run forge/modal_app.py --stage ingest --run-id <id> --params '{...}' --size small
5  Inside the container the repository is mounted read-only at /repo and the persistent
6  volume at /vol; stages write exclusively to /vol/runs/<run_id>/<stage>/.
7  """
8
9  from __future__ import annotations
10
11  import contextlib
12  import io
13  import json
14  import os
15  import sys
16  import traceback
17  from pathlib import Path
18
19  import modal
20
21  _HERE = Path(__file__).resolve()
22  REPO = _HERE.parents[1]
23
24
25  def _load_project() -> dict:
26      for candidate in (REPO / "configs" / "project.json", Path("/repo/configs/project.json")):
27          if candidate.exists():
28              return json.loads(candidate.read_text(encoding="utf-8"))
29      raise FileNotFoundError("configs/project.json not found next to forge/modal_app.py")
30
31
32  PROJECT = _load_project()
33  app = modal.App(PROJECT["app"])
34  volume = modal.Volume.from_name(PROJECT["volume"], create_if_missing=True)
35
36  APT_PACKAGES = [
37      "texlive-xetex",

```

```

38     "texlive-lang-chinese",
39     "texlive-latex-base",
40     "texlive-latex-recommended",
41     "texlive-latex-extra",
42     "texlive-science",
43     "texlive-pictures",
44     "texlive-bibtex-extra",
45     "texlive-fonts-recommended",
46     "fonts-noto-cjk",
47     "fonts-noto-core",
48     "fonts-droid-fallback",
49     "fonts-wqy-microhei",
50     "fontconfig",
51     "poppler-utils",
52     "zip",
53     "unzip",
54 ]
55 PIP_PACKAGES = [
56     "numpy==2.2.6",
57     "scipy==1.15.3",
58     "pandas==2.2.3",
59     "pyarrow==20.0.0",
60     "openpyxl==3.1.5",
61     "matplotlib==3.10.3",
62     "scikit-learn==1.6.1",
63     "statsmodels==0.14.4",
64     "ortools==9.12.4544",
65     "highspy==1.10.0",
66     "numba==0.61.2",
67     "cvxpy==1.6.5",
68     "networkx==3.4.2",
69     "pydantic==2.11.4",
70     "pymupdf==1.26.7",
71     "pytest==8.3.5",
72     "pytest-cov==6.1.1",
73     "hypothesis==6.131.9",
74     "ruff==0.11.10",
75     "mypy==1.15.0",
76     "pandas-stubs==2.2.3.250308",
77     "types-openpyxl==3.1.5.20250602",
78 ]
79 SKIP_PARTS = {
80     ".git",
81     "artifacts",
82     "releases",
83     ".forge",
84     "__pycache__",
85     ".mypy_cache",
86     ".ruff_cache",
87     ".pytest_cache",
88     ".hypothesis",
89     ".DS_Store",
90 }
91
92
93 def _ignore(path: Path) -> bool:
94     return bool(SKIP_PARTS & set(path.parts)) or path.suffix in {".pyc", ".pyo"}
95
96
97 image = (
98     modal.Image.debian_slim(python_version="3.11")
99     .apt_install(*APT_PACKAGES)
100     .pip_install(*PIP_PACKAGES)

```

```

101     .env(
102         {
103             "OMP_NUM_THREADS": "1",
104             "OPENBLAS_NUM_THREADS": "1",
105             "MKL_NUM_THREADS": "1",
106             "MPLBACKEND": "Agg",
107             "MPLCONFIGDIR": "/tmp/mpl",
108             "PYTHONHASHSEED": "0",
109             "PYTHONUNBUFFERED": "1",
110             "PYTHONDONTWRITEBYTECODE": "1",
111             "PYTHONPATH": "/repo",
112             "FORGE_REPO": "/repo",
113             "FORGE_VOL": "/vol",
114             "MODAL_IS_REMOTE": "1",
115             "TEXMFVAR": "/tmp/texmf-var",
116             "TEXMFCONFIG": "/tmp/texmf-config",
117             "HOME": "/root",
118         }
119     )
120     .add_local_dir(REPO, "/repo", ignore=_ignore)
121 )
122
123 VOLUME_MOUNT = {"vol": volume}
124
125
126 def _bootstrap() -> None:
127     if "/repo" not in sys.path:
128         sys.path.insert(0, "/repo")
129     os.chdir("/repo")
130     Path("/tmp/mpl").mkdir(exist_ok=True)
131
132
133 def _run(stage: str, run_id: str, params_json: str, force: bool) -> dict:
134     _bootstrap()
135     volume.reload()
136     from forge.runner import execute
137
138     params = json.loads(params_json or "{}")
139     return execute(
140         stage_name=stage,
141         run_id=run_id,
142         params=params,
143         force=force,
144         repo=Path("/repo"),
145         vol=Path("/vol"),
146         commit=volume.commit,
147     )
148
149
150 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=2, memory=4096, timeout=2 * 3600)
151 def run_small(stage: str, run_id: str, params: str, force: bool) -> dict:
152     return _run(stage, run_id, params, force)
153
154
155 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=8, memory=16384, timeout=6 * 3600)
156 def run_medium(stage: str, run_id: str, params: str, force: bool) -> dict:
157     return _run(stage, run_id, params, force)
158
159
160 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=32, memory=65536, timeout=12 * 3600)
161 def run_large(stage: str, run_id: str, params: str, force: bool) -> dict:
162     return _run(stage, run_id, params, force)
163

```



```

164
165 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=4, memory=8192, timeout=3600)
166 def exec_python(code: str, run_id: str) -> str:
167     """Run an ad-hoc Python snippet in the cloud (exploration, status, debugging). Never local."""
168     _bootstrap()
169     volume.reload()
170     buffer = io.StringIO()
171     scope = {
172         "__name__": "__forge_exec__",
173         "RUN_DIR": Path("/vol/runs") / run_id,
174         "VOL": Path("/vol"),
175         "REPO": Path("/repo"),
176         "RUNS": Path("/vol/runs"),
177     }
178     with contextlib.redirect_stdout(buffer), contextlib.redirect_stderr(buffer):
179         try:
180             exec(compile(code, "<forge-exec>", "exec"), scope) # noqa: S102 - explicit cloud REPL
181         except BaseException:
182             traceback.print_exc(file=buffer)
183     volume.commit()
184     return buffer.getvalue()
185
186
187 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=2, memory=4096, timeout=600)
188 def provision() -> dict:
189     """Prove the toolchain: interpreter, packages, TeX engine, fonts, solver availability."""
190     import importlib.metadata
191     import subprocess
192
193     _bootstrap()
194     info: dict = {"modal_task_id": os.getenv("MODAL_TASK_ID"), "cpu_count": os.cpu_count()}
195     info["python"] = sys.version.split()[0]
196     info["packages"] = {p.split("==")[0]: importlib.metadata.version(p.split("==")[0]) for p in PIP_PACKAGES}
197     info["xelatex"] = subprocess.run(["xelatex", "--version"], capture_output=True, text=True).stdout.splitlines()[0]
198     info["bibtex"] = subprocess.run(["bibtex", "--version"], capture_output=True, text=True).stdout.splitlines()[0]
199     info["fc_cjk"] = subprocess.run(["fc-match", "Noto Serif CJK SC"], capture_output=True, text=True).stdout.strip()
200     info["kpse"] = {
201         name: bool(subprocess.run(["kpsewhich", name], capture_output=True, text=True).stdout.strip())
202         for name in (
203             "ctexart.cls",
204             "gbt7714.sty",
205             "siunitx.sty",
206             "algorithm2e.sty",
207             "booktabs.sty",
208             "tikz.sty",
209             "listings.sty",
210             "cleveref.sty",
211             "tcolorbox.sty",
212             "pgfplots.sty",
213         )
214     }
215     from ortools.sat.python import cp_model
216
217     model = cp_model.CpModel()
218     x = model.NewIntVar(0, 10, "x")
219     model.Add(x >= 3)
220     model.Minimize(x)
221     solver = cp_model.CpSolver()
222     info["cpsat"] = {"status": solver.StatusName(solver.Solve(model)), "x": solver.Value(x)}
223     import highspy
224
225     info["highs"] = highspy.Highs().version()
226     from forge.plotting import cjk_font_family

```

```

227
228     info["figure_cjk_font"] = cjk_font_family()
229     return info
230
231
232 @app.local_entrypoint()
233 def main(
234     stage: str = "ingest",
235     run_id: str = "",
236     params: str = "{}",
237     force: bool = False,
238     size: str = "small",
239     spawn: bool = False,
240 ) -> None:
241     """Submit one stage (or ``all`` with params.stages) to the cloud and print its result."""
242     if stage == "provision":
243         print("PROVISION " + json.dumps(provision.remote(), ensure_ascii=False, default=str))
244         return
245     if stage == "exec":
246         payload = json.loads(params or "{}")
247         print(exec_python.remote(payload.get("code", ""), run_id or "adhoc"))
248         return
249     fn = {"small": run_small, "medium": run_medium, "large": run_large}[size]
250     if spawn:
251         call = fn.spawn(stage, run_id, params, force)
252         print(f"FUNCTION_CALL_ID {call.object_id}")
253         return
254     result = fn.remote(stage, run_id, params, force)
255     print("STAGE_RESULT " + json.dumps(result, ensure_ascii=False, default=str))

```

forge/packaging.py

```

1  """Deterministic packaging of support materials (stable member order and timestamps)."""
2
3  from __future__ import annotations
4
5  import zipfile
6  from collections.abc import Iterable
7  from pathlib import Path
8  from typing import Any
9
10 from .hashing import sha256_file
11
12 FIXED_TIME = (2026, 1, 1, 0, 0, 0)
13 SKIP_PARTS = {
14     ".git",
15     "__pycache__",
16     ".DS_Store",
17     ".mypy_cache",
18     ".ruff_cache",
19     ".pytest_cache",
20     ".hypothesis",
21     "artifacts",
22     "releases",
23     ".forge",
24     "build",
25 }
26
27
28 def code_members(repo: Path, globs: Iterable[str]) -> list[tuple[str, Path]]:
29     found: dict[str, Path] = {}

```

```

30     for pattern in globs:
31         for path in sorted(repo.glob(pattern)):
32             if not path.is_file():
33                 continue
34             rel = path.relative_to(repo)
35             if SKIP_PARTS & set(rel.parts) or path.suffix == ".pyc":
36                 continue
37             found[rel.as_posix()] = path
38     return sorted(found.items())
39
40
41 def run_members(run_dir: Path, stages: Iterable[str], *, prefix: str = "runs") -> list[tuple[str, Path]]:
42     members: list[tuple[str, Path]] = []
43     for stage in stages:
44         directory = run_dir / stage
45         if not directory.is_dir():
46             continue
47         for path in sorted(directory.rglob("*")):
48             rel = path.relative_to(directory)
49             if path.is_file() and not (SKIP_PARTS & set(rel.parts)):
50                 members.append((f"{prefix}/{run_dir.name}/{stage}/{rel.as_posix()}", path))
51     return members
52
53
54 def build_zip(members: Iterable[tuple[str, Path]], out: Path, *, comment: str = "") -> dict[str, Any]:
55     ordered = sorted(dict(members).items())
56     out.parent.mkdir(parents=True, exist_ok=True)
57     total = 0
58     with zipfile.ZipFile(out, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9) as zf:
59         if comment:
60             zf.comment = comment.encode("utf-8")
61         for name, source in ordered:
62             info = zipfile.ZipInfo(name, date_time=FIXED_TIME)
63             info.compress_type = zipfile.ZIP_DEFLATED
64             info.external_attr = 0o644 << 16
65             data = source.read_bytes()
66             total += len(data)
67             zf.writestr(info, data)
68     with zipfile.ZipFile(out) as zf:
69         bad = zf.testzip()
70         count = len(zf.namelist())
71     if bad:
72         raise RuntimeError(f"zip member failed CRC check: {bad}")
73     return {
74         "path": str(out),
75         "members": count,
76         "uncompressed_bytes": total,
77         "bytes": out.stat().st_size,
78         "sha256": sha256_file(out),
79     }

```

forge/pdfqa.py

```

1  """PDF inspection for quality gates (PyMuPDF)."""
2
3  from __future__ import annotations
4
5  import re
6  from collections.abc import Iterable
7  from pathlib import Path
8  from typing import Any

```

```

9
10 A4_PT = (595.276, 841.89)
11
12
13 def inspect_pdf(path: Path, *, with_text: bool = True) -> dict[str, Any]:
14     import fitz # PyMuPDF
15
16     doc = fitz.open(path)
17     pages: list[dict[str, Any]] = []
18     texts: list[str] = []
19     fonts: dict[int, dict[str, Any]] = {}
20     unembedded: list[str] = []
21     for index, page in enumerate(doc, start=1):
22         rect = page.rect
23         pages.append(
24             {
25                 "page": index,
26                 "width_pt": round(rect.width, 2),
27                 "height_pt": round(rect.height, 2),
28                 "a4": abs(rect.width - A4_PT[0]) < 3 and abs(rect.height - A4_PT[1]) < 3,
29             }
30         )
31         if with_text:
32             texts.append(page.get_text("text"))
33         for font in page.get_fonts(full=True):
34             xref, ftype, basefont = font[0], font[2], font[3]
35             if xref in fonts:
36                 continue
37             embedded = ftype == "Type3"
38             if not embedded and xref:
39                 try:
40                     embedded = len(doc.extract_font(xref)[3]) > 0
41                 except Exception:
42                     embedded = False
43             fonts[xref] = {"name": basefont, "type": ftype, "embedded": bool(embedded)}
44             if not embedded:
45                 unembedded.append(basefont)
46     return {
47         "pages": len(doc),
48         "page_boxes": pages,
49         "all_a4": all(p["a4"] for p in pages),
50         "fonts": list(fonts.values()),
51         "unembedded_fonts": sorted(set(unembedded)),
52         "texts": texts,
53         "metadata": doc.metadata,
54     }
55
56
57 def scan_strings(texts: Iterable[str], patterns: Iterable[str], *, context: int = 40) -> list[dict[str, Any]]:
58     compiled = [(p, re.compile(p)) for p in patterns]
59     hits: list[dict[str, Any]] = []
60     for page_no, text in enumerate(texts, start=1):
61         for raw, pattern in compiled:
62             for match in pattern.finditer(text):
63                 start, end = max(0, match.start() - context), min(len(text), match.end() + context)
64                 hits.append({"page": page_no, "pattern": raw, "context": text[start:end].replace("\n", " ")})
65     return hits

```

forge/plotting.py

```
1 """House style for paper figures: vector PDF output, CJK-capable fonts, accessible palette."""
2
3 from __future__ import annotations
4
5 import logging
6 import warnings
7 from pathlib import Path
8 from typing import Any
9
10 OKABE_ITO = ["#0072B2", "#D55E00", "#009E73", "#E69F00", "#CC79A7", "#56B4E9", "#F0E442", "#000000"]
11 # (family name as matplotlib registers it, font file). TTC files expose only their first face,
12 # which is why the Noto CJK collection is addressed by its JP face name.
13 _CJK_CANDIDATES = (
14     ("WenQuanYi Micro Hei", "/usr/share/fonts/truetype/wqy/wqy-microhei.ttc"),
15     ("Noto Sans CJK JP", "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"),
16     ("Droid Sans Fallback", "/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf"),
17 )
18 _PROBE_TEXT = "温度 T 水分 C 药材 频段 关键词 abc 0.25"
19 _STATE: dict[str, Any] = {}
20
21
22 class _Catcher(logging.Handler):
23     def __init__(self) -> None:
24         super().__init__()
25         self.records: list[str] = []
26
27     def emit(self, record: logging.LogRecord) -> None:
28         self.records.append(record.getMessage())
29
30
31 def _probe(family: str) -> bool:
32     """True when ``family`` (with DejaVu Sans as Latin fallback) renders the probe text into PDF cleanly."""
33     from matplotlib import pyplot as plt
34
35     catcher = _Catcher()
36     loggers = [
37         logging.getLogger(n) for n in ("matplotlib.font_manager", "matplotlib.backends.backend_pdf", "matplotlib")
38     ]
39     for lg in loggers:
40         lg.addHandler(catcher)
41     try:
42         with warnings.catch_warnings(record=True) as caught:
43             warnings.simplefilter("always")
44             fig, ax = plt.subplots(figsize=(2.5, 1))
45             ax.text(0.05, 0.5, _PROBE_TEXT, fontfamily=[family, "DejaVu Sans"])
46             ax.set_axis_off()
47             probe = Path("/tmp") / f"forge_font_probe_{family.replace(' ', '_')}.pdf"
48             fig.savefig(probe, format="pdf")
49             plt.close(fig)
50             messages = [str(w.message) for w in caught] + catcher.records
51             bad = [m for m in messages if "missing from font" in m or "not found" in m.lower()]
52             return not bad
53     except Exception:
54         return False
55     finally:
56         for lg in loggers:
57             lg.removeHandler(catcher)
58
59
60 def cjk_font_family() -> str:
61     """Pick a CJK font that renders into PDF without missing glyphs (cached per process)."""
```

```

62     if "family" in _STATE:
63         return str(_STATE["family"])
64     import matplotlib
65
66     matplotlib.use("Agg")
67     from matplotlib import font_manager
68
69     chosen = "DejaVu Sans"
70     for family, path in _CJK_CANDIDATES:
71         if not Path(path).exists():
72             continue
73         try:
74             font_manager.fontManager.addfont(path)
75         except Exception:
76             continue
77         if _probe(family):
78             chosen = family
79             break
80     _STATE["family"] = chosen
81     return chosen
82
83
84 def setup(font_size: float = 9.0) -> str:
85     import matplotlib
86
87     matplotlib.use("Agg")
88     from cycler import cycler
89     from matplotlib import pyplot as plt
90
91     family = cjk_font_family()
92     plt.rcParams.update(
93         {
94             "font.family": [family, "DejaVu Sans"],
95             "font.size": font_size,
96             "axes.titlesize": font_size + 1,
97             "axes.labelsize": font_size,
98             "legend.fontsize": font_size - 1,
99             "xtick.labelsize": font_size - 1,
100            "ytick.labelsize": font_size - 1,
101            "axes.prop_cycle": cycler(color=OKABE_ITO),
102            "axes.spines.top": False,
103            "axes.spines.right": False,
104            "axes.grid": True,
105            "grid.alpha": 0.25,
106            "grid.linewidth": 0.6,
107            "lines.linewidth": 1.4,
108            "figure.dpi": 150,
109            "savefig.dpi": 300,
110            "pdf.fonttype": 42,
111            "ps.fonttype": 42,
112            "axes.unicode_minus": False,
113            "figure.constrained_layout.use": True,
114        }
115     )
116     return family
117
118
119 def new_figure(width_in: float = 6.3, height_in: float = 3.4, **kwargs: Any) -> tuple[Any, Any]:
120     from matplotlib import pyplot as plt
121
122     setup()
123     return plt.subplots(figsize=(width_in, height_in), **kwargs)
124

```

```

125
126 def save(fig: Any, pdf_path: Path, *, png_preview: bool = True, dpi: int = 200) -> dict[str, Any]:
127     from matplotlib import pyplot as plt
128
129     pdf_path.parent.mkdir(parents=True, exist_ok=True)
130     fig.savefig(pdf_path, format="pdf")
131     out = {"pdf": str(pdf_path), "bytes": pdf_path.stat().st_size}
132     if png_preview:
133         png = pdf_path.with_suffix(".png")
134         fig.savefig(png, format="png", dpi=dpi)
135         out["png"] = str(png)
136     plt.close(fig)
137     return out

```

forge/runner.py

```

1  """Stage registry and executor. A stage is a function ``fn(ctx) -> metrics``.
2
3  Guarantees: one manifest per (run, stage); dependencies must be completed in the same run
4  (or copied from ``from_run``); a completed stage is skipped unless ``force``; failures are
5  recorded with a traceback and re-raised; the volume is committed after every stage.
6  """
7
8  from __future__ import annotations
9
10 import importlib
11 import json
12 import shutil
13 import time
14 from collections.abc import Callable, Iterable
15 from dataclasses import dataclass
16 from pathlib import Path
17 from typing import Any
18
19 from .context import StageContext
20 from .hashing import hash_tree, tree_digest
21 from .manifest import Manifest
22
23 StageFn = Callable[[StageContext], dict[str, Any] | None]
24 CODE_IGNORE = (
25     ".git",
26     "artifacts",
27     "releases",
28     "__pycache__",
29     ".mypy_cache",
30     ".ruff_cache",
31     ".pytest_cache",
32     ".hypothesis",
33     ".DS_Store",
34     ".forge",
35 )
36
37
38 @dataclass(frozen=True)
39 class StageSpec:
40     name: str
41     fn: StageFn
42     deps: tuple[str, ...]
43     description: str
44
45

```

```

46 STAGES: dict[str, StageSpec] = {}
47
48
49 def stage(name: str, deps: Iterable[str] = (), description: str = "") -> Callable[[StageFn], StageFn]:
50     def decorator(fn: StageFn) -> StageFn:
51         if name in STAGES:
52             raise ValueError(f"duplicate stage registration: {name}")
53         doc = (fn.__doc__ or "").strip().splitlines()
54         STAGES[name] = StageSpec(name, fn, tuple(deps), description or (doc[0] if doc else ""))
55         return fn
56
57     return decorator
58
59
60 def discover(repo: Path) -> None:
61     project = json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
62     importlib.import_module("pipelines.common.stages")
63     importlib.import_module(f"pipelines.{project['package']}.stages")
64
65
66 def list_stages(repo: Path) -> list[dict[str, Any]]:
67     discover(repo)
68     return [{"name": s.name, "deps": list(s.deps), "description": s.description} for s in STAGES.values()]
69
70
71 def _raw_input_prefixes(repo: Path) -> tuple[str, ...]:
72     project = json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
73     return (project["problem_pdf"], project["attachments_dir"].rstrip("/") + "/")
74
75
76 def execute(
77     stage_name: str,
78     run_id: str,
79     params: dict[str, Any],
80     force: bool,
81     repo: Path,
82     vol: Path,
83     commit: Callable[[], None] | None = None,
84 ) -> dict[str, Any]:
85     discover(repo)
86     if stage_name == "all":
87         results = [execute(s, run_id, params, force, repo, vol, commit) for s in params.get("stages", [])]
88         return {
89             "stage": "all",
90             "run_id": run_id,
91             "results": results,
92             "status": "completed" if all(r.get("status") in {"completed", "skipped"} for r in results) else "failed",
93         }
94     if stage_name not in STAGES:
95         raise KeyError(f"unknown stage {stage_name!r}; known stages: {sorted(STAGES)}")
96     spec = STAGES[stage_name]
97     run_dir = vol / "runs" / run_id
98     stage_dir = run_dir / stage_name
99     manifest_path = stage_dir / "manifest.json"
100
101     if manifest_path.exists() and not force:
102         previous = Manifest.read(manifest_path)
103         if previous.status == "completed":
104             return {
105                 "stage": stage_name,
106                 "run_id": run_id,
107                 "status": "skipped",
108                 "reason": "already completed in this run (use --force to rerun)",

```



```

109         "outputs_digest": previous.outputs_digest,
110     }
111
112     deps_digest: dict[str, str] = {}
113     for dep in spec.deps:
114         dep_manifest = run_dir / dep / "manifest.json"
115         if not dep_manifest.exists() and params.get("from_run"):
116             source = vol / "runs" / str(params["from_run"]) / dep
117             if (source / "manifest.json").exists():
118                 shutil.copytree(source, run_dir / dep, dirs_exist_ok=True)
119             if not dep_manifest.exists():
120                 raise RuntimeError(
121                     f"stage '{stage_name}' requires '{dep}', which has not run in run {run_id} "
122                     f"(run it first, or pass from_run=<run_id> to reuse its outputs)"
123                 )
124             dep_state = Manifest.read(dep_manifest)
125             if dep_state.status != "completed":
126                 raise RuntimeError(f"dependency '{dep}' has status {dep_state.status!r} in run {run_id}")
127             deps_digest[dep] = dep_state.outputs_digest
128
129     if stage_dir.exists():
130         shutil.rmtree(stage_dir)
131     ctx = StageContext(run_id, stage_name, params, repo, vol)
132     code_tree = hash_tree(repo, ignore=CODE_IGNORE)
133     prefixes = _raw_input_prefixes(repo)
134     inputs = {k: v for k, v in code_tree.items() if k.startswith(prefixes)}
135     code_ref = {**dict(params.get("code_ref") or {}), "code_digest": tree_digest(code_tree), "files": len(code_tree)}
136     manifest = Manifest.start(run_id, stage_name, params, code_ref, ctx.config.digest, deps_digest, inputs)
137     manifest.write(manifest_path)
138     ctx.log.info("stage.start", deps=list(spec.deps), description=spec.description, size=params.get("size"))
139     t0 = time.monotonic()
140     try:
141         metrics = spec.fn(ctx) or {}
142         ctx.flush_numbers()
143         manifest.complete(stage_dir, metrics)
144         ctx.log.info("stage.completed", duration_s=round(time.monotonic() - t0, 3), metrics=metrics)
145     except BaseException as exc:
146         manifest.fail(exc)
147         ctx.log.error("stage.failed", error=repr(exc))
148         manifest.write(manifest_path)
149         ctx.log.close()
150         if commit:
151             commit()
152         raise
153     manifest.write(manifest_path)
154     ctx.log.close()
155     if commit:
156         commit()
157     return {
158         "stage": stage_name,
159         "run_id": run_id,
160         "status": "completed",
161         "duration_s": manifest.duration_s,
162         "metrics": manifest.metrics,
163         "outputs_digest": manifest.outputs_digest,
164         "modal_task_id": manifest.runtime.get("modal_task_id"),
165     }

```

forge/tex.py

```
1 """XeLaTeX compilation and log analysis (only ever executed inside Modal)."""
2
3 from __future__ import annotations
4
5 import os
6 import re
7 import subprocess
8 from pathlib import Path
9 from typing import Any
10
11 _UNDEF_REF = re.compile(r"LaTeX Warning: Reference `([~']+)` on page \d+ undefined")
12 _UNDEF_CIT = re.compile(r"LaTeX Warning: Citation `([~']+)` on page \d+ undefined")
13 _OVERFULL = re.compile(r"Overfull \\\hbox \\\(([\d.]+)pt too wide\\")
14 _MISSING = re.compile(r"Missing character: There is no (.[?]) in font")
15 _ERROR = re.compile(r"! (.*?)", re.M)
16 _PAGES = re.compile(r"Output written on .*? \\\((\d+) pages?")
17
18
19 def tex_env() -> dict[str, str]:
20     env = dict(os.environ)
21     env.setdefault("TEXMFVAR", "/tmp/texmf-var")
22     env.setdefault("TEXMFCONFIG", "/tmp/texmf-config")
23     env.setdefault("HOME", "/tmp")
24     env["max_print_line"] = "2000"
25     env["error_line"] = "254"
26     env["half_error_line"] = "238"
27     return env
28
29
30 def summarize_log(log: str) -> dict[str, Any]:
31     overfull = [float(x) for x in _OVERFULL.findall(log)]
32     missing = _MISSING.findall(log)
33     pages = _PAGES.search(log)
34     return {
35         "undefined_references": sorted(set(_UNDEF_REF.findall(log))),
36         "undefined_citations": sorted(set(_UNDEF_CIT.findall(log))),
37         "overfull_count": len(overfull),
38         "overfull_max_pt": max(overfull) if overfull else 0.0,
39         "missing_characters": len(missing),
40         "missing_character_samples": sorted(set(missing))[:10],
41         "errors": _ERROR.findall(log)[:10],
42         "pages_from_log": int(pages.group(1)) if pages else None,
43     }
44
45
46 def aux_page(aux: Path, label: str) -> int | None:
47     """Page number recorded for ``\\label{label}`` in the .aux file (hyperref-compatible)."""
48     if not aux.exists():
49         return None
50     pattern = re.compile(r"\\newlabel\{" + re.escape(label) + r"\}\{\{[~']*\}\}\{(\d+)\}")
51     match = pattern.search(aux.read_text(encoding="utf-8", errors="replace"))
52     return int(match.group(1)) if match else None
53
54
55 def compile_tex(
56     workdir: Path,
57     main: str,
58     jobname: str,
59     *,
60     passes: int = 3,
61     bibtex: bool = True,
```

```

62     timeout: int = 1200,
63 ) -> dict[str, Any]:
64     env = tex_env()
65     runs: list[dict[str, Any]] = []
66     log_path = workdir / f"{jobname}.log"
67
68     def xelatex(tag: str) -> None:
69         proc = subprocess.run(
70             ["xelatex", "-interaction=nonstopmode", "-halt-on-error", "-file-line-error", f"-jobname={jobname}", main],
71             cwd=workdir,
72             capture_output=True,
73             text=True,
74             env=env,
75             timeout=timeout,
76             check=False,
77         )
78         runs.append({"pass": tag, "rc": proc.returncode})
79         if proc.returncode != 0:
80             log = log_path.read_text(encoding="utf-8", errors="replace") if log_path.exists() else proc.stdout
81             errors = _ERROR.findall(log)
82             raise RuntimeError(
83                 f"xelatex failed on pass {tag} for {main}: "
84                 + " | ".join(errors[:5])
85                 + "\n--- log tail ---\n"
86                 + log[-4000:]
87             )
88
89     xelatex("1")
90     aux = workdir / f"{jobname}.aux"
91     if bibtex and aux.exists():
92         aux_text = aux.read_text(encoding="utf-8", errors="replace")
93         if "\\citation" in aux_text and "\\bibdata" in aux_text:
94             proc = subprocess.run(
95                 ["bibtex", jobname], cwd=workdir, capture_output=True, text=True, env=env, timeout=timeout, check=False
96             )
97             runs.append({"pass": "bibtex", "rc": proc.returncode, "tail": proc.stdout[-1500:]})
98     for i in range(2, passes + 1):
99         xelatex(str(i))
100     log_text = log_path.read_text(encoding="utf-8", errors="replace")
101     return {
102         "pdf": workdir / f"{jobname}.pdf",
103         "log": log_path,
104         "aux": aux,
105         "runs": runs,
106         "summary": summarize_log(log_text),
107     }
108
109
110 def tex_escape(text: str) -> str:
111     replacements = {
112         "\\": r"\textbackslash{}",
113         "&": r"&",
114         "%": r"%",
115         "$": r"\$",
116         "#": r"#",
117         "_": r"\_",
118         "{": r"\{",
119         "}": r"\}",
120         "~": r"\textasciitilde{}",
121         "^": r"\textasciicircum{}",
122     }
123     return "".join(replacements.get(ch, ch) for ch in text)
124

```

```

125
126 def tex_path(path: str) -> str:
127     """Typeset a file path so that it can break across lines (hyperref/xurl); falls back to \\texttt."""
128     if any(ch in path for ch in "{}%#\\") or not path.isascii():
129         return f"\\texttt{{{tex_escape(path)}}}" # CJK names need the CJK mono font, not the url font
130     return f"\\nolinkurl{{{path}}}"

```

forge/xlsx.py

```

1 """Workbook I/O: reading attachments, filling result templates, checking result contracts."""
2
3 from __future__ import annotations
4
5 import math
6 from collections.abc import Iterable, Sequence
7 from dataclasses import dataclass
8 from pathlib import Path
9 from typing import Any
10
11 import openpyxl
12 import pandas as pd
13
14 BIG_SHEET_CELLS = 300_000
15
16
17 def read_sheets(path: Path, header: int | None = 0) -> dict[str, pd.DataFrame]:
18     with pd.ExcelFile(path) as xf:
19         return {str(name): xf.parse(name, header=header) for name in xf.sheet_names}
20
21
22 def sheet_headers(path: Path) -> dict[str, list[Any]]:
23     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
24     out: dict[str, list[Any]] = {}
25     for ws in wb.worksheets:
26         row = list(next(ws.iter_rows(min_row=1, max_row=1, values_only=True), ()))
27         while row and row[-1] is None:
28             row.pop()
29         out[str(ws.title)] = row
30     wb.close()
31     return out
32
33
34 def _clean(value: Any, decimals: int | None) -> Any:
35     if hasattr(value, "item") and not isinstance(value, str | bytes):
36         try:
37             value = value.item()
38         except (ValueError, TypeError):
39             pass
40     if isinstance(value, float):
41         if math.isnan(value) or math.isinf(value):
42             return None
43         return round(value, decimals) if decimals is not None else value
44     return value
45
46
47 def write_result(
48     template: Path,
49     out: Path,
50     sheets: dict[str, tuple[Sequence[Any], Iterable[Sequence[Any]]]],
51     *,
52     decimals: int | None = 4,

```

```

53     float_format: str = "0.0000",
54 ) -> dict[str, Any]:
55     """Fill a competition result template.
56
57     ``sheets`` maps sheet name -> (header_row, rows). The template's A1 text is always kept;
58     header cells after A1 are taken from ``header_row[1:]``. Sheets not mentioned keep their
59     template content. Floats are rounded to ``decimals`` and formatted with ``float_format``.
60     Very large sheets fall back to openpyxl's write-only mode (no per-cell number format).
61
62     """
63     template_wb = openpyxl.load_workbook(template)
64     names = list(template_wb.sheetnames)
65     unknown = [n for n in sheets if n not in names]
66     if unknown:
67         raise KeyError(f"sheets {unknown} not in template {template.name} (has {names})")
68     a1 = {ws.title: ws.cell(row=1, column=1).value for ws in template_wb.worksheets}
69     materialised = {name: (list(header), [list(r) for r in rows]) for name, (header, rows) in sheets.items()}
70     cells = sum(max(len(h), 1) * (len(rows) + 1) for h, rows in materialised.values())
71     stats: dict[str, Any] = {}
72     out.parent.mkdir(parents=True, exist_ok=True)
73
74     if cells > BIG_SHEET_CELLS:
75         wb = openpyxl.Workbook(write_only=True)
76         for name in names:
77             ws = wb.create_sheet(name)
78             if name in materialised:
79                 header, rows = materialised[name]
80                 full_header = [a1[name], *header[1:]]
81                 ws.append(full_header)
82                 for row in rows:
83                     ws.append([_clean(v, decimals) for v in row])
84                 stats[name] = {"rows": len(rows), "cols": len(full_header)}
85             else:
86                 for tpl_row in template_wb[name].iter_rows(values_only=True):
87                     ws.append(list(tpl_row))
88         wb.save(out)
89     else:
90         wb = template_wb
91         for name, (header, rows) in materialised.items():
92             index = wb.sheetnames.index(name)
93             wb.remove(wb[name])
94             ws = wb.create_sheet(name, index) # fresh sheet: no stale template cells or dimensions
95             full_header = [a1[name], *header[1:]]
96             for j, value in enumerate(full_header, start=1):
97                 ws.cell(row=1, column=j, value=_clean(value, None))
98             for i, row in enumerate(rows, start=2):
99                 for j, value in enumerate(row, start=1):
100                     cleaned = _clean(value, decimals)
101                     cell = ws.cell(row=i, column=j, value=cleaned)
102                     if isinstance(cleaned, float) and j > 1:
103                         cell.number_format = float_format
104             stats[name] = {"rows": len(rows), "cols": len(full_header)}
105         wb.save(out)
106     return {"path": str(out), "sheets": stats, "write_only": cells > BIG_SHEET_CELLS, "cells": cells}
107
108 @dataclass(frozen=True)
109 class SheetContract:
110     name: str
111     min_rows: int = 1 # data rows (excluding the header)
112     max_rows: int | None = None
113     header_len: int | None = None # exact number of header cells, if known
114     numeric_from_col: int | None = None # 0-based; every data cell from this column must be numeric
115     allow_blank: bool = False # blank numeric cells permitted (e.g. moving-boundary tables)

```

```

116     decimals: int | None = 4 # numeric cells must carry at most this many decimals
117     text_columns: tuple[int, ...] = () # 0-based columns that must be non-empty text
118
119
120 @dataclass(frozen=True)
121 class WorkbookContract:
122     file: str
123     template: str
124     sheets: tuple[SheetContract, ...]
125
126
127 def check_workbook(path: Path, contract: WorkbookContract, template: Path) -> dict[str, Any]:
128     errors: list[str] = []
129     if not path.exists():
130         return {"file": contract.file, "ok": False, "errors": ["file missing"], "sheets": {}}
131     try:
132         tpl_headers = sheet_headers(template)
133     except Exception as exc:
134         tpl_headers = {}
135         errors.append(f"template unreadable: {exc!r}")
136     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
137     report: dict[str, Any] = {}
138     names = wb.sheetnames
139     if tpl_headers and names != list(tpl_headers):
140         errors.append(f"sheet names {names} differ from template {list(tpl_headers)}")
141     for sc in contract.sheets:
142         if sc.name not in names:
143             errors.append(f"sheet {sc.name!r} missing")
144             continue
145         ws = wb[sc.name]
146         rows = ws.iter_rows(values_only=True)
147         first = next(rows, None)
148         header: list[Any] = list(first) if first is not None else []
149         while header and header[-1] is None:
150             header.pop()
151         tpl = tpl_headers.get(sc.name)
152         if tpl and header and header[0] != tpl[0]:
153             errors.append(f"{sc.name}: A1 {header[0]!r} differs from template {tpl[0]!r}")
154         if sc.header_len is not None and len(header) != sc.header_len:
155             errors.append(f"{sc.name}: header has {len(header)} cells, expected {sc.header_len}")
156         n = 0
157         bad_numeric = blank = bad_decimals = bad_text = 0
158         for row in rows:
159             values = list(row)
160             if all(v is None or (isinstance(v, str) and not v.strip()) for v in values):
161                 continue
162             n += 1
163             if sc.numeric_from_col is not None:
164                 for v in values[sc.numeric_from_col : len(header) or None]:
165                     if v is None or (isinstance(v, str) and not v.strip()):
166                         blank += 1
167                     elif not isinstance(v, int | float) or isinstance(v, bool):
168                         bad_numeric += 1
169                     elif sc.decimals is not None and isinstance(v, float) and abs(round(v, sc.decimals) - v) > 1e-12:
170                         bad_decimals += 1
171             for j in sc.text_columns:
172                 v = values[j] if j < len(values) else None
173                 if v is None or not str(v).strip():
174                     bad_text += 1
175         if n < sc.min_rows:
176             errors.append(f"{sc.name}: {n} data rows < {sc.min_rows}")
177         if sc.max_rows is not None and n > sc.max_rows:
178             errors.append(f"{sc.name}: {n} data rows > {sc.max_rows}")

```

```

179         if bad_numeric:
180             errors.append(f"{sc.name}: {bad_numeric} non-numeric cells in numeric region")
181         if blank and not sc.allow_blank:
182             errors.append(f"{sc.name}: {blank} blank cells in numeric region")
183         if bad_decimals:
184             errors.append(f"{sc.name}: {bad_decimals} cells with more than {sc.decimals} decimals")
185         if bad_text:
186             errors.append(f"{sc.name}: {bad_text} empty cells in required text columns")
187         report[sc.name] = {
188             "rows": n,
189             "header": [str(h) for h in header[:6]] + (["..."] if len(header) > 6 else []),
190             "blank": blank,
191         }
192     wb.close()
193     return {"file": contract.file, "ok": not errors, "errors": errors, "sheets": report}

```

pipelines/__init__.py

pipelines/common/__init__.py

pipelines/common/stages.py

```

1  """Problem-agnostic stages: ingest, lint, test, paper, qa, package, release."""
2
3  from __future__ import annotations
4
5  import importlib
6  import json
7  import os
8  import re
9  import shlex
10 import shutil
11 import subprocess
12 import xml.etree.ElementTree as ET
13 from datetime import UTC, datetime
14 from pathlib import Path
15 from typing import Any
16
17 from forge import packaging, pdfqa, xlsx
18 from forge.context import StageContext
19 from forge.hashing import hash_tree, sha256_file
20 from forge.runner import stage
21 from forge.tex import aux_page, compile_tex, tex_escape, tex_path
22
23 PLANNED_STAGE_FILES = {
24     "lint": ["ruff_check.txt", "ruff_format.txt", "mypy.txt", "lint_report.json", "manifest.json", "events.jsonl"],
25     "test": ["junit.xml", "coverage.json", "pytest.txt", "test_report.json", "manifest.json", "events.jsonl"],
26     "qa": ["qa_report.json", "manifest.json", "events.jsonl"],
27 }
28 MEMBER_DESCRIPTIONS = (
29     (r"~forge/", "云端运行时（阶段执行、清单、契约、排版、打包）"),
30     (r"~pipelines/common/", "通用阶段：数据接入、门禁、论文、质量检查、打包发布"),
31     (r"~pipelines/", "本题建模与求解程序"),
32     (r"~tools/", "本地控制面：提交云端任务、下载、校验散列"),

```

```

33     (r"^tests/", "单元 / 性质 / 集成测试"),
34     (r"^configs/", "运行配置与项目元数据"),
35     (r"^manuscript/", "论文 LaTeX 源码"),
36     (r"^docs/", "工程规范、建模标准、决策记录"),
37     (r"^ingest/", "原始数据接入结果 (散列、题目文本、规范化数据)"),
38     (r"^validate/", "数据契约校验报告与数据画像"),
39     (r"^results/", "结果工作簿与求解输出"),
40     (r"^figures/", "论文图 (矢量 PDF 与预览 PNG)"),
41     (r"^tables/", "论文表 (LaTeX 片段与 CSV)"),
42     (r"^lint/", "静态检查报告"),
43     (r"^test/", "测试报告与覆盖率"),
44     (r"^qa/", "论文与结果文件质量门禁报告"),
45     (r"manifest\.json$", "阶段清单 (输入输出散列、环境、参数)"),
46     (r"events\.jsonl$", "结构化运行日志"),
47     (r"^AI工具使用详情\.pdf$", "AI 工具使用详情"),
48     (r"^AI_USAGE\.md$", "AI 工具使用记录 (源文件)"),
49     (r"^Makefile$", "本地快捷命令 (只提交云端任务)"),
50     (r"^pyproject\.toml$", "项目元数据与静态检查配置"),
51     (r"^README\.md$", "仓库说明与复现入口"),
52     (r"^CHANGELOG\.md$", "变更记录"),
53     (r"^UNLICENSE$", "公有领域声明 (Unlicense)"),
54     (r"^REPRODUCE\.md$", "复现说明"),
55     (r"^MANIFEST\.json$", "支撑材料成员清单与 SHA-256"),
56 )
57
58
59 def _describe(name: str) -> str:
60     for pattern, text in MEMBER_DESCRIPTIONS:
61         if re.search(pattern, name):
62             return text
63     return ""
64
65
66 def _to_parquet(df: Any, target: Path) -> None:
67     frame = df.copy()
68     frame.columns = [str(c).strip() for c in frame.columns]
69     try:
70         frame.to_parquet(target, index=False)
71     except Exception:
72         for col in frame.columns:
73             if frame[col].dtype == object:
74                 frame[col] = frame[col].map(
75                     lambda v: None if v is None or (isinstance(v, float) and v != v) else str(v)
76                 )
77         frame.to_parquet(target, index=False)
78
79
80 @stage("ingest", description="Hash raw inputs, extract the problem text, convert every attachment sheet to parquet")
81 def ingest(ctx: StageContext) -> dict[str, Any]:
82     import fitz
83
84     document = fitz.open(ctx.problem_pdf)
85     text = "\n\n".join(page.get_text("text", sort=True) for page in document)
86     ctx.out("problem_text.txt").write_text(text, encoding="utf-8")
87     inventory: dict[str, Any] = {
88         "problem_pdf": {
89             "path": ctx.project["problem_pdf"],
90             "sha256": sha256_file(ctx.problem_pdf),
91             "pages": len(document),
92         },
93         "files": [],
94     }
95     for path in sorted(ctx.attachments.rglob("*.xlsx")):

```



```

96     if path.name.startswith("-$"):
97         continue
98     rel = path.relative_to(ctx.attachments).as_posix()
99     is_template = path.parent == ctx.templates or ctx.templates in path.parents
100     entry: dict[str, Any] = {
101         "path": rel,
102         "sha256": sha256_file(path),
103         "bytes": path.stat().st_size,
104         "template": is_template,
105         "sheets": [],
106     }
107     for name, df in xlsx.read_sheets(path).items():
108         sheet: dict[str, Any] = {"name": name, "rows": len(df), "columns": [str(c) for c in df.columns]}
109         if not is_template:
110             safe = re.sub(r"[^\w.-]+", "_", f"{path.stem}_{name}")
111             target = ctx.out("data", f"{safe}.parquet")
112             _to_parquet(df, target)
113             sheet["parquet"] = target.relative_to(ctx.stage_dir).as_posix()
114             sheet["dtypes"] = {str(c): str(t) for c, t in df.dtypes.items()}
115             sheet["head"] = df.head(5).astype(str).values.tolist()
116             entry["sheets"].append(sheet)
117         inventory["files"].append(entry)
118         ctx.log.info("ingest.file", path=rel, template=is_template, sheets=[s["name"] for s in entry["sheets"]])
119     ctx.write_json("inventory.json", inventory)
120     return {"files": len(inventory["files"]), "problem_chars": len(text)}
121
122
123 @stage("lint", description="Static gates: ruff check, ruff format --check, mypy")
124 def lint(ctx: StageContext) -> dict[str, Any]:
125     returncodes: dict[str, int] = {}
126
127     def run(name: str, cmd: list[str]) -> None:
128         proc = subprocess.run(cmd, cwd=str(ctx.repo), capture_output=True, text=True, check=False)
129         ctx.out(f"{name}.txt").write_text(proc.stdout + ("\n" + proc.stderr if proc.stderr else ""), encoding="utf-8")
130         returncodes[name] = proc.returncode
131         ctx.log.info("lint.tool", tool=name, rc=proc.returncode, tail=(proc.stdout + proc.stderr)[-400:])
132
133     run("ruff_check", ["ruff", "check", "--no-cache", "--output-format", "concise", "."])
134     run("ruff_format", ["ruff", "format", "--check", "--no-cache", "."])
135     run("mypy", ["mypy", "--cache-dir", "/tmp/mypy", *ctx.cfg("gates.mypy_paths", ["forge"])]))
136     required = list(ctx.cfg("gates.lint_required", ["ruff_check", "ruff_format", "mypy"]))
137     passed = all(returncodes.get(name, 1) == 0 for name in required)
138     ctx.write_json("lint_report.json", {"passed": passed, "required": required, "returncodes": returncodes})
139     if not passed:
140         raise RuntimeError(f"lint gate failed: {returncodes}")
141     return {"passed": passed, **returncodes}
142
143
144 @stage("test", description="pytest (unit, property, integration) with JUnit and coverage reports")
145 def test(ctx: StageContext) -> dict[str, Any]:
146     env = {
147         **os.environ,
148         "COVERAGE_FILE": "/tmp/.coverage",
149         "HYPOTHESIS_STORAGE_DIRECTORY": "/tmp/hypothesis",
150         "FORGE_RUN_DIR": str(ctx.run_dir),
151     }
152     cmd = [
153         "python",
154         "-m",
155         "pytest",
156         "tests",
157         "-q",
158         "-p",

```

```

159     "no:cacheprovider",
160     "--rootdir",
161     str(ctx.repo),
162     f"--junitxml={ctx.out('junit.xml')}",
163     "--cov=forge",
164     "--cov=pipelines",
165     f"--cov-report=json:{ctx.out('coverage.json')}",
166     "--cov-report=term",
167 ]
168 extra = ctx.param("pytest_args")
169 if extra:
170     cmd += shlex.split(str(extra))
171 proc = subprocess.run(cmd, cwd=str(ctx.repo), capture_output=True, text=True, env=env, check=False)
172 ctx.out("pytest.txt").write_text(proc.stdout + "\n" + proc.stderr, encoding="utf-8")
173 print(proc.stdout[-5000:], flush=True)
174 summary: dict[str, Any] = {"returncode": proc.returncode}
175 junit = ctx.stage_dir / "junit.xml"
176 if junit.exists():
177     root = ET.parse(junit).getroot()
178     suite = root if root.tag == "testsuite" else root.find("testsuite")
179     if suite is not None:
180         summary.update({k: int(suite.get(k, 0)) for k in ("tests", "failures", "errors", "skipped")})
181 coverage = ctx.stage_dir / "coverage.json"
182 if coverage.exists():
183     summary["coverage_percent"] = round(json.loads(coverage.read_text())["totals"]["percent_covered"], 2)
184 ctx.write_json("test_report.json", summary)
185 if proc.returncode != 0:
186     raise RuntimeError(f"tests failed: {summary}\n{proc.stdout[-2500:]}")
187 return summary
188
189
190 def _support_members(ctx: StageContext, *, planned: bool) -> list[tuple[str, Path]]:
191     members = packaging.code_members(ctx.repo, ctx.cfg("package.code_globs", []))
192     for name in ctx.cfg("package.stages", []):
193         directory = ctx.run_dir / name
194         if (directory / "manifest.json").exists():
195             members += packaging.run_members(ctx.run_dir, [name])
196         elif planned and name in PLANNED_STAGE_FILES:
197             members += [(f"runs/{ctx.run_id}/{name}/{f}", directory / f) for f in PLANNED_STAGE_FILES[name]]
198     members.append(("AI工具使用详情.pdf", ctx.run_dir / "paper" / "AI工具使用详情.pdf"))
199     members.append(("REPRODUCE.md", ctx.run_dir / "package" / "REPRODUCE.md"))
200     members.append(("MANIFEST.json", ctx.run_dir / "package" / "MANIFEST.json"))
201     return sorted(dict(members).items())
202
203
204 def _load_numbers(run_dir: Path) -> tuple[dict[str, str], list[str]]:
205     numbers: dict[str, str] = {}
206     conflicts: list[str] = []
207     for path in sorted(run_dir.glob("*/numbers.json")):
208         for key, value in json.loads(path.read_text(encoding="utf-8")).items():
209             if key in numbers and numbers[key] != value:
210                 conflicts.append(f"{key}: {numbers[key]!r} vs {value!r} ({path.parent.name})")
211             numbers[key] = value
212     return numbers, conflicts
213
214
215 def _stage_rows(ctx: StageContext) -> list[dict[str, Any]]:
216     rows = []
217     for manifest in sorted(ctx.run_dir.glob("*/manifest.json")):
218         data = json.loads(manifest.read_text(encoding="utf-8"))
219         rows.append(
220             {
221                 "stage": data["stage"],

```

```

222         "status": data["status"],
223         "duration_s": data.get("duration_s"),
224         "outputs_digest": data.get("outputs_digest", ""),
225         "modal_task_id": data.get("runtime", {}).get("modal_task_id"),
226     }
227 )
228 return rows
229
230
231 @stage(
232     "paper",
233     description="Assemble generated inputs (numbers, tables, figures, code, support list) and compile with XeLaTeX",
234 )
235 def paper(ctx: StageContext) -> dict[str, Any]:
236     sources = list(ctx.cfg("paper.sources", []))
237     for source in sources:
238         ctx.dep(source)
239     build = Path("/tmp/build")
240     shutil.rmtree(build, ignore_errors=True)
241     shutil.copytree(
242         ctx.repo / "manuscript", build, ignore=shutil.ignore_patterns("generated", "*.aux", "*.log", "*.pdf")
243     )
244     generated = build / "generated"
245     (generated / "tables").mkdir(parents=True, exist_ok=True)
246     (build / "figures").mkdir(exist_ok=True)
247
248     numbers, conflicts = _load_numbers(ctx.run_dir)
249     for conflict in conflicts:
250         ctx.log.warn("paper.number_conflict", detail=conflict)
251     lines = ["% generated by forge — do not edit"] + [
252         f"\newcommand{{\val{key}}}{\tex_escape(value)}" for key, value in sorted(numbers.items())
253     ]
254     (generated / "numbers.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
255
256     n_tables = n_figures = 0
257     for source in sources:
258         directory = ctx.run_dir / source
259         for table in sorted((directory / "tables").glob("*.tex")) if (directory / "tables").is_dir() else []:
260             shutil.copy2(table, generated / "tables" / table.name)
261             n_tables += 1
262         for figure in sorted((directory / "figures").glob("*")) if (directory / "figures").is_dir() else []:
263             if figure.suffix.lower() in {".pdf", ".png"}:
264                 shutil.copy2(figure, build / "figures" / figure.name)
265                 n_figures += 1
266
267     code_ref = ctx.params.get("code_ref") or {}
268     meta = [
269         f"\newcommand{{\RunId}}{\tex_escape(ctx.run_id)}",
270         f"\newcommand{{\CodeSha}}{\tex_escape(str(code_ref.get('git_sha', '')))}",
271         f"\newcommand{{\BuildDate}}{\datetime.now(UTC).strftime('%Y-%m-%d')}",
272     ]
273     (generated / "meta.tex").write_text("\n".join(meta) + "\n", encoding="utf-8")
274
275     files = packaging.code_members(ctx.repo, ctx.cfg("paper.code_globs", []))
276     listing = ["% generated by forge — complete source listing"]
277     for rel, src in files:
278         target = build / "code" / rel
279         target.parent.mkdir(parents=True, exist_ok=True)
280         shutil.copy2(src, target)
281         language = {".py": "Python", ".tex": "[LaTeX]TeX"}.get(src.suffix, "{")
282         listing.append(f"\needspace{{6\baselineskip}}\subsection*{{\tex_path(rel) }}"
283         listing.append(f"\lstinputlisting[language={language}]{code/{rel}}")
284     (generated / "code_listing.tex").write_text("\n".join(listing) + "\n", encoding="utf-8")

```

```

285
286 members = _support_members(ctx, planned=True)
287 rows = [
288     "% generated by forge — support material file list",
289     "\\begin{longtable}{@{}p{0.64\\linewidth}p{0.32\\linewidth}@{}}",
290     "\\toprule 文件 & 说明 \\midrule \\endhead \\bottomrule \\endfoot",
291 ]
292 rows += [f"{tex_path(name)} & {tex_escape(_describe(name))} \\midrule" for name, _ in members]
293 rows.append("\\end{longtable}")
294 (generated / "support_files.tex").write_text("\n".join(rows) + "\n", encoding="utf-8")
295 ctx.write_json("support_members.json", [name for name, _ in members])
296
297 stage_rows = _stage_rows(ctx)
298 repro = [
299     "% generated by forge — reproducibility record",
300     "\\begin{longtable}{@{}l{1}l{1}@{}}",
301     "\\toprule 阶段 & 状态 & 用时/s & 输出摘要 (SHA-256 前 12 位) \\midrule \\endhead \\bottomrule \\endfoot",
302 ]
303 repro += [
304     f"\\texttt{{{tex_escape(r['stage'])}}} & {r['status']} & {r['duration_s']} or 0:.1f & "
305     f"\\texttt{{{r['outputs_digest'][:12]}}} \\midrule"
306     for r in stage_rows
307 ]
308 repro.append("\\end{longtable}")
309 (generated / "repro_table.tex").write_text("\n".join(repro) + "\n", encoding="utf-8")
310
311 result = compile_tex(build, str(ctx.cfg("paper.main", "main.tex")), "main", passes=3, bibtex=True)
312 shutil.copy2(result["pdf"], ctx.out("main.pdf"))
313 shutil.copy2(result["log"], ctx.out("main.log"))
314 if result["aux"].exists():
315     shutil.copy2(result["aux"], ctx.out("main.aux"))
316 body_end = aux_page(result["aux"], "page:body-end")
317 abstract_end = aux_page(result["aux"], "page:abstract-end")
318 ai_summary = None
319 if (build / "ai_usage.tex").exists():
320     ai = compile_tex(build, "ai_usage.tex", "ai_usage", passes=2, bibtex=False)
321     shutil.copy2(ai["pdf"], ctx.out("AI工具使用详情.pdf"))
322     ai_summary = ai["summary"]
323 info = pdfqa.inspect_pdf(ctx.out("main.pdf"), with_text=False)
324 body_pages = (body_end - abstract_end) if body_end and abstract_end else None
325 report = {
326     "pages": info["pages"],
327     "all_a4": info["all_a4"],
328     "abstract_end_page": abstract_end,
329     "body_end_page": body_end,
330     "body_pages": body_pages,
331     "log": result["summary"],
332     "unembedded_fonts": info["unembedded_fonts"],
333     "fonts": info["fonts"],
334     "numbers": len(numbers),
335     "number_conflicts": conflicts,
336     "tables": n_tables,
337     "figures": n_figures,
338     "code_files": len(files),
339     "support_members": len(members),
340     "ai_usage_log": ai_summary,
341     "stages": stage_rows,
342 }
343 ctx.write_json("paper_report.json", report)
344 summary = result["summary"]
345 return {
346     "pages": info["pages"],
347     "body_pages": body_pages,

```

```

348     "undefined_refs": len(summary["undefined_references"]) + len(summary["undefined_citations"]),
349     "overfull_max_pt": summary["overfull_max_pt"],
350     "missing_chars": summary["missing_characters"],
351     "figures": n_figures,
352     "tables": n_tables,
353 }
354
355
356 @stage("qa", deps=("paper",), description="Quality gates over the compiled paper and the result workbooks")
357 def qa(ctx: StageContext) -> dict[str, Any]:
358     for required in ctx.cfg("qa.required_stages", ["lint", "test"]):
359         ctx.dep(required)
360     paper_dir = ctx.dep("paper")
361     report = json.loads((paper_dir / "paper_report.json").read_text(encoding="utf-8"))
362     checks: list[dict[str, Any]] = []
363
364     def check(name: str, ok: bool, detail: Any = "") -> None:
365         checks.append({"name": name, "ok": bool(ok), "detail": detail})
366         ctx.log.info("qa.check", name=name, ok=bool(ok), detail=detail)
367
368     max_body = int(ctx.cfg("paper.max_body_pages", 30))
369     check("abstract_single_page", report["abstract_end_page"] == 1, report["abstract_end_page"])
370     check(
371         "body_pages_within_limit",
372         report["body_pages"] is not None and report["body_pages"] <= max_body,
373         f"{report['body_pages']} <= {max_body}",
374     )
375     log = report["log"]
376     check(
377         "no_undefined_references",
378         not log["undefined_references"] and not log["undefined_citations"],
379         {"refs": log["undefined_references"][:10], "cites": log["undefined_citations"][:10]},
380     )
381     check("no_tex_errors", not log["errors"], log["errors"][:5])
382     check("fonts_embedded", not report["unembedded_fonts"], report["unembedded_fonts"])
383     check("a4_pages", report["all_a4"])
384     check(
385         "overfull_within_limit",
386         log["overfull_max_pt"] <= float(ctx.cfg("paper.max_overfull_pt", 20.0)),
387         f"max {log['overfull_max_pt']} pt, count {log['overfull_count']}",
388     )
389     check("no_missing_characters", log["missing_characters"] == 0, log["missing_character_samples"])
390     check("no_number_conflicts", not report["number_conflicts"], report["number_conflicts"][:5])
391     check("ai_usage_pdf_present", (paper_dir / "AI工具使用详情.pdf").exists())
392
393     info = pdfqa.inspect_pdf(paper_dir / "main.pdf", with_text=True)
394     deny_hits = pdfqa.scan_strings(info["texts"], ctx.cfg("qa.identity_denylist", []))
395     watch_hits = pdfqa.scan_strings(info["texts"], ctx.cfg("qa.identity_watchlist", []))
396     check("identity_denylist_clean", not deny_hits, deny_hits[:5])
397     check("paper_size_under_20mb", (paper_dir / "main.pdf").stat().st_size <= 20 * 1024 * 1024)
398
399     results_reports: list[dict[str, Any]] = []
400     results_stage = str(ctx.cfg("qa.results_stage", "results"))
401     require_results = bool(ctx.param("require_results", ctx.cfg("qa.require_results", True)))
402     if ctx.has_stage(results_stage):
403         contracts = importlib.import_module(f"pipelines.{ctx.project['package']}.contracts")
404         results_dir = ctx.run_dir / results_stage
405         for contract in contracts.RESULT_CONTRACTS:
406             rep = xlsx.check_workbook(results_dir / contract.file, contract, ctx.templates / contract.template)
407             results_reports.append(rep)
408             check(f"result:{contract.file}", rep["ok"], rep["errors"][:5])
409     else:
410         check("results_stage_present", not require_results, f"stage '{results_stage}' not completed in this run")

```

```

411
412 passed = all(c["ok"] for c in checks)
413 ctx.write_json(
414     "qa_report.json",
415     {
416         "passed": passed,
417         "checks": checks,
418         "identity_watchlist_hits": watch_hits,
419         "results": results_reports,
420         "paper": {k: v for k, v in report.items() if k != "fonts"},
421     },
422 )
423 if not passed:
424     raise RuntimeError("QA gate failed: " + ", ".join(c["name"] for c in checks if not c["ok"]))
425 return {"passed": passed, "checks": len(checks), "watchlist_hits": len(watch_hits)}
426
427
428 def _reproduce_md(ctx: StageContext) -> str:
429     rows = _stage_rows(ctx)
430     code_ref = ctx.params.get("code_ref") or {}
431     manifest = json.loads((ctx.run_dir / "ingest" / "manifest.json").read_text(encoding="utf-8"))
432     packages = manifest.get("runtime", {}).get("packages", {})
433     lines = [
434         f"# 复现说明 (run `{ctx.run_id}`)",
435         "",
436         "本目录内所有结果均由 Modal 云端按下列阶段生成；本地只提交任务、下载和核对 SHA-256。",
437         "",
438         f"- 代码版本: git `{code_ref.get('git_sha', '')}` (分支 `{code_ref.get('branch', '')}`)",
439         f"- 运行环境: Python {manifest.get('runtime', {}).get('python', '')}, "
440         f"{manifest.get('runtime', {}).get('platform', '')}",
441         f"- 依赖版本: " + ", ".join(f"{k}=={v}" for k, v in sorted(packages.items())),
442         "",
443         "## 阶段",
444         "",
445         "| 阶段 | 状态 | 用时 (s) | 输出摘要 | Modal 任务 |",
446         "|---|---|---:|---|---|",
447     ]
448     lines += [
449         f"| {r['stage']} | {r['status']} | {r['duration_s'] or 0:.1f} | {r['outputs_digest'][:16]} | "
450         f"{r['modal_task_id'] or ''} | "
451         for r in rows
452     ]
453     lines += [
454         "",
455         "## 复现命令",
456         "",
457         "`bash`,
458         "python3 tools/cli.py provision",
459         "python3 tools/cli.py run ingest,validate --new-run",
460         "python3 tools/cli.py run <科学阶段...> --size medium",
461         "python3 tools/cli.py run lint,test,paper,qa,package,release",
462         "python3 tools/cli.py release --version <tag>",
463         "````,
464         "",
465         "每个阶段目录下的 `manifest.json` 记录输入、输出散列、参数与环境；`events.jsonl` 为结构化日志。",
466         "",
467     ]
468     return "\n".join(lines)
469
470
471 @stage("package", deps=("qa",), description="Build the deterministic support-materials zip and its member manifest")
472 def package(ctx: StageContext) -> dict[str, Any]:
473     planned = set(json.loads((ctx.dep("paper") / "support_members.json").read_text(encoding="utf-8")))

```

```

474 ctx.out("REPRODUCE.md").write_text(_reproduce_md(ctx), encoding="utf-8")
475 members = [(name, path) for name, path in _support_members(ctx, planned=False) if name != "MANIFEST.json"]
476 missing = [name for name, path in members if not path.exists()]
477 if missing:
478     raise RuntimeError(f"support members missing on disk: {missing[:10]}")
479 actual = {name for name, _ in members} | {"MANIFEST.json"}
480 drift = sorted(actual ^ planned)
481 if drift and not ctx.param("allow_member_drift"):
482     raise RuntimeError(f"support material members differ from the list printed in the paper: {drift[:20]}")
483 manifest = {
484     "run_id": ctx.run_id,
485     "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
486     "members": [
487         {"path": name, "sha256": sha256_file(path), "bytes": path.stat().st_size} for name, path in members
488     ],
489 }
490 manifest_path = ctx.out("MANIFEST.json")
491 manifest_path.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
492 members.append(("MANIFEST.json", manifest_path))
493 zip_info = packaging.build_zip(members, ctx.out("支撑材料.zip"), comment=f"forge run {ctx.run_id}")
494 limit = int(ctx.cfg("package.max_bytes", 20 * 1024 * 1024))
495 ctx.write_json("package_report.json", {**zip_info, "limit_bytes": limit, "drift": drift})
496 if zip_info["bytes"] > limit:
497     raise RuntimeError(f"support zip {zip_info['bytes']} bytes exceeds limit {limit}")
498 return {"members": zip_info["members"], "bytes": zip_info["bytes"], "drift": len(drift)}
499
500
501 @stage(
502     "release",
503     deps=("package",),
504     description="Assemble deliverables (paper, result workbooks, support zip) with SHA-256 manifest",
505 )
506 def release(ctx: StageContext) -> dict[str, Any]:
507     qa_report = json.loads((ctx.dep("qa") / "qa_report.json").read_text(encoding="utf-8"))
508     if not qa_report["passed"] and not ctx.param("allow_failed_qa"):
509         raise RuntimeError("release refused: QA gate did not pass")
510     paper_dir = ctx.dep("paper")
511     files: dict[str, Path] = {str(ctx.cfg("release.paper_name", "论文.pdf")): paper_dir / "main.pdf"}
512     results_stage = str(ctx.cfg("qa.results_stage", "results"))
513     if ctx.has_stage(results_stage):
514         for name in ctx.project.get("result_files", []):
515             files[name] = ctx.run_dir / results_stage / name
516     elif not ctx.param("allow_missing_results"):
517         raise RuntimeError("release refused: results stage missing")
518     files["支撑材料.zip"] = ctx.dep("package") / "支撑材料.zip"
519     files["AI工具使用详情.pdf"] = paper_dir / "AI工具使用详情.pdf"
520     manifest: dict[str, Any] = {
521         "run_id": ctx.run_id,
522         "code_ref": ctx.params.get("code_ref"),
523         "files": {},
524         "qa_passed": qa_report["passed"],
525         "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
526     }
527     for name, source in files.items():
528         if not source.exists():
529             raise RuntimeError(f"release file missing: {name} ({source})")
530         target = ctx.out(name)
531         shutil.copy2(source, target)
532         manifest["files"][name] = {"sha256": sha256_file(target), "bytes": target.stat().st_size}
533     notes = [
534         f"# 发布说明 — run {ctx.run_id}",
535         "",
536         f"- 代码: `{(ctx.params.get('code_ref') or {}).get('git_sha', '')}`",
537     ]

```

```

537         f"- QA: {'通过' if qa_report['passed'] else '未通过 (强制发布) '}",
538         "",
539         "| 文件 | 字节 | SHA-256 |",
540         "|---|---:|---|",
541     ]
542     notes += [f"| {n} | {m['bytes']} | {m['sha256']}~` |" for n, m in manifest["files"].items()]
543     ctx.out("RELEASE_NOTES.md").write_text("\n".join(notes) + "\n", encoding="utf-8")
544     ctx.write_json("release_manifest.json", manifest)
545     return {
546         "files": len(files),
547         "paper_bytes": manifest["files"][str(ctx.cfg("release.paper_name", "论文.pdf"))]["bytes"],
548     }
549
550
551 FMT_IGNORE = (".git", "artifacts", "releases", "__pycache__", ".forge", ".DS_Store", ".mypy_cache", ".ruff_cache")
552
553
554 @stage("fmt", description="ruff --fix and ruff format in the cloud; changed files are exported for a local copy-back")
555 def fmt(ctx: StageContext) -> dict[str, Any]:
556     work = Path("/tmp/fmt")
557     shutil.rmtree(work, ignore_errors=True)
558     shutil.copytree(ctx.repo, work, ignore=shutil.ignore_patterns(*FMT_IGNORE, "*.pyc"))
559     before = hash_tree(work, ignore=FMT_IGNORE)
560     fix = subprocess.run(
561         ["ruff", "check", "--fix", "--no-cache", "--output-format", "concise", "."],
562         cwd=work,
563         capture_output=True,
564         text=True,
565         check=False,
566     )
567     form = subprocess.run(["ruff", "format", "--no-cache", "."], cwd=work, capture_output=True, text=True, check=False)
568     after = hash_tree(work, ignore=FMT_IGNORE)
569     changed = sorted(rel for rel in after if rel not in before or after[rel]["sha256"] != before[rel]["sha256"])
570     for rel in changed:
571         shutil.copy2(work / rel, ctx.out("files", rel))
572     ctx.out("ruff_fix.txt").write_text(fix.stdout + fix.stderr, encoding="utf-8")
573     ctx.out("ruff_format.txt").write_text(form.stdout + form.stderr, encoding="utf-8")
574     ctx.write_json("fmt_report.json", {"changed": changed, "remaining": fix.stdout.strip().splitlines()[-1:]})
575     ctx.log.info("fmt.done", changed=len(changed), remaining=fix.stdout.strip().splitlines()[-1:])
576     return {"changed": len(changed)}

```

pipelines/common/validation.py

```

1  """Shared implementation of the ``validate`` stage (input data contracts + profiling)."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import pandas as pd
8
9  from forge.context import StageContext
10 from forge.contracts import FrameContract, normalise_columns, validate_frame
11
12
13 def run_validation(ctx: StageContext, contracts: dict[str, FrameContract]) -> dict[str, Any]:
14     """Validate every ingested parquet named in ``contracts`` (key = parquet stem)."""
15     data_dir = ctx.dep("ingest") / "data"
16     reports: list[dict[str, Any]] = []
17     profile: dict[str, Any] = {}
18     for stem, contract in contracts.items():

```



```

19     path = data_dir / f"{stem}.parquet"
20     if not path.exists():
21         reports.append({"contract": contract.name, "ok": False, "errors": [f"parquet missing: {stem}"], "rows": 0})
22         continue
23     df = normalise_columns(pd.read_parquet(path))
24     report = validate_frame(df, contract)
25     report["parquet"] = path.name
26     reports.append(report)
27     desc = df.describe(include="all").astype(str).to_dict()
28     profile[stem] = {
29         "rows": len(df),
30         "columns": list(df.columns),
31         "describe": desc,
32         "nulls": {c: int(df[c].isna().sum()) for c in df.columns},
33     }
34     ctx.log.info(
35         "validate.frame", contract=contract.name, ok=report["ok"], rows=report["rows"], errors=report["errors"][:5]
36     )
37     ok = all(r["ok"] for r in reports)
38     ctx.write_json("validation_report.json", {"ok": ok, "frames": reports})
39     ctx.write_json("data_profile.json", profile)
40     if not ok:
41         failed = [r["contract"] for r in reports if not r["ok"]]
42         raise RuntimeError(f"input contracts violated: {failed}")
43     return {"ok": ok, "frames": len(reports)}

```

pipelines/e/__init__.py

pipelines/e/allocate.py

```

1  """Problem 3/4 budget allocation: separable concave program, exact water-filling solver, independent verifier."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import numpy as np
8
9
10 def water_filling(a: np.ndarray, gamma: float, caps: np.ndarray, budget: float, tol: float = 1e-12) -> np.ndarray:
11     """maximise  $\sum_i a_i x_i^\gamma$  s.t.  $\sum_i x_i \leq \text{budget}$ ,  $0 \leq x_i \leq \text{caps}_i$  ( $0 < \gamma < 1$ ,  $a_i > 0$ ).
12
13     KKT:  $\gamma a_i x_i^{\gamma-1} = \lambda$  on interior coordinates,  $x_i = \text{cap}_i$  where the marginal at the
14     cap still exceeds  $\lambda$ . The map  $\lambda \rightarrow \sum_i \min(\text{cap}_i, (\gamma a_i / \lambda)^{1/(1-\gamma)})$  is
15     strictly decreasing, so  $\lambda$  is found by bisection; the solution is the unique global optimum.
16     """
17     a = np.asarray(a, dtype=float)
18     caps = np.asarray(caps, dtype=float)
19     if len(a) == 0 or budget <= 0:
20         return np.zeros_like(a)
21     return _bounded_water_filling(a, gamma, np.zeros_like(a), caps, budget, tol)
22
23
24 def _bounded_water_filling(
25     a: np.ndarray, gamma: float, lower: np.ndarray, caps: np.ndarray, budget: float, tol: float = 1e-12
26 ) -> np.ndarray:
27     """Exact solution of  $\max \sum a_i x_i^\gamma$  s.t.  $\sum x_i = \text{budget}$ ,  $\text{lower}_i \leq x_i \leq \text{caps}_i$  (fixed set)."""

```

```

28     expo = 1.0 / (1.0 - gamma)
29     if caps.sum() <= budget:
30         return caps.copy()
31     if lower.sum() >= budget:
32         return lower.copy()
33
34     def total(lam: float) -> np.ndarray:
35         with np.errstate(over="ignore", divide="ignore", invalid="ignore"):
36             raw = np.power(gamma * a / lam, expo)
37             return np.clip(np.nan_to_num(raw, nan=0.0, posinf=np.inf), lower, caps)
38
39     lo, hi = 1e-300, 1.0
40     while total(hi).sum() > budget:
41         hi *= 4.0
42     for _ in range(400):
43         mid = np.sqrt(lo * hi)
44         if total(mid).sum() > budget:
45             lo = mid
46         else:
47             hi = mid
48         if hi / lo - 1.0 < tol:
49             break
50     x = total(np.sqrt(lo * hi))
51     free = (x > lower + 1e-12) & (x < caps - 1e-12)
52     gap = budget - x.sum()
53     if abs(gap) > 1e-9 and free.any(): # exact budget balance on the free coordinates
54         x[free] += gap * x[free] / max(x[free].sum(), 1e-300)
55     return np.clip(x, lower, caps)
56
57
58 def solve_group(
59     w: np.ndarray,
60     s: np.ndarray,
61     gamma: float,
62     caps: np.ndarray,
63     budget: float,
64     min_spend: float = 0.5,
65     tol: float = 1e-12,
66 ) -> dict[str, Any]:
67     """Allocate ``budget`` over keywords with value weights ``w`` (objective sum  $w_i (x_i/s_i)^\gamma$ ).
68
69     With ``min_spend`` =  $m > 0$  every keyword is either not selected ( $x_i = 0$ ) or receives at least  $m$ ,
70     i.e.  $x_i \in \{0\} \cup [m, \text{cap}_i]$ . This non-convex selection problem is solved in two phases:
71     (1) Lagrangian relaxation -- for a budget price  $\lambda$  each keyword takes
72      $x_i(\lambda) = \text{clip}((\gamma a_i / \lambda)^{1/(1-\gamma)}, m, \text{cap}_i)$  and is selected iff its value
73     covers the price,  $a_i x_i^\gamma \geq \lambda x_i$ ; total spend is non-increasing in  $\lambda$ , so the
74     price is found by bisection and fixes the selected set; (2) the convex problem restricted to the
75     selected set with lower bound  $m$  is solved exactly (bounded water filling), so the KKT conditions
76     hold exactly on that set. The convex relaxation ( $m = 0$ ) bounds the optimum from above and is used
77     by the verifier as an optimality certificate.
78     """
79     w = np.asarray(w, dtype=float)
80     s = np.asarray(s, dtype=float)
81     caps = np.asarray(caps, dtype=float)
82     a = w * np.power(s, -gamma)
83     n = len(a)
84     x = np.zeros(n)
85     if n == 0 or budget <= 0:
86         return {"x": x, "selected": x > 0, "objective": 0.0, "spent": 0.0, "budget": float(budget), "n_selected": 0}
87     positive = a > 0
88     if budget < min_spend: # not even one keyword can receive the minimum practical amount
89         return {"x": x, "selected": x > 0, "objective": 0.0, "spent": 0.0, "budget": float(budget), "n_selected": 0}
90     if min_spend <= 0:

```

```

91     x[positive] = water_filling(a[positive], gamma, caps[positive], budget, tol)
92 else:
93     m = float(min_spend)
94     caps_eff = np.maximum(caps, m)
95     expo = 1.0 / (1.0 - gamma)
96
97     def allocation(lam: float) -> np.ndarray:
98         with np.errstate(over="ignore", divide="ignore", invalid="ignore"):
99             interior = np.power(gamma * a / lam, expo)
100             xx = np.clip(np.nan_to_num(interior, nan=0.0, posinf=np.inf), m, caps_eff)
101             include = positive & (a * np.power(xx, gamma) >= lam * xx)
102             return np.where(include, xx, 0.0)
103
104     full = np.where(positive, caps_eff, 0.0)
105     if full.sum() <= budget:
106         x = full
107     else:
108         lo, hi = 1e-300, 1.0
109         while allocation(hi).sum() > budget:
110             hi *= 4.0
111         for _ in range(400):
112             mid = np.sqrt(lo * hi)
113             if allocation(mid).sum() > budget:
114                 lo = mid
115             else:
116                 hi = mid
117             if hi / lo - 1.0 < tol:
118                 break
119         chosen = allocation(hi) > 0
120         if not chosen.any():
121             chosen = allocation(lo) > 0
122         k = int(chosen.sum())
123         x[chosen] = _bounded_water_filling(a[chosen], gamma, np.full(k, m), caps_eff[chosen], budget, tol)
124     objective = float(np.sum(a * np.power(x, gamma)))
125     return {
126         "x": x,
127         "selected": x > 0,
128         "objective": objective,
129         "spent": float(x.sum()),
130         "budget": float(budget),
131         "n_selected": int((x > 0).sum()),
132     }
133
134
135 def objective_value(x: np.ndarray, a: np.ndarray, gamma: float) -> float:
136     return float(np.sum(a * np.power(np.maximum(x, 0.0), gamma)))
137
138
139 def verify_allocation(
140     x: np.ndarray,
141     a: np.ndarray,
142     gamma: float,
143     caps: np.ndarray,
144     budget: float,
145     rng: np.random.Generator,
146     floor: float = 0.0,
147     kkt_tol: float = 1e-6,
148     price_tol: float = 0.05,
149     n_transfers: int = 200,
150     bound_tol: float = 0.05,
151 ) -> dict[str, Any]:
152     """Independent optimality audit of an allocation (does not use the solver).
153

```

```

154 Feasibility (x_i = 0 or floor <= x_i <= cap_i, budget); KKT conditions (equal marginal value on
155 interior coordinates, marginal at capped coordinates not smaller, average value at floored
156 coordinates covering the price up to ``price_tol``); random pairwise transfers, removals and
157 insertions never improve the objective; and, when cvxpy is available, the convex relaxation
158 (no floor) solved by a generic solver bounds the objective from above -- the relative gap is an
159 optimality certificate (not informative for budgets of a few floor units, which are flagged).
160 """
161 x = np.asarray(x, dtype=float)
162 a = np.asarray(a, dtype=float)
163 caps = np.asarray(caps, dtype=float)
164 caps_eff = np.maximum(caps, floor)
165 checks: list[dict[str, Any]] = []
166 sel = x > 0
167 checks.append({"name": "nonnegative", "ok": bool((x >= -1e-9).all())})
168 checks.append({"name": "within_caps", "ok": bool((x <= caps_eff + 1e-6).all())})
169 checks.append({"name": "floor_respected", "ok": bool((x[sel] >= floor - 1e-6).all())})
170 spent = float(x.sum())
171 checks.append({"name": "budget_respected", "ok": bool(spent <= budget + 1e-6 * max(1.0, budget)), "spent": spent})
172 all_capped = np.where(a > 0, caps_eff, 0.0).sum() <= budget + 1e-9
173 exhausted = abs(spent - budget) <= 1e-6 * max(1.0, budget)
174 # with a floor the budget may be unattainable: every selected keyword sits at its cap and the rest < floor
175 stuck = floor > 0 and (budget - spent) < floor + 1e-9 and bool(np.all(x[sel] >= caps_eff[sel] - 1e-6))
176 checks.append({"name": "budget_exhausted_or_all_capped", "ok": bool(all_capped or exhausted or stuck)})
177 marg = np.full(len(x), np.nan)
178 marg[sel] = gamma * a[sel] * np.power(x[sel], gamma - 1.0)
179 avg = np.full(len(x), np.nan)
180 avg[sel] = a[sel] * np.power(x[sel], gamma - 1.0)
181 at_cap = x >= caps_eff - 1e-9
182 at_floor = (x <= floor + 1e-9) & (floor > 0)
183 both = sel & at_cap & at_floor # cap below the floor: both bounds active, only the selection rule applies
184 capped = sel & at_cap & ~at_floor
185 floored = sel & at_floor & ~at_cap
186 interior = sel & ~at_cap & ~at_floor
187 spread = float(np.nanmax(marg[interior]) / np.nanmin(marg[interior]) - 1.0) if interior.sum() >= 2 else 0.0
188 checks.append({"name": "kkt_equal_marginals_interior", "ok": spread <= kkt_tol, "relative_spread": spread})
189 # shadow price: with interior coordinates it is their common marginal value; otherwise any price
190 # between the largest floored marginal and the smallest capped marginal satisfies the KKT system
191 lam_low = float(np.nanmax(marg[floored])) if floored.any() else 0.0
192 lam_high = float(np.nanmin(marg[capped])) if capped.any() else float("inf")
193 if interior.any():
194     lam = float(np.nanmedian(marg[interior]))
195     cap_ok = not capped.any() or bool(np.all(marg[capped] >= lam * (1 - kkt_tol)))
196     floor_marg_ok = not floored.any() or bool(np.all(marg[floored] <= lam * (1 + kkt_tol)))
197     floor_avg_ok = not (floored | both).any() or bool(np.all(avg[floored | both] >= lam * (1 - price_tol)))
198 else:
199     lam = lam_low if np.isfinite(lam_high) is False else min(lam_high, max(lam_low, 0.0))
200     cap_ok = lam_low <= lam_high * (1 + kkt_tol)
201     floor_marg_ok = True
202     floor_avg_ok = not (floored | both).any() or bool(np.all(avg[floored | both] >= lam_low * (1 - price_tol)))
203 checks.append({"name": "kkt_capped_marginal_not_smaller", "ok": bool(cap_ok)})
204 checks.append({"name": "kkt_floored_marginal_not_larger", "ok": bool(floor_marg_ok)})
205 checks.append({"name": "kkt_floored_average_value_covers_price", "ok": bool(floor_avg_ok)})
206 base = objective_value(x, a, gamma)
207 idx = np.flatnonzero(sel)
208 improved = 0
209 if len(idx) >= 2:
210     for _ in range(n_transfers):
211         i, j = rng.choice(idx, size=2, replace=False)
212         y = x.copy()
213         if rng.random() < 0.3: # remove keyword i entirely and hand its money to j
214             delta = min(x[i], caps_eff[j] - x[j])
215             if delta < x[i] - 1e-12 and x[i] - delta < floor - 1e-12:
216                 continue

```

```

217         else:
218             delta = min(x[i] - floor, caps_eff[j] - x[j]) * rng.uniform(0.01, 0.5)
219             if delta <= 0:
220                 continue
221             y[i] -= delta
222             y[j] += delta
223             if objective_value(y, a, gamma) > base * (1 + 1e-9):
224                 improved += 1
225 checks.append({"name": "no_improving_transfer_or_removal", "ok": improved == 0, "improving_moves": improved})
226 inserted = 0
227 out_idx = np.flatnonzero(~sel & (a > 0))
228 if len(out_idx) and floor > 0 and interior.any(): # insert an unselected keyword at the floor
229     donors = np.flatnonzero(interior)
230     slack = price_tol * floor / max(budget, floor) # phase-1 price and phase-2 price differ slightly
231     for k in rng.choice(out_idx, size=min(len(out_idx), 50), replace=False):
232         take = floor * x[donors] / x[donors].sum()
233         if np.any(x[donors] - take < floor):
234             continue
235         y = x.copy()
236         y[donors] -= take
237         y[k] = floor
238         if objective_value(y, a, gamma) > base * (1 + slack):
239             inserted += 1
240 checks.append({"name": "no_improving_insertion", "ok": inserted == 0, "improving_insertions": inserted})
241 cvx: dict[str, Any] = {"available": False}
242 try:
243     import cvxpy as cp
244
245     n = len(x)
246     if n <= 1500:
247         var = cp.Variable(n, nonneg=True)
248         objective = cp.Maximize(cp.sum(cp.multiply(a, cp.power(var, gamma))))
249         prob = cp.Problem(objective, [cp.sum(var) <= budget, var <= caps_eff])
250         prob.solve(solver=cp.CLARABEL)
251         if var.value is not None:
252             relaxed = objective_value(np.asarray(var.value).ravel(), a, gamma)
253             gap = float((relaxed - base) / max(abs(base), 1e-12))
254             tiny = floor > 0 and budget <= 20 * floor
255             cvx = {"available": True, "status": prob.status, "relaxed_objective": relaxed, "relative_gap": gap}
256             tol = bound_tol if floor > 0 else 1e-4
257             checks.append(
258                 {
259                     "name": "within_convex_relaxation_bound",
260                     "ok": bool(gap <= tol or tiny),
261                     "relative_gap": gap,
262                     "tiny_budget": bool(tiny),
263                 }
264             )
265 except Exception as exc: # pragma: no cover - solver availability differs across environments
266     cvx = {"available": False, "error": repr(exc)}
267 return {
268     "ok": all(c["ok"] for c in checks),
269     "checks": checks,
270     "objective": base,
271     "n_selected": int(sel.sum()),
272     "n_capped": int(capped.sum()),
273     "n_floored": int(floored.sum()),
274     "n_both_bounds": int(both.sum()),
275     "cvxpy": cvx,
276 }
277
278
279 def proportional_allocation(s: np.ndarray, budget: float) -> np.ndarray:

```

```

280     """Historical-practice baseline: spend split in proportion to each keyword's mean daily spend."""
281     s = np.asarray(s, dtype=float)
282     return budget * s / s.sum() if s.sum() > 0 else np.zeros_like(s)
283
284
285 def equal_allocation(n: int, budget: float) -> np.ndarray:
286     return np.full(n, budget / n) if n else np.zeros(0)

```

pipelines/e/calendar.py

```

1  """Statutory holiday calendar (State Council notice for 2025) and per-day calendar features."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import pandas as pd
8
9  # (first day, last day) of every statutory holiday block in 2025, inclusive.
10 HOLIDAY_BLOCKS: dict[str, tuple[str, str]] = {
11     "元旦": ("2025-01-01", "2025-01-01"),
12     "春节": ("2025-01-28", "2025-02-04"),
13     "清明节": ("2025-04-04", "2025-04-06"),
14     "劳动节": ("2025-05-01", "2025-05-05"),
15     "端午节": ("2025-05-31", "2025-06-02"),
16     "国庆节中秋节": ("2025-10-01", "2025-10-08"),
17 }
18 # Weekend days declared working days to compensate for the long holidays (调休).
19 ADJUSTED_WORKDAYS: tuple[str, ...] = ("2025-01-26", "2025-02-08", "2025-04-27", "2025-09-28", "2025-10-11")
20 WEEKDAY_NAMES: tuple[str, ...] = ("周一", "周二", "周三", "周四", "周五", "周六", "周日")
21
22
23 def holiday_map() -> dict[pd.Timestamp, str]:
24     """Every statutory holiday date -> holiday name."""
25     out: dict[pd.Timestamp, str] = {}
26     for name, (start, end) in HOLIDAY_BLOCKS.items():
27         for day in pd.date_range(start, end, freq="D"):
28             out[day] = name
29     return out
30
31
32 def day_features(dates: Any) -> pd.DataFrame:
33     """Calendar covariates for a sequence of dates (weekday, month, holiday flags, day type)."""
34     idx = pd.DatetimeIndex(pd.to_datetime(dates)).normalize()
35     holidays = holiday_map()
36     adjusted = {pd.Timestamp(d) for d in ADJUSTED_WORKDAYS}
37     starts = {pd.Timestamp(s) for s, _ in HOLIDAY_BLOCKS.values()}
38     ends = {pd.Timestamp(e) for _, e in HOLIDAY_BLOCKS.values()}
39     one = pd.Timedelta(days=1)
40     rows: list[dict[str, Any]] = []
41     for day in idx:
42         weekday = int(day.dayofweek)
43         is_holiday = day in holidays
44         is_adjusted = day in adjusted
45         is_weekend = weekday >= 5
46         if is_holiday:
47             day_type = "holiday"
48         elif is_adjusted:
49             day_type = "adjusted_workday"
50         elif is_weekend:
51             day_type = "weekend"

```

```

52     else:
53         day_type = "workday"
54     rows.append(
55         {
56             "date": day,
57             "weekday": weekday,
58             "weekday_name": WEEKDAY_NAMES[weekday],
59             "month": int(day.month),
60             "day_of_year": int(day.dayofyear),
61             "is_weekend": is_weekend,
62             "is_holiday": is_holiday,
63             "holiday_name": holidays.get(day, ""),
64             "is_adjusted_workday": is_adjusted,
65             "is_pre_holiday": (day + one) in starts,
66             "is_post_holiday": (day - one) in ends,
67             "is_workday": day_type in {"workday", "adjusted_workday"},
68             "day_type": day_type,
69         }
70     )
71     return pd.DataFrame(rows)

```

pipelines/e/classify.py

```

1  """Problem 2: cost/benefit keyword classification (composite benefit index, transparent thresholds, checker)."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import numpy as np
8  import pandas as pd
9
10 LABELS: tuple[str, ...] = ("黄金词", "重点词", "潜力词", "问题词", "无效词")
11 BENEFIT_COMPONENTS: tuple[str, ...] = ("clicks", "views", "engagement", "reg_attr")
12
13
14 def minmax(x: np.ndarray) -> np.ndarray:
15     lo, hi = float(np.min(x)), float(np.max(x))
16     if hi - lo <= 1e-12:
17         return np.zeros_like(x, dtype=float)
18     return (x - lo) / (hi - lo)
19
20
21 def entropy_weights(X: np.ndarray) -> np.ndarray:
22     """Entropy weight method on a non-negative matrix (rows = items, columns = criteria)."""
23     X = np.asarray(X, dtype=float)
24     n, m = X.shape
25     weights = np.zeros(m)
26     for j in range(m):
27         col = minmax(X[:, j])
28         total = col.sum()
29         if total <= 0:
30             continue
31         p = col / total
32         with np.errstate(divide="ignore", invalid="ignore"):
33             e = -np.nansum(np.where(p > 0, p * np.log(p), 0.0)) / np.log(n)
34         weights[j] = 1.0 - e
35     if weights.sum() <= 0:
36         return np.full(m, 1.0 / m)
37     return weights / weights.sum()
38

```

```

39
40 def benefit_components(kw: pd.DataFrame, rates: dict[int, float]) -> pd.DataFrame:
41     """Log-scaled benefit components: clicks, views, engagement (attention seconds), attributed registrations."""
42     reg_attr = kw["clicks"].to_numpy(dtype=float) * kw["推广单元ID"].map(rates).fillna(0.0).to_numpy(dtype=float)
43     comp = pd.DataFrame(
44         {
45             "clicks": np.log1p(kw["clicks"].to_numpy(dtype=float)),
46             "views": np.log1p(kw["views"].to_numpy(dtype=float)),
47             "engagement": np.log1p(kw["engagement"].to_numpy(dtype=float)),
48             "reg_attr": np.log1p(reg_attr),
49         },
50         index=kw.index,
51     )
52     comp["reg_attr_raw"] = reg_attr
53     return comp
54
55
56 def benefit_index(comp: pd.DataFrame, weights: np.ndarray | None = None) -> tuple[pd.Series, np.ndarray]:
57     """Weighted sum of min-max normalised components (entropy weights unless given)."""
58     X = comp[list(BENEFIT_COMPONENTS)].to_numpy(dtype=float)
59     w = entropy_weights(X) if weights is None else np.asarray(weights, dtype=float)
60     normed = np.column_stack([minmax(X[:, j]) for j in range(X.shape[1])])
61     return pd.Series(normed @ w, index=comp.index, name="benefit"), w
62
63
64 def jenks_break(x: np.ndarray) -> float:
65     """Exact two-class Fisher/Jenks natural break (minimum within-class sum of squares)."""
66     xs = np.sort(np.asarray(x, dtype=float))
67     n = len(xs)
68     if n < 2:
69         return float(xs[0]) if n else 0.0
70     cs = np.cumsum(xs)
71     cs2 = np.cumsum(xs**2)
72     k = np.arange(1, n) # left class = xs[:k]
73     left = cs2[k - 1] - cs[k - 1] ** 2 / k
74     right = (cs2[-1] - cs2[k - 1]) - (cs[-1] - cs[k - 1]) ** 2 / (n - k)
75     best = int(np.argmin(left + right))
76     return float(0.5 * (xs[best] + xs[best + 1]))
77
78
79 def gmm_break(x: np.ndarray, seed: int) -> float:
80     """Threshold where the two-component Gaussian mixture posterior crosses 0.5 between the component means."""
81     from sklearn.mixture import GaussianMixture
82
83     xs = np.asarray(x, dtype=float).reshape(-1, 1)
84     gm = GaussianMixture(n_components=2, random_state=seed, n_init=3).fit(xs)
85     means = gm.means_.ravel()
86     lo, hi = float(means.min()), float(means.max())
87     grid = np.linspace(lo, hi, 2001).reshape(-1, 1)
88     post = gm.predict_proba(grid)[:, int(np.argmax(means))]
89     idx = int(np.argmax(post >= 0.5))
90     return float(grid[idx, 0])
91
92
93 def threshold(x: np.ndarray, method: str, seed: int = 0) -> float:
94     if method == "median":
95         return float(np.median(x))
96     if method == "jenks":
97         return jenks_break(x)
98     if method == "gmm":
99         return gmm_break(x, seed)
100     raise ValueError(f"unknown threshold method {method!r}")
101

```



```

102
103 def assign_labels(kw: pd.DataFrame, benefit: pd.Series, cost_thr: float, benefit_thr: float) -> pd.Series:
104     """Five-way labels: invalid (no spend, no clicks), else the quadrant of (log10 cost > cost_thr, benefit >= thr)."""
105     invalid = (kw["spend"] <= 0) & (kw["clicks"] <= 0)
106     high_cost = np.log10(kw["spend"].clip(lower=1e-9)) > cost_thr
107     high_benefit = benefit >= benefit_thr
108     labels = np.select(
109         [
110             invalid,
111             ~high_cost & high_benefit,
112             high_cost & high_benefit,
113             ~high_cost & ~high_benefit,
114             high_cost & ~high_benefit,
115         ],
116         ["无效词", "黄金词", "重点词", "潜力词", "问题词"],
117         default="问题词",
118     )
119     return pd.Series(labels, index=kw.index, name="label")
120
121
122 def classify_keywords(
123     kw: pd.DataFrame, rates: dict[int, float], methods: tuple[str, ...], main: str, seed: int
124 ) -> dict[str, Any]:
125     """Full classification with every threshold method; ``main`` selects the reported labels."""
126     comp = benefit_components(kw, rates)
127     benefit, weights = benefit_index(comp)
128     benefit_equal, _ = benefit_index(comp, np.full(len(BENEFIT_COMPONENTS), 1.0 / len(BENEFIT_COMPONENTS)))
129     active = kw["spend"] > 0
130     log_cost = np.log10(kw.loc[active, "spend"].to_numpy(dtype=float))
131     thresholds: dict[str, dict[str, float]] = {}
132     labels: dict[str, pd.Series] = {}
133     for method in methods:
134         ct = threshold(log_cost, method, seed)
135         bt = threshold(benefit[active].to_numpy(dtype=float), method, seed)
136         thresholds[method] = {"log10_cost": ct, "cost_yuan": float(10**ct), "benefit": bt}
137         labels[method] = assign_labels(kw, benefit, ct, bt)
138     out = kw[
139         ["序号", "关键词", "方案ID", "推广单元ID", "spend", "clicks", "views", "bounce", "duration_s", "engagement"]
140     ].copy()
141     out["reg_attr"] = comp["reg_attr_raw"]
142     out["benefit"] = benefit
143     out["benefit_equal"] = benefit_equal
144     out["log10_cost"] = np.log10(kw["spend"].clip(lower=1e-9))
145     for method in methods:
146         out[f"label_{method}"] = labels[method]
147     out["label"] = labels[main]
148     agreement = {f"{a}_vs_{b}": float((labels[a] == labels[b]).mean()) for a in methods for b in methods if a < b}
149     agreement["entropy_vs_equal_spearman"] = float(benefit[active].corr(benefit_equal[active], method="spearman"))
150     # equal-weight benefit with the main thresholds re-derived: sensitivity of labels to the weighting
151     bt_equal = threshold(benefit_equal[active].to_numpy(dtype=float), main, seed)
152     labels_equal = assign_labels(kw, benefit_equal, thresholds[main]["log10_cost"], bt_equal)
153     agreement[f"{main}_vs_equal_weights"] = float((labels[main] == labels_equal).mean())
154     out["label_equal_weights"] = labels_equal
155     counts = {m: labels[m].value_counts().reindex(LABELS, fill_value=0).astype(int).to_dict() for m in methods}
156     counts["equal_weights"] = labels_equal.value_counts().reindex(LABELS, fill_value=0).astype(int).to_dict()
157     thresholds["equal_weights"] = {
158         "log10_cost": thresholds[main]["log10_cost"],
159         "cost_yuan": thresholds[main]["cost_yuan"],
160         "benefit": bt_equal,
161     }
162     return {
163         "table": out,
164         "weights": dict(zip(BENEFIT_COMPONENTS, [float(w) for w in weights])),

```

```

165         "thresholds": thresholds,
166         "agreement": agreement,
167         "counts": counts,
168         "main": main,
169     }
170
171
172 def verify_classification(
173     table: pd.DataFrame, cost_thr_log10: float, benefit_thr: float, label_col: str = "label"
174 ) -> dict[str, Any]:
175     """Independent re-derivation of every label from raw columns; returns a report with ``ok``."""
176     checks: list[dict[str, Any]] = []
177     lab = table[label_col]
178     checks.append({"name": "labels_in_vocabulary", "ok": bool(lab.isin(LABELS).all())})
179     invalid_expected = (table["spend"] <= 0) & (table["clicks"] <= 0)
180     invalid_ok = bool(((lab == "无效词") == invalid_expected).all())
181     checks.append({"name": "invalid_iff_no_spend_no_clicks", "ok": invalid_ok})
182     high_cost = table["spend"] > 10**cost_thr_log10
183     high_ben = table["benefit"] >= benefit_thr
184     expected = np.where(
185         invalid_expected,
186         "无效词",
187         np.where(high_cost, np.where(high_ben, "重点词", "问题词"), np.where(high_ben, "黄金词", "潜力词")),
188     )
189     mism = int((lab.to_numpy() != expected).sum())
190     checks.append({"name": "quadrant_rule_reproduced", "ok": mism == 0, "mismatches": mism})
191     counts = lab.value_counts().reindex(LABELS, fill_value=0)
192     checks.append({"name": "counts_sum_to_rows", "ok": int(counts.sum()) == len(table)})
193     gold = table[lab == "黄金词"]
194     prob = table[lab == "问题词"]
195     checks.append(
196         {
197             "name": "gold_cheaper_and_better_than_problem",
198             "ok": bool(
199                 len(gold) == 0
200                 or len(prob) == 0
201                 or (
202                     gold["spend"].max() <= prob["spend"].min() + 1e-9
203                     and gold["benefit"].min() >= prob["benefit"].max() - 1e-12
204                 )
205             ),
206         }
207     )
208     checks.append(
209         {
210             "name": "benefit_in_unit_interval",
211             "ok": bool(((table["benefit"] >= -1e-12) & (table["benefit"] <= 1 + 1e-12)).all()),
212         }
213     )
214     return {"ok": all(c["ok"] for c in checks), "checks": checks, "counts": counts.astype(int).to_dict()}

```

pipelines/e/contracts.py

```

1  """Data contracts for Problem E (SEM advertising strategy)."""
2
3  from __future__ import annotations
4
5  import re
6
7  import pandas as pd
8

```

```

9 from forge.contracts import Column, FrameContract
10 from forge.xlsx import SheetContract, WorkbookContract
11
12 _DURATION = re.compile(r"^\s*d{1,2}:\d{2}:\d{2}\s*$")
13
14
15 def _funnel_consistent(df: pd.DataFrame) -> list[str]:
16     errors: list[str] = []
17     if (df["点击量"] > df["展现量"]).any():
18         errors.append("点击量 > 展现量 in some rows")
19     if (df["上方位展现量"] > df["展现量"]).any():
20         errors.append("上方位展现量 > 展现量 in some rows")
21     if (df["上方首位展现量"] > df["上方位展现量"]).any():
22         errors.append("上方首位展现量 > 上方位展现量 in some rows")
23     if (df["上方位点击量"] > df["点击量"]).any():
24         errors.append("上方位点击量 > 点击量 in some rows")
25     if (df["上方位消费额"] > df["消费额"] + 1e-6).any():
26         errors.append("上方位消费额 > 消费额 in some rows")
27     return errors
28
29
30 def _bounce_and_duration(df: pd.DataFrame) -> list[str]:
31     errors: list[str] = []
32     for value in df["跳出率"].astype(str):
33         v = value.strip()
34         if v == "/":
35             continue
36         try:
37             x = float(v)
38         except ValueError:
39             errors.append(f"跳出率 unparsable {value!r}")
40             continue
41         if not 0 <= x <= 1:
42             errors.append(f"跳出率 out of range {value!r}")
43     for value in df["平均访问时长"].astype(str):
44         v = value.strip()
45         if v == "/" or _DURATION.match(v):
46             continue
47         try: # zero-visit rows carry a bare numeric 0 instead of "00:00:00"
48             if float(v) < 0:
49                 errors.append(f"平均访问时长 negative {value!r}")
50         except ValueError:
51             errors.append(f"平均访问时长 unparsable {value!r}")
52     return errors[:20]
53
54
55 INPUT_CONTRACTS: dict[str, FrameContract] = {
56     "附件1__Sheet1": FrameContract(
57         name="E.附件1.Sheet1.单元日消费",
58         columns=(
59             Column("日期", "datetime"),
60             Column("方案ID", "int"),
61             Column("推广单元ID", "int"),
62             Column("展现量", "int", min=0),
63             Column("点击量", "int", min=0),
64             Column("消费额", "float", min=0),
65             Column("上方位展现量", "int", min=0),
66             Column("上方首位展现量", "int", min=0),
67             Column("上方位点击量", "int", min=0),
68             Column("上方位消费额", "float", min=0),
69         ),
70         min_rows=2000,
71         checks=(_funnel_consistent,),

```

```

72     ),
73     "附件1__Sheet2": FrameContract(
74         name="E.附件1.Sheet2.每日新注册",
75         columns=(Column("日期", "datetime", unique=True), Column("新注册数", "int", min=0)),
76         min_rows=365,
77         max_rows=366,
78     ),
79     "附件1__Sheet3": FrameContract(
80         name="E.附件1.Sheet3.关键词年度统计",
81         columns=(
82             Column("序号", "int", unique=True, min=1),
83             Column("关键词", "int"),
84             Column("方案ID", "int"),
85             Column("推广单元ID", "int"),
86             Column("消费额", "float", min=0),
87             Column("点击量", "int", min=0),
88             Column("浏览量", "int", min=0),
89             Column("跳出率", "any"),
90             Column("平均访问时长", "any"),
91         ),
92         min_rows=2000,
93         checks=(_bounce_and_duration,),
94     ),
95 }
96
97 RESULT_CONTRACTS: list[WorkbookContract] = [
98     WorkbookContract("result2.xlsx", "result2.xlsx", (SheetContract("Sheet1", min_rows=2000, header_len=8))),
99     WorkbookContract("result3.xlsx", "result3.xlsx", (SheetContract("Sheet1", min_rows=16, header_len=9))),
100    WorkbookContract("result4.xlsx", "result4.xlsx", (SheetContract("Sheet1", min_rows=7, header_len=9))),
101 ]

```

pipelines/e/data.py

```

1  """Loading and normalising the three attachment sheets into analysis frames."""
2
3  from __future__ import annotations
4
5  import math
6  import re
7  from pathlib import Path
8  from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13 from pipelines.e.calendar import day_features
14
15 _HMS = re.compile(r"^(?d{1,3}):(?d{2}):(?d{2})$")
16 UNIT_COLS = ["方案ID", "推广单元ID"]
17
18
19 def parse_bounce(value: Any) -> float:
20     """Bounce rate as a fraction in [0, 1]; '/' (no visits) becomes NaN."""
21     text = str(value).strip()
22     if text in {"/", "", "nan", "None"}:
23         return math.nan
24     x = float(text)
25     return x / 100.0 if x > 1.0 else x
26
27
28 def parse_duration(value: Any) -> float:

```

```

29     """Average visit duration in seconds; 'HH:MM:SS', bare numbers (zero) or '/' (no visits -> NaN)."""
30     text = str(value).strip()
31     if text in {"/", "", "nan", "None"}:
32         return math.nan
33     match = _HMS.match(text)
34     if match:
35         h, m, s = (int(g) for g in match.groups())
36         return float(h * 3600 + m * 60 + s)
37     return float(text)
38
39
40 def load_sheets(ingest_dir: Path) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
41     """Sheet1 (unit-day), Sheet2 (daily registrations), Sheet3 (keyword annual statistics)."""
42     data = ingest_dir / "data"
43     s1 = pd.read_parquet(data / "附件1__Sheet1.parquet")
44     s2 = pd.read_parquet(data / "附件1__Sheet2.parquet")
45     s3 = pd.read_parquet(data / "附件1__Sheet3.parquet")
46     for df in (s1, s2, s3):
47         df.columns = [str(c).strip() for c in df.columns]
48         s1["日期"] = pd.to_datetime(s1["日期"]).dt.normalize()
49         s2["日期"] = pd.to_datetime(s2["日期"]).dt.normalize()
50         s1 = s1.sort_values(["日期", *UNIT_COLS]).reset_index(drop=True)
51         s2 = s2.sort_values("日期").reset_index(drop=True)
52         s3 = s3.sort_values("序号").reset_index(drop=True)
53     return s1, s2, s3
54
55
56 def unit_daily(s1: pd.DataFrame) -> pd.DataFrame:
57     """Unit-day records with derived ratios and calendar covariates."""
58     df = s1.copy()
59     clicks = df["点击量"].replace(0, np.nan)
60     imp = df["展现量"].replace(0, np.nan)
61     df["cpc"] = df["消费额"] / clicks
62     df["ctr"] = df["点击量"] / imp
63     df["cpm"] = 1000.0 * df["消费额"] / imp
64     df["top_share"] = df["上方位展现量"] / imp
65     df["first_share"] = df["上方首位展现量"] / imp
66     df["top_click_share"] = df["上方位点击量"] / clicks
67     df["top_cpc"] = df["上方位消费额"] / df["上方位点击量"].replace(0, np.nan)
68     other_clicks = (df["点击量"] - df["上方位点击量"]).replace(0, np.nan)
69     df["other_cpc"] = (df["消费额"] - df["上方位消费额"]) / other_clicks
70     feats = day_features(df["日期"]).drop(columns=["date"])
71     return pd.concat([df.reset_index(drop=True), feats], axis=1)
72
73
74 def daily_totals(s1: pd.DataFrame, s2: pd.DataFrame) -> pd.DataFrame:
75     """All 365 days of 2025: account-level totals, registrations and calendar covariates."""
76     agg = s1.groupby("日期")[
77         ["展现量", "点击量", "消费额", "上方位展现量", "上方首位展现量", "上方位点击量", "上方位消费额"]
78     ].sum()
79     agg.columns = ["imp", "clicks", "spend", "top_imp", "first_imp", "top_clicks", "top_spend"]
80     days = pd.date_range("2025-01-01", "2025-12-31", freq="D")
81     out = agg.reindex(days).fillna(0.0)
82     out["regs"] = s2.set_index("日期")["新注册数"].reindex(days).astype(float)
83     out["active_units"] = s1.groupby("日期")["推广单元ID"].nunique().reindex(days).fillna(0).astype(int)
84     out["cpc"] = out["spend"] / out["clicks"].replace(0, np.nan)
85     out["ctr"] = out["clicks"] / out["imp"].replace(0, np.nan)
86     out["top_share"] = out["top_imp"] / out["imp"].replace(0, np.nan)
87     out["first_share"] = out["first_imp"] / out["imp"].replace(0, np.nan)
88     feats = day_features(days).set_index("date")
89     out = out.join(feats)
90     out.index.name = "date"
91     return out

```

```

92
93
94 def unit_click_matrix(s1: pd.DataFrame, column: str = "点击量") -> pd.DataFrame:
95     """Dates (all of 2025) x units matrix of ``column``; days without a record are zero."""
96     days = pd.date_range("2025-01-01", "2025-12-31", freq="D")
97     mat = s1.pivot_table(index="日期", columns="推广单元ID", values=column, aggfunc="sum")
98     return mat.reindex(days).fillna(0.0)
99
100
101 def keyword_table(s3: pd.DataFrame) -> pd.DataFrame:
102     """Keyword-level annual statistics with parsed engagement fields."""
103     kw = pd.DataFrame(
104         {
105             "序号": s3["序号"].astype(int),
106             "关键词": s3["关键词"].astype(int),
107             "方案ID": s3["方案ID"].astype(int),
108             "推广单元ID": s3["推广单元ID"].astype(int),
109             "spend": s3["消费额"].astype(float),
110             "clicks": s3["点击量"].astype(int),
111             "views": s3["浏览量"].astype(int),
112         }
113     )
114     kw["bounce"] = s3["跳出率"].map(parse_bounce).astype(float)
115     kw["duration_s"] = s3["平均访问时长"].map(parse_duration).astype(float)
116     clicks = kw["clicks"].replace(0, np.nan)
117     kw["cpc"] = kw["spend"] / clicks
118     kw["depth"] = kw["views"] / clicks
119     # visits that did not bounce, and total attention time (person-seconds) they generated
120     kw["engaged_visits"] = (kw["clicks"] * (1.0 - kw["bounce"].fillna(1.0))).clip(lower=0.0)
121     kw["engagement"] = (kw["engaged_visits"] * kw["duration_s"].fillna(0.0)).clip(lower=0.0)
122     kw["has_cost"] = kw["spend"] > 0
123     kw["has_benefit"] = (kw["clicks"] > 0) | (kw["views"] > 0)
124     dup_counts = kw.groupby("关键词")["推广单元ID"].transform("nunique")
125     kw["shared_units"] = dup_counts.astype(int)
126     return kw

```

pipelines/e/eda.py

```

1  """Problem 1 analytics: unit KPIs, keyword concentration, calendar/holiday regressions, registration attribution."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import numpy as np
8  import pandas as pd
9  import statsmodels.api as sm
10 from scipy import stats
11 from scipy.optimize import lsq_linear
12
13 from pipelines.e.calendar import WEEKDAY_NAMES
14
15 WEEKDAY_BASE = 2 # Wednesday is the reference weekday in every regression
16 MONTH_BASE = 1 # January is the reference month
17
18
19 def unit_kpis(s1: pd.DataFrame) -> pd.DataFrame:
20     """Annual funnel KPIs per (plan, unit): CTR, CPC, position shares, top vs. other CPC."""
21     g = s1.groupby(["方案ID", "推广单元ID"])
22     k = pd.DataFrame(
23         {

```

```

24         "days": g.size(),
25         "first_day": g["日期"].min(),
26         "last_day": g["日期"].max(),
27         "imp": g["展现量"].sum(),
28         "clicks": g["点击量"].sum(),
29         "spend": g["消费额"].sum(),
30         "top_imp": g["上方位展现量"].sum(),
31         "first_imp": g["上方首位展现量"].sum(),
32         "top_clicks": g["上方位点击量"].sum(),
33         "top_spend": g["上方位消费额"].sum(),
34     }
35 ).reset_index()
36 imp = k["imp"].replace(0, np.nan)
37 clicks = k["clicks"].replace(0, np.nan)
38 k["ctr"] = k["clicks"] / imp
39 k["cpc"] = k["spend"] / clicks
40 k["cpm"] = 1000.0 * k["spend"] / imp
41 k["top_share"] = k["top_imp"] / imp
42 k["first_share"] = k["first_imp"] / imp
43 k["top_click_share"] = k["top_clicks"] / clicks
44 k["top_cpc"] = k["top_spend"] / k["top_clicks"].replace(0, np.nan)
45 k["other_cpc"] = (k["spend"] - k["top_spend"]) / (k["clicks"] - k["top_clicks"]).replace(0, np.nan)
46 k["top_ctr"] = k["top_clicks"] / k["top_imp"].replace(0, np.nan)
47 k["other_ctr"] = (k["clicks"] - k["top_clicks"]) / (k["imp"] - k["top_imp"]).replace(0, np.nan)
48 k["spend_share"] = k["spend"] / k["spend"].sum()
49 return k
50
51
52 def gini(values: np.ndarray) -> float:
53     """Gini coefficient of a non-negative vector (0 = perfectly even)."""
54     x = np.sort(np.asarray(values, dtype=float))
55     n = len(x)
56     if n == 0 or x.sum() <= 0:
57         return 0.0
58     cum = np.cumsum(x)
59     return float((n + 1 - 2 * np.sum(cum) / cum[-1]) / n)
60
61
62 def keyword_concentration(kw: pd.DataFrame) -> dict[str, Any]:
63     """Long-tail statistics of keyword spend and clicks (global and per unit)."""
64     spend = kw["spend"].to_numpy(dtype=float)
65     order = np.argsort(-spend)
66     cum = np.cumsum(spend[order]) / spend.sum()
67     n = len(spend)
68     lorenz = pd.DataFrame(
69         {
70             "keyword_share": np.arange(1, n + 1) / n,
71             "spend_share": cum,
72             "click_share": np.cumsum(kw["clicks"].to_numpy()[order]) / kw["clicks"].sum(),
73         }
74     )
75     per_unit = (
76         kw.groupby(["方案ID", "推广单元ID"])
77         .apply(
78             lambda d: pd.Series(
79                 {
80                     "keywords": len(d),
81                     "active": int((d["spend"] > 0).sum()),
82                     "zero": int((d["spend"] == 0).sum()),
83                     "spend": d["spend"].sum(),
84                     "clicks": d["clicks"].sum(),
85                     "gini_spend": gini(d["spend"].to_numpy()),
86                     "top5_share": float(np.sort(d["spend"].to_numpy())[:-1][:5].sum() / max(d["spend"].sum(), 1e-9)),

```

```

87         }
88     ),
89     include_groups=False,
90 )
91 .reset_index()
92 )
93 return {
94     "keywords": int(n),
95     "unique_keyword_ids": int(kw["关键词"].nunique()),
96     "shared_keyword_ids": int((kw.groupby("关键词")["推广单元ID"].nunique() > 1).sum()),
97     "zero_cost_zero_benefit": int((kw["spend"] == 0) & (kw["clicks"] == 0).sum()),
98     "active_keywords": int((kw["spend"] > 0).sum()),
99     "gini_spend_all": gini(spend),
100     "gini_spend_active": gini(spend[spend > 0]),
101     "top10_spend_share": float(cum[min(9, n - 1)]),
102     "top50_spend_share": float(cum[min(49, n - 1)]),
103     "top100_spend_share": float(cum[min(99, n - 1)]),
104     "top20pct_spend_share": float(cum[max(int(0.2 * n) - 1, 0)]),
105     "lorenz": lorenz,
106     "per_unit": per_unit,
107 }
108
109
110 def _design(daily: pd.DataFrame) -> pd.DataFrame:
111     """Regression design: constant, weekday and month dummies, holiday and 调休 indicators."""
112     cols: dict[str, Any] = {"const": 1.0}
113     for wd in range(7):
114         if wd != WEEKDAY_BASE:
115             cols[f"wd_{WEEKDAY_NAMES[wd]}"] = (daily["weekday"] == wd).astype(float)
116     for m in range(1, 13):
117         if m != MONTH_BASE and (daily["month"] == m).any():
118             cols[f"month_{m:02d}"] = (daily["month"] == m).astype(float)
119     cols["holiday"] = daily["is_holiday"].astype(float)
120     cols["adjusted_workday"] = daily["is_adjusted_workday"].astype(float)
121     cols["pre_holiday"] = daily["is_pre_holiday"].astype(float)
122     cols["post_holiday"] = daily["is_post_holiday"].astype(float)
123     return pd.DataFrame(cols, index=daily.index)
124
125
126 def calendar_regression(
127     daily: pd.DataFrame, target: str, log: bool, controls: tuple[str, ...] = ()
128 ) -> tuple[pd.DataFrame, dict[str, Any]]:
129     """OLS of ``target`` (optionally log) on calendar dummies (+ log controls) with HC3 robust standard errors."""
130     y = daily[target].astype(float)
131     mask = y.notna() & (y > 0 if log else True)
132     for ctrl in controls:
133         mask &= daily[ctrl].astype(float) > 0
134     y = np.log(y[mask]) if log else y[mask]
135     X = _design(daily.loc[mask])
136     for ctrl in controls:
137         X[f"log_{ctrl}"] = np.log(daily.loc[mask, ctrl].astype(float))
138     name = f"{target}|{'+'.join(controls)}" if controls else target
139     fit = sm.OLS(y, X).fit(cov_type="HC3")
140     ci = fit.conf_int(alpha=0.05)
141     rows = []
142     for term in X.columns:
143         est = float(fit.params[term])
144         rows.append(
145             {
146                 "target": name,
147                 "term": term,
148                 "estimate": est,
149                 "se": float(fit.bse[term]),

```



```

150         "t": float(fit.tvalues[term]),
151         "p": float(fit.pvalues[term]),
152         "ci_low": float(ci.loc[term, 0]),
153         "ci_high": float(ci.loc[term, 1]),
154         "effect_pct": 100.0 * (np.exp(est) - 1.0) if log else np.nan,
155         "effect_pct_low": 100.0 * (np.exp(ci.loc[term, 0]) - 1.0) if log else np.nan,
156         "effect_pct_high": 100.0 * (np.exp(ci.loc[term, 1]) - 1.0) if log else np.nan,
157     }
158 )
159 wd_terms = [c for c in X.columns if c.startswith("wd_")]
160 wald = fit.wald_test(np.eye(len(X.columns))[[list(X.columns).index(c) for c in wd_terms]], scalar=True)
161 summary = {
162     "target": name,
163     "log": log,
164     "n": int(mask.sum()),
165     "r2": float(fit.rsquared),
166     "r2_adj": float(fit.rsquared_adj),
167     "weekday_wald_stat": float(wald.statistic),
168     "weekday_wald_p": float(wald.pvalue),
169     "residual_sd": float(np.sqrt(fit.scale)),
170 }
171 return pd.DataFrame(rows), summary
172
173
174 def holiday_nonparametric(daily: pd.DataFrame, target: str) -> dict[str, Any]:
175     """Mann-Whitney U test of holiday days vs. ordinary workdays for ``target``."""
176     hol = daily.loc[daily["is_holiday"], target].dropna()
177     work = daily.loc[daily["day_type"] == "workday", target].dropna()
178     u = stats.mannwhitneyu(hol, work, alternative="two-sided")
179     return {
180         "target": target,
181         "holiday_n": len(hol),
182         "workday_n": len(work),
183         "holiday_median": float(hol.median()),
184         "workday_median": float(work.median()),
185         "ratio_of_medians": float(hol.median() / work.median()) if work.median() else np.nan,
186         "u_stat": float(u.statistic),
187         "p": float(u.pvalue),
188     }
189
190
191 def day_type_profile(daily: pd.DataFrame, columns: tuple[str, ...]) -> pd.DataFrame:
192     """Mean of ``columns`` by weekday, by month and by day type (long format)."""
193     frames = []
194     for key in ("weekday_name", "month", "day_type"):
195         g = daily.groupby(key)[list(columns)].mean()
196         g["n_days"] = daily.groupby(key).size()
197         g = g.reset_index().rename(columns={key: "level"})
198         g.insert(0, "dimension", key)
199         g["level"] = g["level"].astype(str)
200         frames.append(g)
201     return pd.concat(frames, ignore_index=True)
202
203
204 def attribution_model(
205     clicks: pd.DataFrame,
206     daily: pd.DataFrame,
207     lambdas: tuple[float, ...] = (0.0, 0.2, 0.4, 0.6),
208     min_clicks: float = 10_000.0,
209     band: tuple[float, ...] = (0.25, 4.0),
210 ) -> dict[str, Any]:
211     """Registrations = baseline + sum_u r_u * adstocked clicks_u + calendar effects (r_u >= 0).
212

```

```

213 ``clicks``: dates x units matrix. The adstock decay ``lambda`` is selected by rolling-origin
214 validation MAE. A unit keeps its own rate only when it has enough clicks to identify it and the
215 estimate lies within ``band`` times the pooled rate; otherwise the pooled rate is used.
216 """
217 y = daily["regs"].to_numpy(dtype=float)
218 cal = _design(daily).drop(columns=["const"])
219 units = [int(u) for u in clicks.columns]
220 n = len(y)
221 origins = [int(n * f) for f in (0.5, 0.6, 0.7, 0.8)]
222
223 def adstock(mat: np.ndarray, lam: float) -> np.ndarray:
224     out = mat.copy()
225     for t in range(1, len(out)):
226         out[t] += lam * out[t - 1]
227     return out
228
229 def fit(Xu: np.ndarray, Xc: np.ndarray, yy: np.ndarray) -> np.ndarray:
230     X = np.column_stack([np.ones(len(yy)), Xu, Xc])
231     lo = np.concatenate([[0.0], np.zeros(Xu.shape[1]), np.full(Xc.shape[1], -np.inf)])
232     hi = np.full(X.shape[1], np.inf)
233     return lsq_linear(X, yy, bounds=(lo, hi), lsqr_tol="auto").x
234
235 def predict(beta: np.ndarray, Xu: np.ndarray, Xc: np.ndarray) -> np.ndarray:
236     return np.column_stack([np.ones(len(Xu)), Xu, Xc]) @ beta
237
238 cv_rows = []
239 best_lambda, best_mae = lambdas[0], np.inf
240 raw = clicks.to_numpy(dtype=float)
241 calm = cal.to_numpy(dtype=float)
242 for lam in lambdas:
243     Xu = adstock(raw, lam)
244     errs = []
245     for o in origins:
246         beta = fit(Xu[:,o], calm[:,o], y[:,o])
247         pred = predict(beta, Xu[:,o], calm[:,o])
248         errs.append(np.abs(pred - y[:,o]).mean())
249     mae = float(np.mean(errs))
250     cv_rows.append({"lambda": lam, "cv_mae": mae})
251     if mae < best_mae:
252         best_lambda, best_mae = lam, mae
253 Xu = adstock(raw, best_lambda)
254 beta = fit(Xu, calm, y)
255 pred = predict(beta, Xu, calm)
256 resid = y - pred
257 r_units = {u: float(beta[1 + i]) for i, u in enumerate(units)}
258 # pooled rate: same regression with a single aggregate clicks regressor
259 pooled_beta = fit(Xu.sum(axis=1, keepdims=True), calm, y)
260 pooled_rate = float(pooled_beta[1])
261 totals = clicks.sum(axis=0)
262 rates: dict[int, float] = {}
263 source: dict[int, str] = {}
264 for u in units:
265     if totals[u] >= min_clicks and band[0] * pooled_rate <= r_units[u] <= band[1] * pooled_rate:
266         rates[u], source[u] = r_units[u], "unit"
267     else:
268         rates[u], source[u] = pooled_rate, "pooled"
269 # holdout diagnostics for the selected lambda (train first 70 %, test last 30 %)
270 o = int(n * 0.7)
271 beta_h = fit(Xu[:,o], calm[:,o], y[:,o])
272 pred_h = predict(beta_h, Xu[:,o], calm[:,o])
273 holdout = {
274     "train_days": o,
275     "test_days": n - o,

```

```

276     "mae": float(np.abs(pred_h - y[o:]).mean()),
277     "mape": float(np.mean(np.abs(pred_h - y[o:]) / np.maximum(y[o:], 1.0))),
278     "r2": float(1 - np.sum((y[o:] - pred_h) ** 2) / np.sum((y[o:] - y[o:].mean()) ** 2)),
279 }
280 return {
281     "lambda": float(best_lambda),
282     "cv": cv_rows,
283     "baseline_per_day": float(beta[0]),
284     "unit_rates": r_units,
285     "pooled_rate": pooled_rate,
286     "rates": rates,
287     "rate_source": source,
288     "calendar_terms": {c: float(beta[1 + len(units) + j]) for j, c in enumerate(cal.columns)},
289     "r2": float(1 - np.sum(resid**2) / np.sum((y - y.mean()) ** 2)),
290     "mae": float(np.abs(resid).mean()),
291     "holdout": holdout,
292     "fitted": pd.DataFrame({"date": daily.index, "regs": y, "fitted": pred}),
293     # attributed with the rates actually used downstream (unit rate or pooled fallback); the raw
294     # NNLS rates are kept for transparency (they are zero for units clipped at the bound)
295     "attributed_regs": {u: float((Xu[:, i] * rates[u]).sum()) for i, u in enumerate(units)},
296     "attributed_regs_raw": {u: float((Xu[:, i] * r_units[u]).sum()) for i, u in enumerate(units)},
297     "baseline_regs": float(beta[0] * n),
298 }

```

pipelines/e/figures.py

```

1  """Paper figures for Problem E (vector PDF + PNG preview, Okabe-Ito palette, CJK-capable fonts)."""
2
3  from __future__ import annotations
4
5  import json
6  from typing import Any
7
8  import numpy as np
9  import pandas as pd
10
11  from forge import plotting
12  from forge.context import StageContext
13  from pipelines.e.calendar import HOLIDAY_BLOCKS, WEEKDAY_NAMES
14  from pipelines.e.classify import LABELS
15
16  # fixed hue per keyword class (identity colour never depends on ordering)
17  LABEL_COLORS = {"黄金词": "#E69F00", "重点词": "#0072B2", "潜力词": "#009E73", "问题词": "#D55E00"}
18  LABEL_COLORS["无效词"] = "#999999"
19  STRATEGY_COLORS = {"最优分配": "#0072B2", "按历史比例": "#D55E00", "均匀分配": "#009E73"}
20  MODEL_COLORS = {"calendar": "#0072B2", "seasonal_naive": "#D55E00", "mean28": "#009E73"}
21  MODEL_NAMES = {"calendar": "日历回归", "seasonal_naive": "季节朴素", "mean28": "28 日均值"}
22
23
24  def _shade_holidays(ax: Any) -> None:
25      for start, end in HOLIDAY_BLOCKS.values():
26          ax.axvspan(pd.Timestamp(start), pd.Timestamp(end) + pd.Timedelta(days=1), color="#CC79A7", alpha=0.18, lw=0)
27
28
29  def fig_daily_series(ctx: StageContext, daily: pd.DataFrame) -> dict[str, Any]:
30      fig, axes = plotting.new_figure(6.3, 5.2, nrow=3, sharex=True)
31      labels = ("日消费额 / 元", "日点击量 / 次", "日新注册数 / 人")
32      for ax, col, label in zip(axes, ("spend", "clicks", "regs"), labels):
33          ax.plot(daily["date"], daily[col], lw=0.8, color="#0072B2", alpha=0.55, label="日值")
34          ax.plot(
35              daily["date"],

```

```

36         daily[col].rolling(7, center=True, min_periods=3).mean(),
37         lw=1.8,
38         color="#0072B2",
39         label="7 日滑动均值",
40     )
41     _shade_holidays(ax)
42     for d in daily.loc[daily["is_adjusted_workday"], "date"]:
43         ax.axvline(d, color="#D55E00", lw=0.8, ls=":")
44     ax.set_ylabel(label)
45 axes[0].legend(loc="upper right", ncol=2)
46 axes[-1].set_xlabel("日期 (2025 年; 阴影 = 法定节假日, 虚线 = 调休上班日)")
47 return plotting.save(fig, ctx.out("figures", "fig_daily_series.pdf"))
48
49
50 def fig_calendar_effects(ctx: StageContext, coefs: pd.DataFrame) -> dict[str, Any]:
51     terms = [f"wd_{w}" for w in WEEKDAY_NAMES if w != "周三"] + [
52         "holiday",
53         "adjusted_workday",
54         "pre_holiday",
55         "post_holiday",
56     ]
57     names = {
58         **{f"wd_{w}": w for w in WEEKDAY_NAMES},
59         "holiday": "法定节假日",
60         "adjusted_workday": "调休上班日",
61         "pre_holiday": "假前一日",
62         "post_holiday": "假后一日",
63     }
64     targets = [("regs", "新注册数", "#0072B2"), ("spend", "消费额", "#D55E00")]
65     targets.append(("clicks", "点击量", "#009E73"))
66     fig, ax = plotting.new_figure(6.3, 3.6)
67     y = np.arange(len(terms))
68     for k, (target, label, color) in enumerate(targets):
69         sub = coefs[coefs["target"] == target].set_index("term").reindex(terms)
70         off = (k - 1) * 0.25
71         ax.errorbar(
72             sub["effect_pct"],
73             y + off,
74             xerr=[sub["effect_pct"] - sub["effect_pct_low"], sub["effect_pct_high"] - sub["effect_pct"]],
75             fmt="o",
76             ms=4,
77             color=color,
78             ecolor=color,
79             elinewidth=1.2,
80             capsize=2,
81             label=label,
82         )
83     ax.axvline(0, color="#444444", lw=0.8)
84     ax.set_yticks(y)
85     ax.set_yticklabels([names[t] for t in terms])
86     ax.set_xlabel("相对周三 / 普通日的效应 (%，HC3 稳健 95% 置信区间)")
87     ax.legend(loc="lower right")
88     ax.invert_yaxis()
89     return plotting.save(fig, ctx.out("figures", "fig_calendar_effects.pdf"))
90
91
92 def fig_unit_kpis(ctx: StageContext, kpis: pd.DataFrame) -> dict[str, Any]:
93     k = kpis.sort_values("spend", ascending=False).reset_index(drop=True)
94     fig, axes = plotting.new_figure(6.3, 3.2, ncols=2, gridspec_kw={"width_ratios": [1.1, 1.0]})
95     ax = axes[0]
96     ax.barh(np.arange(len(k)), 100 * k["spend_share"], color="#0072B2", height=0.62)
97     ax.set_yticks(np.arange(len(k)))
98     ax.set_yticklabels([str(u) for u in k["推广单元ID"]], fontsize=7)

```

```

99     ax.invert_yaxis()
100     ax.set_xlabel("年度消费额占比 / %")
101     for i, v in enumerate(100 * k["spend_share"]):
102         if v >= 3:
103             ax.text(v + 0.5, i, f"{v:.1f}", va="center", fontsize=7, color="#333333")
104     ax = axes[1]
105     size = 12 + 300 * np.sqrt(k["spend_share"])
106     ax.scatter(k["cpc"], 100 * k["ctr"], s=size, color="#0072B2", alpha=0.65, edgecolor="white", lw=0.8)
107     for _, r in k.iterrows():
108         ax.annotate(
109             str(r["推广单元ID"])[-4:],
110             (r["cpc"], 100 * r["ctr"]),
111             fontsize=6,
112             xytext=(3, 3),
113             textcoords="offset points",
114             color="#333333",
115         )
116     ax.set_xlabel("平均点击成本 CPC / 元")
117     ax.set_ylabel("点击率 CTR / %")
118     ax.set_yscale("log")
119     return plotting.save(fig, ctx.out("figures", "fig_unit_kpis.pdf"))
120
121
122 def fig_lorenz(ctx: StageContext, lorenz: pd.DataFrame, conc: dict[str, Any]) -> dict[str, Any]:
123     fig, ax = plotting.new_figure(4.2, 3.4)
124     ax.plot(100 * lorenz["keyword_share"], 100 * lorenz["spend_share"], color="#0072B2", label="消费额")
125     ax.plot(100 * lorenz["keyword_share"], 100 * lorenz["click_share"], color="#D55E00", label="点击量")
126     ax.plot([0, 100], [0, 100], color="#888888", lw=0.8, ls="--", label="均匀分布")
127     ax.set_xlabel("关键词累计占比 / % (按消费额降序)")
128     ax.set_ylabel("累计占比 / %")
129     ax.annotate(
130         f"前 10 个词占 {100 * conc['top10_spend_share']:.1f}% 消费",
131         (100 * 10 / conc["keywords"], 100 * conc["top10_spend_share"]),
132         xytext=(25, 60),
133         textcoords="data",
134         fontsize=8,
135         arrowprops={"arrowstyle": "-", "color": "#555555", "lw": 0.7},
136     )
137     ax.legend(loc="lower right")
138     return plotting.save(fig, ctx.out("figures", "fig_lorenz.pdf"))
139
140
141 def fig_response_curves(ctx: StageContext, ud: pd.DataFrame, units: pd.DataFrame) -> dict[str, Any]:
142     fig, axes = plotting.new_figure(6.3, 6.0, nrows=3, ncols=4)
143     for ax, (_, row) in zip(axes.ravel(), units.sort_values("推广单元ID").iterrows()):
144         unit = int(row["推广单元ID"])
145         d = ud[(ud["推广单元ID"] == unit) & (ud["消费额"] > 0) & (ud["点击量"] > 0)]
146         ax.scatter(d["消费额"], d["点击量"], s=6, color="#0072B2", alpha=0.45, lw=0)
147         xs = np.geomspace(max(d["消费额"].min(), 0.1), d["消费额"].max(), 50)
148         ax.plot(xs, row["mean_daily_clicks"] * (xs / row["mean_daily_spend"]) ** row["gamma"], color="#D55E00", lw=1.4)
149         ax.set_xscale("log")
150         ax.set_yscale("log")
151         ax.set_title(f"{unit}  gamma={row['gamma']:.2f}" + (" if row['gamma_source'] == 'unit' else '*"), fontsize
152             ↳ =7.5)
153         ax.tick_params(labelsize=6)
154         fig.supxlabel("日消费额 / 元 (对数)", fontsize=9)
155         fig.supylabel("日点击量 / 次 (对数)", fontsize=9)
156     return plotting.save(fig, ctx.out("figures", "fig_response_curves.pdf"))
157
158 def fig_attribution(ctx: StageContext, fit: pd.DataFrame, attribution: dict[str, Any]) -> dict[str, Any]:
159     fig, ax = plotting.new_figure(6.3, 2.8)
160     fit["date"] = pd.to_datetime(fit["date"])

```

```

161 ax.plot(fit["date"], fit["regs"], color="#0072B2", lw=1.0, label="实际新注册数")
162 ax.plot(fit["date"], fit["fitted"], color="#D55E00", lw=1.2, label="归因模型拟合")
163 split = fit["date"].iloc[attribution["holdout"]["train_days"]]
164 ax.axvline(split, color="#444444", lw=0.8, ls="--")
165 ax.text(split, ax.get_ylim()[1] * 0.95, " 留出期 →", fontsize=7, va="top")
166 _shade_holidays(ax)
167 ax.set_ylabel("日新注册数 / 人")
168 ax.legend(loc="upper left", ncol=2)
169 return plotting.save(fig, ctx.out("figures", "fig_attribution.pdf"))
170
171
172 def fig_cost_benefit(ctx: StageContext, table: pd.DataFrame, thr: dict[str, float]) -> dict[str, Any]:
173     fig, ax = plotting.new_figure(6.3, 4.0)
174     act = table[table["spend"] > 0]
175     for label in LABELS[:4]:
176         sub = act[act["label"] == label]
177         ax.scatter(
178             sub["spend"],
179             sub["benefit"],
180             s=9,
181             color=LABEL_COLORS[label],
182             alpha=0.65,
183             lw=0,
184             label=f"{label} ({len(sub)}) ",
185         )
186     ax.axvline(thr["cost_yuan"], color="#444444", lw=0.9, ls="--")
187     ax.axhline(thr["benefit"], color="#444444", lw=0.9, ls="--")
188     ax.set_xscale("log")
189     ax.set_xlabel("年度消费额 / 元 (对数轴; 无效词 = 零消费, 未画出)")
190     ax.set_ylabel("综合效益指数")
191     ax.legend(loc="upper left", markerscale=1.8)
192     return plotting.save(fig, ctx.out("figures", "fig_cost_benefit.pdf"))
193
194
195 def fig_class_thresholds(
196     ctx: StageContext, table: pd.DataFrame, thresholds: dict[str, dict[str, float]]
197 ) -> dict[str, Any]:
198     act = table[table["spend"] > 0]
199     fig, axes = plotting.new_figure(6.3, 2.8, ncols=2)
200     styles = {
201         "jenks": ("-", "#0072B2", "自然断点"),
202         "median": ("--", "#D55E00", "中位数"),
203         "gmm": (":", "#009E73", "高斯混合"),
204     }
205     axes[0].hist(act["log10_cost"], bins=40, color="#BBBBBB")
206     axes[1].hist(act["benefit"], bins=40, color="#BBBBBB")
207     for method, (ls, color, name) in styles.items():
208         if method in thresholds:
209             axes[0].axvline(thresholds[method]["log10_cost"], ls=ls, color=color, label=name)
210             axes[1].axvline(thresholds[method]["benefit"], ls=ls, color=color, label=name)
211     axes[0].set_xlabel("log10 年度消费额 / 元")
212     axes[1].set_xlabel("综合效益指数")
213     axes[0].set_ylabel("关键词数")
214     axes[1].legend(loc="upper right")
215     return plotting.save(fig, ctx.out("figures", "fig_class_thresholds.pdf"))
216
217
218 def fig_q3_gain(ctx: StageContext, summary: pd.DataFrame, boot: pd.DataFrame) -> dict[str, Any]:
219     fig, axes = plotting.new_figure(6.3, 3.0, ncols=3, gridspec_kw={"width_ratios": [1.2, 1.2, 0.9]})
220     for ax, (window, title) in zip(axes[:2], (("feb", "2025-02-01 至 02-08"), ("aug", "2025-08-01 至 08-08"))):
221         d = summary[summary["window"] == window].groupby("date")[["opt_regs", "prop_regs", "equal_regs"]].sum()
222         x = np.arange(len(d))
223         ax.plot(x, d["opt_regs"], marker="o", ms=3.5, color=STRATEGY_COLORS["最优分配"], label="最优分配")

```

```

224     prop_color = STRATEGY_COLORS["按历史比例"]
225     ax.plot(x, d["prop_regs"], marker="s", ms=3.5, color=prop_color, label="按历史比例")
226     ax.plot(x, d["equal_regs"], marker="^", ms=3.5, color=STRATEGY_COLORS["均匀分配"], label="均匀分配")
227     ax.set_xticks(x)
228     ax.set_xticklabels([s[-2:] for s in d.index], fontsize=7)
229     ax.set_title(title, fontsize=8.5)
230     ax.set_xlabel("日")
231     axes[0].set_ylabel("预期注册量 / 人")
232     axes[0].legend(loc="upper left", fontsize=7)
233     ax = axes[2]
234     for window, color in (("feb", "#0072B2"), ("aug", "#D55E00")):
235         b = boot[boot["window"] == window]
236         ax.hist(
237             100 * (b["ratio"] - 1),
238             bins=20,
239             color=color,
240             alpha=0.6,
241             label={ "feb": "2 月窗口", "aug": "8 月窗口" }[window],
242         )
243     ax.set_xlabel("相对历史比例的提升 / %")
244     ax.set_ylabel("自助抽样次数")
245     ax.legend(loc="upper right", fontsize=7)
246     return plotting.save(fig, ctx.out("figures", "fig_q3_gain.pdf"))
247
248
249 def fig_allocation_shift(
250     ctx: StageContext, allocation: pd.DataFrame, params: pd.DataFrame, cap_mult: float
251 ) -> dict[str, Any]:
252     """August window: where the optimum moves money -- optimal window-mean daily spend per keyword against its
253     2025 mean daily spend (log-log, class colours), and how many candidates of each class are ever selected."""
254     a = allocation[allocation["window"] == "aug"]
255     p = params[(params["window"] == "aug") & params["eligible"]]
256     n_days = max(a["date"].nunique(), 1)
257     opt = a.groupby("序号").sum() / n_days # zero on days when the keyword is not selected
258     df = p[["序号", "label", "s"]].copy()
259     df["x_opt"] = df["序号"].map(opt).fillna(0.0)
260     labels = list(LABELS[:4])
261     fig, axes = plotting.new_figure(6.3, 3.0, ncols=2, gridspec_kw={"width_ratios": [1.35, 1.0]})
262     ax = axes[0]
263     sel = df[df["x_opt"] > 0]
264     lo = max(min(sel["s"].min(), sel["x_opt"].min()), 1e-3)
265     hi = max(sel["s"].max(), sel["x_opt"].max())
266     ax.plot([lo, hi], [lo, hi], color="#888888", lw=0.8, ls="--", label="投放不变")
267     ax.plot([lo, hi], [cap_mult * lo, cap_mult * hi], color="#888888", lw=0.8, ls=":", label=f"上限 {cap_mult:g} 倍")
268     for label in labels:
269         sub = sel[sel["label"] == label]
270         if len(sub):
271             ax.scatter(
272                 sub["s"], sub["x_opt"], s=9, color=LABEL_COLORS[label], alpha=0.7, lw=0, label=f"{label} ({len(sub)}) "
273             )
274     ax.set_xscale("log")
275     ax.set_yscale("log")
276     ax.set_xlabel("2025 年日均消费 / 元 (对数)")
277     ax.set_ylabel("最优分配的窗口日均投放 / 元 (对数)")
278     ax.legend(loc="upper left", fontsize=6.5)
279     ax = axes[1]
280     counts = df.groupby("label").agg(candidates=("序号", "size"), selected=("x_opt", lambda v: int((v > 0).sum())))
281     counts = counts.reindex(labels).fillna(0)
282     x = np.arange(len(labels))
283     ax.bar(x - 0.2, counts["candidates"], width=0.38, color="#BBBBBB", label="候选词")
284     ax.bar(x + 0.2, counts["selected"], width=0.38, color=[LABEL_COLORS[lab] for lab in labels], label="至少入选一天")
285     for i, (c, s) in enumerate(zip(counts["candidates"], counts["selected"])):
286         ax.text(i - 0.2, c, f"{int(c)}", ha="center", va="bottom", fontsize=6.5, color="#333333")

```

```

287         ax.text(i + 0.2, s, f"{int(s)}", ha="center", va="bottom", fontsize=6.5, color="#333333")
288     ax.set_xticks(x)
289     ax.set_xticklabels(labels, fontsize=7.5)
290     ax.set_ylabel("关键词数 (8 月窗口)")
291     ax.legend(loc="upper right", fontsize=7)
292     return plotting.save(fig, ctx.out("figures", "fig_allocation_shift.pdf"))
293
294
295 def fig_backtest(ctx: StageContext, bt: pd.DataFrame) -> dict[str, Any]:
296     targets = [
297         ("log_eff", "效率乘子"),
298         ("log_cpc", "CPC"),
299         ("logit_ctr", "CTR"),
300         ("logit_top", "上方位占比"),
301         ("log_clicks", "点击量"),
302     ]
303     fig, axes = plotting.new_figure(6.3, 2.7, ncols=2)
304     x = np.arange(len(targets))
305     for k, model in enumerate(("calendar", "seasonal_naive", "mean28")):
306         sub = bt[bt["model"] == model].set_index("target").reindex([t for t, _ in targets])
307         axes[0].bar(x + (k - 1) * 0.27, sub["mae"], width=0.26, color=MODEL_COLORS[model], label=MODEL_NAMES[model])
308         axes[1].bar(x + (k - 1) * 0.27, 100 * sub["coverage80"], width=0.26, color=MODEL_COLORS[model])
309     axes[1].axhline(80, color="#444444", lw=0.9, ls="--")
310     for ax in axes:
311         ax.set_xticks(x)
312         ax.set_xticklabels([n for _, n in targets], fontsize=7)
313     axes[0].set_ylabel("滚动回测 MAE (变换尺度)")
314     axes[1].set_ylabel("80% 区间覆盖率 / %")
315     axes[0].legend(loc="upper left", fontsize=7)
316     return plotting.save(fig, ctx.out("figures", "fig_backtest.pdf"))
317
318
319 def fig_q4_plan(ctx: StageContext, days: pd.DataFrame) -> dict[str, Any]:
320     d = days.groupby("date").agg(
321         clicks=("clicks_mean", "sum"),
322         clicks_lo=("clicks_q10", "sum"),
323         clicks_hi=("clicks_q90", "sum"),
324         regs=("regs_mean", "sum"),
325         regs_lo=("regs_q10", "sum"),
326         regs_hi=("regs_q90", "sum"),
327         spend=("spend", "sum"),
328     )
329     d["cpc"] = d["spend"] / d["clicks"]
330     d["cpc_lo"] = d["spend"] / d["clicks_hi"]
331     d["cpc_hi"] = d["spend"] / d["clicks_lo"]
332     fig, axes = plotting.new_figure(6.3, 2.8, ncols=3)
333     x = np.arange(len(d))
334     for ax, (col, ylabel) in zip(
335         axes, (("clicks", "预期点击量 / 次"), ("regs", "预期注册量 / 人"), ("cpc", "预期 CPC / 元"))
336     ):
337         ax.fill_between(x, d[f"{col}_lo"], d[f"{col}_hi"], color="#0072B2", alpha=0.2, lw=0, label="10%-90% 分位")
338         ax.plot(x, d[col], marker="o", ms=3.5, color="#0072B2", label="期望")
339         ax.set_xticks(x)
340         ax.set_xticklabels([s[-2:] for s in d.index], fontsize=7)
341         ax.set_ylabel(ylabel)
342         ax.set_xlabel("2026 年 9 月")
343     axes[0].legend(loc="upper right", fontsize=7)
344     return plotting.save(fig, ctx.out("figures", "fig_q4_plan.pdf"))
345
346
347 def fig_q4_units(ctx: StageContext, per_unit: list[dict[str, Any]]) -> dict[str, Any]:
348     df = pd.DataFrame(per_unit).sort_values("regs_mean", ascending=True)
349     fig, ax = plotting.new_figure(5.4, 3.2)

```



```

350     y = np.arange(len(df))
351     ax.errorbar(
352         df["regs_mean"],
353         y,
354         xerr=[df["regs_mean"] - df["regs_q10"], df["regs_q90"] - df["regs_mean"]],
355         fmt="o",
356         ms=4,
357         color="#0072B2",
358         ecolor="#0072B2",
359         elinewidth=1.2,
360         capsize=2,
361     )
362     ax.set_yticks(y)
363     ax.set_yticklabels([str(u) for u in df["推广单元ID"]], fontsize=7)
364     ax.set_xlabel("2026-09-11 至 09-17 预期注册量 / 人 (点 = 期望, 横线 = 10%-90% 分位) ")
365     ax.set_xscale("log")
366     return plotting.save(fig, ctx.out("figures", "fig_q4_units.pdf"))
367
368
369 def build_figures(ctx: StageContext) -> dict[str, Any]:
370     eda_dir, cls_dir, alloc_dir, fc_dir = ctx.dep("eda"), ctx.dep("classify"), ctx.dep("allocate"), ctx.dep("forecast")
371     daily = pd.read_parquet(eda_dir / "daily_totals.parquet")
372     daily["date"] = pd.to_datetime(daily["date"])
373     made = [
374         fig_daily_series(ctx, daily),
375         fig_calendar_effects(ctx, pd.read_parquet(eda_dir / "calendar_regression.parquet")),
376         fig_unit_kpis(ctx, pd.read_parquet(eda_dir / "unit_kpis.parquet")),
377         fig_lorenz(
378             ctx,
379             pd.read_parquet(eda_dir / "lorenz.parquet"),
380             json.loads((eda_dir / "concentration.json").read_text(encoding="utf-8")),
381         ),
382         fig_response_curves(
383             ctx, pd.read_parquet(eda_dir / "unit_daily.parquet"), pd.read_parquet(eda_dir / "unit_response.parquet")
384         ),
385         fig_attribution(
386             ctx,
387             pd.read_parquet(eda_dir / "attribution_fit.parquet"),
388             json.loads((eda_dir / "attribution.json").read_text(encoding="utf-8")),
389         ),
390     ]
391     summary = json.loads((cls_dir / "classification_summary.json").read_text(encoding="utf-8"))
392     table = pd.read_parquet(cls_dir / "classification.parquet")
393     made += [
394         fig_cost_benefit(ctx, table, summary["thresholds"][summary["main"]]),
395         fig_class_thresholds(ctx, table, summary["thresholds"]),
396     ]
397     made += [
398         fig_q3_gain(
399             ctx,
400             pd.read_parquet(alloc_dir / "allocation_summary.parquet"),
401             pd.read_parquet(alloc_dir / "bootstrap.parquet"),
402         ),
403         fig_allocation_shift(
404             ctx,
405             pd.read_parquet(alloc_dir / "allocation.parquet"),
406             pd.read_parquet(alloc_dir / "keyword_parameters.parquet"),
407             float(ctx.cfg("allocate.cap_multiplier", 4.0)),
408         ),
409         fig_backtest(ctx, pd.read_parquet(fc_dir / "backtest_summary.parquet")),
410         fig_q4_plan(ctx, pd.read_parquet(fc_dir / "plan_by_unit_day.parquet")),
411         fig_q4_units(ctx, json.loads((fc_dir / "plan_by_unit.json").read_text(encoding="utf-8"))),
412     ]

```

```

413     ctx.write_json("figures_index.json", made)
414     return {"figures": len(made)}

```

pipelines/e/forecast.py

```

1  """Problem 4: unit-level daily forecasts with rolling-origin backtests, scenario generation and Monte Carlo ranges."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import numpy as np
8  import pandas as pd
9  from scipy import stats
10
11 from pipelines.e.calendar import day_features
12
13 TARGETS: dict[str, dict[str, Any]] = {
14     "log_eff": {"label": "效率乘子 log_eff", "transform": "exp"},
15     "log_cpc": {"label": "log CPC", "transform": "exp"},
16     "logit_ctr": {"label": "logit CTR", "transform": "expit"},
17     "logit_top": {"label": "logit 上方位占比", "transform": "expit"},
18     "logit_first": {"label": "logit 首位占比", "transform": "expit"},
19     "log_clicks": {"label": "log 点击量", "transform": "exp"},
20 }
21 Z80 = float(stats.norm.ppf(0.9))
22
23
24 def _logit(p: np.ndarray) -> np.ndarray:
25     p = np.clip(np.asarray(p, dtype=float), 0.005, 0.995)
26     return np.log(p / (1 - p))
27
28
29 def back_transform(values: np.ndarray, transform: str) -> np.ndarray:
30     values = np.asarray(values, dtype=float)
31     return np.exp(values) if transform == "exp" else 1 / (1 + np.exp(-values))
32
33
34 def unit_series(ud_unit: pd.DataFrame, gamma: float, mean_spend: float, mean_clicks: float) -> pd.DataFrame:
35     """Daily targets for one unit on days with positive spend and clicks."""
36     d = ud_unit[(ud_unit["消费额"] > 0) & (ud_unit["点击量"] > 0) & (ud_unit["展现量"] > 0)].copy()
37     d = d.sort_values("日期")
38     spend = d["消费额"].to_numpy(dtype=float)
39     clicks = d["点击量"].to_numpy(dtype=float)
40     out = pd.DataFrame({"date": d["日期"].to_numpy()})
41     out["log_eff"] = np.log(clicks) - gamma * np.log(spend / mean_spend) - np.log(mean_clicks)
42     out["log_cpc"] = np.log(spend / clicks)
43     out["logit_ctr"] = _logit(clicks / d["展现量"].to_numpy(dtype=float))
44     out["logit_top"] = _logit(d["top_share"].to_numpy(dtype=float))
45     out["logit_first"] = _logit(d["first_share"].to_numpy(dtype=float))
46     out["log_clicks"] = np.log(clicks)
47     feats = day_features(out["date"]).drop(columns=["date"])
48     return pd.concat([out.reset_index(drop=True), feats], axis=1)
49
50
51 def _design(feats: pd.DataFrame, months: list[int]) -> np.ndarray:
52     cols = [np.ones(len(feats))]
53     for wd in range(7):
54         if wd != 2:
55             cols.append((feats["weekday"] == wd).to_numpy(dtype=float))
56     for m in months:

```

```

57     cols.append((feats["month"] == m).to_numpy(dtype=float))
58     cols.append(feats["is_holiday"].to_numpy(dtype=float))
59     cols.append(feats["is_adjusted_workday"].to_numpy(dtype=float))
60     return np.column_stack(cols)
61
62
63 class CalendarModel:
64     """Ridge-stabilised least squares of a target on weekday, month and holiday indicators."""
65
66     def __init__(self, ridge: float = 1e-3) -> None:
67         self.ridge = ridge
68         self.months: list[int] = []
69         self.beta = np.zeros(0)
70         self.sd = float("nan")
71         self.month_sd = 0.0
72
73     def fit(self, train: pd.DataFrame, target: str) -> CalendarModel:
74         months = sorted(int(m) for m in train["month"].unique())
75         self.months = [m for m in months if m != months[0]]
76         X = _design(train, self.months)
77         y = train[target].to_numpy(dtype=float)
78         reg = self.ridge * np.eye(X.shape[1])
79         reg[0, 0] = 0.0
80         self.beta = np.linalg.solve(X.T @ X + reg, X.T @ y)
81         resid = y - X @ self.beta
82         dof = max(len(y) - X.shape[1], 1)
83         self.sd = float(np.sqrt(np.sum(resid**2) / dof))
84         n_wd = 6
85         month_effects = self.beta[1 + n_wd : 1 + n_wd + len(self.months)]
86         self.month_sd = float(np.std(np.concatenate([[0.0], month_effects]))) if len(month_effects) else 0.0
87         return self
88
89     def predict(self, feats: pd.DataFrame, month_known: bool = True) -> np.ndarray:
90         f = feats.copy()
91         if not month_known:
92             f["month"] = -1 # reference-month behaviour when the target month never appeared in training
93         return _design(f, self.months) @ self.beta
94
95
96 def seasonal_naive(train: pd.DataFrame, target: str, future: pd.DataFrame) -> np.ndarray:
97     """Last observed value on the same weekday (falls back to the last observation)."""
98     out = np.empty(len(future))
99     last = train[target].to_numpy(dtype=float)[-1]
100     by_wd = train.groupby("weekday")[target].last()
101     for i, wd in enumerate(future["weekday"].to_numpy()):
102         out[i] = float(by_wd.get(int(wd), last))
103     return out
104
105
106 def pinball(y: np.ndarray, q: np.ndarray, tau: float) -> float:
107     diff = y - q
108     return float(np.mean(np.maximum(tau * diff, (tau - 1) * diff)))
109
110
111 def rolling_backtest(
112     series: pd.DataFrame, target: str, origins: list[pd.Timestamp], horizon: int = 7, min_train: int = 40
113 ) -> pd.DataFrame:
114     """Rolling-origin evaluation of the calendar model vs. seasonal-naive vs. 28-day mean."""
115     rows = []
116     info = TARGETS[target]
117     for origin in origins:
118         train = series[series["date"] < origin]
119         test = series[(series["date"] >= origin) & (series["date"] < origin + pd.Timedelta(days=horizon))]

```

```

120     if len(train) < min_train or len(test) == 0:
121         continue
122     y = test[target].to_numpy(dtype=float)
123     y_raw = back_transform(y, info["transform"])
124     preds: dict[str, tuple[np.ndarray, np.ndarray, np.ndarray]] = {}
125     model = CalendarModel().fit(train, target)
126     known = int(test["month"].iloc[0]) in {*model.months, sorted(train["month"].unique())[0]}
127     mu = model.predict(test, month_known=known)
128     preds["calendar"] = (mu, mu - Z80 * model.sd, mu + Z80 * model.sd)
129     sn = seasonal_naive(train, target, test)
130     tr = train[target].to_numpy(dtype=float)
131     sn_err = tr[7:] - tr[:-7] if len(tr) > 7 else np.array([0.0])
132     preds["seasonal_naive"] = (sn, sn + np.quantile(sn_err, 0.1), sn + np.quantile(sn_err, 0.9))
133     m28 = float(tr[-28:].mean())
134     dev = tr[-28:] - m28
135     preds["mean28"] = (np.full(len(y), m28), m28 + np.quantile(dev, 0.1), m28 + np.quantile(dev, 0.9))
136     for name, (p, lo, hi) in preds.items():
137         p_raw = back_transform(p, info["transform"])
138         rows.append(
139             {
140                 "target": target,
141                 "model": name,
142                 "origin": origin,
143                 "n": len(y),
144                 "mae": float(np.mean(np.abs(y - p))),
145                 "rmse": float(np.sqrt(np.mean((y - p) ** 2))),
146                 "mape_raw": float(np.mean(np.abs(y_raw - p_raw) / np.maximum(np.abs(y_raw), 1e-9))),
147                 "pinball10": pinball(y, lo, 0.1),
148                 "pinball90": pinball(y, hi, 0.9),
149                 "coverage80": float(np.mean((y >= lo) & (y <= hi))),
150             }
151         )
152     return pd.DataFrame(rows)
153
154
155 def forecast_unit(series: pd.DataFrame, target: str, future: pd.DataFrame) -> dict[str, Any]:
156     """Point forecast, residual sd and between-month sd for the future days (calendar model on all data)."""
157     model = CalendarModel().fit(series, target)
158     known = all(int(m) in {*model.months, sorted(series["month"].unique())[0]} for m in future["month"].unique())
159     mu = model.predict(future, month_known=known)
160     return {"mean": mu, "sd": model.sd, "month_sd": model.month_sd, "month_known": known, "n": len(series)}
161
162
163 def simulate_group(
164     x: np.ndarray,
165     params: pd.DataFrame,
166     gamma: float,
167     eff_mean: float,
168     eff_sd: float,
169     kw_sd: float,
170     ctr_mean_logit: float,
171     ctr_sd: float,
172     pos_top: dict[str, Any],
173     pos_first: dict[str, Any],
174     top_sd: float,
175     rng: np.random.Generator,
176     n_scen: int = 500,
177 ) -> dict[str, np.ndarray]:
178     """Monte Carlo of clicks, CPC, impressions, position, views and registrations for one unit-day allocation.
179
180     Common day shock (log-normal efficiency), keyword idiosyncratic log-normal noise, CTR and position
181     day shocks, Poisson registrations. Returns arrays of shape (n_scen, n_keywords).
182     """

```

```

183     from pipelines.e.response import expected_position, position_shares
184
185     n = len(x)
186     c = params["c"].to_numpy(dtype=float)
187     s = params["s"].to_numpy(dtype=float)
188     rho = params["rho"].to_numpy(dtype=float)
189     depth = params["depth"].to_numpy(dtype=float)
190     base = c * np.power(np.maximum(x, 0.0) / s, gamma)
191     # eff_mean is the calendar model's conditional mean of log eff, so the day multiplier is
192     # log-normal with mean exp(eff_mean + eff_sd^2 / 2) -- the same expectation the allocation uses;
193     # the keyword noise is a mean-one multiplicative disturbance around that expectation
194     day = np.exp(eff_mean + eff_sd * rng.standard_normal((n_scen, 1)))
195     idio = np.exp(kw_sd * rng.standard_normal((n_scen, n)) - 0.5 * kw_sd**2)
196     clicks = base[None, :] * day * idio
197     with np.errstate(divide="ignore", invalid="ignore"):
198         cpc = np.where(clicks > 0, x[None, :] / clicks, np.nan)
199     ctr = 1 / (1 + np.exp(-(ctr_mean_logit + ctr_sd * rng.standard_normal((n_scen, 1)))))
200     imp = clicks / ctr
201     p_top, p_first = position_shares(np.nan_to_num(cpc, nan=1.0), pos_top, pos_first)
202     shock = top_sd * rng.standard_normal((n_scen, 1))
203     p_top = 1 / (1 + np.exp(-(np.log(p_top / (1 - p_top)) + shock)))
204     p_first = np.minimum(p_first, p_top)
205     position = expected_position(p_top, p_first)
206     views = clicks * depth[None, :]
207     regs = rng.poisson(np.maximum(rho[None, :] * clicks, 0.0)).astype(float)
208     return {
209         "clicks": clicks,
210         "cpc": cpc,
211         "imp": imp,
212         "position": position,
213         "views": views,
214         "regs": regs,
215         "p_top": p_top,
216     }
217
218
219 def summarise(samples: np.ndarray, axis: int = 0) -> dict[str, np.ndarray]:
220     return {
221         "mean": np.nanmean(samples, axis=axis),
222         "q10": np.nanquantile(samples, 0.1, axis=axis),
223         "q50": np.nanquantile(samples, 0.5, axis=axis),
224         "q90": np.nanquantile(samples, 0.9, axis=axis),
225     }

```

pipelines/e/response.py

```

1  """Keyword response model (concave power law calibrated per unit), position model and keyword parameters."""
2
3  from __future__ import annotations
4
5  from typing import Any
6
7  import numpy as np
8  import pandas as pd
9  import statsmodels.api as sm
10
11  GAMMA_BOUNDS = (0.3, 0.95)
12  MIN_DAYS_FOR_UNIT_GAMMA = 60
13  POSITION_NON_TOP = 4.0 # ranking index assigned to impressions outside the top region (positions > 3)
14  POSITION_TOP_OTHER = 2.5 # mean ranking of top-region impressions that are not in first place
15

```

```

16
17 def fit_unit_gamma(df: pd.DataFrame) -> dict[str, Any]:
18     """log clicks = const + gamma * log spend + weekday dummies, on days with positive spend and clicks."""
19     d = df[(df["消费额"] > 0) & (df["点击量"] > 0)]
20     n = len(d)
21     if n < 10:
22         return {"n": n, "gamma": np.nan, "se": np.nan, "r2": np.nan}
23     X = pd.DataFrame({"const": 1.0, "log_spend": np.log(d["消费额"].to_numpy(dtype=float))}, index=d.index)
24     for wd in range(1, 7):
25         X[f"wd{wd}"] = (d["weekday"] == wd).astype(float)
26     fit = sm.OLS(np.log(d["点击量"].to_numpy(dtype=float)), X).fit(cov_type="HC3")
27     return {
28         "n": int(n),
29         "gamma": float(fit.params["log_spend"]),
30         "se": float(fit.bse["log_spend"]),
31         "r2": float(fit.rsquared),
32     }
33
34
35 def pooled_gamma(ud: pd.DataFrame) -> dict[str, Any]:
36     """Same regression pooled over units with unit fixed effects."""
37     d = ud[(ud["消费额"] > 0) & (ud["点击量"] > 0)]
38     X = pd.DataFrame({"const": 1.0, "log_spend": np.log(d["消费额"].to_numpy(dtype=float))}, index=d.index)
39     units = sorted(d["推广单元ID"].unique())
40     for u in units[1:]:
41         X[f"u{u}"] = (d["推广单元ID"] == u).astype(float)
42     for wd in range(1, 7):
43         X[f"wd{wd}"] = (d["weekday"] == wd).astype(float)
44     fit = sm.OLS(np.log(d["点击量"].to_numpy(dtype=float)), X).fit(cov_type="HC3")
45     return {
46         "n": len(d),
47         "gamma": float(fit.params["log_spend"]),
48         "se": float(fit.bse["log_spend"]),
49         "r2": float(fit.rsquared),
50     }
51
52
53 def select_gamma(est: dict[str, Any], pooled: float) -> tuple[float, str]:
54     """Unit estimate when well identified and inside the concave range, else the pooled elasticity."""
55     g = est.get("gamma", np.nan)
56     if est.get("n", 0) >= MIN_DAYS_FOR_UNIT_GAMMA and np.isfinite(g) and GAMMA_BOUNDS[0] <= g <= GAMMA_BOUNDS[1]:
57         return float(g), "unit"
58     return float(np.clip(pooled, *GAMMA_BOUNDS)), "pooled"
59
60
61 def unit_response_table(ud: pd.DataFrame) -> pd.DataFrame:
62     """Per-unit elasticity table (estimate, SE, n, R2, selected gamma and its source)."""
63     pooled = pooled_gamma(ud)
64     rows = []
65     for (plan, unit), df in ud.groupby(["方案ID", "推广单元ID"]):
66         est = fit_unit_gamma(df)
67         gamma, source = select_gamma(est, pooled["gamma"])
68         active = df[df["消费额"] > 0]
69         rows.append(
70             {
71                 "方案ID": int(plan),
72                 "推广单元ID": int(unit),
73                 "n_days": est["n"],
74                 "gamma_hat": est["gamma"],
75                 "gamma_se": est["se"],
76                 "r2": est["r2"],
77                 "gamma": gamma,
78                 "gamma_source": source,

```

```

79         "active_days": len(active),
80         "mean_daily_spend": float(active["消费额"].mean()) if len(active) else 0.0,
81         "mean_daily_clicks": float(active["点击量"].mean()) if len(active) else 0.0,
82         "p95_spend_ratio": float(active["消费额"].quantile(0.95) / max(active["消费额"].mean(), 1e-9))
83         if len(active)
84         else 1.0,
85         "pooled_gamma": pooled["gamma"],
86         "pooled_se": pooled["se"],
87     }
88 )
89 return pd.DataFrame(rows)
90
91
92 def clicks_response(x: np.ndarray, c: np.ndarray, s: np.ndarray, gamma: float) -> np.ndarray:
93     """Expected clicks at spend ``x`` for keywords with mean daily spend ``s`` and clicks ``c``:  $c (x/s)^\gamma$ """
94     x = np.asarray(x, dtype=float)
95     return c * np.power(np.maximum(x, 0.0) / s, gamma)
96
97
98 def cpc_response(x: np.ndarray, c: np.ndarray, s: np.ndarray, gamma: float) -> np.ndarray:
99     """Average CPC at spend ``x`` (rises with spend when  $\gamma < 1$ )."""
100     clicks = clicks_response(x, c, s, gamma)
101     with np.errstate(divide="ignore", invalid="ignore"):
102         return np.where(clicks > 0, np.asarray(x, dtype=float) / clicks, np.nan)
103
104
105 def _logit(p: np.ndarray) -> np.ndarray:
106     p = np.clip(p, 0.005, 0.995)
107     return np.log(p / (1 - p))
108
109
110 def fit_position_model(df: pd.DataFrame, share_col: str, min_imp: int = 30) -> dict[str, Any]:
111     """logit(share) = a + b log CPC on unit-days (impression-weighted); slope forced non-negative."""
112     d = df[(df["展现量"] >= min_imp) & (df["点击量"] > 0) & (df["消费额"] > 0)]
113     mean_share = float((df[share_col] * df["展现量"]).sum() / max(df["展现量"].sum(), 1.0)) if len(df) else 0.0
114     if len(d) < 20:
115         return {
116             "a": float(_logit(np.array([mean_share]))[0]),
117             "b": 0.0,
118             "n": len(d),
119             "r2": np.nan,
120             "mean_share": mean_share,
121             "source": "constant",
122         }
123     y = _logit(d[share_col].to_numpy(dtype=float))
124     X = sm.add_constant(np.log(d["cpc"].to_numpy(dtype=float)))
125     fit = sm.WLS(y, X, weights=np.sqrt(d["展现量"].to_numpy(dtype=float))).fit()
126     a, b = float(fit.params[0]), float(fit.params[1])
127     if b < 0: # a negative slope has no causal reading (bid up -> lower position); use the constant model
128         return {
129             "a": float(_logit(np.array([mean_share]))[0]),
130             "b": 0.0,
131             "n": len(d),
132             "r2": float(fit.rsquared),
133             "mean_share": mean_share,
134             "source": "constant",
135         }
136     return {"a": a, "b": b, "n": len(d), "r2": float(fit.rsquared), "mean_share": mean_share, "source": "logit"}
137
138
139 def position_shares(
140     cpc: np.ndarray, model_top: dict[str, Any], model_first: dict[str, Any]
141 ) -> tuple[np.ndarray, np.ndarray]:

```

```

142     """Expected top-region and first-place impression shares at a given CPC."""
143     lc = np.log(np.maximum(np.asarray(cpc, dtype=float), 1e-6))
144     p_top = 1 / (1 + np.exp(-(model_top["a"] + model_top["b"] * lc)))
145     p_first = 1 / (1 + np.exp(-(model_first["a"] + model_first["b"] * lc)))
146     p_first = np.minimum(p_first, p_top)
147     return p_top, p_first
148
149
150 def expected_position(p_top: np.ndarray, p_first: np.ndarray) -> np.ndarray:
151     """Expected ranking index: 1 for first place, 2.5 for other top-region slots, 4 outside the top region."""
152     return 1.0 * p_first + POSITION_TOP_OTHER * (p_top - p_first) + POSITION_NON_TOP * (1.0 - p_top)
153
154
155 def keyword_parameters(
156     kw: pd.DataFrame,
157     units: pd.DataFrame,
158     rates: dict[int, float],
159     kappa_bounds: tuple[float, float] = (0.25, 2.5),
160 ) -> pd.DataFrame:
161     """Per-keyword response parameters for active keywords (spend > 0).
162
163     s_i, c_i: mean daily spend / clicks over the unit's active days; depth_i views per click (unit
164     fallback); kappa_i bounce-quality multiplier normalised to the unit's click-weighted mean bounce;
165     rho_i = r_u * kappa_i expected registrations per click.
166     """
167     u = units.set_index("推广单元ID")
168     active = kw[kw["spend"] > 0].copy()
169     days = active["推广单元ID"].map(u["active_days"]).astype(float)
170     active["s"] = active["spend"] / days
171     active["c"] = active["clicks"] / days
172     active["gamma"] = active["推广单元ID"].map(u["gamma"]).astype(float)
173     # keywords with clicks but no recorded page views (bounce reported as 0) are tracking failures:
174     # both engagement fields are treated as missing and fall back to the unit's click-weighted means.
175     untracked = (active["views"] <= 0) & (active["clicks"] > 0)
176     active.loc[untracked, "depth"] = np.nan
177     active.loc[untracked, "bounce"] = np.nan
178     tracked = active[~untracked]
179     unit_depth = tracked.groupby("推广单元ID").apply(
180         lambda d: d["views"].sum() / max(d["clicks"].sum(), 1), include_groups=False
181     )
182     active["depth"] = active["depth"].fillna(active["推广单元ID"].map(unit_depth)).fillna(1.0)
183     unit_bounce = tracked.groupby("推广单元ID").apply(
184         lambda d: float(np.average(d["bounce"].fillna(1.0), weights=np.maximum(d["clicks"], 1))), include_groups=False
185     )
186     b_i = active["bounce"].fillna(active["推广单元ID"].map(unit_bounce)).fillna(0.5)
187     kappa = (1.0 - b_i) / (1.0 - active["推广单元ID"].map(unit_bounce)).clip(lower=0.02)
188     active["kappa"] = kappa.clip(*kappa_bounds).fillna(1.0)
189     active["rate"] = active["推广单元ID"].map(rates).astype(float)
190     active["rho"] = active["rate"] * active["kappa"]
191     active["cpc"] = active["spend"] / active["clicks"]
192     return active

```

pipelines/e/stages.py

```

1 """Problem E stages: validate, eda (Q1), classify (Q2), allocate (Q3), forecast (Q4), results, figures, tables.
2
3 Conventions (see docs/ENGINEERING_STANDARD.md):
4 * every stage is ``@stage(name, deps=(...))`` and returns a JSON-serialisable metrics dict;
5 * outputs go only to ``ctx.out(...)``; paper numbers via ``ctx.number("Key", value)``;
6 * ``results`` writes result*.xlsx with ``forge.xlsx.write_result`` from the organisers' templates;
7 * every forecast carries a backtest and every allocation an independent optimality audit.

```



```

8  """
9
10 from __future__ import annotations
11
12 from typing import Any
13
14 import numpy as np
15 import pandas as pd
16
17 from forge import xlsx
18 from forge.context import StageContext
19 from forge.runner import stage
20 from pipelines.common.validation import run_validation
21 from pipelines.e import allocate as alloc
22 from pipelines.e import classify as cls
23 from pipelines.e import eda as eda_mod
24 from pipelines.e import forecast as fc
25 from pipelines.e import response as resp
26 from pipelines.e.calendar import day_features
27 from pipelines.e.contracts import INPUT_CONTRACTS
28 from pipelines.e.data import daily_totals, keyword_table, load_sheets, unit_click_matrix, unit_daily
29
30 Q3_WINDOWS: dict[str, tuple[str, str]] = {"feb": ("2025-02-01", "2025-02-08"), "aug": ("2025-08-01", "2025-08-08")}
31 Q4_DATES = ("2026-09-11", "2026-09-17")
32 Q4_REFERENCE = ("2025-09-11", "2025-09-17")
33 RESULT_HEADER = [
34     "日期",
35     "方案ID",
36     "推广单元",
37     "关键词",
38     "投入金额",
39     "预期展位",
40     "预期点击量",
41     "预期浏览量",
42     "预期注册量",
43 ]
44
45
46 @stage("validate", deps=("ingest",), description="Validate the three SEM sheets against their contracts")
47 def validate(ctx: StageContext) -> dict[str, Any]:
48     return run_validation(ctx, INPUT_CONTRACTS)
49
50
51 def _save(ctx: StageContext, df: pd.DataFrame, name: str) -> None:
52     df.to_parquet(ctx.out(f"{name}.parquet"), index=False)
53     df.to_csv(ctx.out(f"{name}.csv"), index=False)
54
55
56 # ----- Q1
57 @stage(
58     "eda",
59     deps=("validate",),
60     description="Q1: KPIs, keyword concentration, calendar/holiday effects, registration attribution",
61 )
62 def eda(ctx: StageContext) -> dict[str, Any]:
63     ctx.seed_everything("eda")
64     s1, s2, s3 = load_sheets(ctx.dep("ingest"))
65     ud = unit_daily(s1)
66     daily = daily_totals(s1, s2)
67     kw = keyword_table(s3)
68     _save(ctx, ud, "unit_daily")
69     _save(ctx, daily.reset_index(), "daily_totals")
70

```

```

71 kpis = eda_mod.unit_kpis(s1)
72 _save(ctx, kpis, "unit_kpis")
73 conc = eda_mod.keyword_concentration(kw)
74 _save(ctx, conc.pop("lorenz"), "lorenz")
75 kw_per_unit = conc.pop("per_unit")
76 _save(ctx, kw_per_unit, "keyword_per_unit")
77 ctx.write_json("concentration.json", conc)
78
79 coef_frames, summaries = [], []
80 specs: list[tuple[str, bool, tuple[str, ...]]] = [
81     ("regs", True, ()),
82     ("regs", True, ("spend",)),
83     ("spend", True, ()),
84     ("clicks", True, ()),
85     ("imp", True, ()),
86     ("cpc", False, ()),
87     ("ctr", False, ()),
88     ("top_share", False, ()),
89 ]
90 for target, log, controls in specs:
91     coef, summary = eda_mod.calendar_regression(daily, target, log, controls)
92     coef_frames.append(coef)
93     summaries.append(summary)
94 coefs = pd.concat(coef_frames, ignore_index=True)
95 _save(ctx, coefs, "calendar_regression")
96 nonparam = [eda_mod.holiday_nonparametric(daily, t) for t in ("regs", "spend", "clicks")]
97 ctx.write_json("calendar_regression_summary.json", {"models": summaries, "mann_whitney": nonparam})
98 _save(
99     ctx,
100     eda_mod.day_type_profile(daily, ("spend", "clicks", "imp", "regs", "cpc", "ctr", "top_share")),
101     "day_type_profile",
102 )
103
104 attribution = eda_mod.attribution_model(unit_click_matrix(s1), daily)
105 _save(ctx, attribution.pop("fitted"), "attribution_fit")
106 ctx.write_json("attribution.json", attribution)
107 units = resp.unit_response_table(ud)
108 _save(ctx, units, "unit_response")
109
110 total_spend = float(s1["消费额"].sum())
111 total_clicks, total_imp = int(s1["点击量"].sum()), int(s1["展现量"].sum())
112 ctx.number("TotalSpend", total_spend, ".2f")
113 ctx.number("TotalSpendWan", total_spend / 1e4, ".2f")
114 ctx.number("TotalClicks", total_clicks, ",d")
115 ctx.number("TotalImpressions", total_imp, ",d")
116 ctx.number("TotalRegs", int(daily["regs"].sum()), ",d")
117 ctx.number("OverallCtrPct", 100 * total_clicks / total_imp, ".2f")
118 ctx.number("OverallCpc", total_spend / total_clicks, ".4f")
119 ctx.number("TopSharePct", 100 * s1["上方位展现量"].sum() / total_imp, ".2f")
120 ctx.number("FirstSharePct", 100 * s1["上方首位展现量"].sum() / total_imp, ".2f")
121 ctx.number("TopClickSharePct", 100 * s1["上方位点击量"].sum() / total_clicks, ".2f")
122 ctx.number("TopSpendSharePct", 100 * s1["上方位消费额"].sum() / total_spend, ".2f")
123 ctx.number("TopCpc", s1["上方位消费额"].sum() / s1["上方位点击量"].sum(), ".4f")
124 other_clicks = total_clicks - s1["上方位点击量"].sum()
125 ctx.number("OtherCpc", (total_spend - s1["上方位消费额"].sum()) / other_clicks, ".4f")
126 top_imp = s1["上方位展现量"].sum()
127 ctx.number("TopRegionCtrPct", 100 * s1["上方位点击量"].sum() / top_imp, ".2f")
128 ctx.number("OtherRegionCtrPct", 100 * other_clicks / (total_imp - top_imp), ".2f")
129 ctx.number("UnitDays", len(s1))
130 ctx.number("UnitsCount", int(s1["推广单元ID"].nunique()))
131 ctx.number("PlansCount", int(s1["方案ID"].nunique()))
132 ctx.number("ZeroSpendUnitDays", int((s1["消费额"] == 0).sum()))
133 ctx.number("KeywordRows", conc["keywords"])

```

```

134 ctx.number("UniqueKeywordIds", conc["unique_keyword_ids"])
135 ctx.number("SharedKeywordIds", conc["shared_keyword_ids"])
136 ctx.number("ActiveKeywords", conc["active_keywords"])
137 ctx.number("ZeroKeywords", conc["zero_cost_zero_benefit"])
138 ctx.number("ZeroKeywordSharePct", 100 * conc["zero_cost_zero_benefit"] / conc["keywords"], ".1f")
139 ctx.number("SharedKeywordRows", int((kw["shared_units"] > 1).sum()))
140 ctx.number("GiniSpendActive", conc["gini_spend_active"], ".4f")
141 ctx.number("TopTenSpendSharePct", 100 * conc["top10_spend_share"], ".2f")
142 ctx.number("TopHundredSpendSharePct", 100 * conc["top100_spend_share"], ".2f")
143 big = kpis.sort_values("spend", ascending=False).iloc[0]
144 ctx.number("BiggestUnitSpendSharePct", 100 * big["spend_share"], ".2f")
145 ctx.number("BiggestUnitId", int(big["推广单元ID"]))
146 ctx.number("MaxUnitCpc", float(kpis["cpc"].max()), ".2f")
147 ctx.number("MinUnitCpc", float(kpis["cpc"].min()), ".2f")
148 weak = kpis.sort_values("ctr", ascending=True).iloc[0]
149 ctx.number("MinCtrUnitId", int(weak["推广单元ID"]))
150 ctx.number("MinCtrUnitCtrPct", 100 * weak["ctr"], ".2f")
151 brand = kpis.sort_values("ctr", ascending=False).iloc[0] # navigational/brand-type unit: extreme CTR
152 ctx.number("TopCtrUnitId", int(brand["推广单元ID"]))
153 ctx.number("TopCtrUnitCtrPct", 100 * brand["ctr"], ".1f")
154 ctx.number("TopCtrUnitCpc", brand["cpc"], ".2f")
155 ctx.number("TopCtrUnitTopSharePct", 100 * brand["top_share"], ".1f")
156 ctx.number(
157     "TopCtrUnitKeywords", int(kw_per_unit.loc[kw_per_unit["推广单元ID"] == brand["推广单元ID"], "keywords"].iloc[0])
158 )
159
160 def term(target: str, name: str) -> pd.Series:
161     return coefs[(coefs["target"] == target) & (coefs["term"] == name)].iloc[0]
162
163 for target, key in (("regs", "Reg"), ("spend", "Spend"), ("clicks", "Clicks"), ("imp", "Imp")):
164     row = term(target, "holiday")
165     ctx.number(f"Holiday{key}EffectPct", row["effect_pct"], ".2f")
166     ctx.number(f"Holiday{key}CiLow", row["effect_pct_low"], ".2f")
167     ctx.number(f"Holiday{key}CiHigh", row["effect_pct_high"], ".2f")
168     ctx.number(f"Holiday{key}P", row["p"], ".4f")
169     ctx.number(f"Holiday{key}PSci", row["p"], ".1e")
170 for name, key in (
171     ("adjusted_workday", "AdjustedWorkday"),
172     ("pre_holiday", "PreHoliday"),
173     ("post_holiday", "PostHoliday"),
174     ("wd_周一", "Monday"),
175     ("wd_周六", "Saturday"),
176     ("wd_周日", "Sunday"),
177 ):
178     row = term("regs", name)
179     ctx.number(f"{key}RegEffectPct", row["effect_pct"], ".2f")
180     ctx.number(f"{key}RegP", row["p"], ".4f")
181 for name, key in ("holiday", "Holiday"), ("wd_周一", "Monday")):
182     row = term("cpc", name)
183     ctx.number(f"{key}CpcEffect", row["estimate"], ".4f")
184     ctx.number(f"{key}CpcP", row["p"], ".4f")
185 ctrl = term("regs|spend", "holiday")
186 ctx.number("HolidayRegGivenSpendEffectPct", ctrl["effect_pct"], ".2f")
187 ctx.number("HolidayRegGivenSpendCiLow", ctrl["effect_pct_low"], ".2f")
188 ctx.number("HolidayRegGivenSpendCiHigh", ctrl["effect_pct_high"], ".2f")
189 ctx.number("HolidayRegGivenSpendP", ctrl["p"], ".4f")
190 # marginal-return equalisation under the log-log response: optimal holiday/normal-day spend ratio
191 #  $\exp(\Delta_H / (1 - \theta))$ , where  $\Delta_H$  is the demand-side holiday effect and  $\theta$  the elasticity
192 theta = float(term("regs|spend", "log_spend")["estimate"])
193 holiday_ratio = float(np.exp(float(ctrl["estimate"]) / max(1.0 - theta, 1e-6)))
194 ctx.number("HolidayOptimalSpendRatioPct", 100 * holiday_ratio, ".1f")
195 ctx.number("HolidayOptimalSpendCutPct", 100 * (1 - holiday_ratio), ".1f")
196 ctx.number("MondayRegGivenSpendEffectPct", term("regs|spend", "wd_周一")["effect_pct"], ".2f")

```

```

197 ctx.number("MondayRegGivenSpendP", term("regs|spend", "wd_周一")["p"], ".4f")
198 ctx.number("SundayRegGivenSpendEffectPct", term("regs|spend", "wd_周日")["effect_pct"], ".2f")
199 ctx.number("SundayRegGivenSpendP", term("regs|spend", "wd_周日")["p"], ".4f")
200 ctx.number("RegSpendElasticity", term("regs|spend", "log_spend")["estimate"], ".4f")
201 ctx.number("RegSpendElasticitySe", term("regs|spend", "log_spend")["se"], ".4f")
202 ctx.number("HolidayTopEffectPp", 100 * term("top_share", "holiday")["estimate"], ".2f")
203 ctx.number("HolidayTopP", term("top_share", "holiday")["p"], ".4f")
204 ctx.number("HolidayCtrEffectPp", 100 * term("ctr", "holiday")["estimate"], ".3f")
205 ctx.number("HolidayCtrP", term("ctr", "holiday")["p"], ".4f")
206 reg_summary = next(s for s in summaries if s["target"] == "regs")
207 ctx.number("RegCalendarRsqr", reg_summary["r2"], ".4f")
208 ctx.number("RegWeekdayWaldP", reg_summary["weekday_wald_p"], ".2e")
209 ctx.number("SpendCalendarRsqr", next(s for s in summaries if s["target"] == "spend")["r2"], ".4f")
210 ctx.number("RegGivenSpendRsqr", next(s for s in summaries if s["target"] == "regs|spend")["r2"], ".4f")
211 mw = next(m for m in nonparam if m["target"] == "regs")
212 ctx.number("HolidayRegMedianRatio", mw["ratio_of_medians"], ".4f")
213 ctx.number("HolidayRegMannWhitneyP", mw["p"], ".2e")
214 ctx.number("HolidayRegMedian", mw["holiday_median"], ".0f")
215 ctx.number("WorkdayRegMedian", mw["workday_median"], ".0f")
216 ctx.number("HolidayDays", mw["holiday_n"])
217 ctx.number("WorkdayDays", mw["workday_n"])
218 ctx.number("AttributionLambda", attribution["lambda"], ".1f")
219 ctx.number("AttributionRsqr", attribution["r2"], ".4f")
220 ctx.number("AttributionHoldoutRsqr", attribution["holdout"]["r2"], ".4f")
221 ctx.number("AttributionHoldoutMape", 100 * attribution["holdout"]["mape"], ".2f")
222 ctx.number("BaselineRegsPerDay", attribution["baseline_per_day"], ".2f")
223 ctx.number("PooledRegRatePerHundredClicks", 100 * attribution["pooled_rate"], ".4f")
224 attributed = sum(attribution["attributed_regs"].values())
225 ctx.number("AttributedRegs", attributed, ".0f")
226 ctx.number("AttributedRegsSharePct", 100 * attributed / float(daily["regs"].sum()), ".2f")
227 ctx.number("CostPerAttributedReg", total_spend / attributed, ".2f")
228 ctx.number("UnitsWithOwnRate", sum(1 for v in attribution["rate_source"].values() if v == "unit"))
229 ctx.number("PooledGamma", units["pooled_gamma"].iloc[0], ".4f")
230 ctx.number("PooledGammaSe", units["pooled_se"].iloc[0], ".4f")
231 ctx.number("MinUnitGamma", units["gamma"].min(), ".4f")
232 ctx.number("MaxUnitGamma", units["gamma"].max(), ".4f")
233 ctx.number("UnitsWithOwnGamma", int((units["gamma_source"] == "unit").sum()))
234 ctx.number("MinUnitGammaRsqr", float(units["r2"].min()), ".2f")
235 ctx.number("MaxUnitGammaRsqr", float(units["r2"].max()), ".2f")
236 return {
237     "unit_days": len(s1),
238     "holiday_reg_effect_pct": float(term("regs", "holiday")["effect_pct"]),
239     "holiday_reg_p": float(term("regs", "holiday")["p"]),
240     "attribution_r2": attribution["r2"],
241     "pooled_gamma": float(units["pooled_gamma"].iloc[0]),
242 }
243
244
245 def _rates(ctx: StageContext) -> dict[int, float]:
246     attribution = ctx.dep_data("eda", "attribution.json")
247     return {int(k): float(v) for k, v in attribution["rates"].items()}
248
249
250 # ----- Q2
251 @stage(
252     "classify", deps=("eda",), description="Q2: five-way cost/benefit keyword classification with threshold robustness"
253 )
254 def classify(ctx: StageContext) -> dict[str, Any]:
255     seed = ctx.seed_everything("classify")
256     _, _, s3 = load_sheets(ctx.dep("ingest"))
257     kw = keyword_table(s3)
258     methods = tuple(ctx.cfg("classify.methods", ["jenks", "median", "gmm"]))
259     main = str(ctx.cfg("classify.main_method", "jenks"))

```

```

260 result = cls.classify_keywords(kw, _rates(ctx), methods, main, seed)
261 table = result.pop("table")
262 _save(ctx, table, "classification")
263 thr = result["thresholds"][main]
264 report = cls.verify_classification(table, thr["log10_cost"], thr["benefit"])
265 ctx.write_json("classification_check.json", report)
266 per_unit = (
267     table.groupby(["方案ID", "推广单元ID", "label"])
268     .agg(n=("序号", "size"), spend=("spend", "sum"), clicks=("clicks", "sum"))
269     .reset_index()
270 )
271 _save(ctx, per_unit, "classification_per_unit")
272 by_label = (
273     table.groupby("label")
274     .agg(
275         n=("序号", "size"),
276         spend=("spend", "sum"),
277         clicks=("clicks", "sum"),
278         views=("views", "sum"),
279         reg_attr=("reg_attr", "sum"),
280     )
281     .reindex(cls.LABELS)
282     .fillna(0)
283 )
284 by_label["spend_share"] = by_label["spend"] / by_label["spend"].sum()
285 by_label["cpc"] = by_label["spend"] / by_label["clicks"].replace(0, np.nan)
286 _save(ctx, by_label.reset_index(), "classification_by_label")
287 ctx.write_json("classification_summary.json", result)
288 if not report["ok"]:
289     raise RuntimeError(f"classification checker failed: {report}")
290 keys = {"黄金词": "Gold", "重点词": "Key", "潜力词": "Potential", "问题词": "Problem"}
291 keys["无效词"] = "Invalid"
292 for label, key in keys.items():
293     ctx.number(f"{key}Count", int(by_label.loc[label, "n"]))
294     ctx.number(f"{key}SpendSharePct", 100 * float(by_label.loc[label, "spend_share"]), ".2f")
295     ctx.number(f"{key}Cpc", float(by_label.loc[label, "cpc"]) if by_label.loc[label, "clicks"] > 0 else 0.0, ".4f")
296 ctx.number("CostThresholdYuan", thr["cost_yuan"], ".2f")
297 ctx.number("CostThresholdLog", thr["log10_cost"], ".4f")
298 ctx.number("BenefitThreshold", thr["benefit"], ".4f")
299 for comp, key in (("clicks", "Clicks"), ("views", "Views"), ("engagement", "Engagement"), ("reg_attr", "Reg")):
300     ctx.number(f"Weight{key}", result["weights"][comp], ".4f")
301 ctx.number("AgreementJenksMedianPct", 100 * result["agreement"]["jenks_vs_median"], ".2f")
302 ctx.number("AgreementGmmJenksPct", 100 * result["agreement"]["gmm_vs_jenks"], ".2f")
303 ctx.number("AgreementJenksEqualWeightsPct", 100 * result["agreement"]["jenks_vs_equal_weights"], ".2f")
304 ctx.number("EntropyEqualSpearman", result["agreement"]["entropy_vs_equal_spearman"], ".4f")
305 ctx.number("MedianCostThresholdYuan", result["thresholds"]["median"]["cost_yuan"], ".2f")
306 ctx.number("GmmCostThresholdYuan", result["thresholds"]["gmm"]["cost_yuan"], ".2f")
307 return {"counts": result["counts"][main], "checker_ok": report["ok"], "thresholds": thr}
308
309
310 # ----- Q3
311 def _load_params(
312     ctx: StageContext,
313 ) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame, dict[int, dict[str, Any]], dict[int, float]]:
314     s1, _, s3 = load_sheets(ctx.dep("ingest"))
315     ud = unit_daily(s1)
316     units = pd.read_parquet(ctx.dep("eda") / "unit_response.parquet")
317     rates = _rates(ctx)
318     kw = keyword_table(s3)
319     labels = pd.read_parquet(ctx.dep("classify") / "classification.parquet")[["序号", "label"]]
320     params = resp.keyword_parameters(kw, units, rates).merge(labels, on="序号", how="left")
321     params["efficiency"] = params["rho"] / params["cpc"]
322     pos: dict[int, dict[str, Any]] = {}

```

```

323     for unit, df in ud.groupby("推广单元ID"):
324         pos[int(unit)] = {
325             "top": resp.fit_position_model(df, "top_share"),
326             "first": resp.fit_position_model(df, "first_share"),
327         }
328     return ud, units, params, pos, rates
329
330
331 def _eligible(params: pd.DataFrame, active_units: set[int], problem_cap_factor: float) -> pd.DataFrame:
332     """Eligibility inside a window: invalid words never enter, problem words keep half the normal cap.
333
334     Keyword IDs shared by several active units are flagged (`dup_best` marks the most efficient copy)
335     for reporting; the copies stay eligible because the data cannot show cross-unit cannibalisation."""
336     p = params[params["推广单元ID"].isin(active_units)].copy()
337     p["cap_factor"] = np.where(p["label"] == "问题词", problem_cap_factor, 1.0)
338     p["dup_best"] = True
339     for _, grp in p[p["shared_units"] > 1].groupby("关键词"):
340         if len(grp) > 1:
341             best = grp["efficiency"].idxmax()
342             p.loc[grp.index.difference([best]), "dup_best"] = False
343     p["eligible"] = p["label"].isin(["黄金词", "重点词", "潜力词", "问题词"])
344     return p
345
346
347 def _evaluate(x: np.ndarray, p: pd.DataFrame, gamma: float, eff: float, pos: dict[str, Any]) -> pd.DataFrame:
348     c, s = p["c"].to_numpy(dtype=float), p["s"].to_numpy(dtype=float)
349     clicks = eff * resp.clicks_response(x, c, s, gamma)
350     with np.errstate(divide="ignore", invalid="ignore"):
351         cpc = np.where(clicks > 0, x / clicks, np.nan)
352     p_top, p_first = resp.position_shares(np.nan_to_num(cpc, nan=1.0), pos["top"], pos["first"])
353     return pd.DataFrame(
354         {
355             "序号": p["序号"].to_numpy(),
356             "关键词": p["关键词"].to_numpy(),
357             "方案ID": p["方案ID"].to_numpy(),
358             "推广单元ID": p["推广单元ID"].to_numpy(),
359             "label": p["label"].to_numpy(),
360             "spend": x,
361             "clicks": clicks,
362             "cpc": cpc,
363             "p_top": p_top,
364             "p_first": p_first,
365             "position": resp.expected_position(p_top, p_first),
366             "views": clicks * p["depth"].to_numpy(dtype=float),
367             "regs": clicks * p["rho"].to_numpy(dtype=float),
368         }
369     )
370
371
372 def _caps(p: pd.DataFrame, budget: float, cap_mult: float) -> np.ndarray:
373     """Per-keyword daily caps: multiplier x mean daily spend x type factor, never below the historical
374     proportional share (so the proportional baseline is always feasible) and jointly attainable."""
375     s = p["s"].to_numpy(dtype=float)
376     caps = np.maximum(cap_mult * s * p["cap_factor"].to_numpy(dtype=float), alloc.proportional_allocation(s, budget))
377     if caps.sum() < 1.2 * budget: # keep the cap structure but make the budget attainable
378         caps = caps * (1.2 * budget / caps.sum())
379     return caps
380
381
382 def _solve_unit_day(
383     p: pd.DataFrame, gamma: float, budget: float, cap_mult: float, min_spend: float
384 ) -> tuple[np.ndarray, dict[str, Any], np.ndarray, np.ndarray]:
385     w = (p["rho"] * p["c"]).to_numpy(dtype=float)

```

```

386     s = p["s"].to_numpy(dtype=float)
387     caps = _caps(p, budget, cap_mult)
388     res = alloc.solve_group(w, s, gamma, caps, budget, min_spend=min_spend)
389     return res["x"], res, w * np.power(s, -gamma), caps
390
391
392 def _day_efficiency(ud: pd.DataFrame, units: pd.DataFrame, dates: pd.DatetimeIndex) -> dict[int, dict[str, Any]]:
393     """Calendar-model expectation of the day efficiency multiplier  $E[\text{eff}_{\{u,d\}}]$  for each unit and date."""
394     feats = day_features(dates)
395     out: dict[int, dict[str, Any]] = {}
396     for _, row in units.iterrows():
397         unit = int(row["推广单元ID"])
398         series = fc.unit_series(
399             ud[ud["推广单元ID"] == unit],
400             float(row["gamma"]),
401             float(row["mean_daily_spend"]),
402             float(row["mean_daily_clicks"]),
403         )
404         if len(series) < 40:
405             out[unit] = {"eff": np.ones(len(dates)), "sd": 0.0, "n": len(series), "mean_log": np.zeros(len(dates))}
406             continue
407         f = fc.forecast_unit(series, "log_eff", feats)
408         out[unit] = {"eff": np.exp(f["mean"] + 0.5 * f["sd"] ** 2), "sd": f["sd"], "n": f["n"], "mean_log": f["mean"]}
409     return out
410
411
412 def _sensitivity(
413     all_groups: dict[str, dict[tuple[int, pd.Timestamp], dict[str, Any]]],
414     elig_by_window: dict[str, pd.DataFrame],
415     unit_index: pd.DataFrame,
416     cap_mult: float,
417     min_spend: float,
418     problem_cap: float,
419 ) -> pd.DataFrame:
420     """Gain of the optimal allocation over the proportional baseline under perturbed modelling choices."""
421     scenarios: list[tuple[str, dict[str, Any]]] = [
422         ("基准", {}),
423         ("上限倍数 2", {"cap": 2.0}),
424         ("上限倍数 8", {"cap": 8.0}),
425         ("弹性 gamma x0.9", {"gamma_scale": 0.9}),
426         ("弹性 gamma x1.1", {"gamma_scale": 1.1}),
427         ("预算 -20%", {"budget_scale": 0.8}),
428         ("预算 +20%", {"budget_scale": 1.2}),
429         ("剔除问题词", {"exclude_problem": True}),
430         ("问题词不降上限", {"problem_cap": 1.0}),
431         ("最小投放 0", {"min_spend": 0.0}),
432     ]
433     rows = []
434     for name, opt in scenarios:
435         for window, groups in all_groups.items():
436             elig = elig_by_window[window]
437             opt_total = prop_total = 0.0
438             selected = []
439             for (unit, _date), g in groups.items():
440                 gamma = float(np.clip(g["gamma"] * opt.get("gamma_scale", 1.0), *resp.GAMMA_BOUNDS))
441                 budget = g["budget"] * opt.get("budget_scale", 1.0)
442                 p = g["p"]
443                 if opt.get("exclude_problem"):
444                     p = p[p["label"] != "问题词"]
445                     if p.empty:
446                         p = g["p"]
447                 p = p.assign(cap_factor=np.where(p["label"] == "问题词", opt.get("problem_cap", problem_cap), 1.0))
448                 w = (p["rho"] * p["c"]).to_numpy(dtype=float)

```

```

449         s = p["s"].to_numpy(dtype=float)
450         caps = _caps(p, budget, opt.get("cap", cap_mult))
451         r = alloc.solve_group(w, s, gamma, caps, budget, min_spend=opt.get("min_spend", min_spend))
452         opt_total += g["eff"] * r["objective"]
453         selected.append(r["n_selected"])
454         p_all = elig[elig["推广单元ID"] == unit]
455         xb = alloc.proportional_allocation(p_all["s"].to_numpy(dtype=float), budget)
456         prop_total += g["eff"] * float(
457             np.sum((p_all["rho"] * p_all["c"]).to_numpy() * np.power(xb / p_all["s"].to_numpy(), gamma))
458         )
459         rows.append(
460             {
461                 "scenario": name,
462                 "window": {"feb": "2 月", "aug": "8 月"}[window],
463                 "opt_regs": opt_total,
464                 "prop_regs": prop_total,
465                 "gain": 100 * (opt_total / max(prop_total, 1e-9) - 1),
466                 "selected": float(np.mean(selected)) if selected else 0.0,
467             }
468         )
469     return pd.DataFrame(rows)
470
471
472 @stage(
473     "allocate",
474     deps=("eda", "classify"),
475     description="Q3: keyword selection and daily budget allocation for 2025-02-01..08 and 08-01..08 with audit",
476 )
477 def allocate(ctx: StageContext) -> dict[str, Any]:
478     rng = np.random.default_rng(ctx.seed_everything("allocate"))
479     ud, units, params, pos, rates = _load_params(ctx)
480     cap_mult = float(ctx.cfg("allocate.cap_multiplier", 4.0))
481     min_spend = float(ctx.cfg("allocate.min_spend", 0.5))
482     problem_cap = float(ctx.cfg("allocate.problem_cap_factor", 0.5))
483     n_boot = int(ctx.param("bootstrap", ctx.cfg("allocate.bootstrap", 100)))
484     unit_index = units.set_index("推广单元ID")
485     s1 = ud[["日期", "方案ID", "推广单元ID", "消费额", "点击量", "展现量"]]
486     ctx.write_json("position_models.json", pos)
487
488     rows, summaries, audits = [], [], []
489     boot_records: list[dict[str, Any]] = []
490     window_alt: list[dict[str, Any]] = []
491     eligible_tables = []
492     all_groups: dict[str, dict[tuple[int, pd.Timestamp], dict[str, Any]]] = {}
493     elig_by_window: dict[str, pd.DataFrame] = {}
494     for window, (start, end) in Q3_WINDOWS.items():
495         dates = pd.date_range(start, end, freq="D")
496         day_eff = _day_efficiency(ud, units, dates)
497         actual = s1[(s1["日期"] >= start) & (s1["日期"] <= end) & (s1["消费额"] > 0)]
498         active_units = {int(u) for u in actual["推广单元ID"].unique()}
499         elig = _eligible(params, active_units, problem_cap)
500         elig["window"] = window
501         eligible_tables.append(elig)
502         groups: dict[tuple[int, pd.Timestamp], dict[str, Any]] = {}
503         for _, rec in actual.iterrows():
504             unit, date, budget = int(rec["推广单元ID"]), pd.Timestamp(rec["日期"]), float(rec["消费额"])
505             p = elig[(elig["推广单元ID"] == unit) & elig["eligible"]]
506             if p.empty:
507                 continue
508             gamma = float(unit_index.loc[unit, "gamma"])
509             eff = float(day_eff[unit]["eff"][dates.get_loc(date)])
510             x, res, a, caps = _solve_unit_day(p, gamma, budget, cap_mult, min_spend)
511             ev = _evaluate(x, p, gamma, eff, pos[unit])

```



```

512     ev.insert(0, "date", date.strftime("%Y-%m-%d"))
513     ev["window"] = window
514     rows.append(ev[ev["spend"] > 0])
515     # baselines evaluated under the same response model and day multiplier
516     p_all = elig[elig["推广单元ID"] == unit]
517     base = _evaluate(
518         alloc.proportional_allocation(p_all["s"].to_numpy(dtype=float), budget), p_all, gamma, eff, pos[unit]
519     )
520     equal = _evaluate(alloc.equal_allocation(len(p), budget), p, gamma, eff, pos[unit])
521     sel = x > 0
522     audit = alloc.verify_allocation(x, a, gamma, caps, budget, rng, floor=min_spend)
523     audits.append(
524         {
525             "window": window,
526             "date": date.strftime("%Y-%m-%d"),
527             "推广单元ID": unit,
528             **{k: v for k, v in audit.items() if k != "checks"},
529             "checks": audit["checks"],
530         }
531     )
532     groups[(unit, date)] = {"p": p, "gamma": gamma, "eff": eff, "budget": budget, "x": x, "a": a, "caps": caps}
533     all_groups[window] = groups
534     elig_by_window[window] = elig
535     summaries.append(
536         {
537             "window": window,
538             "date": date.strftime("%Y-%m-%d"),
539             "方案ID": int(rec["方案ID"]),
540             "推广单元ID": unit,
541             "budget": budget,
542             "spent": float(x.sum()),
543             "eligible": len(p),
544             "selected": int(sel.sum()),
545             "capped": audit["n_capped"],
546             "floored": audit["n_floored"],
547             "day_eff": eff,
548             "actual_clicks": float(rec["点击量"]),
549             "actual_regs_expected": float(rec["点击量"]) * rates[unit],
550             "opt_clicks": float(ev["clicks"].sum()),
551             "opt_regs": float(ev["regs"].sum()),
552             "opt_views": float(ev["views"].sum()),
553             "opt_position": float(np.average(ev["position"], weights=np.maximum(ev["clicks"], 1e-9))),
554             "opt_cpc": float(x.sum() / max(ev["clicks"].sum(), 1e-9)),
555             "prop_clicks": float(base["clicks"].sum()),
556             "prop_regs": float(base["regs"].sum()),
557             "prop_cpc": float(budget / max(base["clicks"].sum(), 1e-9)),
558             "equal_clicks": float(equal["clicks"].sum()),
559             "equal_regs": float(equal["regs"].sum()),
560             "audit_ok": audit["ok"],
561             "cvx_gap": audit["cvxpy"].get("relative_gap", np.nan),
562         }
563     )
564     # bootstrap over the elasticity uncertainty: improvement ratio of optimal vs proportional
565     for b in range(n_boot):
566         opt = prop = 0.0
567         for (unit, _date), g in groups.items():
568             row = unit_index.loc[unit]
569             mu = row["gamma_hat"] if row["gamma_source"] == "unit" else row["pooled_gamma"]
570             se = row["gamma_se"] if row["gamma_source"] == "unit" else row["pooled_se"]
571             gb = float(np.clip(rng.normal(mu, se), *resp.GAMMA_BOUNDS))
572             p = g["p"]
573             w = (p["rho"] * p["c"]).to_numpy(dtype=float)
574             s = p["s"].to_numpy(dtype=float)

```

```

575     caps = g["caps"]
576     r = alloc.solve_group(w, s, gb, caps, g["budget"], min_spend=min_spend)
577     opt += g["eff"] * r["objective"]
578     p_all = elig[elig["推广单元ID"] == unit]
579     xb = alloc.proportional_allocation(p_all["s"].to_numpy(dtype=float), g["budget"])
580     prop += g["eff"] * float(
581         np.sum((p_all["rho"] * p_all["c"]).to_numpy() * np.power(xb / p_all["s"].to_numpy(), gb))
582     )
583     boot_records.append(
584         {"window": window, "draw": b, "opt_regs": opt, "prop_regs": prop, "ratio": opt / max(prop, 1e-9)}
585     )
586     # alternative budget reading: the window total per unit is also allocated across days
587     for unit in sorted(active_units):
588         unit_days = [(u, d) for (u, d) in groups if u == unit]
589         if not unit_days:
590             continue
591         gamma = float(unit_index.loc[unit, "gamma"])
592         B = float(sum(groups[k]["budget"] for k in unit_days))
593         eff_days = np.array([day_eff[unit]["eff"][dates.get_loc(d)] for _, d in unit_days])
594         day_cap = 1.5 * max(groups[k]["budget"] for k in unit_days)
595         X = alloc.water_filling(eff_days, gamma, np.full(len(unit_days), day_cap), B)
596         total = 0.0
597         for (u, d), Xd in zip(unit_days, X):
598             g = groups[(u, d)]
599             r = alloc.solve_group(
600                 (g["p"]["rho"] * g["p"]["c"]).to_numpy(dtype=float),
601                 g["p"]["s"].to_numpy(dtype=float),
602                 gamma,
603                 g["caps"],
604                 float(Xd),
605                 min_spend=min_spend,
606             )
607             total += g["eff"] * r["objective"]
608             window_alt.append(
609                 {
610                     "window": window,
611                     "推广单元ID": unit,
612                     "date": d.strftime("%Y-%m-%d"),
613                     "daily_budget": g["budget"],
614                     "window_budget_day": float(Xd),
615                     "day_eff": g["eff"],
616                     "regs_window": g["eff"] * r["objective"],
617                 }
618             )
619     sens = _sensitivity(all_groups, elig_by_window, unit_index, cap_mult, min_spend, problem_cap)
620     _save(ctx, sens, "sensitivity")
621     allocation = pd.concat(rows, ignore_index=True)
622     _save(ctx, allocation, "allocation")
623     summary = pd.DataFrame(summaries)
624     _save(ctx, summary, "allocation_summary")
625     _save(ctx, pd.concat(eligible_tables, ignore_index=True), "keyword_parameters")
626     boot = pd.DataFrame(boot_records)
627     _save(ctx, boot, "bootstrap")
628     alt = pd.DataFrame(window_alt)
629     _save(ctx, alt, "window_budget_alternative")
630     ctx.write_json(
631         "allocation_audit.json",
632         {
633             "ok": all(a["ok"] for a in audits),
634             "groups": len(audits),
635             "failed": [a for a in audits if not a["ok"]],
636             "cvx_available": sum(1 for a in audits if a["cvxpy"].get("available")),
637             "max_cvx_gap": float(np.nanmax([a["cvxpy"].get("relative_gap", np.nan) for a in audits])),

```

```

638         "aggregate_gap": float(
639             (
640                 sum(a["cvxpy"].get("relaxed_objective", a["objective"]) for a in audits)
641                 - sum(a["objective"] for a in audits)
642             )
643             / sum(a["objective"] for a in audits)
644         ),
645         "audits": audits,
646     },
647 )
648 # response-model calibration check: proportional allocation at the actual budget should reproduce actual clicks
649 calib = summary[["window", "推广单元ID", "actual_clicks", "prop_clicks"]].copy()
650 calib["ape"] = np.abs(calib["prop_clicks"] - calib["actual_clicks"]) / np.maximum(calib["actual_clicks"], 1.0)
651 ctx.write_json(
652     "calibration_check.json",
653     {
654         "mape": float(calib["ape"].mean()),
655         "median_ape": float(calib["ape"].median()),
656         "n": len(calib),
657         "spend_weighted_mape": float(np.average(calib["ape"], weights=summary["budget"])),
658     },
659 )
660 if not all(a["ok"] for a in audits):
661     raise RuntimeError("allocation audit failed for some unit-days; see allocation_audit.json")
662
663 win_tot = summary.groupby("window")[
664     [
665         "budget",
666         "opt_clicks",
667         "opt_regs",
668         "prop_clicks",
669         "prop_regs",
670         "equal_regs",
671         "actual_clicks",
672         "actual_regs_expected",
673     ]
674 ].sum()
675 for window, key in (("feb", "Feb"), ("aug", "Aug")):
676     w = win_tot.loc[window]
677     ctx.number(f"QthreeBudget{key}", w["budget"], ".2f")
678     ctx.number(f"QthreeOptRegs{key}", w["opt_regs"], ".2f")
679     ctx.number(f"QthreePropRegs{key}", w["prop_regs"], ".2f")
680     ctx.number(f"QthreeOptClicks{key}", w["opt_clicks"], ".1f")
681     ctx.number(f"QthreePropClicks{key}", w["prop_clicks"], ".1f")
682     ctx.number(f"QthreeActualClicks{key}", w["actual_clicks"], ".0f")
683     ctx.number(f"QthreeRegGainPct{key}", 100 * (w["opt_regs"] / w["prop_regs"] - 1), ".2f")
684     ctx.number(f"QthreeClickGainPct{key}", 100 * (w["opt_clicks"] / w["prop_clicks"] - 1), ".2f")
685     ctx.number(f"QthreeEqualRegGainPct{key}", 100 * (w["opt_regs"] / w["equal_regs"] - 1), ".2f")
686     bw = boot[boot["window"] == window]["ratio"]
687     ctx.number(f"QthreeRegGainCiLow{key}", 100 * (bw.quantile(0.025) - 1), ".2f")
688     ctx.number(f"QthreeRegGainCiHigh{key}", 100 * (bw.quantile(0.975) - 1), ".2f")
689     sw = summary[summary["window"] == window]
690     ctx.number(f"QthreeUnitDays{key}", len(sw))
691     ctx.number(f"QthreeUnits{key}", int(sw["推广单元ID"].nunique()))
692     ctx.number(f"QthreeSelectedKeywords{key}", int(allocation[allocation["window"] == window]["序号"].nunique()))
693     ctx.number(f"QthreeEligibleKeywords{key}", int(sw.groupby("推广单元ID")["eligible"].first().sum()))
694     ctx.number(f"QthreeAvgCpc{key}", w["budget"] / w["opt_clicks"], ".4f")
695     ctx.number(f"QthreePropCpc{key}", w["budget"] / w["prop_clicks"], ".4f")
696     aw = alt[alt["window"] == window]
697     if len(aw):
698         ctx.number(f"QthreeWindowRegs{key}", aw["regs_window"].sum(), ".2f")
699         ctx.number(f"QthreeWindowGainPct{key}", 100 * (aw["regs_window"].sum() / w["opt_regs"] - 1), ".2f")
700     ctx.number(f"QthreeAuditGroups", len(audits))

```

```

701     ctx.number("QthreeMaxCvxGap", float(np.nanmax([a["cvxpy"].get("relative_gap", np.nan) for a in audits])), ".2e")
702
703     def cvx_check(a: dict[str, Any]) -> dict[str, Any]:
704         return next((c for c in a["checks"] if c["name"] == "within_convex_relaxation_bound"), {})
705
706     tiny_groups = [a for a in audits if cvx_check(a).get("tiny_budget")]
707     informative = [
708         float(cvx_check(a)["relative_gap"]) for a in audits if cvx_check(a) and not cvx_check(a).get("tiny_budget")
709     ]
710     ctx.number("QthreeCvxTinyGroups", len(tiny_groups))
711     ctx.number("QthreeMaxCvxGapInformativePct", 100 * max(informative) if informative else 0.0, ".3f")
712     ctx.number(
713         "QthreeMaxCvxGapPct", 100 * float(np.nanmax([a["cvxpy"].get("relative_gap", np.nan) for a in audits])), ".1f"
714     )
715     ctx.number("QthreeCappedKeywordDays", int(sum(a["n_capped"] for a in audits)))
716     ctx.number("QthreeSelectedKeywordDays", int(sum(a["n_selected"] for a in audits)))
717     # long-tail words whose cap is the floor itself: how much money and value they carry (August window)
718     aug = allocation[allocation["window"] == "aug"]
719     at_floor = aug["spend"] <= min_spend + 1e-9
720     ctx.number("QthreeFloorKeywordDaysAug", int(at_floor.sum()))
721     ctx.number("QthreeFloorSpendSharePctAug", 100 * float(aug.loc[at_floor, "spend"].sum() / aug["spend"].sum()), ".2f"
722     ↪ )
723     ctx.number("QthreeFloorRegsSharePctAug", 100 * float(aug.loc[at_floor, "regs"].sum() / aug["regs"].sum()), ".2f")
724     sw_aug = summary[summary["window"] == "aug"]
725     ctx.number("QthreeMeanSelectedAug", float(sw_aug["selected"].mean()), ".1f")
726     unit_gain = sw_aug.groupby("推广单元ID")[["opt_regs", "prop_regs"]].sum()
727     unit_gain_pct = 100 * (unit_gain["opt_regs"] / unit_gain["prop_regs"] - 1)
728     ctx.number("QthreeMinUnitGainPctAug", float(unit_gain_pct.min()), ".1f")
729     ctx.number("QthreeMaxUnitGainPctAug", float(unit_gain_pct.max()), ".1f")
730     ctx.number("QthreeMinGainUnitAug", int(unit_gain_pct.idxmin()))
731     ctx.number("QthreeMaxGainUnitAug", int(unit_gain_pct.idxmax()))
732     relaxed = sum(a["cvxpy"].get("relaxed_objective", a["objective"]) for a in audits)
733     achieved = sum(a["objective"] for a in audits)
734     ctx.number("QthreeAggregateGapPct", 100 * (relaxed - achieved) / achieved, ".3f")
735     ctx.number("QthreeCalibrationMape", 100 * float(calib["ape"].mean()), ".2f")
736     ctx.number("QthreeCalibrationMedianApe", 100 * float(calib["ape"].median()), ".2f")
737     ctx.number("QthreeRows", len(allocation))
738     ctx.number("QthreeCapMultiplier", cap_mult, ".1f")
739     ctx.number("QthreeMinSpend", min_spend, ".2f")
740     return {"rows": len(allocation), "audit_ok": True, "windows": win_tot.to_dict(orient="index")}
741
742 # ----- Q4
743 @stage(
744     "forecast",
745     deps=("eda", "classify", "allocate"),
746     description="Q4: 2026-09-11..17 forecasts, backtests, stochastic allocation and expected ranges",
747 )
748 def forecast(ctx: StageContext) -> dict[str, Any]:
749     rng = np.random.default_rng(ctx.seed_everything("forecast"))
750     ud, units, params, pos, _ = _load_params(ctx)
751     cap_mult = float(ctx.cfg("allocate.cap_multiplier", 4.0))
752     min_spend = float(ctx.cfg("allocate.min_spend", 0.5))
753     problem_cap = float(ctx.cfg("allocate.problem_cap_factor", 0.5))
754     n_scen = int(ctx.param("scenarios", ctx.cfg("forecast.scenarios", 500)))
755     unit_index = units.set_index("推广单元ID")
756     dates = pd.date_range(*Q4_DATES, freq="D")
757     future = day_features(dates)
758     origins = list(pd.date_range("2025-10-09", "2025-12-25", freq="7D"))
759
760     series_by_unit: dict[int, pd.DataFrame] = {}
761     for _, row in units.iterrows():
762         unit = int(row["推广单元ID"])

```

```

763     series_by_unit[unit] = fc.unit_series(
764         ud[ud["推广单元ID"] == unit],
765         float(row["gamma"]),
766         float(row["mean_daily_spend"]),
767         float(row["mean_daily_clicks"]),
768     )
769     bt_frames = []
770     for unit, series in series_by_unit.items():
771         for target in fc.TARGETS:
772             bt = fc.rolling_backtest(series, target, origins)
773             if len(bt):
774                 bt.insert(0, "推广单元ID", unit)
775                 bt_frames.append(bt)
776     backtest = pd.concat(bt_frames, ignore_index=True)
777     _save(ctx, backtest, "backtest")
778     bt_summary = (
779         backtest.groupby(["target", "model"])[["mae", "rmse", "mape_raw", "pinball10", "pinball90", "coverage80"]]
780         .mean()
781         .reset_index()
782     )
783     _save(ctx, bt_summary, "backtest_summary")
784
785     fc_rows = []
786     unit_fc: dict[int, dict[str, dict[str, Any]]] = {}
787     for unit, series in series_by_unit.items():
788         unit_fc[unit] = {}
789         for target, info in fc.TARGETS.items():
790             if len(series) < 40:
791                 continue
792             f = fc.forecast_unit(series, target, future)
793             unit_fc[unit][target] = f
794             sd_total = float(np.sqrt(f["sd"] ** 2 + f["month_sd"] ** 2))
795             for i, d in enumerate(dates):
796                 fc_rows.append(
797                     {
798                         "推广单元ID": unit,
799                         "date": d.strftime("%Y-%m-%d"),
800                         "target": target,
801                         "mean_t": float(f["mean"][i]),
802                         "sd_t": sd_total,
803                         "point": float(fc.back_transform(np.array([f["mean"][i]]), info["transform"])[0]),
804                         "q10": float(
805                             fc.back_transform(np.array([f["mean"][i] - fc.Z80 * sd_total]), info["transform"])[0]
806                         ),
807                         "q90": float(
808                             fc.back_transform(np.array([f["mean"][i] + fc.Z80 * sd_total]), info["transform"])[0]
809                         ),
810                         "month_known": f["month_known"],
811                     }
812                 )
813     forecasts = pd.DataFrame(fc_rows)
814     _save(ctx, forecasts, "unit_forecasts")
815
816     # budgets: 2025 same-period spend per unit (main) and the annual-average alternative
817     s1 = ud[["日期", "方案ID", "推广单元ID", "消费额"]]
818     ref = s1[(s1["日期"] >= Q4_REFERENCE[0]) & (s1["日期"] <= Q4_REFERENCE[1])]
819     budget_main = ref.groupby("推广单元ID")["消费额"].sum()
820     day_cap_ref = ref.groupby("推广单元ID")["消费额"].max()
821     annual = s1.groupby("推广单元ID")["消费额"].sum()
822     budget_alt = annual / 365.0 * len(dates)
823     budgets = (
824         pd.DataFrame({"budget_same_period": budget_main, "budget_annual_avg": budget_alt}).fillna(0.0).reset_index()
825     )

```

```

826 _save(ctx, budgets, "budgets")
827 active_units = {int(u) for u, b in budget_main.items() if b > 0}
828 elig = _eligible(params, active_units, problem_cap)
829 elig_all = _eligible(params, {int(u) for u in units["推广单元ID"]}, problem_cap)
830 _save(ctx, elig, "keyword_parameters")
831
832 rows, day_rows, audits, unit_rows = [], [], [], []
833 for unit in sorted(active_units):
834     p = elig[(elig["推广单元ID"] == unit) & elig["eligible"]]
835     if p.empty or unit not in unit_fc or "log_eff" not in unit_fc[unit]:
836         continue
837     gamma = float(unit_index.loc[unit, "gamma"])
838     f_eff = unit_fc[unit]["log_eff"]
839     sd_eff = float(np.sqrt(f_eff["sd"] ** 2 + f_eff["month_sd"] ** 2))
840     eff_mean = np.exp(f_eff["mean"] + 0.5 * sd_eff**2)
841     B = float(budget_main[unit])
842     day_cap = float(max(1.5 * day_cap_ref[unit], B / len(dates)))
843     # day split with the same floor: a day is either skipped or receives at least the minimum amount
844     day_caps = np.full(len(dates), day_cap)
845     X = alloc.solve_group(eff_mean, np.ones(len(dates)), gamma, day_caps, B, min_spend=min_spend)["x"]
846     day_audit = alloc.verify_allocation(X, eff_mean, gamma, day_caps, B, rng, floor=min_spend)
847     audits.append({"推广单元ID": unit, "level": "days", **{k: v for k, v in day_audit.items() if k != "cvxpy"}})
848     f_ctr = unit_fc[unit]["logit_ctr"]
849     f_top = unit_fc[unit]["logit_top"]
850     f_cpc = unit_fc[unit]["log_cpc"]
851     kw_sd = float(f_cpc["sd"])
852     totals = {k: np.zeros(n_scen) for k in ("clicks", "imp", "views", "regs", "spend")}
853     for i, d in enumerate(dates):
854         if X[i] <= 0:
855             continue
856         w = (p["rho"] * p["c"]).to_numpy(dtype=float) * float(eff_mean[i])
857         s = p["s"].to_numpy(dtype=float)
858         caps = _caps(p, float(X[i]), cap_mult)
859         res = alloc.solve_group(w, s, gamma, caps, float(X[i]), min_spend=min_spend)
860         x = res["x"]
861         sel = x > 0
862         a = w * np.power(s, -gamma)
863         audit = alloc.verify_allocation(x, a, gamma, caps, float(X[i]), rng, floor=min_spend)
864         audits.append(
865             {
866                 "推广单元ID": unit,
867                 "level": "keywords",
868                 "date": d.strftime("%Y-%m-%d"),
869                 **{k: v for k, v in audit.items() if k != "checks"},
870                 "checks": audit["checks"],
871             }
872         )
873     sim = fc.simulate_group(
874         x[sel],
875         p[sel],
876         gamma,
877         float(f_eff["mean"][i]),
878         sd_eff,
879         kw_sd,
880         float(f_ctr["mean"][i]),
881         float(np.sqrt(f_ctr["sd"] ** 2 + f_ctr["month_sd"] ** 2)),
882         pos[unit]["top"],
883         pos[unit]["first"],
884         float(np.sqrt(f_top["sd"] ** 2 + f_top["month_sd"] ** 2)),
885         rng,
886         n_scen,
887     )
888     ps = p[sel]

```

```

889 stats = {k: fc.summarise(v) for k, v in sim.items()}
890 for j in range(int(sel.sum())):
891     rows.append(
892         {
893             "date": d.strftime("%Y-%m-%d"),
894             "方案ID": int(ps["方案ID"].iloc[j]),
895             "推广单元ID": unit,
896             "关键词": int(ps["关键词"].iloc[j]),
897             "序号": int(ps["序号"].iloc[j]),
898             "label": ps["label"].iloc[j],
899             "spend": float(x[sel][j]),
900             **{
901                 f"{k}_{q}": float(stats[k][q][j])
902                 for k in ("clicks", "cpc", "imp", "position", "views", "regs", "p_top")
903                 for q in ("mean", "q10", "q50", "q90")
904             },
905         }
906     )
907 for k in ("clicks", "imp", "views", "regs"):
908     totals[k] += np.nansum(sim[k], axis=1)
909 totals["spend"] += float(x.sum())
910 day_tot = {k: np.nansum(sim[k], axis=1) for k in ("clicks", "imp", "views", "regs")}
911 day_cpc = float(x.sum()) / np.maximum(day_tot["clicks"], 1e-9)
912 pos_w = np.nansum(sim["position"] * sim["clicks"], axis=1) / np.maximum(day_tot["clicks"], 1e-9)
913 day_rows.append(
914     {
915         "date": d.strftime("%Y-%m-%d"),
916         "方案ID": int(ps["方案ID"].iloc[0]),
917         "推广单元ID": unit,
918         "budget_day": float(X[i]),
919         "spend": float(x.sum()),
920         "selected": int(sel.sum()),
921         "eff_mean": float(eff_mean[i]),
922         **{
923             f"{k}_{q}": float(v)
924             for k, arr in day_tot.items()
925             for q, v in zip(
926                 ("mean", "q10", "q90"), (arr.mean(), np.quantile(arr, 0.1), np.quantile(arr, 0.9))
927             )
928         },
929         **{
930             f"cpc_{q}": float(v)
931             for q, v in zip(
932                 ("mean", "q10", "q90"),
933                 (day_cpc.mean(), np.quantile(day_cpc, 0.1), np.quantile(day_cpc, 0.9)),
934             )
935         },
936         **{
937             f"position_{q}": float(v)
938             for q, v in zip(
939                 ("mean", "q10", "q90"), (pos_w.mean(), np.quantile(pos_w, 0.1), np.quantile(pos_w, 0.9))
940             )
941         },
942     }
943 )
944 unit_rows.append(
945     {
946         "推广单元ID": unit,
947         "budget": B,
948         "spent": float(totals["spend"][0]),
949         "gamma": gamma,
950         "eligible": len(p),
951         "day_split": [float(v) for v in X],

```

```

952         **{
953             f"{k}_{q}": float(v)
954             for k, arr in totals.items()
955             if k != "spend"
956             for q, v in zip(("mean", "q10", "q90"), (arr.mean(), np.quantile(arr, 0.1), np.quantile(arr, 0.9)))
957         },
958     }
959 )
960 alt_expected: dict[str, float] = {}
961 for name, budget_series in ((("same_period", budget_main), ("annual_avg", budget_alt)):
962     total_regs = 0.0
963     for unit in sorted(int(u) for u, b in budget_series.items() if b > 0):
964         p = elig_all[(elig_all["推广单元ID"] == unit) & elig_all["eligible"]]
965         if p.empty or unit not in unit_fc or "log_eff" not in unit_fc[unit]:
966             continue
967         gamma = float(unit_index.loc[unit, "gamma"])
968         f_eff = unit_fc[unit]["log_eff"]
969         sd_eff = float(np.sqrt(f_eff["sd"] ** 2 + f_eff["month_sd"] ** 2))
970         eff_mean = np.exp(f_eff["mean"] + 0.5 * sd_eff**2)
971         B = float(budget_series[unit])
972         cap_day = float(max(1.5 * day_cap_ref.get(unit, B / len(dates)), B / len(dates)))
973         X = alloc.solve_group(
974             eff_mean, np.ones(len(dates)), gamma, np.full(len(dates), cap_day), B, min_spend=min_spend
975         )["x"]
976         for i in range(len(dates)):
977             if X[i] <= 0:
978                 continue
979             w = (p["rho"] * p["c"]).to_numpy(dtype=float) * float(eff_mean[i])
980             s = p["s"].to_numpy(dtype=float)
981             total_regs += alloc.solve_group(
982                 w, s, gamma, _caps(p, float(X[i]), cap_mult), float(X[i]), min_spend=min_spend
983             )["objective"]
984         alt_expected[name] = total_regs
985 ctx.write_json("budget_alternatives.json", alt_expected)
986 plan = pd.DataFrame(rows)
987 _save(ctx, plan, "plan")
988 days = pd.DataFrame(day_rows)
989 _save(ctx, days, "plan_by_unit_day")
990 per_unit = pd.DataFrame(unit_rows)
991 per_unit.to_json(ctx.out("plan_by_unit.json"), orient="records", force_ascii=False, indent=2)
992 audit_ok = all(a["ok"] for a in audits)
993 ctx.write_json(
994     "allocation_audit.json",
995     {"ok": audit_ok, "groups": len(audits), "failed": [a for a in audits if not a["ok"]], "audits": audits},
996 )
997 if not audit_ok:
998     raise RuntimeError("Q4 allocation audit failed; see allocation_audit.json")
999
1000 cal = bt_summary[bt_summary["model"] == "calendar"].set_index("target")
1001 sn = bt_summary[bt_summary["model"] == "seasonal_naive"].set_index("target")
1002 m28 = bt_summary[bt_summary["model"] == "mean28"].set_index("target")
1003 for target, key in (
1004     ("log_eff", "Eff"),
1005     ("log_cpc", "Cpc"),
1006     ("logit_ctr", "Ctr"),
1007     ("logit_top", "Top"),
1008     ("log_clicks", "Clicks"),
1009 ):
1010     ctx.number(f"BacktestCalendarMae{key}", cal.loc[target, "mae"], ".4f")
1011     ctx.number(f"BacktestNaiveMae{key}", sn.loc[target, "mae"], ".4f")
1012     ctx.number(f"BacktestMeanMae{key}", m28.loc[target, "mae"], ".4f")
1013     ctx.number(f"BacktestCalendarCoverage{key}", 100 * cal.loc[target, "coverage80"], ".2f")
1014     ctx.number(f"BacktestNaiveCoverage{key}", 100 * sn.loc[target, "coverage80"], ".2f")

```



```

1015     ctx.number(f"BacktestMeanCoverage{key}", 100 * m28.loc[target, "coverage80"], ".2f")
1016     ctx.number(f"BacktestCalendarPinball{key}", cal.loc[target, "pinball10"] + cal.loc[target, "pinball90"], ".4f")
1017     ctx.number(f"BacktestNaivePinball{key}", sn.loc[target, "pinball10"] + sn.loc[target, "pinball90"], ".4f")
1018     tot = per_unit[
1019         [
1020             "budget",
1021             "spent",
1022             "clicks_mean",
1023             "clicks_q10",
1024             "clicks_q90",
1025             "imp_mean",
1026             "imp_q10",
1027             "imp_q90",
1028             "views_mean",
1029             "views_q10",
1030             "views_q90",
1031             "regs_mean",
1032             "regs_q10",
1033             "regs_q90",
1034         ]
1035     ].sum()
1036     ctx.number("QfourBudget", tot["budget"], ".2f")
1037     ctx.number("QfourBudgetAnnualAlt", float(budgets["budget_annual_avg"].sum()), ".2f")
1038     ctx.number("QfourRegsExpectedSamePeriod", alt_expected["same_period"], ".1f")
1039     ctx.number("QfourRegsExpectedAnnualAlt", alt_expected["annual_avg"], ".1f")
1040     ctx.number(
1041         "QfourRegsGainAnnualAltPct",
1042         100 * (alt_expected["annual_avg"] / max(alt_expected["same_period"], 1e-9) - 1),
1043         ".1f",
1044     )
1045     biggest = per_unit.sort_values("budget", ascending=False).iloc[0]
1046     ctx.number("QfourBiggestUnitId", int(biggest["推广单元ID"]))
1047     ctx.number("QfourBiggestUnitBudgetSharePct", 100 * float(biggest["budget"]) / float(tot["budget"]), ".1f")
1048     ctx.number("QfourUnits", len(per_unit))
1049     ctx.number("QfourSkippedUnits", int(len(units) - len(per_unit)))
1050     ctx.number("QfourRows", len(plan))
1051     ctx.number("QfourSelectedKeywords", int(plan["序号"].nunique()))
1052     for k, key in (("clicks", "Clicks"), ("imp", "Imp"), ("views", "Views"), ("regs", "Regs")):
1053         ctx.number(f"Qfour{key}Mean", tot[f"{k}_mean"], ".1f")
1054         ctx.number(f"Qfour{key}Low", tot[f"{k}_q10"], ".1f")
1055         ctx.number(f"Qfour{key}High", tot[f"{k}_q90"], ".1f")
1056     ctx.number("QfourCpcMean", tot["spent"] / tot["clicks_mean"], ".4f")
1057     ctx.number("QfourCpcLow", tot["spent"] / tot["clicks_q90"], ".4f")
1058     ctx.number("QfourCpcHigh", tot["spent"] / tot["clicks_q10"], ".4f")
1059     ctx.number(
1060         "QfourPositionMean",
1061         float(np.average(days["position_mean"], weights=np.maximum(days["clicks_mean"], 1e-9))),
1062         ".4f",
1063     )
1064     ctx.number("QfourScenarios", n_scen)
1065     ctx.number("QfourAuditGroups", len(audits))
1066     ref_clicks = float(ud[(ud["日期"] >= Q4_REFERENCE[0]) & (ud["日期"] <= Q4_REFERENCE[1])]["点击量"].sum())
1067     ctx.number("QfourRefClicks", ref_clicks, ".0f")
1068     ctx.number("QfourClickGainVsRefPct", 100 * (tot["clicks_mean"] / ref_clicks - 1), ".2f")
1069     return {"rows": len(plan), "units": len(per_unit), "regs_mean": float(tot["regs_mean"]), "audit_ok": audit_ok}
1070
1071
1072 # ----- results
1073 @stage(
1074     "results",
1075     deps=("classify", "allocate", "forecast"),
1076     description="Fill result2/3/4.xlsx from the organisers' templates",
1077 )

```

```

1078 def results(ctx: StageContext) -> dict[str, Any]:
1079     table = pd.read_parquet(ctx.dep("classify") / "classification.parquet").sort_values("序号")
1080     header2 = ["方案ID", "推广单元", "序号", *cls.LABELS]
1081     rows2 = [
1082         [int(r["方案ID"]), int(r["推广单元ID"]), int(r["序号"]), *[int(r["label"] == lab) for lab in cls.LABELS]]
1083         for _, r in table.iterrows()
1084     ]
1085     info2 = xlsx.write_result(ctx.templates / "result2.xlsx", ctx.out("result2.xlsx"), {"Sheet1": (header2, rows2)})
1086
1087 def plan_rows(df: pd.DataFrame, clicks: str, views: str, regs: str, position: str) -> list[list[Any]]:
1088     df = df.sort_values(["date", "方案ID", "推广单元ID", "spend"], ascending=[True, True, True, False])
1089     return [
1090         [
1091             str(r["date"]),
1092             int(r["方案ID"]),
1093             int(r["推广单元ID"]),
1094             int(r["关键词"]),
1095             float(r["spend"]),
1096             float(r[position]),
1097             float(r[clicks]),
1098             float(r[views]),
1099             float(r[regs]),
1100         ]
1101         for _, r in df.iterrows()
1102     ]
1103
1104 allocation = pd.read_parquet(ctx.dep("allocate") / "allocation.parquet")
1105 info3 = xlsx.write_result(
1106     ctx.templates / "result3.xlsx",
1107     ctx.out("result3.xlsx"),
1108     {"Sheet1": (RESULT_HEADER, plan_rows(allocation, "clicks", "views", "regs", "position"))},
1109 )
1110 plan = pd.read_parquet(ctx.dep("forecast") / "plan.parquet")
1111 info4 = xlsx.write_result(
1112     ctx.templates / "result4.xlsx",
1113     ctx.out("result4.xlsx"),
1114     {"Sheet1": (RESULT_HEADER, plan_rows(plan, "clicks_mean", "views_mean", "regs_mean", "position_mean"))},
1115 )
1116 summary = {"result2": info2["sheets"], "result3": info3["sheets"], "result4": info4["sheets"]}
1117 ctx.write_json("results_summary.json", summary)
1118 return {k: v["Sheet1"]["rows"] for k, v in summary.items()}
1119
1120
1121 @stage(
1122     "figures", deps=("eda", "classify", "allocate", "forecast"), description="Paper figures (vector PDF + PNG preview)"
1123 )
1124 def figures(ctx: StageContext) -> dict[str, Any]:
1125     from pipelines.e.figures import build_figures
1126
1127     return build_figures(ctx)
1128
1129
1130 @stage("tables", deps=("eda", "classify", "allocate", "forecast"), description="Paper tables (booktabs .tex + CSV)")
1131 def tables(ctx: StageContext) -> dict[str, Any]:
1132     from pipelines.e.tables import build_tables
1133
1134     return build_tables(ctx)

```

pipelines/e/tables.py

```
1 """Paper tables for Problem E: booktabs LaTeX fragments (tabular only) plus identical CSV files."""
2
3 from __future__ import annotations
4
5 import json
6 from collections.abc import Sequence
7 from typing import Any
8
9 import numpy as np
10 import pandas as pd
11
12 from forge.context import StageContext
13 from forge.tex import tex_escape
14 from pipelines.e.classify import LABELS
15
16
17 def _fmt(value: Any, spec: str) -> str:
18     if value is None or (isinstance(value, float) and not np.isfinite(value)):
19         return "--"
20     if spec == "raw": # already valid LaTeX (math, %); inserted verbatim
21         return str(value)
22     if spec == "s":
23         return tex_escape(str(value))
24     if spec == "d":
25         return f"{round(float(value)):.}"
26     return format(float(value), spec)
27
28
29 def write_table(
30     ctx: StageContext, name: str, df: pd.DataFrame, columns: Sequence[tuple[str, str, str]], align: str | None = None
31 ) -> str:
32     """`columns` = (dataframe column, header, format spec). Writes tables/<name>.tex and tables/<name>.csv."""
33     keys = [c for c, _, _ in columns]
34     headers = [h for _, h, _ in columns]
35     specs = [s for _, _, s in columns]
36     if align is None:
37         align = "".join("l" if s == "s" else "r" for s in specs)
38     lines = [f"\begin{{tabular}}{{@{{}}{align}@{{}}}}", "\\toprule", " & ".join(headers) + " \\\\", "\\midrule"]
39     for _, row in df.iterrows():
40         lines.append(" & ".join(_fmt(row[k], s) for k, s in zip(keys, specs)) + " \\\\")
41     lines += ["\\bottomrule", "\\end{{tabular}}"]
42     text = "\n".join(lines) + "\n"
43     ctx.out("tables", f"{name}.tex").write_text(text, encoding="utf-8")
44     df[keys].to_csv(ctx.out("tables", f"{name}.csv"), index=False)
45     return text
46
47
48 def build_tables(ctx: StageContext) -> dict[str, Any]:
49     eda_dir, cls_dir, alloc_dir, fc_dir = ctx.dep("eda"), ctx.dep("classify"), ctx.dep("allocate"), ctx.dep("forecast")
50     names: list[str] = []
51
52     kpis = pd.read_parquet(eda_dir / "unit_kpis.parquet").sort_values("spend", ascending=False)
53     kpis["spend_share_pct"] = 100 * kpis["spend_share"]
54     kpis["ctr_pct"] = 100 * kpis["ctr"]
55     kpis["top_pct"] = 100 * kpis["top_share"]
56     kpis["first_pct"] = 100 * kpis["first_share"]
57     write_table(
58         ctx,
59         "tab_unit_kpis",
60         kpis,
61         [
```

```

62         ("方案ID", "方案ID", "s"),
63         ("推广单元ID", "推广单元ID", "s"),
64         ("days", "投放天数", "d"),
65         ("spend", "消费额/元", ".0f"),
66         ("spend_share_pct", "占比/\\%", ".1f"),
67         ("imp", "展现量", "d"),
68         ("clicks", "点击量", "d"),
69         ("ctr_pct", "CTR/\\%", ".2f"),
70         ("cpc", "CPC/元", ".2f"),
71         ("top_pct", "上方位/\\%", ".1f"),
72         ("first_pct", "首位/\\%", ".1f"),
73     ],
74 )
75 names.append("tab_unit_kpis")
76
77 coefs = pd.read_parquet(eda_dir / "calendar_regression.parquet")
78 term_names = {
79     "holiday": "法定节假日",
80     "adjusted_workday": "调休上班日",
81     "pre_holiday": "假前一日",
82     "post_holiday": "假后一日",
83     "wd_周一": "周一",
84     "wd_周二": "周二",
85     "wd_周四": "周四",
86     "wd_周五": "周五",
87     "wd_周六": "周六",
88     "wd_周日": "周日",
89 }
90 rows = []
91 for term, tname in term_names.items():
92     row: dict[str, Any] = {"term": tname}
93     for target, key in (("regs", "reg"), ("spend", "spend"), ("clicks", "clicks")):
94         r = coefs[(coefs["target"] == target) & (coefs["term"] == term)].iloc[0]
95         row[f"{key}_eff"] = r["effect_pct"]
96         row[f"{key}_ci"] = f"{r['effect_pct_low']:.1f}, {r['effect_pct_high']:.1f}"
97         row[f"{key}_p"] = r["p"]
98     rows.append(row)
99 write_table(
100     ctx,
101     "tab_calendar_effects",
102     pd.DataFrame(rows),
103     [
104         ("term", "因素", "s"),
105         ("reg_eff", "注册 效应/\\%", ".1f"),
106         ("reg_ci", "95\\% CI", "s"),
107         ("reg_p", "$p$", ".3f"),
108         ("spend_eff", "消费 效应/\\%", ".1f"),
109         ("spend_ci", "95\\% CI", "s"),
110         ("spend_p", "$p$", ".3f"),
111         ("clicks_eff", "点击 效应/\\%", ".1f"),
112         ("clicks_p", "$p$", ".3f"),
113     ],
114 )
115 names.append("tab_calendar_effects")
116
117 attribution = json.loads((eda_dir / "attribution.json").read_text(encoding="utf-8"))
118 units = pd.read_parquet(eda_dir / "unit_response.parquet")
119 att_rows = []
120 for _, u in units.sort_values("推广单元ID").iterrows():
121     unit = str(int(u["推广单元ID"]))
122     spend = float(kpis.loc[kpis["推广单元ID"] == int(unit), "spend"].iloc[0])
123     attributed = attribution["attributed_regs"].get(unit, 0.0)
124     att_rows.append(

```

```

125         {
126             "unit": unit,
127             "rate": 100 * attribution["rates"][unit],
128             "source": {"unit": "单元估计", "pooled": "合并估计"}[attribution["rate_source"][unit]],
129             "attributed": attributed,
130             "cost_per_reg": spend / attributed if attributed > 0 else np.nan,
131             "gamma_hat": u["gamma_hat"],
132             "gamma_se": u["gamma_se"],
133             "r2": u["r2"],
134             "gamma": u["gamma"],
135             "gsrc": {"unit": "单元", "pooled": "合并"}[u["gamma_source"]],
136         }
137     )
138     write_table(
139         ctx,
140         "tab_attribution_response",
141         pd.DataFrame(att_rows),
142         [
143             ("unit", "推广单元ID", "s"),
144             ("rate", "注册率/(人/百次点击)", ".3f"),
145             ("source", "来源", "s"),
146             ("attributed", "归因注册/人", ".0f"),
147             ("cost_per_reg", "注册成本/元", ".1f"),
148             ("gamma_hat", "$\\hat{\\gamma}_u$", ".3f"),
149             ("gamma_se", "SE", ".3f"),
150             ("r2", "$R^2$", ".2f"),
151             ("gamma", "采用 $\\gamma_u$", ".3f"),
152             ("gsrc", "$\\gamma$ 来源", "s"),
153         ],
154     )
155     names.append("tab_attribution_response")
156
157     by_label = pd.read_parquet(cls_dir / "classification_by_label.parquet")
158     table = pd.read_parquet(cls_dir / "classification.parquet")
159     by_label["spend_share_pct"] = 100 * by_label["spend_share"]
160     by_label["share_n"] = 100 * by_label["n"] / by_label["n"].sum()
161     by_label["mean_benefit"] = by_label["label"].map(table.groupby("label")["benefit"].mean())
162     by_label["median_spend"] = by_label["label"].map(table.groupby("label")["spend"].median())
163     write_table(
164         ctx,
165         "tab_class_summary",
166         by_label,
167         [
168             ("label", "类别", "s"),
169             ("n", "数量", "d"),
170             ("share_n", "占比/\\%", ".1f"),
171             ("spend", "消费额/元", ".0f"),
172             ("spend_share_pct", "消费占比/\\%", ".1f"),
173             ("median_spend", "消费中位数/元", ".1f"),
174             ("clicks", "点击量", "d"),
175             ("cpc", "CPC/元", ".2f"),
176             ("mean_benefit", "平均效益指数", ".3f"),
177             ("reg_attr", "归因注册/人", ".0f"),
178         ],
179     )
180     names.append("tab_class_summary")
181
182     summary = json.loads((cls_dir / "classification_summary.json").read_text(encoding="utf-8"))
183     rob_rows = []
184     method_names = {
185         "jenks": "自然断点 (主)",
186         "median": "中位数",
187         "gmm": "高斯混合",

```

```

188     "equal_weights": "等权效益 + 自然断点",
189 }
190 for method, counts in summary["counts"].items():
191     thr = summary["thresholds"].get(method, {})
192     rob_rows.append(
193         {
194             "method": method_names.get(method, method),
195             "cost_thr": thr.get("cost_yuan", np.nan),
196             "ben_thr": thr.get("benefit", np.nan),
197             **{lab: counts.get(lab, 0) for lab in LABELS},
198         }
199     )
200 write_table(
201     ctx,
202     "tab_class_robustness",
203     pd.DataFrame(rob_rows),
204     [
205         ("method", "阈值方法", "s"),
206         ("cost_thr", "成本阈值/元", ".2f"),
207         ("ben_thr", "效益阈值", ".3f"),
208         *[(lab, lab, "d") for lab in LABELS],
209     ],
210 )
211 names.append("tab_class_robustness")
212
213 per_unit = (
214     table.groupby(["推广单元ID", "label"])
215     .size()
216     .unstack(fill_value=0)
217     .reindex(columns=LABELS, fill_value=0)
218     .reset_index()
219 )
220 per_unit["total"] = per_unit[list(LABELS)].sum(axis=1)
221 per_unit["推广单元ID"] = per_unit["推广单元ID"].astype(str)
222 write_table(
223     ctx,
224     "tab_class_per_unit",
225     per_unit,
226     [("推广单元ID", "推广单元ID", "s"), *[(lab, lab, "d") for lab in LABELS], ("total", "合计", "d")],
227 )
228 names.append("tab_class_per_unit")
229
230 asum = pd.read_parquet(alloc_dir / "allocation_summary.parquet")
231 calib = json.loads((alloc_dir / "calibration_check.json").read_text(encoding="utf-8"))
232 audit = json.loads((alloc_dir / "allocation_audit.json").read_text(encoding="utf-8"))
233 boot = pd.read_parquet(alloc_dir / "bootstrap.parquet")
234 q3_rows = []
235 for window, wname in (("feb", "2025-02-01 至 02-08"), ("aug", "2025-08-01 至 08-08")):
236     w = asum[asum["window"] == window]
237     for unit, g in w.groupby("推广单元ID"):
238         q3_rows.append(
239             {
240                 "window": wname,
241                 "unit": str(int(unit)),
242                 "days": len(g),
243                 "budget": g["budget"].sum(),
244                 "eligible": int(g["eligible"].iloc[0]),
245                 "selected": g["selected"].mean(),
246                 "opt_regs": g["opt_regs"].sum(),
247                 "prop_regs": g["prop_regs"].sum(),
248                 "gain": 100 * (g["opt_regs"].sum() / g["prop_regs"].sum() - 1),
249                 "opt_cpc": g["budget"].sum() / g["opt_clicks"].sum(),
250                 "prop_cpc": g["budget"].sum() / g["prop_clicks"].sum(),

```

```

251         }
252     )
253     b = boot[boot["window"] == window]["ratio"]
254     q3_rows.append(
255         {
256             "window": wname,
257             "unit": "合计",
258             "days": len(w),
259             "budget": w["budget"].sum(),
260             "eligible": int(w.groupby("推广单元ID")["eligible"].first().sum()),
261             "selected": w["selected"].mean(),
262             "opt_regs": w["opt_regs"].sum(),
263             "prop_regs": w["prop_regs"].sum(),
264             "gain": 100 * (w["opt_regs"].sum() / w["prop_regs"].sum() - 1),
265             "opt_cpc": w["budget"].sum() / w["opt_clicks"].sum(),
266             "prop_cpc": w["budget"].sum() / w["prop_clicks"].sum(),
267             "ci": f"[100 * (b.quantile(0.025) - 1):.1f], {100 * (b.quantile(0.975) - 1):.1f}]",
268         }
269     )
270     q3 = pd.DataFrame(q3_rows)
271     q3["ci"] = q3["ci"].fillna("")
272     write_table(
273         ctx,
274         "tab_q3_summary",
275         q3,
276         [
277             ("window", "窗口", "s"),
278             ("unit", "推广单元", "s"),
279             ("days", "天数", "d"),
280             ("budget", "预算/元", ".0f"),
281             ("eligible", "候选词", "d"),
282             ("selected", "平均入选词", ".1f"),
283             ("prop_regs", "历史比例注册/人", ".1f"),
284             ("opt_regs", "最优注册/人", ".1f"),
285             ("gain", "提升\\%", ".1f"),
286             ("ci", "95\\% 自助区间", "s"),
287             ("prop_cpc", "历史 CPC/元", ".2f"),
288             ("opt_cpc", "最优 CPC/元", ".2f"),
289         ],
290     )
291     names.append("tab_q3_summary")
292
293     daily_rows = (
294         asum.groupby(["window", "date"])
295         .agg(
296             budget=("budget", "sum"),
297             units=("推广单元ID", "nunique"),
298             opt_clicks=("opt_clicks", "sum"),
299             prop_clicks=("prop_clicks", "sum"),
300             actual_clicks=("actual_clicks", "sum"),
301             opt_regs=("opt_regs", "sum"),
302             prop_regs=("prop_regs", "sum"),
303             equal_regs=("equal_regs", "sum"),
304         )
305         .reset_index()
306     )
307     daily_rows["gain"] = 100 * (daily_rows["opt_regs"] / daily_rows["prop_regs"] - 1)
308     write_table(
309         ctx,
310         "tab_q3_daily",
311         daily_rows,
312         [
313             ("date", "日期", "s"),

```

```

314         ("units", "单元数", "d"),
315         ("budget", "预算/元", ".1f"),
316         ("actual_clicks", "实际点击", "d"),
317         ("prop_clicks", "历史比例点击", ".1f"),
318         ("opt_clicks", "最优点击", ".1f"),
319         ("prop_regs", "历史比例注册", ".1f"),
320         ("equal_regs", "均匀注册", ".1f"),
321         ("opt_regs", "最优注册", ".1f"),
322         ("gain", "提升/\\%", ".1f"),
323     ],
324 )
325 names.append("tab_q3_daily")
326
327 alt = pd.read_parquet(alloc_dir / "window_budget_alternative.parquet")
328 alt_rows = []
329 for window, wname in (("feb", "2 月窗口"), ("aug", "8 月窗口")):
330     a = alt[alt["window"] == window]
331     opt = asum[asum["window"] == window]["opt_regs"].sum()
332     alt_rows.append(
333         {
334             "window": wname,
335             "regs_daily": opt,
336             "regs_window": a["regs_window"].sum(),
337             "gain": 100 * (a["regs_window"].sum() / opt - 1),
338             "moved": 100 * (a["window_budget_day"] - a["daily_budget"]).abs().sum() / 2 / a["daily_budget"].sum(),
339         }
340     )
341 write_table(
342     ctx,
343     "tab_q3_window_budget",
344     pd.DataFrame(alt_rows),
345     [
346         ("window", "窗口", "s"),
347         ("regs_daily", "逐日预算最优注册/人", ".1f"),
348         ("regs_window", "窗口预算最优注册/人", ".1f"),
349         ("gain", "额外提升/\\%", ".1f"),
350         ("moved", "跨日调动预算比例/\\%", ".1f"),
351     ],
352 )
353 names.append("tab_q3_window_budget")
354
355 sens = (
356     pd.read_parquet(alloc_dir / "sensitivity.parquet")
357     if (alloc_dir / "sensitivity.parquet").exists()
358     else pd.DataFrame()
359 )
360 if len(sens):
361     write_table(
362         ctx,
363         "tab_q3_sensitivity",
364         sens,
365         [
366             ("scenario", "情形", "s"),
367             ("window", "窗口", "s"),
368             ("opt_regs", "最优注册/人", ".1f"),
369             ("prop_regs", "历史比例注册/人", ".1f"),
370             ("gain", "提升/\\%", ".1f"),
371             ("selected", "平均入选词", ".1f"),
372         ],
373     )
374     names.append("tab_q3_sensitivity")
375
376 def cvx_check(a: dict[str, Any]) -> dict[str, Any]:

```



```

377         return next((c for c in a["checks"] if c["name"] == "within_convex_relaxation_bound"), {})
378
379 tiny = sum(1 for a in audit["audits"] if cvx_check(a).get("tiny_budget"))
380 informative = [
381     float(cvx_check(a)["relative_gap"])
382     for a in audit["audits"]
383     if cvx_check(a) and not cvx_check(a).get("tiny_budget")
384 ]
385 ver_rows = [
386     {"item": "Q3 单元-日分配问题数", "value": f"{audit['groups']}"},
387     {
388         "item": "Q3 独立校验 (可行性 / KKT / 配对转移与插入 / 通用凸求解器上界) ",
389         "value": "全部通过" if audit["ok"] else "存在失败",
390     },
391     {
392         "item": "Q3 凸松弛总体最优性界  $(\sum \text{上界} - \sum \text{目标值}) / \sum \text{目标值}$ ",
393         "value": f"{100 * audit['aggregate_gap']:.3f}%",
394     },
395     {
396         "item": f"Q3 单个问题的最大相对差 (不含 {tiny} 个预算低于 20 倍最小投放额的微小问题) ",
397         "value": f"{100 * max(informative):.3f}%" if informative else "--",
398     },
399     {
400         "item": "Q3 微小问题的最大相对差 (松弛界在此处不具信息量, 见正文) ",
401         "value": f"{100 * audit['max_cvx_gap']:.1f}%",
402     },
403     {
404         "item": "响应模型校准: 历史比例分配预测点击 vs 实际 (MAPE / 中位 APE) ",
405         "value": f"{100 * calib['mape']:.1f}% / {100 * calib['median_ape']:.1f}%",
406     },
407 ]
408 q4_audit = json.loads((fc_dir / "allocation_audit.json").read_text(encoding="utf-8"))
409 ver_rows.append({"item": "Q4 分配问题数 (跨日 + 单元-日) ", "value": f"{q4_audit['groups']}"})
410 ver_rows.append({"item": "Q4 独立校验", "value": "全部通过" if q4_audit["ok"] else "存在失败"})
411 cls_check = json.loads((cls_dir / "classification_check.json").read_text(encoding="utf-8"))
412 ver_rows.append(
413     {"item": "Q2 分类独立复核 (规则重推 / 无效词定义 / 计数) ", "value": "通过" if cls_check["ok"] else "失败"},
414 )
415 write_table(
416     ctx,
417     "tab_verification",
418     pd.DataFrame(ver_rows),
419     [{"item", "检验项", "raw"}, {"value", "结果", "raw"}],
420     align="p{0.72\\linewidth}l",
421 )
422 names.append("tab_verification")
423
424 bt = pd.read_parquet(fc_dir / "backtest_summary.parquet")
425 tnames = {
426     "log_eff": "效率乘子  $\\log \\var{epsilon}$ ",
427     "log_cpc": " $\\log$  CPC",
428     "logit_ctr": "logit CTR",
429     "logit_top": "logit 上方位占比",
430     "logit_first": "logit 首位占比",
431     "log_clicks": " $\\log$  点击量",
432 }
433 mnames = {"calendar": "日历回归", "seasonal_naive": "季节朴素", "mean28": "28 日均值"}
434 bt["target_name"] = bt["target"].map(tnames)
435 bt["model_name"] = bt["model"].map(mnames)
436 bt["cov_pct"] = 100 * bt["coverage80"]
437 bt["pinball"] = bt["pinball10"] + bt["pinball90"]
438 bt = bt.sort_values(["target", "model"])
439 write_table(

```

```

440     ctx,
441     "tab_backtest",
442     bt,
443     [
444         ("target_name", "目标", "raw"),
445         ("model_name", "模型", "s"),
446         ("mae", "MAE", ".3f"),
447         ("rmse", "RMSE", ".3f"),
448         ("pinball", "Pinball(0.1+0.9)", ".3f"),
449         ("cov_pct", "80\\% 覆盖率/\\%", ".1f"),
450     ],
451 )
452 names.append("tab_backtest")
453
454 days = pd.read_parquet(fc_dir / "plan_by_unit_day.parquet")
455 d = (
456     days.groupby("date")
457     .agg(
458         units=("推广单元ID", "nunique"),
459         spend=("spend", "sum"),
460         clicks=("clicks_mean", "sum"),
461         clicks_lo=("clicks_q10", "sum"),
462         clicks_hi=("clicks_q90", "sum"),
463         imp=("imp_mean", "sum"),
464         imp_lo=("imp_q10", "sum"),
465         imp_hi=("imp_q90", "sum"),
466         views=("views_mean", "sum"),
467         regs=("regs_mean", "sum"),
468         regs_lo=("regs_q10", "sum"),
469         regs_hi=("regs_q90", "sum"),
470     )
471     .reset_index()
472 )
473 d["cpc"] = d["spend"] / d["clicks"]
474 d["cpc_range"] = [f"[{s} / {hi:.2f}], {s} / {lo:.2f}]" for s, lo, hi in zip(d["spend"], d["clicks_lo"], d["clicks_hi"])]
475 d["clicks_range"] = [f"[{lo:.0f}, {hi:.0f}]" for lo, hi in zip(d["clicks_lo"], d["clicks_hi"])]
476 d["imp_range"] = [f"[{lo:.0f}, {hi:.0f}]" for lo, hi in zip(d["imp_lo"], d["imp_hi"])]
477 d["regs_range"] = [f"[{lo:.0f}, {hi:.0f}]" for lo, hi in zip(d["regs_lo"], d["regs_hi"])]
478 d["position"] = [
479     float(np.average(g["position_mean"], weights=np.maximum(g["clicks_mean"], 1e-9)))
480     for _, g in days.groupby("date")
481 ]
482 write_table(
483     ctx,
484     "tab_q4_daily",
485     d,
486     [
487         ("date", "日期", "s"),
488         ("units", "单元数", "d"),
489         ("spend", "投入/元", ".1f"),
490         ("cpc", "CPC/元", ".2f"),
491         ("cpc_range", "CPC 范围", "s"),
492         ("imp", "展现量", "d"),
493         ("imp_range", "展现范围", "s"),
494         ("position", "展位", ".2f"),
495         ("clicks", "点击量", "d"),
496         ("clicks_range", "点击范围", "s"),
497         ("views", "浏览量", "d"),
498         ("regs", "注册量", ".0f"),
499         ("regs_range", "注册范围", "s"),
500     ],
501 )
502 names.append("tab_q4_daily")

```

```

503
504 per = pd.DataFrame(json.loads((fc_dir / "plan_by_unit.json").read_text(encoding="utf-8")))
505 per["unit"] = per["推广单元ID"].astype(str)
506 per["regs_range"] = [f"[{lo:.0f}, {hi:.0f}]" for lo, hi in zip(per["regs_q10"], per["regs_q90"])]
507 per["clicks_range"] = [f"[{lo:.0f}, {hi:.0f}]" for lo, hi in zip(per["clicks_q10"], per["clicks_q90"])]
508 per["cpc"] = per["spent"] / per["clicks_mean"]
509 per = per.sort_values("budget", ascending=False)
510 write_table(
511     ctx,
512     "tab_q4_units",
513     per,
514     [
515         ("unit", "推广单元ID", "s"),
516         ("budget", "7 日预算/元", ".1f"),
517         ("gamma", "$\\gamma$", ".3f"),
518         ("eligible", "候选词", "d"),
519         ("cpc", "CPC/元", ".2f"),
520         ("clicks_mean", "点击量", ".0f"),
521         ("clicks_range", "点击范围", "s"),
522         ("regs_mean", "注册量", ".1f"),
523         ("regs_range", "注册范围", "s"),
524     ],
525 )
526 names.append("tab_q4_units")
527
528 ctx.write_json("tables_index.json", names)
529 return {"tables": len(names)}

```

pyproject.toml

```

1 [project]
2 name = "cumcm2026-e"
3 version = "0.1.0"
4 description = "SEM 广告投放策略 — cloud-only reproducible modelling study (CUMCM 2026 Problem E)"
5 requires-python = ">=3.11"
6 license = "Unlicense"
7
8 [tool.ruff]
9 line-length = 120
10 target-version = "py311"
11 extend-exclude = ["artifacts", "releases", "manuscript"]
12
13 [tool.ruff.lint]
14 select = ["E", "F", "W", "I", "B", "UP", "NPY", "RUF", "S102"]
15 ignore = ["RUF001", "RUF002", "RUF003", "E741", "B905"]
16
17 [tool.ruff.lint.pycodestyle]
18 # E501 is measured in display width (CJK characters count double); Chinese paper strings need headroom
19 max-line-length = 150
20
21 [tool.ruff.lint.per-file-ignores]
22 "tests/**" = ["S101", "B011"]
23
24 [tool.mypy]
25 python_version = "3.11"
26 ignore_missing_imports = true
27 check_untyped_defs = true
28 warn_unused_ignores = false
29 no_implicit_optional = true
30 exclude = "(artifacts|releases|manuscript)/"
31

```

```

32 [tool.pytest.ini_options]
33 testpaths = ["tests"]
34 addopts = "-p no:cacheprovider -q"
35 markers = ["integration: needs the mounted repository inputs (runs in Modal)"]
36
37 [tool.coverage.run]
38 source = ["forge", "pipelines"]

```

tests/__init__.py

tests/conftest.py

```

1  """Shared test configuration: deterministic hypothesis profile, no on-disk example database."""
2
3  from __future__ import annotations
4
5  import os
6  from pathlib import Path
7
8  import pytest
9  from hypothesis import HealthCheck, settings
10
11 settings.register_profile(
12     "forge", database=None, deadline=None, max_examples=60, suppress_health_check=[HealthCheck.too_slow]
13 )
14 settings.load_profile("forge")
15
16 REPO = Path(os.environ.get("FORGE_REPO", Path(__file__).resolve().parents[1]))
17
18
19 @pytest.fixture(scope="session")
20 def repo() -> Path:
21     return REPO
22
23
24 @pytest.fixture
25 def tmp_repo(tmp_path: Path) -> Path:
26     """A minimal repository layout (configs only) for runtime tests that must not touch real data."""
27     (tmp_path / "configs").mkdir()
28     (tmp_path / "configs" / "default.toml").write_text("[run]\nseed = 7\n[paper]\nsources = []\n", encoding="utf-8")
29     (tmp_path / "configs" / "project.json").write_text(
30         '{"letter": "T", "package": "t", "problem_pdf": "T.pdf", "attachments_dir": "att", '
31         '"templates_dir": "att/tpl", "result_files": []}',
32         encoding="utf-8",
33     )
34     (tmp_path / "att" / "tpl").mkdir(parents=True)
35     return tmp_path

```

tests/integration/__init__.py

tests/integration/test_ingest_validate.py

```
1 """End-to-end ingest + validate against the real attachments (runs inside Modal where /repo exists)."""
2
3 from __future__ import annotations
4
5 import json
6 import os
7 from pathlib import Path
8
9 import pytest
10
11 from forge import runner
12
13 REPO = Path(os.environ.get("FORGE_REPO", "/repo"))
14 pytestmark = pytest.mark.integration
15
16
17 @pytest.mark.skipif(not (REPO / "configs" / "project.json").exists(), reason="repository inputs not mounted")
18 def test_ingest_then_validate_on_real_inputs(tmp_path: Path) -> None:
19     vol = tmp_path / "vol"
20     ingest = runner.execute("ingest", "itest", {"code_ref": {"git_sha": "test"}}, False, REPO, vol)
21     assert ingest["status"] == "completed" and ingest["metrics"]["files"] >= 1
22     inventory = json.loads((vol / "runs" / "itest" / "ingest" / "inventory.json").read_text(encoding="utf-8"))
23     assert any(not f["template"] for f in inventory["files"])
24     validate = runner.execute("validate", "itest", {}, False, REPO, vol)
25     assert validate["status"] == "completed" and validate["metrics"]["ok"]
```

tests/unit/__init__.py

tests/unit/test_config.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import pytest
6
7 from forge.config import Config, deep_merge, load_config
8
9
10 def test_deep_merge_is_recursive_and_non_destructive() -> None:
11     base = {"a": {"x": 1, "y": 2}, "b": 1}
12     merged = deep_merge(base, {"a": {"y": 3}, "c": 4})
13     assert merged == {"a": {"x": 1, "y": 3}, "b": 1, "c": 4}
14     assert base == {"a": {"x": 1, "y": 2}, "b": 1}
15
16
17 def test_config_dotted_access_and_digest() -> None:
18     cfg = Config({"paper": {"sources": ["q1"], "max_body_pages": 30}})
19     assert cfg.get("paper.max_body_pages") == 30
20     assert cfg.get("paper.missing", "d") == "d"
21     assert cfg["paper.sources"] == ["q1"]
22     with pytest.raises(KeyError):
23         cfg["nope.nope"]
24     assert cfg.digest == Config({"paper": {"max_body_pages": 30, "sources": ["q1"]}}).digest
25
```

```

26
27 def test_load_config_with_profile_and_overrides(tmp_repo: Path) -> None:
28     (tmp_repo / "configs" / "fast.toml").write_text("[run]\nseed = 99\n", encoding="utf-8")
29     cfg = load_config(tmp_repo, profile="fast", overrides={"paper": {"sources": ["x"]}})
30     assert cfg.get("run.seed") == 99
31     assert cfg.get("paper.sources") == ["x"]

```

tests/unit/test_contracts.py

```

1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from forge.contracts import Column, FrameContract, validate_frame
9
10 CONTRACT = FrameContract(
11     name="t",
12     columns=(
13         Column("t", "int", min=0, unique=True, monotonic="increasing", step=60),
14         Column("v", "float", min=0, max=1),
15     ),
16     min_rows=2,
17 )
18
19
20 def test_valid_frame_passes() -> None:
21     df = pd.DataFrame({"t": [0, 60, 120], "v": [0.1, 0.2, 0.3]})
22     report = validate_frame(df, CONTRACT)
23     assert report["ok"], report["errors"]
24
25
26 def test_violations_are_reported_precisely() -> None:
27     df = pd.DataFrame({"t": [0, 60, 60], "v": [0.1, 1.5, np.nan], "extra": [1, 2, 3]})
28     report = validate_frame(df, CONTRACT)
29     joined = " | ".join(report["errors"])
30     assert not report["ok"]
31     assert "unexpected columns" in joined
32     assert "duplicated" in joined
33     assert "max 1.5 > 1" in joined
34     assert "null" in joined
35     assert "not increasing" in joined
36
37
38 def test_column_names_are_stripped() -> None:
39     df = pd.DataFrame({"t ": [0, 60], " v": [0.5, 0.6]})
40     assert validate_frame(df, CONTRACT)["ok"]
41
42
43 @given(st.lists(st.floats(min_value=0, max_value=1, allow_nan=False), min_size=2, max_size=40))
44 def test_property_ranges_within_bounds_always_pass(values: list[float]) -> None:
45     df = pd.DataFrame({"t": np.arange(len(values)) * 60, "v": values})
46     assert validate_frame(df, CONTRACT)["ok"]
47
48
49 def test_regex_and_custom_checks() -> None:
50     contract = FrameContract(
51         "ids", (Column("id", "str", regex=r"[ABC]\d{3}"),), checks=(lambda df: ["custom"] if len(df) > 1 else [],)
52     )

```

```

53     assert validate_frame(pd.DataFrame({"id": ["A001"]}), contract)["ok"]
54     report = validate_frame(pd.DataFrame({"id": ["A001", "Z9"]}), contract)
55     assert any("violate" in e for e in report["errors"]) and "custom" in report["errors"]

```

tests/unit/test_e_allocate.py

```

1  from __future__ import annotations
2
3  import itertools
4
5  import numpy as np
6  import pytest
7  from hypothesis import given, settings
8  from hypothesis import strategies as st
9
10 from pipelines.e.allocate import (
11     equal_allocation,
12     objective_value,
13     proportional_allocation,
14     solve_group,
15     verify_allocation,
16     water_filling,
17 )
18 from pipelines.e.response import clicks_response, cpc_response, expected_position
19
20
21 def test_water_filling_matches_grid_search_on_small_instance() -> None:
22     a = np.array([3.0, 1.0, 2.0])
23     gamma = 0.7
24     caps = np.array([4.0, 10.0, 10.0])
25     budget = 9.0
26     x = water_filling(a, gamma, caps, budget)
27     assert abs(x.sum() - budget) < 1e-8 and (x <= caps + 1e-9).all()
28     best = -np.inf
29     grid = np.linspace(0, 9, 181)
30     for x1, x2 in itertools.product(grid, grid):
31         x3 = budget - x1 - x2
32         if x3 < 0 or x1 > caps[0] or x2 > caps[1] or x3 > caps[2]:
33             continue
34         best = max(best, objective_value(np.array([x1, x2, x3]), a, gamma))
35     assert objective_value(x, a, gamma) >= best - 1e-6
36
37
38 def test_water_filling_closed_form_without_caps() -> None:
39     a = np.array([1.0, 2.0, 4.0])
40     gamma = 0.5
41     x = water_filling(a, gamma, np.full(3, 1e9), 7.0)
42     # x_i proportional to a_i^(1/(1-gamma)) = a_i^2
43     expected = 7.0 * a**2 / (a**2).sum()
44     assert np.allclose(x, expected, rtol=1e-6)
45
46
47 def test_caps_bind_and_budget_not_exceeded() -> None:
48     a = np.array([10.0, 1.0])
49     x = water_filling(a, 0.8, np.array([1.0, 5.0]), 3.0)
50     assert abs(x[0] - 1.0) < 1e-9 and abs(x.sum() - 3.0) < 1e-9
51     x = water_filling(a, 0.8, np.array([1.0, 1.0]), 3.0)
52     assert np.allclose(x, [1.0, 1.0])
53
54
55 @settings(max_examples=40)

```

```

56 @given(
57     st.lists(st.floats(min_value=0.05, max_value=50.0), min_size=2, max_size=25),
58     st.floats(min_value=0.35, max_value=0.95),
59     st.floats(min_value=1.0, max_value=500.0),
60 )
61 def test_property_verifier_accepts_solver_output(weights: list[float], gamma: float, budget: float) -> None:
62     rng = np.random.default_rng(0)
63     w = np.array(weights)
64     s = np.linspace(1.0, 3.0, len(w))
65     caps = 4.0 * s
66     res = solve_group(w, s, gamma, caps, budget, min_spend=0.0)
67     a = w * s ** (-gamma)
68     sel = res["x"] > 0
69     report = verify_allocation(res["x"][sel], a[sel], gamma, caps[sel], budget, rng, n_transfers=30)
70     assert report["ok"], report["checks"]
71
72
73 def test_verifier_rejects_a_suboptimal_allocation() -> None:
74     rng = np.random.default_rng(1)
75     a = np.array([5.0, 1.0, 1.0])
76     caps = np.full(3, 100.0)
77     report = verify_allocation(np.array([1.0, 4.0, 4.0]), a, 0.6, caps, 9.0, rng)
78     assert not report["ok"]
79     names = {c["name"]: c["ok"] for c in report["checks"]}
80     assert not names["kkt_equal_marginals_interior"] or not names["no_improving_transfer_or_removal"]
81
82
83 def test_min_spend_drops_tiny_keywords_and_responds_budget() -> None:
84     w = np.array([100.0, 100.0, 0.001])
85     s = np.array([10.0, 10.0, 10.0])
86     res = solve_group(w, s, 0.8, 4 * s, 20.0, min_spend=0.5)
87     assert res["x"][2] == 0.0 and abs(res["spent"] - 20.0) < 1e-6 and res["n_selected"] == 2
88     assert (res["x"][res["x"] > 0] >= 0.5).all()
89
90
91 def test_min_spend_solution_beats_enumeration_of_selections() -> None:
92     rng = np.random.default_rng(5)
93     w = rng.uniform(0.5, 5.0, 6)
94     s = rng.uniform(0.5, 3.0, 6)
95     gamma, budget, m = 0.75, 4.0, 0.6
96     caps = 4 * s
97     res = solve_group(w, s, gamma, caps, budget, min_spend=m)
98     a = w * s ** (-gamma)
99     # brute force over selections: on a fixed subset the problem is convex; solve it with an independent
100     # general-purpose NLP solver (SLSQP) under the lower bound m
101     from scipy.optimize import minimize
102
103     best = -np.inf
104     for mask in range(1, 2**6):
105         idx = [i for i in range(6) if mask >> i & 1]
106         if m * len(idx) > budget + 1e-12:
107             continue
108         sub_a, sub_caps = a[idx], caps[idx]
109         x0 = np.clip(np.full(len(idx), budget / len(idx)), m, sub_caps)
110         res_nlp = minimize(
111             lambda y, sa=sub_a: -float(np.sum(sa * np.power(np.maximum(y, 1e-12), gamma))),
112             x0,
113             method="SLSQP",
114             bounds=[(m, c) for c in sub_caps],
115             constraints=[{"type": "ineq", "fun": lambda y: budget - y.sum()}],
116             options={"ftol": 1e-12, "maxiter": 500},
117         )
118         if res_nlp.success:

```



```

119         best = max(best, -float(res_nlp.fun))
120     assert res["objective"] >= best * (1 - 1e-6)
121     report = verify_allocation(res["x"], a, gamma, caps, budget, rng, floor=m)
122     assert report["ok"], report["checks"]
123
124
125 def test_baselines_and_response_helpers() -> None:
126     s = np.array([1.0, 3.0])
127     assert np.allclose(proportional_allocation(s, 8.0), [2.0, 6.0])
128     assert np.allclose(equal_allocation(2, 8.0), [4.0, 4.0])
129     c = np.array([10.0, 30.0])
130     assert np.allclose(clicks_response(s, c, s, 0.8), c) # passes through the operating point
131     assert cpc_response(np.array([2.0, 6.0]), c, s, 0.8)[0] > cpc_response(np.array([1.0, 3.0]), c, s, 0.8)[0]
132     pos = expected_position(np.array([1.0, 0.0, 0.5]), np.array([1.0, 0.0, 0.25]))
133     assert pos[0] == pytest.approx(1.0) and pos[1] == pytest.approx(4.0) and 1.0 < pos[2] < 4.0

```

tests/unit/test_e_calendar_data.py

```

1 from __future__ import annotations
2
3 import math
4
5 import pandas as pd
6
7 from pipelines.e.calendar import ADJUSTED_WORKDAYS, HOLIDAY_BLOCKS, day_features, holiday_map
8 from pipelines.e.data import parse_bounce, parse_duration
9
10
11 def test_holiday_calendar_matches_state_council_notice() -> None:
12     hm = holiday_map()
13     assert len(hm) == 1 + 8 + 3 + 5 + 3 + 8
14     assert hm[pd.Timestamp("2025-01-28")] == "春节" and hm[pd.Timestamp("2025-10-08")] == "国庆节中秋节"
15     assert pd.Timestamp("2025-02-08") not in hm and "2025-02-08" in ADJUSTED_WORKDAYS
16     assert all(pd.Timestamp(s) <= pd.Timestamp(e) for s, e in HOLIDAY_BLOCKS.values())
17
18
19 def test_day_features_flags() -> None:
20     f = day_features(
21         ["2025-01-27", "2025-01-28", "2025-02-04", "2025-02-05", "2025-02-08", "2025-02-09", "2026-09-11"]
22     ).set_index("date")
23     assert bool(f.loc["2025-01-27", "is_pre_holiday"]) and not bool(f.loc["2025-01-27", "is_holiday"])
24     assert bool(f.loc["2025-01-28", "is_holiday"]) and f.loc["2025-01-28", "day_type"] == "holiday"
25     assert bool(f.loc["2025-02-05", "is_post_holiday"]) and f.loc["2025-02-05", "day_type"] == "workday"
26     assert f.loc["2025-02-08", "day_type"] == "adjusted_workday" and bool(f.loc["2025-02-08", "is_workday"])
27     assert f.loc["2025-02-09", "day_type"] == "weekend"
28     assert f.loc["2026-09-11", "weekday"] == 4 and f.loc["2026-09-11", "day_type"] == "workday"
29
30
31 def test_parsers_handle_every_encoding_in_sheet3() -> None:
32     assert math.isnan(parse_bounce("/")) and parse_bounce("0.6109") == 0.6109 and parse_bounce("1") == 1.0
33     assert parse_bounce(" 45 ") == 0.45
34     assert math.isnan(parse_duration("/")) and parse_duration(" 00:03:16 ") == 196.0 and parse_duration("0") == 0.0
35     assert parse_duration(0) == 0.0

```

tests/unit/test_e_classify.py

```
1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from pipelines.e.classify import (
9     LABELS,
10     assign_labels,
11     benefit_components,
12     benefit_index,
13     entropy_weights,
14     jenks_break,
15     verify_classification,
16 )
17
18
19 def _frame(n: int, rng: np.random.Generator) -> pd.DataFrame:
20     spend = np.where(rng.random(n) < 0.3, 0.0, rng.lognormal(2, 1.5, n))
21     clicks = np.where(spend > 0, rng.poisson(spend / 1.2) + 1, 0)
22     views = np.where(clicks > 0, rng.poisson(clicks * 1.5), 0)
23     bounce = np.where(clicks > 0, rng.random(n), np.nan)
24     return pd.DataFrame(
25         {
26             "序号": np.arange(1, n + 1),
27             "关键词": np.arange(1, n + 1),
28             "方案ID": 1,
29             "推广单元ID": 7,
30             "spend": spend,
31             "clicks": clicks,
32             "views": views,
33             "bounce": bounce,
34             "duration_s": np.where(clicks > 0, rng.random(n) * 300, np.nan),
35             "engagement": np.where(clicks > 0, clicks * (1 - np.nan_to_num(bounce)) * 60, 0.0),
36         }
37     )
38
39
40 def test_jenks_break_matches_brute_force() -> None:
41     rng = np.random.default_rng(1)
42     x = np.concatenate([rng.normal(0, 1, 40), rng.normal(6, 1, 30)])
43     thr = jenks_break(x)
44     xs = np.sort(x)
45     best = min(range(1, len(xs)), key=lambda k: np.var(xs[:k]) * k + np.var(xs[k:]) * (len(xs) - k))
46     assert xs[best - 1] < thr < xs[best]
47     assert 1.5 < thr < 4.5
48
49
50 def test_entropy_weights_sum_to_one_and_ignore_constant_columns() -> None:
51     rng = np.random.default_rng(2)
52     X = np.column_stack([rng.random(50), np.ones(50), rng.random(50) ** 4])
53     w = entropy_weights(X)
54     assert abs(w.sum() - 1) < 1e-12 and w[1] == 0.0 and (w >= 0).all()
55
56
57 def test_labels_follow_quadrant_rule_and_checker_agrees() -> None:
58     rng = np.random.default_rng(3)
59     kw = _frame(300, rng)
60     comp = benefit_components(kw, {7: 0.02})
61     benefit, _ = benefit_index(comp)
```

```

62     active = kw["spend"] > 0
63     ct = float(np.median(np.log10(kw.loc[active, "spend"])))
64     bt = float(np.median(benefit[active]))
65     labels = assign_labels(kw, benefit, ct, bt)
66     table = kw.assign(benefit=benefit, label=labels)
67     report = verify_classification(table, ct, bt)
68     assert report["ok"], report
69     assert set(labels.unique()) <= set(LABELS)
70     assert (labels[~active] == "无效词").all()
71     gold = table["label"] == "黄金词"
72     assert (gold["spend"] <= 10**ct).all() and (gold["benefit"] >= bt).all()
73
74
75 @given(st.integers(min_value=20, max_value=200), st.integers(min_value=0, max_value=1000))
76 def test_property_every_keyword_gets_exactly_one_label(n: int, seed: int) -> None:
77     rng = np.random.default_rng(seed)
78     kw = _frame(n, rng)
79     comp = benefit_components(kw, {7: 0.01})
80     benefit, w = benefit_index(comp)
81     assert abs(w.sum() - 1) < 1e-9
82     labels = assign_labels(kw, benefit, 1.0, 0.5)
83     assert len(labels) == n and labels.isin(LABELS).all()
84     counts = labels.value_counts()
85     assert counts.sum() == n

```

tests/unit/test_e_forecast.py

```

1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5
6 from pipelines.e.calendar import day_features
7 from pipelines.e.forecast import (
8     CalendarModel,
9     back_transform,
10    pinball,
11    rolling_backtest,
12    seasonal_naive,
13    simulate_group,
14    summarise,
15 )
16
17
18 def _series(n: int = 200, seed: int = 0) -> pd.DataFrame:
19     rng = np.random.default_rng(seed)
20     dates = pd.date_range("2025-03-01", periods=n, freq="D")
21     f = day_features(dates)
22     y = 0.3 * (f["weekday"] >= 5).to_numpy() - 0.5 * f["is_holiday"].to_numpy() + 0.1 * rng.standard_normal(n)
23     out = pd.DataFrame({"date": dates, "log_eff": y})
24     return pd.concat([out, f.drop(columns=["date"])], axis=1)
25
26
27 def test_calendar_model_recovers_weekend_effect() -> None:
28     s = _series()
29     m = CalendarModel().fit(s, "log_eff")
30     future = day_features(pd.date_range("2026-09-11", "2026-09-17"))
31     pred = m.predict(future)
32     weekend = future["weekday"].to_numpy() >= 5
33     assert pred[weekend].mean() - pred[~weekend].mean() > 0.2
34     assert 0.05 < m.sd < 0.2

```

```

35
36
37 def test_backtest_prefers_calendar_model_on_calendar_driven_series() -> None:
38     s = _series(260)
39     origins = list(pd.date_range("2025-08-01", "2025-10-30", freq="7D"))
40     bt = rolling_backtest(s, "log_eff", origins)
41     agg = bt.groupby("model")["mae"].mean()
42     assert agg["calendar"] < agg["mean28"]
43     assert 0.5 <= bt[bt["model"] == "calendar"]["coverage80"].mean() <= 1.0
44
45
46 def test_pinball_and_transforms() -> None:
47     y = np.array([1.0, 2.0])
48     assert pinball(y, np.array([0.0, 3.0]), 0.5) == 0.5 * (1.0 + 1.0) / 2
49     assert np.allclose(back_transform(np.log([2.0, 3.0]), "exp"), [2.0, 3.0])
50     assert np.allclose(back_transform(np.array([0.0]), "expit"), [0.5])
51     s = _series(60)
52     assert len(seasonal_naive(s, "log_eff", s.tail(7))) == 7
53
54
55 def test_simulation_shapes_and_monotone_ranges() -> None:
56     rng = np.random.default_rng(3)
57     params = pd.DataFrame({"c": [10.0, 5.0], "s": [20.0, 8.0], "rho": [0.02, 0.05], "depth": [1.5, 2.0]})
58     sim = simulate_group(
59         np.array([20.0, 8.0]),
60         params,
61         0.8,
62         0.0,
63         0.2,
64         0.2,
65         0.0,
66         0.1,
67         {"a": 0.0, "b": 0.3},
68         {"a": -1.0, "b": 0.3},
69         0.2,
70         rng,
71         n_scen=400,
72     )
73     assert sim["clicks"].shape == (400, 2)
74     st = summarise(sim["clicks"])
75     assert (st["q10"] <= st["q50"]).all() and (st["q50"] <= st["q90"]).all()
76     assert abs(st["mean"][0] / 10.0 - 1) < 0.15
77     assert ((sim["position"] >= 1.0) & (sim["position"] <= 4.0)).all()

```

tests/unit/test_hashing.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from forge.hashing import hash_tree, sha256_bytes, sha256_json, tree_digest
9
10
11 @given(st.binary())
12 def test_sha256_bytes_is_deterministic(data: bytes) -> None:
13     assert sha256_bytes(data) == sha256_bytes(data)
14     assert len(sha256_bytes(data)) == 64
15

```

```

16
17 @given(
18     st.dictionaries(
19         st.text(min_size=1, max_size=8), st.integers() | st.floats(allow_nan=False) | st.text(max_size=5), max_size=6
20     )
21 )
22 def test_sha256_json_ignores_key_order(mapping: dict) -> None:
23     reversed_mapping = dict(reversed(list(mapping.items())))
24     assert sha256_json(mapping) == sha256_json(reversed_mapping)
25
26
27 def test_hash_tree_ignores_records_and_is_sorted(tmp_path: Path) -> None:
28     (tmp_path / "b.txt").write_text("b")
29     (tmp_path / "a.txt").write_text("a")
30     (tmp_path / "manifest.json").write_text("{}")
31     (tmp_path / "__pycache__").mkdir()
32     (tmp_path / "__pycache__" / "x.pyc").write_bytes(b"\x00")
33     tree = hash_tree(tmp_path, ignore=("manifest.json", "__pycache__"))
34     assert list(tree) == ["a.txt", "b.txt"]
35     digest = tree.digest()
36     (tmp_path / "manifest.json").write_text('{"changed": true}')
37     assert tree.digest(hash_tree(tmp_path, ignore=("manifest.json", "__pycache__"))) == digest

```

tests/unit/test_manifest.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from forge.manifest import Manifest
6
7
8 def test_manifest_lifecycle_and_verification(tmp_path: Path) -> None:
9     stage_dir = tmp_path / "stage"
10     stage_dir.mkdir()
11     (stage_dir / "out.json").write_text("{}")
12     (stage_dir / "events.jsonl").write_text("{}\n")
13     manifest = Manifest.start("run1", "stage", {"p": 1}, {"git_sha": "abc"}, "cfg", {"ingest": "d"}, {})
14     assert manifest.status == "running" and manifest.runtime["python"]
15     manifest.complete(stage_dir, {"m": 1.5})
16     assert manifest.status == "completed"
17     assert list(manifest.outputs) == ["out.json"]
18     manifest.write(stage_dir / "manifest.json")
19     loaded = Manifest.read(stage_dir / "manifest.json")
20     assert loaded.outputs_digest == manifest.outputs_digest
21     assert loaded.verify_outputs(stage_dir) == []
22     (stage_dir / "out.json").write_text('{"x": 1}')
23     assert loaded.verify_outputs(stage_dir) == ["sha256 mismatch: out.json"]
24
25
26 def test_manifest_failure_records_traceback() -> None:
27     manifest = Manifest.start("run1", "stage", {}, {}, "cfg", {}, {})
28     try:
29         raise ValueError("boom")
30     except ValueError as exc:
31         manifest.fail(exc)
32     assert manifest.status == "failed"
33     assert manifest.error and manifest.error["type"] == "ValueError"
34     assert manifest.duration_s is not None

```

tests/unit/test_packaging_tex.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from forge.packaging import build_zip, code_members, run_members
6 from forge.tex import aux_page, summarize_log, tex_escape
7
8
9 def test_zip_is_deterministic(tmp_path: Path) -> None:
10     src = tmp_path / "src"
11     src.mkdir()
12     (src / "b.py").write_text("print(2)")
13     (src / "a.py").write_text("print(1)")
14     (src / "__pycache__").mkdir()
15     (src / "__pycache__" / "a.pyc").write_bytes(b"x")
16     members = code_members(src, ["**/*.py"])
17     assert [m[0] for m in members] == ["a.py", "b.py"]
18     one = build_zip(members, tmp_path / "one.zip")
19     two = build_zip(list(reversed(members)), tmp_path / "two.zip")
20     assert one["sha256"] == two["sha256"] and one["members"] == 2
21
22
23 def test_run_members_prefix(tmp_path: Path) -> None:
24     run = tmp_path / "runs" / "r1"
25     (run / "q1").mkdir(parents=True)
26     (run / "q1" / "out.json").write_text("{}")
27     assert run_members(run, ["q1", "missing"]) == [(("runs/r1/q1/out.json", run / "q1" / "out.json"))]
28
29
30 def test_log_summary_and_aux_page(tmp_path: Path) -> None:
31     log = (
32         "LaTeX Warning: Reference `fig:x' on page 3 undefined on input line 10.\n"
33         "Overfull \hbox (12.5pt too wide) in paragraph\n"
34         "Missing character: There is no  in font Latin Modern!\n"
35         "! Undefined control sequence.\n"
36         "Output written on main.pdf (7 pages).\n"
37     )
38     summary = summarize_log(log)
39     assert summary["undefined_references"] == ["fig:x"]
40     assert summary["overfull_max_pt"] == 12.5 and summary["missing_characters"] == 1
41     assert summary["errors"] == ["Undefined control sequence."] and summary["pages_from_log"] == 7
42     aux = tmp_path / "main.aux"
43     aux.write_text("\n\\newlabel{page:body-end}{{}{27}}{{page:body-end}}\n\\newlabel{sec:a}{{2.1}{5}}{{}}\n")
44     assert aux_page(aux, "page:body-end") == 27 and aux_page(aux, "sec:a") == 5 and aux_page(aux, "nope") is None
45
46
47 def test_tex_escape() -> None:
48     assert tex_escape("a_b & 100% #1") == r"a\_b \& 100\% \#1"
```

tests/unit/test_runner.py

```
1 from __future__ import annotations
2
3 import json
4 from pathlib import Path
5
6 import pytest
7
8 from forge import runner
```

```

9 from forge.context import StageContext
10
11
12 @pytest.fixture(autouse=True)
13 def isolated_registry(monkeypatch: pytest.MonkeyPatch) -> None:
14     monkeypatch.setattr(runner, "STAGES", {})
15     monkeypatch.setattr(runner, "discover", lambda repo: None)
16
17
18 def test_execute_records_manifest_skips_reruns_and_enforces_deps(tmp_repo: Path, tmp_path: Path) -> None:
19     vol = tmp_path / "vol"
20
21     @runner.stage("first", description="first stage")
22     def first(ctx: StageContext) -> dict:
23         ctx.out("a.txt").write_text("hello")
24         ctx.number("Answer", 42)
25         return {"n": 1}
26
27     @runner.stage("second", deps=("first",))
28     def second(ctx: StageContext) -> dict:
29         assert (ctx.dep("first") / "a.txt").read_text() == "hello"
30         return {"ok": True}
31
32     with pytest.raises(RuntimeError):
33         runner.execute("second", "r1", {}, False, tmp_repo, vol)
34     result = runner.execute("first", "r1", {"code_ref": {"git_sha": "x"}}, False, tmp_repo, vol)
35     assert result["status"] == "completed" and result["metrics"] == {"n": 1}
36     manifest = json.loads((vol / "runs" / "r1" / "first" / "manifest.json").read_text())
37     assert (
38         manifest["status"] == "completed" and "a.txt" in manifest["outputs"] and "numbers.json" in manifest["outputs"]
39     )
40     assert json.loads((vol / "runs" / "r1" / "first" / "numbers.json").read_text()) == {"Answer": "42"}
41     assert runner.execute("first", "r1", {}, False, tmp_repo, vol)["status"] == "skipped"
42     assert runner.execute("second", "r1", {}, False, tmp_repo, vol)["status"] == "completed"
43     combined = runner.execute("all", "r1", {"stages": ["first", "second"]}, False, tmp_repo, vol)
44     assert combined["status"] == "completed" and [r["status"] for r in combined["results"]] == ["skipped", "skipped"]
45
46
47 def test_failed_stage_writes_failed_manifest(tmp_repo: Path, tmp_path: Path) -> None:
48     vol = tmp_path / "vol"
49
50     @runner.stage("bad")
51     def bad(ctx: StageContext) -> dict:
52         raise ValueError("boom")
53
54     with pytest.raises(ValueError):
55         runner.execute("bad", "r1", {}, False, tmp_repo, vol)
56     manifest = json.loads((vol / "runs" / "r1" / "bad" / "manifest.json").read_text())
57     assert manifest["status"] == "failed" and manifest["error"]["message"] == "boom"
58
59
60 def test_from_run_copies_dependency_outputs(tmp_repo: Path, tmp_path: Path) -> None:
61     vol = tmp_path / "vol"
62
63     @runner.stage("base")
64     def base(ctx: StageContext) -> dict:
65         ctx.out("x.txt").write_text("x")
66         return {}
67
68     @runner.stage("child", deps=("base",))
69     def child(ctx: StageContext) -> dict:
70         return {"seen": (ctx.dep("base") / "x.txt").read_text()}
71

```

```

72 runner.execute("base", "old", {}, False, tmp_repo, vol)
73 result = runner.execute("child", "new", {"from_run": "old"}, False, tmp_repo, vol)
74 assert result["metrics"] == {"seen": "x"}

```

tests/unit/test_xlsx.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import openpyxl
6 import pytest
7
8 from forge.xlsx import SheetContract, WorkbookContract, check_workbook, read_sheets, sheet_headers, write_result
9
10
11 def _template(path: Path) -> Path:
12     wb = openpyxl.Workbook()
13     ws = wb.active
14     ws.title = "温度"
15     ws.append(["时间\\到药材中心的距离", 0, 0.1, "...", 2])
16     ws.append([1, None, None, None, None])
17     ws2 = wb.create_sheet("水分浓度")
18     ws2.append(["时间\\到药材中心的距离", 0, 0.1, "...", 2])
19     wb.save(path)
20     return path
21
22
23 def test_write_result_keeps_all_rounds_and_passes_contract(tmp_path: Path) -> None:
24     template = _template(tmp_path / "tpl.xlsx")
25     header = ["ignored", 0.0, 0.1, 0.2]
26     rows = [[t, 28.123456, 28.2, 28.3] for t in range(1, 6)]
27     info = write_result(template, tmp_path / "out.xlsx", {"温度": (header, rows), "水分浓度": (header, rows)})
28     assert info["sheets"]["温度"]["rows"] == 5
29     headers = sheet_headers(tmp_path / "out.xlsx")
30     assert headers["温度"][0] == "时间\\到药材中心的距离"
31     assert headers["温度"][1:] == [0.0, 0.1, 0.2]
32     frames = read_sheets(tmp_path / "out.xlsx")
33     assert float(frames["温度"].iloc[0, 1]) == pytest.approx(28.1235)
34     contract = WorkbookContract(
35         "out.xlsx",
36         "tpl.xlsx",
37         (
38             SheetContract("温度", min_rows=5, max_rows=5, header_len=4, numeric_from_col=0),
39             SheetContract("水分浓度", min_rows=5, header_len=4, numeric_from_col=0),
40         ),
41     )
42     report = check_workbook(tmp_path / "out.xlsx", contract, template)
43     assert report["ok"], report["errors"]
44
45
46 def test_check_workbook_flags_blank_and_text_cells(tmp_path: Path) -> None:
47     template = _template(tmp_path / "tpl.xlsx")
48     rows = [[1, 1.0, None, "x"], [2, 2.0, 2.0, 2.0]]
49     write_result(template, tmp_path / "out.xlsx", {"温度": ([ "h", 0, 0.1, 0.2 ], rows)})
50     contract = WorkbookContract("out.xlsx", "tpl.xlsx", (SheetContract("温度", min_rows=2, numeric_from_col=0),))
51     report = check_workbook(tmp_path / "out.xlsx", contract, template)
52     joined = " ".join(report["errors"])
53     assert not report["ok"] and "blank" in joined and "non-numeric" in joined
54
55

```



```

56 def test_write_result_rejects_unknown_sheet(tmp_path: Path) -> None:
57     template = _template(tmp_path / "tpl.xlsx")
58     with pytest.raises(KeyError):
59         write_result(template, tmp_path / "out.xlsx", {"nope": ([ "h" ], [])})
60
61
62 def test_write_only_path_for_large_sheets(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> None:
63     import forge.xlsx as mod
64
65     monkeypatch.setattr(mod, "BIG_SHEET_CELLS", 10)
66     template = _template(tmp_path / "tpl.xlsx")
67     rows = [[t, 1.0, 2.0] for t in range(1, 8)]
68     info = write_result(template, tmp_path / "big.xlsx", {"温度": ([ "h" ], 0, 0.1), rows})
69     assert info["write_only"]
70     headers = sheet_headers(tmp_path / "big.xlsx")
71     assert list(headers) == ["温度", "水分浓度"]
72     assert headers["温度"][0] == "时间\\到药材中心的距离"

```

tools/__init__.py

tools/cli.py

```

1  #!/usr/bin/env python3
2  """Forge control plane (local). Never computes: submits cloud jobs, downloads, verifies hashes.
3
4  Examples
5      python3 tools/cli.py run ingest                                # new run id, small container
6      python3 tools/cli.py run validate --run-id <id>              # continue the same run
7      python3 tools/cli.py run q1,q2 --size medium                 # sequential stages in one container
8      python3 tools/cli.py run q3 --size large --spawn             # detached; poll with `wait`/`status`
9      python3 tools/cli.py exec --code 'print(sorted(p.name for p in RUN_DIR.iterdir()))'
10     python3 tools/cli.py status                                    # manifests of the last run
11     python3 tools/cli.py download --stage results                 # into artifacts/, verified against manifests
12     python3 tools/cli.py release --version v1.0.0                 # deliverables into releases/v1.0.0/
13
14
15  from __future__ import annotations
16
17  import argparse
18  import hashlib
19  import json
20  import os
21  import shutil
22  import subprocess
23  import sys
24  from datetime import datetime
25  from pathlib import Path
26
27  REPO = Path(__file__).resolve().parents[1]
28  PROJECT = json.loads((REPO / "configs" / "project.json").read_text(encoding="utf-8"))
29  APP_FILE = REPO / "forge" / "modal_app.py"
30  STATE_DIR = REPO / ".forge"
31  LAST_RUN = STATE_DIR / "last_run"
32  MODAL = shutil.which("modal") or str(Path.home() / ".local" / "bin" / "modal")
33
34
35  def _sh(cmd: list[str], **kw) -> subprocess.CompletedProcess:

```

```

36     print("$ " + " ".join(cmd), flush=True)
37     return subprocess.run(cmd, cwd=REPO, check=False, **kw)
38
39
40 def git_ref() -> dict:
41     def out(*args: str) -> str:
42         try:
43             return subprocess.run(["git", *args], cwd=REPO, capture_output=True, text=True, check=False).stdout.strip()
44         except OSError:
45             return ""
46
47     sha = out("rev-parse", "HEAD") or "nogit"
48     return {
49         "git_sha": sha,
50         "branch": out("rev-parse", "--abbrev-ref", "HEAD"),
51         "dirty": bool(out("status", "--porcelain")),
52     }
53
54
55 def new_run_id() -> str:
56     return datetime.now().strftime("%Y%m%d-%H%M%S") + "-" + git_ref()["git_sha"][:7]
57
58
59 def remember(run_id: str) -> None:
60     STATE_DIR.mkdir(exist_ok=True)
61     LAST_RUN.write_text(run_id, encoding="utf-8")
62
63
64 def resolve_run_id(explicit: str | None, create: bool) -> str:
65     if explicit:
66         remember(explicit)
67         return explicit
68     if create or not LAST_RUN.exists():
69         run_id = new_run_id()
70         remember(run_id)
71         return run_id
72     return LAST_RUN.read_text(encoding="utf-8").strip()
73
74
75 def parse_params(items: list[str] | None) -> dict:
76     params: dict = {}
77     for item in items or []:
78         key, _, raw = item.partition("=")
79         try:
80             params[key] = json.loads(raw)
81         except json.JSONDecodeError:
82             params[key] = raw
83     return params
84
85
86 def modal_run(args: list[str], detach: bool = False) -> int:
87     cmd = [MODAL, "run"] + (["--detach"] if detach else []) + [str(APP_FILE)] + args
88     return _sh(cmd).returncode
89
90
91 def cmd_run(ns: argparse.Namespace) -> int:
92     stages = [s for s in ns.stage.split(",") if s]
93     run_id = resolve_run_id(ns.run_id, create=ns.new_run)
94     params = parse_params(ns.param)
95     params.update(
96         {"code_ref": git_ref(), "size": ns.size, "submitted_at": datetime.now().isoformat(timespec="seconds")}
97     )
98     if ns.from_run:

```

```

99     params["from_run"] = ns.from_run
100 if ns.profile:
101     params["profile"] = ns.profile
102 stage = stages[0] if len(stages) == 1 else "all"
103 if stage == "all":
104     params["stages"] = stages
105 args = ["--stage", stage, "--run-id", run_id, "--params", json.dumps(params, ensure_ascii=False), "--size", ns.size
↵ ]
106 if ns.force:
107     args.append("--force")
108 if ns.spawn:
109     args.append("--spawn")
110 print(f"run_id={run_id} stages={stages} size={ns.size} spawn={ns.spawn}")
111 return modal_run(args, detach=ns.spawn)
112
113
114 def cmd_exec(ns: argparse.Namespace) -> int:
115     code = Path(ns.file).read_text(encoding="utf-8") if ns.file else ns.code
116     if not code:
117         print("provide --code or --file", file=sys.stderr)
118         return 2
119     run_id = resolve_run_id(ns.run_id, create=False)
120     return modal_run(
121         ["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": code}, ensure_ascii=False)]
122     )
123
124
125 def cmd_provision(_: argparse.Namespace) -> int:
126     return modal_run(["--stage", "provision"])
127
128
129 STATUS_SNIPPET = r"""
130 import json
131 from pathlib import Path
132 run = RUN_DIR
133 if not run.exists():
134     print("run directory missing:", run)
135 else:
136     rows = []
137     for m in sorted(run.glob("*/manifest.json")):
138         d = json.loads(m.read_text())
139         err = (d.get("error") or {}).get("message", "")[:80]
140         digest = d.get("outputs_digest", "")[:12]
141         rows.append((d["stage"], d["status"], d.get("duration_s"), digest, len(d.get("outputs", {})), err))
142     print(f'{"stage":14s} {"status":10s} {"seconds":>9s} {"digest":12s} {"files":>5s} error')
143     for r in rows:
144         print(f"{r[0]:14s} {r[1]:10s} {str(r[2]):>9s} {r[3]:12s} {r[4]:5d} {r[5]}")
145     print("run_dir:", run)
146 """
147
148
149 def cmd_status(ns: argparse.Namespace) -> int:
150     run_id = resolve_run_id(ns.run_id, create=False)
151     return modal_run(["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": STATUS_SNIPPET})])
152
153
154 def cmd_runs(_: argparse.Namespace) -> int:
155     return _sh([MODAL, "volume", "ls", PROJECT["volume"], "runs"]).returncode
156
157
158 def cmd_logs(ns: argparse.Namespace) -> int:
159     run_id = resolve_run_id(ns.run_id, create=False)
160     code = (

```

```

161     "from pathlib import Path\n"
162     f"p = RUN_DIR / {ns.stage!r} / 'events.jsonl'\n"
163     "print(p.read_text()[-%d:] if p.exists() else 'no events for stage')\n" % ns.tail
164 )
165 return modal_run(["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": code})])
166
167
168 def sha256(path: Path) -> str:
169     h = hashlib.sha256()
170     with path.open("rb") as fh:
171         for chunk in iter(lambda: fh.read(1 << 20), b""):
172             h.update(chunk)
173     return h.hexdigest()
174
175
176 def verify_dir(root: Path) -> int:
177     """Verify every downloaded stage directory against its manifest (transport check only)."""
178     problems = 0
179     manifests = sorted(root.rglob("manifest.json"))
180     if not manifests:
181         print(f"no manifests under {root}")
182         return 1
183     for manifest in manifests:
184         data = json.loads(manifest.read_text(encoding="utf-8"))
185         stage_dir = manifest.parent
186         bad = []
187         for rel, meta in data.get("outputs", {}).items():
188             file = stage_dir / rel
189             if not file.exists():
190                 bad.append(f"missing {rel}")
191             elif sha256(file) != meta["sha256"]:
192                 bad.append(f"sha256 mismatch {rel}")
193         status = "OK " if not bad else "BAD"
194         print(
195             f"[{status}] {data.get('stage'):14s} {data.get('status'):10s} "
196             f"files={len(data.get('outputs', {}))} {stage_dir}"
197         )
198         for b in bad:
199             print("    " + b)
200         problems += len(bad)
201     return 0 if problems == 0 else 1
202
203
204 def volume_get(remote: str, local_parent: Path) -> int:
205     """Download a remote directory into ``local_parent/<basename>`` (replacing any previous copy)."""
206     target = local_parent / Path(remote).name
207     shutil.rmtree(target, ignore_errors=True)
208     local_parent.mkdir(parents=True, exist_ok=True)
209     return _sh([MODAL, "volume", "get", "--force", PROJECT["volume"], remote, str(local_parent) + "/"]).returncode
210
211
212 def cmd_download(ns: argparse.Namespace) -> int:
213     run_id = resolve_run_id(ns.run_id, create=False)
214     if ns.stage:
215         remote, parent = f"runs/{run_id}/{ns.stage}", REPO / ns.dest / "runs" / run_id
216     else:
217         remote, parent = f"runs/{run_id}", REPO / ns.dest / "runs"
218     rc = volume_get(remote, parent)
219     if rc != 0:
220         return rc
221     return verify_dir(parent / Path(remote).name)
222
223

```

```

224 def cmd_verify(ns: argparse.Namespace) -> int:
225     return verify_dir(Path(ns.path))
226
227
228 def cmd_release(ns: argparse.Namespace) -> int:
229     run_id = resolve_run_id(ns.run_id, create=False)
230     dest = REPO / "releases" / ns.version
231     if dest.exists() and not ns.force:
232         print(f"{dest} exists; pass --force to overwrite", file=sys.stderr)
233         return 2
234     staging = REPO / "artifacts" / "release_staging" / run_id
235     rc = volume_get(f"runs/{run_id}/release", staging)
236     if rc != 0:
237         return rc
238     shutil.rmtree(dest, ignore_errors=True)
239     shutil.copytree(staging / "release", dest)
240     manifest = json.loads((dest / "release_manifest.json").read_text(encoding="utf-8"))
241     bad = 0
242     for name, meta in manifest["files"].items():
243         file = dest / name
244         ok = file.exists() and sha256(file) == meta["sha256"]
245         bad += 0 if ok else 1
246         print(f"[{'OK' if ok else 'BAD'}] {name} {meta['bytes']} bytes sha256={meta['sha256'][:16]}")
247     print(f"release {ns.version} from run {run_id}: {'verified' if not bad else f'{bad} problems'} -> {dest}")
248     return 0 if not bad else 1
249
250
251 def cmd_wait(ns: argparse.Namespace) -> int:
252     interpreter = (
253         Path(MODAL).read_text(encoding="utf-8", errors="ignore").splitlines()[0].lstrip("#!").strip()
254         if Path(MODAL).exists()
255         else sys.executable
256     )
257     code = (
258         "import json, modal, sys\n"
259         f"call = modal.FunctionCall.from_id({ns.call_id!r})\n"
260         "print('STAGE_RESULT ' + json.dumps(call.get(), ensure_ascii=False, default=str))\n"
261     )
262     return _sh([interpreter, "-c", code]).returncode
263
264
265 def cmd_fmt(ns: argparse.Namespace) -> int:
266     """Format in the cloud, then copy the exported files back into the working tree."""
267     run_id = resolve_run_id(ns.run_id, create=False)
268     params = {"code_ref": git_ref(), "size": "small"}
269     rc = modal_run(["--stage", "fmt", "--run-id", run_id, "--params", json.dumps(params), "--force"])
270     if rc != 0:
271         return rc
272     dest = REPO / "artifacts" / "runs" / run_id / "fmt"
273     rc = volume_get(f"runs/{run_id}/fmt", dest.parent)
274     if rc != 0:
275         return rc
276     files = dest / "files"
277     changed = sorted(p for p in files.rglob("*") if p.is_file()) if files.exists() else []
278     for src in changed:
279         rel = src.relative_to(files)
280         target = REPO / rel
281         target.parent.mkdir(parents=True, exist_ok=True)
282         shutil.copy2(src, target)
283         print(f"updated {rel}")
284     report = json.loads((dest / "fmt_report.json").read_text(encoding="utf-8"))
285     print(f"{len(changed)} files formatted; remaining ruff findings: {report.get('remaining')}")
286     return 0

```

```

287
288
289 def cmd_new_run(_: argparse.Namespace) -> int:
290     run_id = new_run_id()
291     remember(run_id)
292     print(run_id)
293     return 0
294
295
296 def build_parser() -> argparse.ArgumentParser:
297     p = argparse.ArgumentParser(prog="forge", description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter
↳ )
298     sub = p.add_subparsers(dest="command", required=True)
299
300     r = sub.add_parser("run", help="run one stage or a comma-separated sequence in one container")
301     r.add_argument("stage")
302     r.add_argument("--run-id")
303     r.add_argument("--new-run", action="store_true", help="start a fresh run id")
304     r.add_argument("--size", default="small", choices=["small", "medium", "large"])
305     r.add_argument("--force", action="store_true", help="re-run even if the stage already completed")
306     r.add_argument("--spawn", action="store_true", help="detach: return immediately, poll with status/wait")
307     r.add_argument("--from-run", help="copy missing dependency outputs from this earlier run")
308     r.add_argument("--profile", help="overlay configs/<profile>.toml")
309     r.add_argument("--param", action="append", help="key=value (JSON-parsed) forwarded to the stage")
310     r.set_defaults(fn=cmd_run)
311
312     e = sub.add_parser("exec", help="run an ad-hoc Python snippet in the cloud against the run directory")
313     e.add_argument("--code")
314     e.add_argument("--file")
315     e.add_argument("--run-id")
316     e.set_defaults(fn=cmd_exec)
317
318     sub.add_parser("provision", help="verify the cloud toolchain").set_defaults(fn=cmd_provision)
319
320     s = sub.add_parser("status", help="print manifests of a run")
321     s.add_argument("--run-id")
322     s.set_defaults(fn=cmd_status)
323
324     lg = sub.add_parser("logs", help="tail the event log of a stage")
325     lg.add_argument("stage")
326     lg.add_argument("--run-id")
327     lg.add_argument("--tail", type=int, default=6000)
328     lg.set_defaults(fn=cmd_logs)
329
330     sub.add_parser("runs", help="list runs on the volume").set_defaults(fn=cmd_runs)
331
332     d = sub.add_parser("download", help="download a run (or one stage) into artifacts/ and verify hashes")
333     d.add_argument("--run-id")
334     d.add_argument("--stage")
335     d.add_argument("--dest", default="artifacts")
336     d.set_defaults(fn=cmd_download)
337
338     v = sub.add_parser("verify", help="verify a downloaded directory against its manifests")
339     v.add_argument("path")
340     v.set_defaults(fn=cmd_verify)
341
342     rel = sub.add_parser("release", help="download the release stage into releases/<version>/ and verify")
343     rel.add_argument("--version", required=True)
344     rel.add_argument("--run-id")
345     rel.add_argument("--force", action="store_true")
346     rel.set_defaults(fn=cmd_release)
347
348     w = sub.add_parser("wait", help="block on a spawned function call id and print its result")

```

```

349     w.add_argument("call_id")
350     w.set_defaults(fn=cmd_wait)
351
352     f = sub.add_parser("fmt", help="ruff --fix + format in the cloud and copy the results back")
353     f.add_argument("--run-id")
354     f.set_defaults(fn=cmd_fmt)
355
356     sub.add_parser("new-run", help="mint a run id").set_defaults(fn=cmd_new_run)
357     return p
358
359
360 def main(argv: list[str] | None = None) -> int:
361     os.chdir(REPO)
362     ns = build_parser().parse_args(argv)
363     return int(ns.fn(ns))
364
365
366 if __name__ == "__main__":
367     sys.exit(main())

```